Foundational ideas of modern AI

Sutskever's List



Richard Heimann

Foreword by Sebastian Raschka

MANNING

Praise for Sutskever's List

If you want to learn where today's methods came from, what motivated them, and how a relatively small set of ideas helped define the trajectory of the field, this is an excellent place to begin.

—From the Foreword by Sebastian Raschka
author of *Build a Large Language Model (From Scratch)*

Provides a clear, concise and well-contextualized overview of the most important breakthroughs in AI research since the 1990s. It's a highly enjoyable read, with the author weaving the personalities of Sutskever and his peers into the broader narrative.

—Nico Smuts, Data Science Executive

The author refuses to treat Sutskever's List as what it superficially appears to be—just a reading list, a paper after a paper that you have to treat in isolation. Instead, the author reconstructs it as a carefully argued intellectual journey.

—Francisco Perez-Sorrosal, independent AI Engineer and Advisor

The book is instrumental in forming a mental picture of the main ideas that shaped today's achievements in AI and their interplay.

—Stefano Lottini, Software Engineer

The author has an exquisitely deep, detailed, and nuanced knowledge of the subject area.

—Wendy Langer, Good Stuff! Tutoring

Sutskever's List *is all you need if you want to learn the history, and breadth and depth of deep learning by understanding foundational research papers! Author's intuitions at the end of each chapter have so much wisdom and insights—you will love reading them.*

—Bhavin Thaker, New Relic

I like how the author made this book into a story. I enjoyed it thoroughly.

—Jay Kelkar, Kelkar Systems

Chapter 8 gave me a clear conceptualization of complexity and grokking research I didn't have before.

—Leo Huovinen, Tampere University

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Sutskever's List

FOUNDATIONAL IDEAS OF MODERN AI

RICHARD HEIMANN

FOREWORD BY SEBASTIAN RASCHKA

MANNING

SHELTER ISLAND

Licensed to Hermilo Cortés <hermilocg@gmail.com>

For online information and ordering of this and other Manning books, please visit www.manning.com. The publisher offers discounts on this book when ordered in quantity. For more information, please contact

Special Sales Department
Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964
Email: orders@manning.com

©2026 by Manning Publications Co. All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by means electronic, mechanical, photocopying, or otherwise, without prior written permission of the publisher.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in the book, and Manning Publications was aware of a trademark claim, the designations have been printed in initial caps or all caps.

♾ Recognizing the importance of preserving what has been written, it is Manning's policy to have the books we publish printed on acid-free paper, and we exert our best efforts to that end. Recognizing also our responsibility to conserve the resources of our planet, Manning books are printed on paper that is at least 15 percent recycled and processed without the use of elemental chlorine.

The author and publisher have made every effort to ensure that the information in this book was correct at press time. The author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause, or from any usage of the information herein.


Manning Publications Co.
20 Baldwin Road
PO Box 761
Shelter Island, NY 11964

Development editor: Doug Rudder
Technical editor: Tulika Bhatt
Review editor: Kishor Rit
Production editor: Kathy Rossland
Copy editor: Kari Lucke
Proofreader: Olga Milanko
Typesetter and cover designer: Marija Tudor

ISBN 9781633434790
Printed in the United States of America

Licensed to Hermilo Cortés <hermilocg@gmail.com>

To my wife and three boys:
Thank you for feigning credulity every time I said,
"just one more hour of writing."

Licensed to Hermilo Cortés <hermilocg@gmail.com>

brief contents

1 ■ What did Ilya see? 1
2 ■ The AlexNet moment 17
3 ■ ResNet revolution 42
4 ■ Deep learning accelerates 66
5 ■ Attention is all you need 95
6 ■ The birth of hyperscale 129
7 ■ The pivot to reasoning 153
8 ■ Simplicity, hidden in complexity 187
9 ■ Safe superintelligence 216

epilogue *The missing pieces 245*

appendix *Design patterns for engineers 261*

references 267

index 307

Licensed to Hermilo Cortés <hermilocg@gmail.com>

contents

foreword xi
preface xiii
acknowledgments xiv
about this book xvi
about the author xix
about the cover illustration xx

1 ***What did Ilya see?*** 1
1.1 Ilya's rise 3
1.2 The GPT-2 controversy 7
1.3 The list 12

2 ***The AlexNet moment*** 17
2.1 Feature engineering 20
2.2 Pre-AlexNet skepticism 22
2.3 ImageNet 25
2.4 Why training is hard 27
2.5 AlexNet 30
Network architecture 30 ▪ *Training innovations* 33 *Data augmentation* 36 ▪ *Efficient and scalable training* 37 ▪ *The effect* 39

Licensed to Hermilo Cortés <hermilocg@gmail.com>

3 *ResNet revolution 42*

3.1 Telephone game 43

3.2 Comparison with other major architectures 49

ResNet vs. AlexNet and ZFNet 49 ▪ ResNet vs. VGGNet 49 ▪ ResNet vs. GoogLeNet 50

3.3 Real-world adoption 51

3.4 ResNet v2 51

Post-activation to pre-activation 52 ▪ Ablation studies 52

3.5 Extending ResNet with dense prediction 55

3.6 Human measuring sticks 59

3.7 CS231n 62

3.8 Scale what? 63

4 *Deep learning accelerates 66*

4.1 The unreasonable effectiveness of RNNs 67

4.2 Understanding LSTM networks 74

4.3 Recurrent neural network regularization 79

4.4 Deep Speech 2 83

Core architecture 83 ▪ Training techniques 85 Language models and decoding 87 ▪ Significance and broader influence 88 ▪ An engineering shift 90

5 *Attention is all you need 95*

5.1 Transformers 96

Empirical results 102 ▪ Universal transformer 103

5.2 The Annotated Transformer 106

5.3 Bahdanau attention 107

5.4 Just point to it! 113

Experimental results 114 ▪ Significance 117

5.5 Neural Turing Machines 117

Significance 119

5.6 Order matters 119

5.7 Reversing input sentences 122

5.8 Effects 125

Licensed to Hermilo Cortés <hermilocg@gmail.com>

6 *The birth of hyperscale 129*

6.1 Scaling canon 130

Kaplan scaling laws 131 ▪ Does shape matter? 133 Scaling with data and model size 134 ▪ Scaling compute 135 ▪ Architecture comparisons 137 The scaling hypothesis 137 ▪ Chinchilla and post-Chinchilla 138

6.2 Scaling detours 140

Jaggedness 141 ▪ Contamination 143

6.3 Parallelism 145

GPipe 146 ▪ Results 147 ▪ Wise to pipeline? 149

6.4 Rounding errors 152

7 *The pivot to reasoning 153*

7.1 Variational lossy autoencoder 154

What Is VLAE? 156 ▪ Results 157 Autoregressive prior 158

7.2 Relational reasoning 159

Benchmark evaluation 160

7.3 Neural message passing for quantum chemistry 162

Accuracy as a bottleneck 163 ▪ Message passing neural networks 164 ▪ Training and results 166 Towers architecture 167 ▪ Influence 168

7.4 Relational recurrent neural networks 169

Experimental methodology and empirical results 172 Historical context 175

7.5 Reasoning models 176

Distributional brittleness 177 ▪ Reversal curse 179 ▪ Prompting as a reasoning interface 180

7.6 Paper doubts 182

8 *Simplicity, hidden in complexity 187*

8.1 Coffee automaton 188

Methodology 189 ▪ Results 191

Licensed to Hermilo Cortés <hermilocg@gmail.com>

8.2 Kolmogorov complexity and algorithmic randomness 194
Algorithmic statistics 195 ▪ Significance 198
8.3 A tutorial introduction to the minimum description length principle 198
8.4 Keeping artificial neural networks simple 202
Methodology 202 ▪ Results 203 ▪ Limitations 204 Cultural influence 205
8.5 Grokking 205
Compression 208 ▪ Theory to practice to theory 209 Double descent 212
8.6 Blurry JPEG 215

9 Safe superintelligence 216
9.1 Machine superintelligence 217
Defining intelligence 217 ▪ Universal intelligence measure 219 ▪ AIXI 221 ▪ Environment taxonomy 223 ▪ Agents and limits 224
9.2 AI safety 225
Intelligence explosion 226 ▪ Technological singularity 226 ▪ AGI (as a field) is born 227 Superintelligence 228 ▪ AI foom 229 ▪ Effects 229
9.3 Explaining too little and promising too much 231
Legg's functionalism 232 ▪ A hint of behaviorism 234 ▪ Meaning in use 235 ▪ Turing's test 236 ▪ Definitions masquerade as explanations 238 ▪ Public tests 240 ▪ Dirty hands, clean(er) claims 241

epilogue The missing pieces 245
appendix Design patterns for engineers 261
references 267
index 307

Licensed to Hermilo Cortés <hermilocg@gmail.com>

foreword

I have always liked reading papers. In my view, a good paper does more than present a result. It explains what problems the researchers wanted to solve, what assumptions they made, what was not working at the time, and why a new idea seemed worth pursuing. In that sense, papers are one of the best ways to understand a field, because they not only present the methods but also the motivations that produced them.

That is one of the reasons I enjoyed and recommend this book. It takes a good mix of influential papers and turns them into a guided tour through the development of modern AI. And rather than treating these works as a mere list, it shows how they connect. You can see how one line of work created the conditions for the next and how practical issues shaped research priorities that ultimately led to the methods we use today.

This perspective is especially useful today, because modern AI moves quickly, and it is easy to encounter current methods only in their applied form. For example, AlexNet was a response to the limitations of handcrafted features and the difficulty of making deep learning work for computer vision at scale. Residual networks addressed the optimization problems that came with the increasing depth of these deep neural nets. Sequence models, attention mechanisms, and Transformers emerged because earlier approaches ran into concrete bottlenecks in capturing and using information over long contexts. Later, scaling laws helped formalize how performance improves in systematic ways when model size, data, and compute are increased together.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

One of the strengths of this book is that it focuses on the motivations and not just the results. That's because once we understand the original motivation behind a method, we are in a much better position to judge whether its core idea still matters and which lessons continue to generalize. This is also why reading older papers remains so valuable for practitioners and researchers getting started in a field. Even when specific implementation details become outdated, the problem formulations and design choices often remain highly relevant, and they help us learn to formulate our own ideas and approaches to design the next generation of methods.

For readers who like papers, this book offers a particularly rewarding kind of experience, and it helps restore context to works that are often reduced to citations, memes, or benchmark references. And for readers who are newer to this literature, it provides an accessible path into a body of work that can otherwise feel scattered across subfields and eras.

In that sense, this book is both historical and practical. It looks back at the papers that shaped modern AI, but it also gives readers a better framework for understanding the present. If you want to learn where today's methods came from, what motivated them, and how a relatively small set of ideas helped define the trajectory of the field, this is an excellent place to begin.

—Sebastian Raschka
Author of *Build a Large Language Model (From Scratch)*
Founder and Principal AI/LLM Research Engineer, RAIR Lab

Licensed to Hermilo Cortés <hermilocg@gmail.com>

preface

In late 2023, a question ricocheted online: "What did Ilya see?" The backdrop was a dramatic power struggle at OpenAI, where Ilya Sutskever had reportedly been alarmed enough to support ousting the CEO. Elon Musk's public musings on the matter fueled speculation. Amid the commotion, one intriguing clue to Sutskever's mindset resurfaced: a personal reading list of research papers he had shared with John Carmack, informally known as "Sutskever's List." Sutskever allegedly claimed it captured "90% of what matters today" in AI. The notion that a single collection of papers could hold the keys to modern AI lent the list an aura of mystery and importance.

I first learned of Sutskever's List through these whispers and online treasure hunts. Enthusiasts pieced together clues, and eventually a reconstructed version of the list appeared, quickly gaining almost a million views. It became a cultural touchstone. To ask, "Have you read Sutskever's List?" was shorthand for claiming a handle on the fundamentals of contemporary AI. And yet, this question was often more signaling, as many knew of the list without truly engaging with the papers on it. In a field racing forward with speed, the idea of a stable canon of ideas felt refreshing and necessary. I realized that exploring these works and the context in which they arose could provide invaluable insight into how AI evolved into what we see today.

This book grew out of that realization.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

acknowledgments

My deepest thanks go to the researchers and engineers whose work animates *Sutskever's List*. Their hard-won victories have moved the field forward. I am grateful to the pioneers of deep learning who challenged conventional wisdom and proved skeptics wrong, to the problem-solvers who engineered systems that made AI work in the real world, and to the educators and communicators who clarified these complex ideas for a broader audience.

I am especially grateful to those whose skepticism rose above paper doubt into a living demand for better evidence—those who did not merely declare the limits of AI but sharpened benchmarks, exposed brittle claims, and compelled the field to earn confidence through results. Their doubts strengthened the field more than paper hopes and doubts could ever.

I am also deeply grateful to Manning Publications for giving this project a home and for guiding it with care from a rough proposal to a finished book. Jonathan Gennick, my acquisitions editor, believed in the project early and helped bring it into being. Doug Rudder, my developmental editor, helped sharpen its structure, pacing, world-building, and voice. My thanks also go to Tulika Bhatt, whose technical review improved the accuracy of the book. Tulika is a senior software engineer at Netflix, where she works on real-time data systems and personalization infrastructure at scale. Lastly, I am honored that Sebastian Raschka contributed the foreword; his work as a researcher, educator, and communicator inspires me and has helped make modern machine learning more accessible to countless readers.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

To all the reviewers: Adi Shavit, Anirban Majumder, Arslan Gabdulkhakov, Arturo Geigel, Bhavin Thaker, Bin Hu, Bryan Cardillo, Casey Robinson, Jérémie Clos, Thomas Briegel, Emin Tahirovic, Frances Buontempo, Francisco Perez-Sorrosal, Georgerobert Freeman, Giovanni Alzetta, Hardev Ranglani, Henry Beveridge, Ian Yang, Ioannis Atsonios, Iurii Iurchenko, Jay Kelkar, Jean-François Morin, Jerry Kuch, Joaquin Gracia, John Williams, Jonathan Thoms, Joseph Catanzarite, Julian Squires, Kostas Passadis, Krishna Kandi, Krzysztof Kamyczek, Leo Huovinen, Lucas Roberts, M. Milan Loseke, Manoj Agarwal, Marielle Dado, Mengyue (J.P.) Li, Milorad Imbra, Natasha Chong, Nico Smuts, Nicolas Bievre, Payam Pourashraf, Praveen Nair, Prof. Hyung-Jong (John) Kim, Ravi Kiran Bamidi, Recep Erol, Ruben Gonzalez-Rubio, Sanny Mulyono, Sidharth Mahotra, Stefano Lottini, Stephen Wolff, Surbhi Madan, Vitosh K. Doynov, Wendy Langer, and William Springer: your suggestions helped made this a better book.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

about this book

The goal of *Sutskever's List* is to illuminate the connections between key breakthroughs, placing each into a narrative that spans the deep learning revolution of the 2010s and early 2020s. Rather than treating each paper as an isolated achievement, the chapters weave them together to tell a larger story that connects technological innovation with shifting paradigms, cultural milestones, and evolving philosophies within AI. In these pages, you will discover what problems each solved, what doors they opened, and even what controversies or questions they raised. Examining the landmark research through the eyes of Ilya Sutskever, the book offers a cohesive framework for understanding how we arrived at today's state of AI and where we might be heading.

You will meet the researchers who carried the torch of deep learning, the engineers who scaled algorithms, and even the critics and skeptics who challenged the hype. We also highlight how ideas build on one another: how a trick in one experiment enabled a breakthrough in the next and how a theoretical insight found its way into practical systems. In doing so, *Sutskever's List* serves both as a map of modern AI's intellectual terrain and as a commentary on its journey. Whether you are here to learn the backstory of famous algorithms or to grasp the mindset of an AI pioneer, the book's guiding purpose is the same: to shed light on how foundational AI research became the world-transforming force we now witness.

Intended audience

Sutskever's List is the kind of book people might actually enjoy reading—and finish—which is more than can be said for most books in machine

Licensed to Hermilo Cortés <hermilocg@gmail.com>

learning and AI, which are generally written to be referenced, not read. Specifically, this book is written for a broad but technically curious audience. Its primary audience is software engineers, data scientists, and machine learning practitioners. If you build or work with AI models, this book will enrich your understanding of why those models exist in their current form and the key engineering patterns that enabled them.

However, *Sutskever's List* is also meant to be accessible to motivated general readers. We assume only a basic familiarity with computing and math. If you have ever read popular science accounts of technology or enjoyed learning the stories behind scientific breakthroughs, you will find a similar narrative approach here, albeit one that doesn't shy away from the technical heart of each idea. In short, this book welcomes anyone eager to learn how AI evolved, offering different layers of insight for different readers. An engineer might appreciate the finer technical points or historical references, while a non-specialist will come away with a solid conceptual understanding.

Code, software, and resources

One of the distinguishing features of this book is that no software or coding is required to engage with it. This is not a hands-on programming guide but rather a conceptual journey through ideas and history. No code downloads, libraries, or sandbox environments are required. All of the important algorithms and experiments are described and discussed within the text, so you won't need to run anything on your own machine. The "output" is understanding, not software.

Guidance for readers

The chapters are organized in a logical order, and reading them sequentially is recommended, especially if you're new to AI. If you start from chapter 1 and move forward, you'll follow a storyline that gradually builds context, including concepts and terminology that later chapters will build upon or reference. For example, understanding how convolutional networks triumphed in vision (chapter 2) will add depth to your appreciation of the challenges in training deeper networks (chapter 3); knowing what RNNs and LSTMs are (chapter 4) sets the stage for the Transformer's innovation (chapter 5); and so on. The narrative is designed to be cumulative, revealing recurring themes and evolving perspectives as you progress. Reading straight through will give you a cohesive understanding of Sutskever's worldview and how the field of AI transformed over time.

That said, the book is also structured to allow selective reading. Each chapter focuses on a distinct theme and a specific set of papers, so if you

Licensed to Hermilo Cortés <hermilocg@gmail.com>

have a particular interest in one topic—say, you're most curious about how Transformers work, or you want to jump directly into the discussion of AI safety—you can head directly to the relevant chapter. We have made an effort to ensure that each chapter provides enough background to stand on its own. Key concepts are reintroduced when necessary, and whenever we refer to an idea from an earlier chapter, we provide a brief recap or a pointer to where it was first explained. So, a motivated reader can certainly dip into chapter 5 on attention or chapter 9 on superintelligence and still follow the argument. If you choose this approach, you might occasionally skip some cross-references, but you will still gain insight into that slice of the story.

Whether you read it cover-to-cover or jump around, consider using the references as a tool for deeper exploration. If a particular idea fascinates you, the references can direct you to the primary source for more detailed treatment. In addition, the appendix includes key takeaways and engineering principles for each chapter, which can help reinforce what you've learned.

Finally, the field of AI is full of debates. As you read, you'll encounter these threads woven through the narrative. They include the balance between empiricism and theory, ethics and risks, safety and progress, and what "intelligence" even means. Engaging with them actively will enrich your experience. By the end, you will have not only a clearer picture of where AI's core ideas came from but also a framework to think about where AI is going and how we, as readers and practitioners, might navigate that future.

liveBook discussion forum

Purchase of *Sutskever's List* includes free access to liveBook, Manning's online reading platform. Using liveBook's exclusive discussion features, you can attach comments to the book globally or to specific sections or paragraphs. It's a snap to make notes for yourself, ask and answer technical questions, and receive help from the author and other users. To access the forum, go to https://livebook.manning.com/book/sutskevers-list/discussion.

Manning's commitment to our readers is to provide a venue where a meaningful dialogue between individual readers and between readers and the author can take place. It is not a commitment to any specific amount of participation on the part of the author, whose contribution to the forum remains voluntary (and unpaid). We suggest you try asking the author some challenging questions lest his interest stray! The forum and the archives of previous discussions will be accessible from the publisher's website as long as the book is in print.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

about the author



RICH HEIMANN is a researcher, practitioner, educator, and communicator. With over a decade of experience with machine learning, he has witnessed firsthand the field's rapid transformation. The author has been involved in projects spanning from fundamental algorithm development to applied AI systems. This hands-on experience is complemented by a passion for pedagogy and public engagement in science and technology. In academia, the author has taught machine learning courses and mentored students, earning a reputation for demystifying complex concepts without dumbing them down.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

about the cover illustration

The figure on the cover of *Sutskever's List*, titled "Le Maraicher," or "The Market Gardener," is taken from a book by Louis Curmer published in 1841. Each illustration is finely drawn and colored by hand.

In those days, it was easy to identify where people lived and what their trade or station in life was just by their dress. Manning celebrates the inventiveness and initiative of the computer business with book covers based on the rich diversity of regional culture centuries ago, brought back to life by pictures from collections such as this one.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

1 What did Ilya see?

This chapter covers

- Ilya Sutskever's journey to prominence
- The founding of OpenAI as a counterbalance to Google
- GPT-2's surprising capabilities and controversy
- The OpenAI board crisis and Altman's ouster with Ilya at the center
- The enduring mystery and influence of Sutskever's List

In November 2023, Elon Musk tweeted, "Something scared Ilya enough to want to fire Sam. What was it?" Soon after, the phrase "What did Ilya see?" went viral, sparking curiosity among AI researchers and enthusiasts [1][2]. Had Ilya Sutskever, co-founder of OpenAI, glimpsed an existential risk that others missed? Many thought so.

These moments reflect a deeper tension at AI's frontier, but to understand them, we must look at the broader intellectual landscape that shaped Ilya's thinking. Specifically, we need to examine "Sutskever's List," a collection of nearly 30 papers, blogs, and books

Licensed to Hermilo Cortés <hermilocg@gmail.com>

(for the sake of simplicity, we'll call all of them "papers" in this book), blogs, and books (for the sake of simplicity, we'll call all of them "papers" in this book) he once described to legendary programmer John Carmack as containing "90% of what matters today."

Such clarity is elusive, though. Rather than treating each paper as an isolated contribution, this book explores them as interconnected threads in a larger narrative. In this way, the book is more than an anthology. It's for those who want to understand how these ideas connect, what they originally meant and still mean today, and what they reveal about one of AI's most transformative periods. The book doesn't just analyze technical milestones. It weaves together cultural moments, paradigm shifts, blind spots, and the field's ambitions and anxieties. In doing so, it transforms what could be a dry reference work into a cohesive and engaging narrative.

We'll follow Ilya's intellectual evolution, tracing the shifting perspectives that shaped his work. Each item on Sutskever's List—spanning research papers, blog posts, and even a college course—offers insight into what matters most to him. Through this list, we create a portrait of its compiler and a map of its code, surfacing themes including the tension between accelerationism and caution, scaling and alignment, success and failure, and ambition and uncertainty.

While the book centers on Ilya, it is not a biography. Instead, his intellectual journey serves as a narrative throughline as we explore broader developments in AI. His decisions, from his work on AlexNet to his time at Google, the founding of OpenAI, and his eventual departure, intersect with and mirror the field's own trajectory.

In chapter 1, we trace Ilya's rise to become one of AI's most influential and enigmatic thinkers. We introduce *Sutskever's List* and outline Ilya's mental model. The chapter focuses on GPT-2 (a model notably absent from the list) because it illustrates the real-world implications of Ilya's work and philosophy. When GPT-2 was declared "too dangerous to release," it marked the first public clash between early AI scaling and safety. This tension would deepen, shaping OpenAI's culture and eventually leading to a public crisis. We examine GPT-2 not because it is the beginning or end of the story, but because it marks a point at which the trajectory of modern AI became more visible.

This chapter provides context for the deeper technical discussions that follow. It clarifies why innovations such as AlexNet, ResNet, and Transformers matter. As the book progresses, the intensity and urgency of this opening chapter inevitably give way to more technical detail. But it's here that we begin to establish the concerns, assumptions, and direction

Licensed to Hermilo Cortés <hermilocg@gmail.com>

of travel. As we move through the list, we'll explore Ilya's cautionary vision and why it continues to resonate today.

1.1 Ilya's rise

Ilya Sutskever pursued graduate studies at the University of Toronto, where he studied under the "godfather of AI," Geoffrey Hinton [3]. At the time, Hinton was widely respected but had not yet reached the legendary status he holds today, due in small part to Sutskever's work. In September 2012, Alex Krizhevsky, Sutskever, and Hinton trained a convolutional neural network (CNN) to classify images. Their model, AlexNet, dramatically reduced error rates in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) and decimated the competition [4].

This single achievement largely dispelled decades of skepticism. The victory provided empirical evidence skeptics could not ignore [5]. As people discussed the results, even Yann LeCun, a longtime advocate of CNNs, was astonished. He called the victory "an unequivocal turning point in the history of computer vision" [6]. Decades of handcrafted vision approaches had been toppled by an artificial neural network trained end-to-end on raw images [7].

In 2013, Google recognized the significance of the team's work and acquired DNNresearch, the startup founded by Hinton, Sutskever, and Krizhevsky. The acquisition brought that expertise in-house as deep learning began to reshape the field [8].

At Google Brain, Sutskever became central to advancing deep learning [9][10]. He was an early collaborator on TensorFlow, Google's open source deep learning library, and co-authored the seminal AlphaGo paper [11]. In addition, several projects on *Sutskever's List* emerged directly out of his years at Google, including pioneering work on recurrent neural network regularization and indirect support for influential architectures such as Pointer Networks and Neural Turing Machines [12].

Yet by 2015, a new opportunity was taking shape. Sam Altman, then the president of Y Combinator, invited Sutskever to a private dinner with Elon Musk and others [13][14]. Unbeknownst to Sutskever, he was the guest of honor. The dinner conversation centered on the future of AI, including whether incumbents like Google were unbeatable or if a new research lab could be created as a "counterbalance."

The idea of a counterbalance had gained urgency in the weeks prior. At a party, Musk clashed with Larry Page, Google's co-founder, after Page shared his vision of a future in which humans and machines compete for resources. Musk responded bluntly, "If that happens, we're doomed."

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Page dismissed the concern and called Musk a "specieist" [15]. Shaken by Page's surreal claim that preferring humans to machines amounted to speciesism, Musk wanted to build a counterweight.

The whole idea resonated with Sutskever. On *The Robot Brains* podcast, Ilya recalled: "I really wanted it to be engineering-heavy. And seeing that Elon was going to be involved, I thought, well, who better? I couldn't imagine a better person from whom to learn the big engineering project side of things" [16]. After some hesitation and months of counteroffers, Sutskever left Google. Musk later described Ilya as the "linchpin" of the new venture, underscoring how critical his involvement was to the project's credibility [17][18].

OpenAI was founded in late 2015. Ilya became OpenAI's chief scientist and the "soul of the company and its driving force in research," shaping OpenAI's research agenda and culture [19][20][21]. Sutskever's influence ensured that the work remained ambitious, prompting bold, even grandiose claims, such as building "systems that can do things humans were not capable of doing before" [22]. But this statement lacked a clearly articulated goal. OpenAI discussed balancing openness with safety and debated the pace at which it should pursue artificial general intelligence (AGI). These debates reveal an organization shaped by broad aspirations rather than precise strategic direction.

But the discussions didn't stay theoretical forever. OpenAI soon began translating its philosophical musings into practical milestones. The lab developed OpenAI Gym, a platform to accelerate reinforcement learning research [23]. It also achieved early breakthroughs in AI-driven gameplay, notably mastering the complex multiplayer game Dota 2 [24]. By 2018, these efforts had converged into focused experiments with Transformers, culminating in the release of a language model called GPT-1. This shift from broad uncertainty to targeted exploration marked a pivotal moment for OpenAI. It also coincided with an internal reconfiguration, as Elon Musk parted ways due to conflicts with his role at Tesla [25].

Behind the scenes, though, it emerged that Musk proposed taking more direct control of OpenAI because he was concerned it was falling behind Google [26]. The other co-founders rejected Musk's bid, leaving Altman and Sutskever firmly in charge of OpenAI's direction [27]. Sutskever chose to stand by the nonprofit rather than cede control to Musk. This reflected a principled stance that the company should stay focused on its long-term, safe AGI goals.

OpenAI's research accelerated. In 2019, Sutskever oversaw GPT-2 [28]. By 2020, the lab had embraced scaling. That June, it unveiled GPT-3, a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

175-billion-parameter model that stunned observers with its versatility [29]. The project demanded extensive engineering, and under Sutskever's vision, the team built the infrastructure to make it possible. He is widely credited as a chief architect of GPT-3.5, which was the first model behind ChatGPT [30].

As OpenAI's creations grew more interesting, Sutskever's reputation as one of the foremost research minds in the field grew. This period also solidified his image as a bit of a recluse: "I lead a very simple life," Ilya said in one interview. "I go to work, then I go home. I don't do much else. There are a lot of social activities one could engage in, lots of events one could go to. Which I don't" [31].

Sutskever also became more publicly reflective about the implications of OpenAI's research. In February 2022, nearly a year before ChatGPT was released, he sparked significant debate by posting a curious statement on Twitter (now X): "It may be that today's large neural networks are slightly conscious" [32]. The tweet was provocative and broached a topic many researchers treat skeptically: machine consciousness. Sutskever offered no further explanation and did not delete the remark, even after it drew criticism and incredulity.

While some researchers, such as Andrej Karpathy, supported the assertion by responding, "agree … consciousness is a useful insight for compression," others were more skeptical. Turing Award winner Yann LeCun responded with "nope," and Murray Shanahan, a professor at Imperial College London and principal scientist at Google DeepMind, added, "in the same sense that it may be that a large field of wheat is slightly pasta" [33][34][35]. The episode highlighted Sutskever's willingness to voice unconventional thoughts. While most experts dismissed the idea, the comment foreshadowed Ilya's increasing focus on AI's philosophical and safety dimensions.

Increasingly, Ilya became a spiritual evangelist for safe AGI within the company. OpenAI launched a new "Superalignment" team co-led by Sutskever and researcher Jan Leike. The team's goal was to steer a potentially superintelligent AI and prevent it from "going rogue" within four years. In a widely read blog post published eight months after the release of ChatGPT, Sutskever stressed the urgency, suggesting that while superintelligence might feel distant, it "could happen this decade" [36][37].

Reports from this period portray him as a passionate, if eccentric, figure who rallied the company around the mission of creating beneficial AGI. He reportedly encouraged employees to "feel the AGI," treating the quest for artificial general intelligence with an almost reverential fervor

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[38]. In one notable anecdote, Sutskever commissioned a wooden effigy representing an "unaligned" superintelligent AI that could harm humanity and ceremonially burned it before his colleagues.

These ritualistic displays, described in an exposé of OpenAI's internal culture, were unusual even by tech industry standards. They illustrated Sutskever's intense conviction and the sense of gravity he brought to the alignment task. Some insiders saw this as inspirational, while others found it worrying [39]. Either way, by late 2023, Sutskever had shifted toward a more urgent, cautionary stance that would soon compel him to confront OpenAI's trajectory head-on.

Tensions came to a head on November 17, 2023, when the OpenAI board, on which Sutskever served, abruptly ousted Sam Altman [40]. Sutskever was reportedly a key instigator behind this decision [41]. Publicly, the board attributed Altman's dismissal to a vague "breakdown of communications," an ironically uncommunicative phrase that barely concealed a deeper strategic rift [42]. In Ilya's view, OpenAI was moving fast and might soon cross a dangerous threshold if left unchecked.

Reports emerged that, just before Altman's termination, several OpenAI researchers, likely from Sutskever's alignment team, had written a letter to the board warning of a new breakthrough they believed could pose an existential threat [43]. They referenced a secret research project codenamed Q* (pronounced Q-star) that, according to insiders, showed an ability to solve math problems at a grade-school level. While long division hardly seems to justify such extreme action, some researchers felt it could scale into a more general reasoning machine and urged the board to consider the safety implications [44].

The Altman firing echoed the unresolved tensions first glimpsed during the GPT-2 release, a pivotal event we'll explore in greater depth in the next section. In both instances, OpenAI took preemptive actions based on anticipated risks, not immediate threats. The thread that connects them is a deeply held belief, championed by Sutskever and many AI safety researchers: that acting too late is worse than acting prematurely. Yet while the GPT-2 decision was largely symbolic, Altman's termination had real consequences, transforming hypothetical concerns into an institutional crisis.

After the event, the question "What did Ilya see?" captured the industry's sudden existential dread [45][46]. But, as with GPT-2, the feared catastrophe never arrived, leaving OpenAI to reckon with a crisis entirely of its own making. The irony is that the impulse to guard against catastrophe caused one.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The aftermath was chaotic. OpenAI's staff were shocked. At a tense all-hands meeting, Ilya attempted to justify the decision, telling employees that the board was "doing its duty" [47]. The move backfired, with no clear communication about which specific safety concern or misstep by Altman had prompted it. Within 48 hours, more than 700 of OpenAI's 770 employees had signed an open letter threatening to quit and follow Altman to whatever new venture he pursued unless the board reversed course [48].

Sutskever became the target of intense criticism following what many saw as a coup that jeopardized the company's stability. Under this pressure, and with OpenAI facing a likely collapse, Sutskever reversed his stance. On November 20, just three days after the board's decision, Ilya apologized, tweeting: "I deeply regret my participation in the board's actions," adding, "I never intended to harm OpenAI" [49][50].

The mea culpa was extraordinary: it's rare for a board member to disavow a decision, underscoring how dramatically the situation had spun out of control. For Sutskever, the episode was personally and professionally bruising. He stepped down from the OpenAI board in the wake of the fiasco [51]. For months afterward, reports suggested Sutskever had not been actively working at OpenAI. Some said he was on leave; others said he was quietly leading his team from afar. Either way, it was evident his internal influence had waned [52]. The Altman saga illustrated Sutskever's profound commitment to AI safety and his willingness to risk everything on principle. But it also illustrated the perils of such a unilateral approach, with a hint of religious fanaticism.

In May 2024, Ilya and OpenAI "parted ways," an outcome many saw as inevitable [53]. Sam Altman announced Sutskever's exit in a respectful internal message, calling him "easily one of the greatest minds of our generation" and lamenting the loss of a once-in-a-generation intellect whose influence had shaped OpenAI from the start [54]. That same week, Jan Leike, Sutskever's co-lead on the Superalignment team, also resigned, citing an erosion of trust in OpenAI's direction [55][56]. With both leaders gone, OpenAI dismantled the Superalignment team [57]. Ilya's departure transformed the unease that began with GPT-2 into something personal and significant.

1.2 The GPT-2 controversy

On February 14, 2019, *The Guardian* published an article titled "New AI Fake Text Generator May Be Too Dangerous to Release, Say Creators" [58]. The piece went viral and highlighted OpenAI's decision to withhold the full-size GPT-2 model, citing fears of malicious use. Accompanying the article was a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

black-and-white photograph of George Orwell, cigarette in mouth, evoking a dystopian sensibility that echoed the themes of 1984. The reference to Orwell and the phrase "deepfakes for text" in the second line of the article establishes a connection between the emerging capabilities of generative artificial intelligence and the potential for misinformation.

GPT-2 is a transformer-based language model with 1.5 billion parameters, trained on about 8 million web pages scraped from Reddit. It was a direct scale-up of GPT-1, with more than 10 times the parameters and training data. Vox's Kelsey Piper wrote that GPT-2 was "one of the coolest AI systems I've ever seen," so humanlike that it "may also be the one that will kick me out of my job" [59]. Reporters marveled at GPT-2's zero-shot abilities. Without task-specific training, it could generate news articles from headlines or answer reading comprehension questions, achieving results that seemed unimaginable for a language model [60].

TechCrunch reported that GPT-2 produces "longer text with greater coherence" than prior models, calling it a vast improvement over the original GPT-1 model [61]. Even OpenAI's researchers were taken aback when the model produced a persuasive essay arguing a counterintuitive viewpoint, describing it as "something you could have submitted to the SAT and get a good score on" [62]. Such anecdotes spread quickly, fueling excitement that AI had reached a new level of sophistication, which it had.

The model's technical significance crystallized in a cherry-picked sample known as "Ovid's Unicorn." Starting from a whimsical prompt about English-speaking unicorns discovered in the Andes, GPT-2 generated multiple paragraphs of stylistically plausible text, complete with a fictional scientist, a university affiliation, and journalistic conventions such as introducing the scientist by his full name and then referring to him by surname, along with fabricated quotes. This was all without an explicit memory mechanism. Although GPT-2 still hallucinated "four-horned" unicorns and "fires burning underwater," revealing a fragile underlying "world model," the prose's convincing form and long-range coherence reset expectations of what language models could achieve. The model's fluency was an eye-opener for experts accustomed to earlier models that would quickly go off the rails.

But some researchers noted that while GPT-2's underlying techniques were not new, its scale made the difference. "There is no algorithmic contribution here ... they are 'just scaling up' previous research, but seeing exactly how strong these scaled-up models are is an important contribution in its own right," one researcher noted, adding that "if I had shown the [GPT-2] unicorn story to a colleague ... and told them that it was

Licensed to Hermilo Cortés <hermilocg@gmail.com>

AI-generated, I don't think they would have believed me" [63]. The community recognized GPT-2 as proof that scaling can yield startling new performance levels.

OpenAI described GPT-2 as "so good… and the risk of malicious use so high" that it was "breaking from its normal practice" of open release and publication. GPT-2 is a primer for what's to come, including GPT-3, scaling laws, emergent behavior, and the OpenAI board coup. It also reflects what came before, when publishing was the default and sharing models was the norm.

Instead of releasing the full model, OpenAI provided a much smaller 124-million-parameter version, roughly the same size as GPT-1, and a research paper as "an experiment in responsible disclosure" [64]. The full GPT-2 model, training data, and code were all withheld, representing a significant departure from OpenAI's earlier commitment to open publication and a shift toward a more guarded, selective strategy.

Interestingly, this change in posture had been anticipated. In 2018, nearly a year before the GPT-2 release, OpenAI quietly updated its charter, reflecting an evolving perspective on its role in an increasingly precarious technological landscape. The revised charter stated, "We are committed to providing public goods that help society navigate the path to AGI. Today, this includes publishing most of our AI research, but we expect that safety and security concerns will reduce our traditional publishing in the future while increasing the importance of sharing safety, policy, and standards research." As a result, GPT-2's limited release seemed less like a sudden reaction to a technological breakthrough and more like the first public signal of an institutional pivot already underway.

Nevertheless, OpenAI presented its decision as grounded in specific risks. An official blog post outlined several scenarios that justified a more cautious release. The concerns included generating misleading news articles, impersonating individuals online, producing abusive or fake social media content, and automating spam and phishing campaigns. OpenAI argued that these scenarios meant a model like GPT-2 could be dangerous in the wrong hands. "Technologies are reducing the cost of generating fake content and waging disinformation campaigns," the blog warned, noting the need for skepticism of text just as "deepfakes" have taught skepticism of images.

Deepfakes are realistic AI-generated synthetic videos or images of people doing or saying things they never actually did. By 2019, they had already sparked public concern. Based on the generative adversarial network (GAN) framework introduced by Ian Goodfellow in 2014, these

Licensed to Hermilo Cortés <hermilocg@gmail.com>

models pit a generator against a discriminator to produce increasingly convincing forgeries [65]. Imagine a forger painting counterfeit masterpieces (generator) while an art expert attempts to identify the fakes (discriminator). Each time the expert uncovers a fake, the forger learns and improves. Over time, this competition makes it increasingly difficult to distinguish fakes from genuine artworks. GANs quickly became associated with ethical risks, including nonconsensual pornography, misinformation, and manipulated media. "Deepfakes for text" served as a caution against convincing synthetic text produced at scale.

The decision to withhold the model was presented as a proactive release strategy to give the community time to discuss its implications. OpenAI acknowledged, "We are not sure that it is the right decision today" but expressed hope that it would encourage more nuanced discussions about publication norms in AI, akin to debates in fields like biotechnology and cybersecurity. In 2019, treating a model as dual-use technology rather than as an academic publication and emphasizing safety was unconventional. Unsurprisingly, the decision was met with shock, skepticism, and, at times, derision.

The media portrayed GPT-2 as a dangerous "fake news generator," drawing public attention to both its capabilities and risks. *Wired* ran a story titled "The AI Text Generator That's Too Dangerous to Make Public" and quoted an OpenAI engineer who marveled at GPT-2's realism: "It looks pretty darn real," fearing it could enable "someone with malicious intent … to generate high-quality fake news" [66]. *TechCrunch* echoed the story with the blunt title "OpenAI Built a Text Generator So Good, It's Considered Too Dangerous to Release" [67].

Conversely, many experts believed that the narrative overshadowed more nuanced discussions. Anima Anandkumar, an AI professor at Caltech and director of machine learning research at NVIDIA at the time, told *The Verge* that OpenAI's approach was unnecessary from a safety standpoint and ultimately detrimental to the progress of AI research. Others in the field shared this perspective, noting that open publication is especially important when risks are involved because transparency enables researchers to identify and develop safeguards [68].

Many joked that OpenAI had become "ClosedAI," a comment on the organization's shift away from its open source roots. The "ClosedAI" meme was widely shared on X and Reddit in 2019 [69]. The community suspected OpenAI's caution was a publicity move. Skeptics noted that the dramatic framing generated media hype, benefiting OpenAI's profile, and "accusations that OpenAI is motivated by hype generation" emerged [70].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Although the controversy attracted attention, it was minor compared with the immense public focus OpenAI would soon receive from models like GPT-3 and ChatGPT.

Despite the criticism, OpenAI believed that a cautious approach was necessary. Policy researcher Miles Brundage emphasized that the objective wasn't merely to focus on GPT-2 but to raise awareness about broader risks: "We tried to highlight both the current capabilities of GPT-2 and the risks of a broader class of systems." Of course, this stance seems contradictory: it claims that GPT-2 was withheld in the name of caution, yet the move was justified by pointing to risks in future systems. If GPT-2 itself did not pose that risk, then withholding it cannot be defended on GPT-2-specific safety grounds.

OpenAI clarified its stance in a blog post titled "GPT-2: 6-Month Follow-Up," published on August 20, 2019 [71]. The post outlined a staged release strategy, beginning with a 124-million-parameter model in February, followed by a 355-million-parameter model in May, and then a 774-million-parameter model along with the six-month follow-up.

This staggered approach soon influenced others in the field. By the end of 2019, what had begun as a decision by one lab had evolved into a community-wide dialogue about managing increasingly general-purpose AI systems that might be misused. In collaboration with the University of Washington, Hugging Face and the Allen Institute for Artificial Intelligence adopted a similar strategy for releasing their language model, Grover [72][73]. In 2022, Google withheld public release of the Imagen diffusion model, citing ethical concerns [74]. In 2023, Meta restricted access to LLaMA 1, making it available only to approved academic researchers, government agencies, and civil society groups.

Looking back, the controversy surrounding GPT-2 stands out not because the model was especially good. It wasn't, at least by today's standards. GPT-2 wasn't particularly dangerous either. OpenAI acknowledged as much when it released the full 1.5 billion-parameter GPT-2 model in November 2019, noting, "We've seen no strong evidence of misuse so far" [75]. But GPT-2 was an early exemplar of risks that had not yet arrived. Its significance lies in its role as a harbinger of things to come.

We now know that far more powerful models would soon emerge. Indeed, some were withdrawn shortly after public release. In November 2022, Meta pulled Galactica three days after its release, amid concerns over its authoritative-sounding misinformation [76]. In 2024, Google followed a similar pattern, pulling Gemini days after apologizing for its historically inaccurate outputs [77]. In retrospect, what once appeared to be

Licensed to Hermilo Cortés <hermilocg@gmail.com>

premature caution about GPT-2 increasingly seems like a preview of the challenging decisions ahead.

GPT-2 marked the first time the AI community publicly confronted the tension between innovation and acceleration on one hand and risk mitigation and safety on the other. Today, as language models write code, summarize meetings, and impersonate public figures, the GPT-2 moment feels almost quaint. Still, it marked the first occasion in 75 years of research when anyone pressed pause, even though speculating about AI risk and control had long been a favorite pastime for researchers ranging from Turing and Wiener to Good and Minsky to Bostrom and Yudkowsky. These discussions largely remained theoretical, philosophical, or relegated to footnotes. In that sense, GPT-2 symbolizes a cultural moment, signaling AI's first significant fracture.

But the GPT-2 decision did not relieve any tensions. In fact, by declaring the model "too dangerous to release," OpenAI opened Pandora's box. By publicly acknowledging risks that neither existed nor were understood, OpenAI triggered a chain of events. This moment foreshadowed the crisis that later unfolded, leading to a slow march toward OpenAI's board firing CEO Sam Altman over vague safety concerns.

Theories swirled that Sutskever had pushed Altman out after discovering a dangerous capability or existential risk. The community latched onto the mystery, spawning countless posts and speculative threads [78][79]. Many fixated on the haunting question of what had frightened Ilya. The meme became shorthand for something deeper that underscores the historical anxieties at the heart of AI. It reflects a growing suspicion that those closest to the frontier may be seeing behaviors or trajectories that are not yet widely understood. Combined with the lore around Sutskever's List, it gives a picture of how much of the community views him. To some, he is a quiet, almost reclusive figure with unusual insight into the direction of modern AI; to others, he is one of the field's most accomplished engineers navigating uncertainty [80].

1.3 The list

Legendary game developer John Carmack faced a challenge. After reshaping the gaming world with iconic titles like *Wolfenstein*, *Quake*, and *DOOM*, Carmack turned his curiosity toward AI. But AI was uncharted territory. During a recruiting meeting with OpenAI's founders, Carmack asked how he could accelerate his learning. In response, Ilya Sutskever handed him something that quickly took on a life of its own: a curated reading list that Sutskever promised would deliver "90% of what matters today" in AI.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Carmack took the challenge to heart. “And I did. I plowed through all those things, and it all started sorting out [AI] in my head.”

When Carmack revealed that he had the list, it immediately captured the imagination of the AI community. The endorsement from a renowned technologist lent weight to the list’s importance. It promised to cut through the noise of thousands of publications and point straight to the most pivotal ideas. Carmack was struck by how the key advances were “not extreme black-magic mathematical wizardries.” He added, “A lot of them are relatively simple techniques that make perfect sense to me now that I understand them” [81]. The comforting hindsight in which complexity collapses into simplicity once understood encouraged newcomers and made AI seem approachable.

The notion that readers could access “90% of what matters” from so few references in a rapidly advancing field was tantalizing. The fact that Sutskever personally curated the list added to the allure, especially given his measured online presence. Despite having more than 481,000 followers on X, he posts infrequently, which only adds to the mystique. Sutskever’s reticence turns what might otherwise be a routine reading list into a totem.

Yet, despite Carmack frequently mentioning the list in interviews, its contents remain undisclosed. What’s interesting is that the secrecy didn’t cause people to lose interest. In fact, it had the opposite effect. As word spread on forums and social media, excitement grew, especially after ChatGPT became popular. ChatGPT was released in November 2022 and reached more than 100 million users within two months of its launch [82]. This acted as a catalyst, reigniting curiosity about the list. By early 2023, interest had reached a fever pitch. A Hacker News thread, “What were the papers on the list Ilya Sutskever gave John Carmack?” drew more than 130 comments of speculation and crowdsourced guesses [83]. So many people wanted the list that Carmack posted on X, expressing his hope that Ilya would make it public. Carmack wrote that “a canonical list of references from a leading figure would be appreciated by many” [82] . But that never happened.

The mystery sparked a treasure hunt as AI enthusiasts sought to reconstruct the list using clues and secondhand reports. In 2023, a researcher compiled a likely version and shared it online, reigniting the conversation. It has since been viewed almost a million times [84][85][86]. This book adopts this carefully reconstructed version.

The intrigue surrounding Sutskever’s List grew over time, and it became a common cultural touchstone. “Have you read Sutskever’s List?”

Licensed to Hermilo Cortés <hermilocg@gmail.com>

became shorthand for asking whether someone know the fundamentals of modern AI research, even if they hadn't actually read the items on the list. Just knowing about the list carried a certain cachet.

Today, a certain mythos still clings to it, partly because Ilya has never officially published it, despite Carmack's public nudging. Paradoxically, Sutskever's silence reinforces its appeal, drawing in everyone from eager enthusiasts to intrigued book publishers, some of whom, I can personally attest, found the temptation irresistible. I suspect the reason is that—in a field where breakthroughs arrive relentlessly—there's something uniquely comforting about the notion of a stable canon. Yet for many, it's more than a reading list. Many look at Sutskever's List and try to decode his worldview. What exactly does Ilya Sutskever believe we need to understand about AI? Is the list for safety hawks? alignment believers? friends of AGI? practitioners? In a word, yes.

The list includes papers that defined the deep learning revolution in computer vision, such as AlexNet and ResNet. It also highlights attention-based neural networks like Pointer Networks and Transformers. Recurrent and long short-term memory (LSTM) networks are also prominent, reflecting the dominant natural language processing paradigms before attention-based architectures. The list also includes pedagogical works such as Stanford's CS231n, Andrej Karpathy's "The Unreasonable Effectiveness of Recurrent Neural Networks," Chris Olah's influential essay "Understanding LSTM Networks," and Alexander "Sasha" Rush's "The Annotated Transformer."

The list highlights engineering innovations and includes papers like GPipe, which details pipeline parallelism for training large models. It also covers scaling laws, including OpenAI's 2020 study, which demonstrates how model performance improves as scale increases. These papers underscore Sutskever's interest in how far brute-force scaling can push the frontier and at what cost.

The list doesn't stop with engineering feats or new architectural designs. It also includes papers on minimum description length, Kolmogorov complexity, and algorithmic randomness—all topics Ilya regularly discusses in interviews. The presence of Scott Aaronson's blog, "The First Law of Complexodynamics" and his whimsical "Coffee Automaton" signals an openness to complexity theory.

In short, Sutskever's List is best read as a compact syllabus of ideas that have shaped modern deep learning, from CNNs and sequence models to attention, scaling, and the engineering patterns that made these systems

Licensed to Hermilo Cortés <hermilocg@gmail.com>

work in practice, with detours into complexity and compression-based intuitions.

This book analyzes the papers on the list primarily by theme, though it still progresses in a loose chronological order as it connects conceptual threads. Each paper is given dedicated attention, including its technical and cultural significance, though some are treated more concisely than others. Rest assured that every paper on the list will be explored without overlooking any essential insights.

Additionally, while the list guides the book's core narrative, covering roughly a decade from 2012 to 2022, our exploration extends beyond this window. The book selectively introduces earlier and later research to provide critical context, illuminate blind spots, clarify Sutskever's broader philosophical perspective, and trace the threads connecting older research to modern practice. Still, the primary focus remains securely tethered to these core papers and their immediate intellectual surroundings. Consequently, the book is not intended to be a complete history of AI or to solve the vexing problem of who should receive credit for various innovations.

Also, Manning books often adopt a "mental model" approach, offering readers a clear conceptual framework shaped by an author's experiences and insights. But this book is different. The mental model presented here is not exactly my own; instead, it reflects Ilya Sutskever's worldview, derived directly from the themes and papers in the list and shaped further by his public statements and interviews. By unpacking Ilya's mental model, readers gain more than historical context: they acquire a conceptual toolkit for interpreting past developments, anticipating future shifts, and guiding practical engineering trade-offs. In short, the book provides conceptual clarity and, where applicable, practical insights, all viewed through Ilya's eyes.

With this context in mind, let's introduce the foundational themes of Sutskever's worldview. Consider the following initial insights not as definitive conclusions but as starting points we'll refine as we explore more deeply the ideas shaping modern AI:

- *Don't bet against deep learning*—Ilya explicitly believes that "one doesn't bet against deep learning" [87]. This belief is reflected in the list, which highlights deep learning and its successes while omitting references to older symbolic approaches and classical planning. Remarkably, reinforcement learning (RL) is also missing, despite Sutskever leading OpenAI's early work on RL and its role in projects like AlphaGo and OpenAI Five. Instead, the list focuses almost entirely on supervised and self-supervised deep learning.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

- *Engineering pragmatism*—The list prioritizes large-scale engineering efforts over purely theoretical innovation. Influential projects such as AlexNet and Deep Speech 2 illustrate this philosophy, combining established techniques with significant computational resources to deliver unprecedented real-world performance gains. While critics dismiss this approach as merely incremental "engineering" rather than innovative "science," Ilya argues that tangible progress comes from pragmatic experimentation, iterative improvement, and real-world deployment rather than from isolated theoretical insights.
- *Do more with less at scale*—Sutskever's research philosophy is that genuine progress emerges from refining ideas at scale and uncovering what works through experimentation. For example, ResNets introduced a simple architectural trick called residual connections that unlocked the training of extremely deep models [88]. Transformers replaced recurrent architectures with parallelizable attention. Even a paper like "Order Matters," one of the more niche papers on the list, explores how rearranging sequence data can make training more efficient. These examples emphasize conceptual simplicity and practical results at scale and align with Ilya's preference for "minimum innovation for maximum results" [89].
- *Emergence and compression*—Sutskever's List explores Kolmogorov complexity, minimum description length (MDL), and even an essay pondering how complexity rises and then falls in closed systems, hinting at a philosophical bent in his thinking. Why would a busy AI engineer recommend reading about algorithmic randomness or the "coffee automaton"? Likely because Sutskever sees intelligence as a compression process that yields simpler, more abstract representations of complex phenomena. In this view, intelligence emerges when a system can distill raw experience into a minimal, generalizable form.

With that conceptual framing in mind, let's turn next to Sutskever's List. Before residual networks, transformers, and scaling laws, there was AlexNet. It marked the end of one era and the beginning of another.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

2 The AlexNet moment

This chapter covers

- Skepticism about artificial neural networks
- Feature engineering before AlexNet
- The difficulty of training artificial neural networks
- ImageNet's role in AlexNet's success
- The AlexNet moment and technical innovations

Papers

- *ImageNet Classification with Deep Convolutional Neural Networks* (2012) Krizhevsky, Sutskever, and Hinton

In 2012, Ilya, alongside Alex Krizhevsky and Geoffrey Hinton, trained a convolutional neural network to classify images. Their network stunned the AI community by dramatically reducing error rates in the emerging ImageNet Large Scale Visual Recognition Challenge (ILSVRC). AlexNet achieved a top-5 error rate of 15%, significantly outperforming con-

Licensed to Hermilo Cortés <hermilocg@gmail.com>

ventional methods reliant on handcrafted feature engineering, which had an error rate of around 26%. The top-5 error rate measures how often the correct answer is not among the model's top five guesses.

Although the original paper did not explicitly name the model, the community adopted "AlexNet," following an informal tradition of naming influential networks after their lead authors, as previously seen with *Hopfield Networks* (1982) after John Hopfield, *LeNet* (1998) after Yann LeCun, and *DanNet* (2011) after Dan Ciresan [1][2][3]. Later models like VGGNet from the Visual Geometry Group at the University of Oxford and ResNet (chapter 3) would shift toward more descriptive or brand-like names, often chosen directly by their authors. Nevertheless, after years of false starts, AlexNet demonstrated that artificial neural networks (ANNs) were remarkably effective at pattern recognition [4].

The AlexNet paper has become highly cited. By spring 2025, it had been cited over 174,000 times. While some reports cautioned that these models remained fragile, requiring substantial data and sometimes making peculiar errors, the overall tone during that period was celebratory. The revolution, it seemed, had arrived. AlexNet transformed ANNs from speculative curiosity into a strategic investment, marking a fundamental shift across academia, industry, and public perception. It is not an overstatement to say that this moment turned deep learning from a niche interest into a global phenomenon.

Yet to feel the full weight of the AlexNet moment, we must understand the world it entered. This includes the assumptions it defied, the failures that preceded it, and the quiet persistence of those who believed it would work.

Today, ANNs are taken for granted, yet their history has been characterized by brief moments of enthusiasm and long periods of disillusionment. A revival began in the 1980s when David Rumelhart, Geoffrey Hinton, and Ronald Williams popularized backpropagation, which provided a practical algorithm for training multilayer ANNs [5]. An ANN is learning when it adjusts its weights and biases based on feedback from training data to minimize prediction error. This learning process is guided by a loss function that measures the difference between the model's predictions and the correct answers. The model updates its parameters iteratively through gradient descent to minimize this loss and improve performance. Backpropagation computes the gradient of the loss with respect to each weight in the network, allowing the model to reduce errors during training. This breakthrough renewed optimism and helped lay the foundation for modern deep learning.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

A few years later, LeNet-5, named for its five-layer architecture, demonstrated the potential of a specialized ANN known as a convolutional neural network (CNN). The impressive performance in handwritten digit recognition indicated that CNNs could scale to real-world tasks, but that promise did not materialize. Training deeper networks was challenging, large datasets were scarce, and computing power remained limited. While connectionist techniques had shown promise on small problems, they did not generalize, and momentum faded. By the late 1990s, ANNs were seen as a dead end. One observer recalled that "[artificial] neural networks had become a backwater" and "many researchers had moved on." [6]. Indeed, throughout the 1990s and 2000s, ANNs were "almost entirely dismissed as researchers embraced other approaches" [7].

The academic community organized around methods with a solid theoretical foundation and good performance on limited data. For example, support vector machines (SVMs) are classifiers that draw a sharp boundary between categories. SVMs gained popularity due to convex optimization, an appealing mathematical property that ensures globally optimal solutions can be found efficiently [8].

In contrast to SVMs, ANNs lack theoretical guarantees. They were difficult to analyze and were correctly perceived as slow black boxes. Training a multilayer network on high-dimensional data could take days, often leading to problems like exploding or vanishing gradients, which we will discuss in detail in subsequent chapters.

Moreover, these networks were prone to overfitting without large datasets, which were nearly impossible to obtain. They also tended to get stuck in poor local minima or points where the error is low, but not the lowest possible, preventing further improvement. Lacking today's data, compute resources, and architectural enhancements, ANNs of the era failed to deliver the promised gains, prompting many researchers to abandon them [9]. As a 2015 deep learning review by Geoffrey Hinton, Yann LeCun, and Yoshua Bengio noted, even CNNs, which had some success in the 1990s, were "forsaken by the mainstream computer vision and machine learning communities until the ImageNet competition in 2012" [10].

Only a small group of believers kept the research alive throughout this period. Hinton, LeCun, Bengio, Jürgen Schmidhuber, and a handful of others continued to develop ideas during the 2000s. Their efforts, including Hinton's research on deep belief nets in 2006 and Bengio's greedy layer-wise training in 2007, were acknowledged but failed to persuade the broader academic community, which remained skeptical [11][12][13]. The prevailing wisdom favored more practical methods. In hindsight,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

these researchers sustained the field during the AI winter. Yet AlexNet's crucial optimization insights finally bridged the gap between these dormant theoretical ideas and practical breakthroughs.

2.1 Feature engineering

To appreciate the significance of AlexNet's breakthrough, we must understand the dominant paradigm it disrupted. During the 2000s, all high-performing computer vision systems relied on handcrafted hierarchical feature pipelines. Local descriptors such as SIFT (Scale-Invariant Feature Transform) and SURF (Speeded-Up Robust Features) were used to detect and describe image patches [14][15][16]. Meanwhile, bag-of-visual-words models, histograms of quantized SIFT descriptors, and spatial pyramids encoded global representations [17]. These global representations transformed local patch descriptors such as SIFT and SURF into a single histogram "vocabulary" for an image. Researchers then fed these engineered features into shallow learning algorithms like SVMs and boosted decision trees, which map input features directly to outputs without learning intermediate or hierarchical representations. This approach remained remarkably stable and continued to deliver state-of-the-art results throughout that period. Even as late as 2012, the winning entry in the PASCAL Visual Object Classes (VOC) challenge used this approach.

Computer vision researchers preferred handcrafted features because they incorporated prior knowledge about visual invariances. For example, SIFT was designed to account for scale and rotation variations, whereas HOG (Histogram of Oriented Gradients) effectively captures edge patterns for shape detection. These features were considered reliable and interpretable precisely because researchers could explain what they were measuring.

The stability of hand-designed features came with a hidden cost. It hard-coded a particular idea of what "signal" should look like, leaving the model with no way to revise those assumptions when reality didn't cooperate. A few people, including Ilya Sutskever, had begun to notice that [18]. A HOG-based detector may respond strongly to irrelevant edge patterns or struggle with minor image distortions, such as detecting a "car" in a patch of water, whereas a learned CNN feature could better handle such cases [19]. In situations like this, the classifier faithfully separates the inputs it is given. The failure is upstream, where the feature representation confused car-like edges with a car. See figure 2.1.

In contrast, automatic feature learning with ANNs was opaque and depended on large datasets. The prevailing wisdom was that effective

Licensed to Hermilo Cortés <hermilocg@gmail.com>

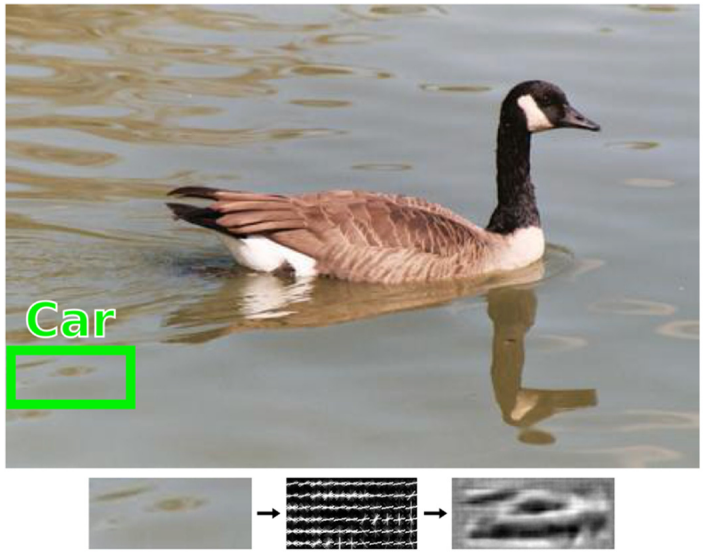


Figure 2.1 This figure shows a goose swimming (top), where an object detector mistakenly labels a small patch of rippled water as a car. Below the image are three visualizations that clarify the source of this error: a close-up of the water patch (left), the corresponding HOG features (middle), and a specialized visualization (right) showing that the rippled water's features resemble those associated with cars in HOG feature space. ***Used with permission granted by the lead author (Carl Vondrick).***

features *needed* to be created by hand [20]. A few contrarians argued that features should be learned directly from data, but none succeeded. The arguments remained theoretical because no learned features could match the performance of methods like SURF, HOG, or SIFT [21]. The field placed more trust in human intuition than in data and believed features were better designed than discovered.

The Deformable Parts Model (DPM) exemplifies the mindset of the pre-deep learning era [22][23]. HOG and SIFT captured the specific shape information present in the training data. When combined with an SVM, they achieved strong performance on tasks such as pedestrian detection [24]. By the late 2000s, this approach had been refined into the DPM, which used HOG features and a latent structured SVM to represent objects as collections of parts with learnable spatial relationships [25]. While DPM added some geometric flexibility, it also introduced hard negative mining, in which every false positive was added back to the training set as a hard negative. The model was then retrained to carve out a new decision boundary around that mistake. DPM wasn't just trained on

Licensed to Hermilo Cortés <hermilocg@gmail.com>

PASCAL. Its geometry was literally sculpted around the dataset's statistical quirks. This high performance masked poor generalization. In hindsight, DPM was the high-water mark of hand-engineered features [26].

These weaknesses inadvertently shaped research priorities. In 2011, Alexei "Alyosha" Efros and Antonio Torralba highlighted such concerns, casting doubt on the robustness of even the best handcrafted pipelines [27][28]. Computer vision had become benchmark-driven and dataset-bound, optimized for performance on contrived benchmarks rather than real-world generalization. Whether knowingly or not, computer vision was optimizing for a definition of success of its own choosing. Efros and Torralba exposed a blind spot in the entire field, and everything collapsed once multilayer ANNs could be trained end-to-end. But the breakthroughs that would eventually validate their promise still lay ahead, requiring not only algorithmic innovations but also a fundamental shift in mindset.

2.2 Pre-AlexNet skepticism

During this era, many leading figures in machine learning doubted that ANNs could rival established methods. Jitendra Malik, a prominent computer vision researcher at the University of California, Berkeley, exemplified the academic skepticism that defined the field. His stance reflected a broader "show-me" attitude, in which researchers demanded clear empirical evidence. Aware of this skepticism, Geoffrey Hinton approached Malik in 2011 and asked what kind of evidence would be convincing. Malik replied, "You really want to impress me? Show me they can handle the PASCAL VOC challenge" [29].

Malik's reply reflects a philosophical stance similar to Russell's Teapot, where the burden of proof rests with the claimant rather than on others to disprove it. After all, Malik's skepticism was reasonable at the time. Despite a few promising wins in smaller European vision challenges (including DanNet), ANNs had not yet delivered a clear breakthrough on the field's most-watched benchmarks, where handcrafted feature pipelines remained the standard. The vision community was waiting for evidence strong enough to overcome well-founded doubts. While this is a healthy posture in any scientific discipline, the tone of the exchange may also reveal a field that had been burned before.

Many felt, "We tried artificial neural networks in the 1990s, but they didn't work. Why would they work now?" Past failures loomed large, and disappointments hardened this widespread sentiment. The community hadn't forgotten the "second wave" of ANNs that began in the 1980s and "ran out of steam" by the late 1990s [30]. Younger researchers inherited

Licensed to Hermilo Cortés <hermilocg@gmail.com>

this skepticism, often seeing ANNs as anachronistic and unorthodox [31]. By 2011, the field had become incredulous, adopting a "show me, don't tell me" stance. No amount of theoretical promise or anecdotal success would be enough; researchers demanded to be overwhelmed, but not by improvements driven by data alone.

One researcher who clearly articulated this skepticism was Alyosha Efros, who argued that many apparent breakthroughs in ANNs were driven more by larger datasets than by genuine algorithmic innovation. Like many at the time, Efros was unconvinced that ANNs benefited from scale in a way fundamentally different from simpler models. He believed that, given enough data, a simpler method could perform just as well. In effect, his critique suggests that ANN researchers (often called connectionist researchers) attributed too much of their limited success to their architectures and too little to their training data [32].

Efros recalls reviewing a 2008 paper in which an ANN outperformed his earlier work on image localization. Suspecting that the gain resulted from larger datasets, he asked the authors to evaluate a simple nearest neighbor algorithm on the same dataset. This method works by comparing a new point to all known examples and labeling it according to the closest match. It relies purely on data similarity rather than learned features. The authors found that the ANN performed no better than this simple approach. "If you keep the data size constant, the fancy schmancy [artificial] neural network was doing no better than a simple nearest neighbor," Efros said. He added, "I'm not saying 'forget [artificial] neural networks … but in this particular setting, it was really data that was doing all the main lifting" [33]. This stance gained traction as the field began to focus on larger datasets. The rise of "big data" reinforced the view that data scale, not architecture, was the primary engine of progress, as data was dubbed "the new oil" [34][35].

In 2009, while at Google, Alon Halevy, Peter Norvig, and Fernando Pereira published "The Unreasonable Effectiveness of Data," a short essay that has since been cited more than 2,700 times [36]. They argue that for many natural language processing problems, the limiting factor isn't the elegance of our linguistic theories or the sophistication of our modeling machinery but coverage. Thus, scaling up data, even if it's noisy and unannotated, makes it possible to learn important tail events that smaller, curated corpora simply don't contain. Their provocation isn't that modeling choices never matter but that once training sets become sufficiently large, the marginal gains from added structure shrink. The paper bluntly states, "Simple models and a lot of data trump more elaborate models based on less data."

Licensed to Hermilo Cortés <hermilocg@gmail.com>

This position aligns with Efros' critique that, without data, "fancy schmancy" models amount to little more than elaborate scaffolding, while simple baselines such as nearest neighbors can appear surprisingly competitive. In this spirit, "The Unreasonable Effectiveness of Data" treated scale, not architecture, as decisive, framing data as a universal equalizer and suggesting that any statistical approach benefited from scale. The paper made no mention of ANNs; instead, it foregrounded n-gram language models. These simple statistical models predict words based on short sequences of preceding words (for example, "the cat" ? "sat"). The concept appears seven times in the paper [37]. At the time, even Google recognized the futility of pursuing ANNs.

The lesson was clear: ANNs weren't alchemical. They didn't conjure intelligence from structure alone. At best, they reflected the data they were trained on, leading many to conclude that any method with more data might suffice. Both "The Unreasonable Effectiveness of Data" and Efros's broader critique argue that researchers should first amass significantly more data. In that view, better performance came from giving even ordinary statistical methods enough material to work with.

In the folktale Stone Soup, a hungry traveler arrives at a village whose residents insist they have no food to spare. Undeterred, the traveler begins boiling water with a few stones and casually remarks that it would taste much better with carrots, onions, barley, and even chicken. One by one, the villagers contribute the missing ingredients. By the end, the pot transforms into a delightful communal meal, and the villagers marvel at how such a wonderful soup emerged from just a few stones.

The modern retelling features researchers who promise to create AI [38][39]. The researchers suggest it would perform better with all their data, including images, text, reinforcement learning from human feedback, and increasingly careful prompting. Users readily comply. The resulting system is praised as if it were the sole source of intelligence, even though the substance came from collective contributions poured into the pot [40].

There is considerable truth in this modern retelling. The field needed more ingredients, and data wasn't getting enough credit. Yet, unlike the tale's universal recipe, not all algorithms benefit equally. While the belief that more data is better was rightfully gaining momentum, the idea that data alone is a great equalizer masked a deeper truth: some architectures benefit disproportionately from scale. ImageNet would soon provide abundant data, but rather than democratizing the field, it crowned a single champion, and it wasn't SIFT+SVM, nearest neighbors, or even n-grams.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

2.3 ImageNet

Jitendra Malik's challenge to Hinton was explicit: "Show me they can handle the PASCAL VOC challenge" [41]. PASCAL VOC was a modest-sized but meticulously annotated dataset that also served as a fierce proving ground for computer vision algorithms. For Malik, VOC was the serious benchmark that could separate hype from substance. "Show me they [artificial neural networks] can handle something serious," Malik added. Hinton responded, "No luck. It's just too small." Hinton pressed for an alternative. Malik told Hinton, "Fine, if not PASCAL, then go win ImageNet." If ANNs were to surpass the handcrafted paradigm, this would be where they'd have to prove it [42]. The gauntlet was thrown.

AI researcher Fei-Fei Li began developing ImageNet at Princeton in 2006. The project was inspired by cognitive psychologist Irving Biederman, who sought to explain the remarkable efficiency of human visual recognition. Biederman proposed that recognizable objects such as tables, lamps, and mugs are composed of a limited set of basic shapes, or "geons" (geometric icons), including cylinders, blocks, wedges, cones, and donut-shaped rings [43]. Having identified 36 such geons, he suggested that humans perceive roughly 30,000 meaningful shape combinations, a number known as the "Biederman number." This convergence between cognitive theory and large-scale visual categorization provided a conceptual foundation for ImageNet [44]. The ImageNet dataset and accompanying paper were released in 2009, with the first major competition taking place the following year.

Eleven teams participated in the first competition, and the winning entry combined handcrafted descriptors—including SIFT and Local Binary Patterns (LBP)—with an SVM [45][46]. The 2011 competition saw only four teams compete and fewer registrations overall. It was again won by an SVM-based approach from Xerox Research Centre Europe, which replaced LBP with compressed Fisher vectors based on SIFT [47]. Fei-Fei Li had bet that more data would yield better performance, but the early results were underwhelming. Traditional techniques dominated but delivered only minor incremental improvements over PASCAL VOC rather than breakthroughs. Researchers were still crafting features by hand, and the scale of ImageNet offered diminishing returns without an architecture capable of learning representations from its increased size and diversity. ImageNet was at serious risk of becoming a beautiful but useless monument to scale.

Fei-Fei Li built ImageNet to push the field of computer vision and the limits of visual recognition algorithms through scale. But Li's ambitious

Licensed to Hermilo Cortés <hermilocg@gmail.com>

endeavor was met with skepticism from peers, including Malik, who cautioned her against leaping too far ahead of the field. Malik was one of Li's heroes, but he warned his protégé, "I think you've taken this idea way too far … The trick is to grow with your field. Not to leap so far ahead of it" [48][49]. Such a pursuit was seen as overkill. Researchers were not ready for or even interested in this kind of scale.

Meanwhile, early signs of end-to-end learning were clearly emerging. In 2011, Dan Cireşan's "DanNet" showed that CNNs consistently outperformed hand-engineered pipelines. Not only was it the first CNN to win computer vision competitions, it won four in a row, including the International Conference on Document Analysis and Recognition (ICDAR, 2011), the International Joint Conference on Neural Networks (IJCNN, 2011), the IEEE International Symposium on Biomedical Imaging (ISBI, 2012), and the International Association for Pattern Recognition (IAPR, 2012) [50][51][52]. In the IJCNN traffic sign challenge, DanNet achieved 99.46% accuracy, outperforming the runner-up by 1.15 percentage points (~68% reduction in error). Still, DanNet's success landed softly because these were small, specialized benchmarks. It demonstrated that CNNs could outperform feature engineering, but it didn't address whether they would still perform at the ImageNet scale.

Krizhevsky, Sutskever, and Hinton recognized that ImageNet was the perfect battleground. It was large enough to allow a multilayer ANN to learn feature representations. In 2012, ImageNet was smaller in absolute terms than it is today. Upon its release in 2009, the dataset contained 3.2 million images and more than 5,000 categories. In 2012, ImageNet-1k, a widely used subset in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) from 2012 to 2017, contained approximately 1.2 million images across 1,000 categories. Yet in relative terms, ImageNet was enormous. For comparison, in 2012, PASCAL VOC included 20 categories and approximately 11,000 images; MNIST had 10 categories and 70,000 images; CIFAR-10 had 10 categories and 60,000 images; and the German Traffic Sign benchmark had approximately 50,000 images [53]. Today, ImageNet contains more than 14 million labeled images across more than 22,000 categories, converging on the Biederman number.

While the DanNet victories were technically impressive, they were easier to dismiss as niche demonstrations than as a paradigm shift. Meanwhile, AlexNet didn't just win a contest; it beat a benchmark considered "ahead of its time." Hinton chose ImageNet not because the community saw it as the ultimate benchmark or proving ground for computer vision algorithms but because no one thought it would work. ImageNet was

Licensed to Hermilo Cortés <hermilocg@gmail.com>

considered excessive and impractical. By decisively winning the ILSVRC, AlexNet created the stage where its victory mattered. That difference best explains why history remembers AlexNet as the breakthrough and DanNet as an essential precursor [54].

2.4 Why training is hard

A critical factor in training an ANN is how the network's weights are initialized. Weight initialization sets the starting values for each parameter before training begins. Random weights are commonly used to ensure that each computational block in the network, known as a neuron, learns something unique. If all weights are initialized to the same value—such as zero or any constant—every neuron in a layer receives the same gradient during backpropagation and updates identically. This prevents neurons from diverging and learning distinct features, effectively collapsing the layer into a single unit. Random initialization breaks this symmetry, allowing neurons to specialize and the network to model more complex patterns [55].

But bad initialization distorts the signal as it moves through a network. If weights start too large, neuron outputs can grow excessively after applying an activation function, causing gradients to explode or neurons to saturate. This instability prevents the model from reaching a stable solution, or "converging" [56]. Gradients can also vanish if weights are initialized too small, leading to slow or stalled learning [57]. These problems become particularly pronounced as networks grow deeper.

To understand why, consider how gradients are computed during backpropagation. They are the product of multiple partial derivatives as you move backward through the layers. If these derivatives are less than 1, the product shrinks with depth, leading to vanishing gradients. The result is that weights in earlier layers receive updates so small that they effectively stop learning. This vanishing gradient problem led many to conclude that depth itself was the problem.

Conversely, if the derivatives exceed 1, the gradient can explode, leading to erratic weight updates that prevent convergence. Researchers at the time believed that multilayer networks were fundamentally difficult to train, not only because of limitations in data and compute but also because the optimization landscapes were riddled with poor local minima and saddle points, making learning unstable. While GPUs would later accelerate training, they did not make optimization easier.

These problems were compounded by early activation functions such as the sigmoid and the hyperbolic tangent (tanh), which further weakened gradients during backpropagation. As a result, deeper networks not only

Licensed to Hermilo Cortés <hermilocg@gmail.com>

required more time to train but also performed worse. All of this reinforced the belief that multilayer networks were inherently unstable and required special tricks to train. If some of these ideas feel dense, it's okay. We'll return to them several times throughout the book, each time in a new context, so that the intuition behind them develops naturally. What's important here is that, before AlexNet, training networks with any real depth was considered impractical.

To escape this dilemma, the dominant approach to training ANNs was to use layer-wise unsupervised pretraining followed by supervised fine-tuning. Instead of training the entire network at once, each layer was trained individually. The output of one layer served as the input to the next, building the network incrementally. Pretraining brought the network weights much closer to a good solution, allowing backpropagation to succeed where it had previously failed. Once all layers had been pretrained this way, they were "unrolled" into a single architecture [58]. This initialization helped the model avoid poor local minima and improved convergence during training. After unrolling, the entire network could be fine-tuned end-to-end using backpropagation.

This strategy was introduced by Geoffrey Hinton in 2006 and expanded by Yoshua Bengio in 2007. Bengio showed that unsupervised pretraining could effectively unlock deep architectures that were otherwise untrainable with gradient-based methods [59][60]. These results were striking, but they also gave rise to a new orthodoxy. Researchers began to believe that deep networks would inevitably fail without pretraining. Against this backdrop of skepticism and stalled progress, James Martens's 2010 research paper stood out. It significantly influenced Ilya, who has frequently cited it as a key inspiration for AlexNet [61]. Martens's work directly challenged the prevailing belief that unsupervised pretraining was necessary for training multilayer networks. Instead, he showed that models could be trained from scratch with random initialization.

Martens reframed the problem. The difficulty wasn't due to an abundance of local minima, as widely assumed, but rather to limitations of first-order methods such as gradient descent, which update weights using only the slope of the loss function. These methods move in the direction of steepest descent but are blind to the landscape's curvature. In contrast, second-order methods also use information about how that slope changes, the curvature or "bending" of the loss surface, allowing them to adjust step size and direction more intelligently.

Martens demonstrated that ANNs often contain regions of "pathological curvature." These regions (see figure 2.2) are long, narrow valleys that

Licensed to Hermilo Cortés <hermilocg@gmail.com>

cause optimization algorithms to stall. Gradient-based methods lack curvature awareness. They either oscillate uselessly or make tiny, incremental steps, which dramatically slows training. Martens proposed a second-order approach that uses curvature to traverse these valleys efficiently. He applied it successfully to autoencoders—models that, until then, had been trainable only with unsupervised layer-wise pretraining. His insights revealed that the actual barrier wasn't the loss landscape itself but the inadequate tools for navigating its geometry [62]. The results were striking. As Martens noted in the paper's abstract, "Without using pre-training, we obtain results superior to those reported by Hinton & Salakhutdinov (2006) on the same tasks they considered" [63].

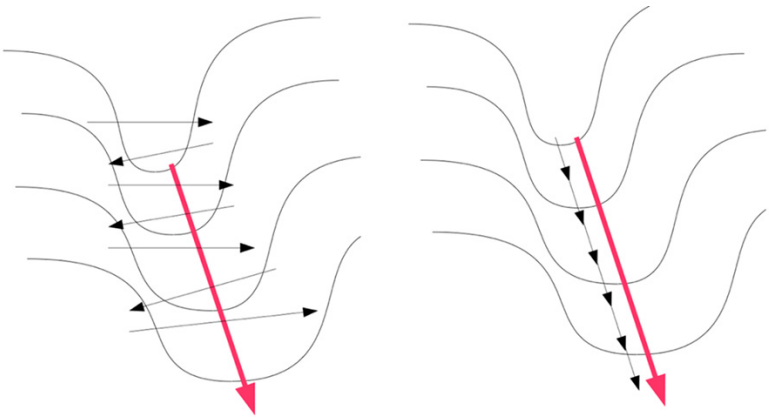


Figure 2.2 The challenge of optimizing functions shaped like long, narrow valleys. The contour lines represent a valley, with the optimal direction of progress indicated by an arrow running along the valley's base. Smaller arrows represent steps taken by gradient descent: the left diagram shows large steps oscillating inefficiently across the valley, while the right diagram shows small steps making slow progress along the base. The "pathological" nature does not arise from high or low curvature, but from the combination of high curvature across the valley and low curvature along it. *Used with permission granted by the author (James Martens).*

More important than the results was the underlying principle. The success wasn't due to any specific model architecture or clever initialization scheme. It was due to a more capable optimizer that could navigate the complex loss surfaces. Where traditional methods like gradient descent would stall, Martens's approach could better understand the shape of the loss landscape and follow more direct paths toward lower error. Martens had proven that unsupervised pretraining wasn't necessary. The real obstacle wasn't depth but the lack of tools to deal with curvature.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Martens acknowledged Sutskever in the 2010 paper, and the two even collaborated in 2011 on another notoriously difficult architecture to train: the long short-term memory (LSTM) [64][65]. But Martens's method was computationally heavy and better suited to smaller problems. Yet Sutskever understood the broader implication. If a second-order method could train a deep network from scratch, then a first-order method with appropriate refinements might be able to do the same.

That became the bet behind AlexNet. Improved initialization, better activation functions, and sufficient computational power allowed gradient descent to overcome obstacles that had once made deep learning seem impossible. In later interviews, Sutskever claimed he *knew* AlexNet would work. To some, it might sound like hindsight bias, but it wasn't. Martens had already shown that deep networks could converge. For Sutskever, the confidence wasn't theoretical; it was empirical. He had already seen it happen [66].

2.5 AlexNet

AlexNet redefined the ImageNet competition with a top-5 error of 15.3%, meaning the correct label appeared among its top five predictions more than 84% of the time. AlexNet didn't merely surpass the runner-up, which had a 26.2% error; it obliterated it. These results emerged from carefully engineered architectural innovations detailed in the seminal paper "ImageNet Classification with Deep Convolutional Neural Networks," often abbreviated as KSH, after the authors Krizhevsky, Sutskever, and Hinton [67].

2.5.1 Network architecture

AlexNet significantly increased the depth and width of the standard network configuration. Structurally, AlexNet comprises 60 million parameters and approximately 650,000 so-called "neurons." As previously discussed, parameters are the model's weights and biases, which it tunes during training. Neurons, meanwhile, are the model's computational blocks: they take an input, multiply it by weights, potentially add a bias term, and pass the result through an activation function that rescales the raw output to a specific range (0 to 1 or -1 to 1).

A single neuron isn't very useful. But connecting many neurons across multiple layers allows the network to learn complex patterns from data, such as recognizing a cat or translating languages. Typically, researchers emphasize parameter counts because they directly influence how challenging the model is to train, how expensive it is to run, and how easily it

Licensed to Hermilo Cortés <hermilocg@gmail.com>

might overfit. Neuron counts, by contrast, are more abstract and ambiguous, particularly in convolutional layers where parameters are shared.

Despite this ambiguity, KSH highlighted its neuron count. In 2012, artificial neural network researchers often emphasized analogies to biological systems to provide intuition and motivation for their architectural choices. AlexNet firmly positioned itself within this biologically inspired tradition by referencing neuron counts and promoting local response normalization. The authors claim this concept was inspired by the brain's lateral inhibition, despite its limited practical effect on AlexNet's performance. Such parallels to biology provided essential legitimacy in an era before dramatic improvements on standardized benchmarks rendered them largely unnecessary. Today, ANNs are generally described in pragmatic, engineering-focused terms. Still, AlexNet's explicit neuron count reflects a time when biological intuition was crucial to establishing credibility and inspiring confidence in novel approaches.

At the core of AlexNet's success are its first five convolutional layers, which enable automatic, hierarchical feature extraction. Unlike traditional handcrafted methods, convolutional layers progressively capture increasingly complex patterns. They begin with simple patterns—such as edges and textures in the earliest layers—and evolve into intricate shapes, corners, and recognizable object parts as the network progresses. For example, an early layer might detect horizontal and vertical edges in a cat image; the next layer might combine those edges into whisker-like or ear-shaped contours, and deeper layers might assemble these parts into a coherent representation of the cat's face.

Each convolutional layer employs multiple small filters that slide across the input image, comparing each segment to a learned pattern. You can think of this as of measuring how much a region of the image resembles the filter. Mathematically, this comparison is performed through a dot product between the filter's weights and the input pixels. The resulting values form feature maps that highlight where specific visual patterns are most strongly detected. These activations indicate which filters respond most strongly to different parts of the image. Crucially, the same filter is applied at every location in the input, dramatically reducing the parameter count and enabling translation invariance, or the ability to recognize a pattern regardless of its location in an image. For instance, AlexNet's first convolutional layer applies 96 distinct filters, generating 96 feature maps that feed into deeper layers for further abstraction and analysis.

AlexNet's convolutional layers weren't merely deep. They were strategically wide too. As the network went deeper, the number of filters rose

Licensed to Hermilo Cortés <hermilocg@gmail.com>

from 96 in the first layer to 256 and then 384, stayed at 384, and ended at 256, expanding the network's capacity. At the same time, filter sizes decreased from 11 × 11 in the initial layers to 5 × 5 and then to 3 × 3 deeper in the network. See figure 2.3.

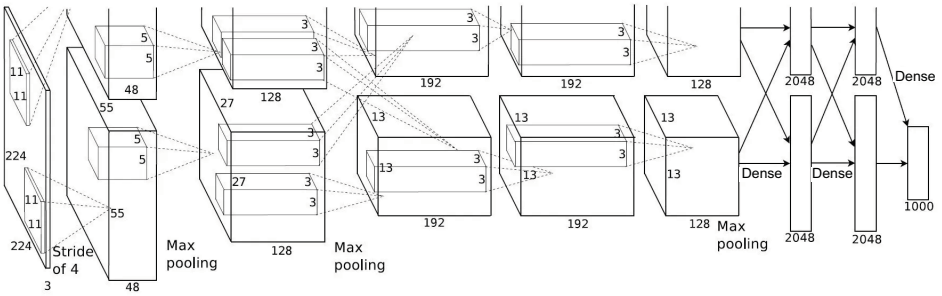


Figure 2.3 The AlexNet architecture (cropped in the original paper). The numbers indicate spatial dimensions and channel depth at each stage. The input image (224 × 224 × 3 RGB) passes through a series of convolutional and pooling layers that progressively reduce spatial resolution (224→55→27→13) while increasing channel depth. Early downsampling is driven by the first convolutional layer (11 × 11 filters with a stride of 4), followed by max pooling layers (3 × 3 windows with a stride of 2) that further reduce spatial dimensions. The stride determines how far a filter moves across the input at each step, controlling how densely the image is sampled and how quickly spatial resolution is reduced. Filter sizes (11, 5, 3) are depicted as cubes and define the local receptive field of each convolution. The two horizontal streams reflect the model's original two-GPU training setup, where feature maps were split across devices (e.g., 48 + 48 = 96 channels). The fully connected layers (4096 neurons, implemented as 2048 + 2048 across GPUs) feed into a 1000-way output for ImageNet classification. *Used with permission granted by the lead author (Alex Krizhevsky).*

This progression enabled AlexNet to initially focus on coarse features and then refine them into increasingly intricate, fine-grained patterns. Max pooling complemented this strategy by systematically reducing feature map dimensions and selecting only the strongest activations within small spatial regions, typically 3 × 3 windows. This selective reduction decreased computational complexity and enhanced the network's robustness to minor positional variations and input distortions. By narrowing filter size and increasing the number of feature maps in deeper layers, AlexNet expanded each neuron's receptive field, or the region of the input image that influences that neuron's activation. This allowed the network to build layered representations that captured increasingly complex visual abstractions across broader image regions.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

After the convolutional layers extract features, AlexNet transitions to three fully connected (FC) layers. This is the network's final critical step in image classification. Unlike convolutional layers, FC layers connect every neuron to every neuron in the preceding layer. This dense connectivity flattens the feature maps into a single vector, intentionally discarding spatial structure to integrate high-level features into a unified representation. In doing so, the FC layers can synthesize information across the entire image, identifying global patterns essential for accurate classification. This shift from spatially aware feature extraction to spatially agnostic global interpretation equips AlexNet to map complex visual features into precise categorical predictions.

AlexNet's first two FC layers account for most of the model's roughly 60 million parameters. This scale dramatically increased the model's representational capacity but also introduced the risk of overfitting. We will discuss this further in the next section [68].

The final FC layer outputs a 1,000-dimensional vector corresponding directly to the 1,000 categories in the ImageNet dataset. A softmax function transforms these raw scores into predicted probabilities, with the highest probability determining the network's final classification. In sum, convolutional layers extract visual patterns, and fully connected layers interpret those patterns as categorical predictions. Together, they form a computational pipeline that, while devoid of genuine phenomenological experience or visual understanding, enables AlexNet to translate raw pixels into object classifications with groundbreaking accuracy.

2.5.2 Training innovations

A critical innovation in AlexNet, informed by insights from Ilya Sutskever, was the adoption of the rectified linear unit (ReLU) activation after each convolutional layer [69]. Unlike traditional activation functions such as sigmoid or tanh—which compress outputs to a limited range and often lead to neuron saturation and vanishing gradients—ReLU preserves strong gradient flow throughout training. Figure 2.4 shows that ReLU achieves this by mapping negative inputs to zero while leaving positive inputs unchanged, effectively acting as an identity function in that region. Because its derivative is constant for positive values, gradients can propagate through many layers without being repeatedly diminished. This seemingly simple modification enabled AlexNet to scale effectively to previously unseen depths.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

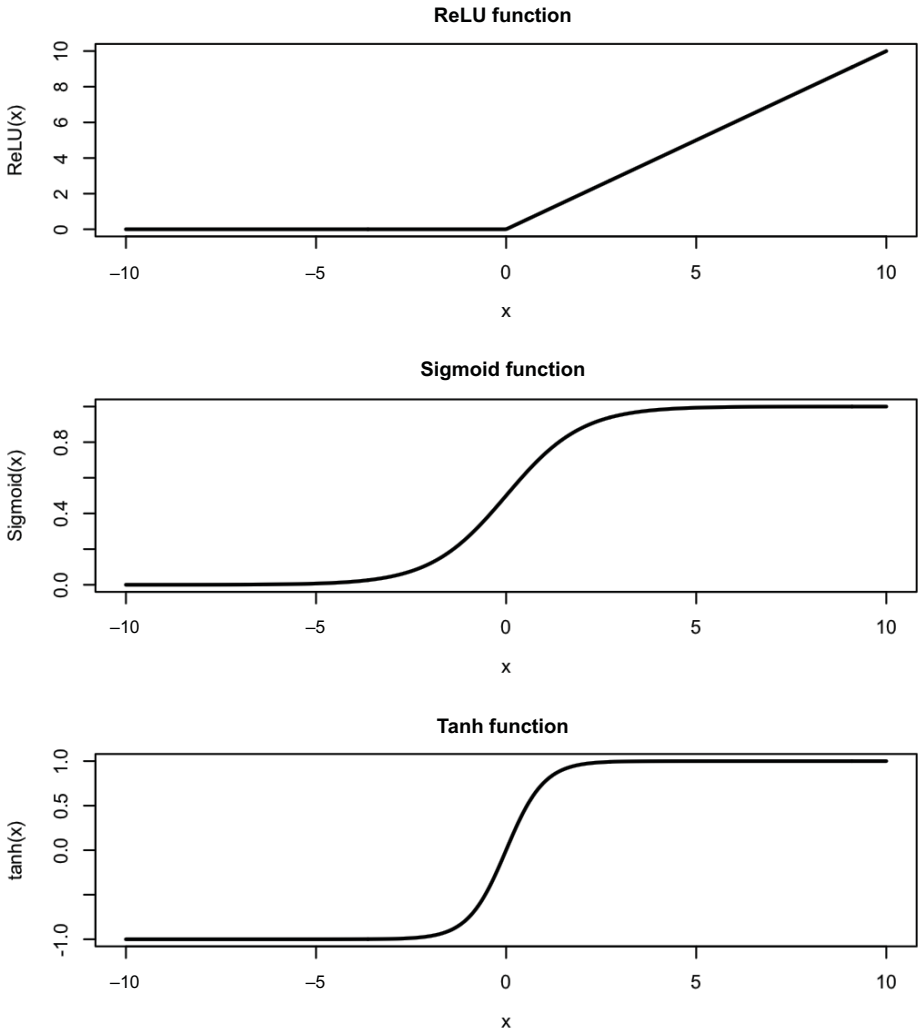


Figure 2.4 Plot of ReLU (top), sigmoid (middle), and tanh (bottom) activation functions generated in R by the author. Activation functions map input values (on the x-axis) to output values (on the y-axis). For example, ReLU produces 0 for negative inputs; for positive inputs, the output equals the input (0 for $x < 0$ and linear for $x \geq 0$). The sigmoid function maps inputs to values between 0 and 1, resembling an S-curve. Similarly, the tanh function provides a smooth S-shaped curve but maps inputs to values between –1 and 1, centered at 0.

In a controlled experiment, KSH demonstrated that switching from traditional tanh to ReLU activations accelerated convergence by a factor of 6. The authors recognized how critical this efficiency gain was, noting explicitly that "we would not have been able to experiment with such large [artificial] neural networks for this work if we had used traditional

Licensed to Hermilo Cortés <hermilocg@gmail.com>

saturating neuron models (i.e., sigmoid or tanh activation functions)." This improvement was possible because ReLU activations help flatten the optimization landscape by avoiding saturation and alleviating what Martens called "pathological curvature" [70].

AlexNet's success firmly established ReLU as the default activation for deep networks. Later networks, such as VGGNet, GoogLeNet, and ResNet, all employed ReLU or its variants, confirming that nonsaturating activation was critical for scaling to greater depth.

Fully connected layers contained most of AlexNet's parameters, making them especially prone to overfitting. To address this vulnerability, the authors employed dropout, which randomly disables a fraction of neurons during each training iteration. Dropout is a regularization technique that imposes a constraint during training to prevent overfitting and preserve the model's ability to generalize. In AlexNet's case, 50% of the neurons in the first two FC layers (FC6 and FC7) were dropped at each iteration. Geoffrey Hinton, the godfather of analogies, famously explained dropout's intuition using the "co-conspirator" analogy [71]. If you want to find out who committed a crime, you interrogate suspects separately, not together. Similarly, dropout forced neurons to learn useful features independently rather than relying on tightly coupled groups. See figure 2.5.

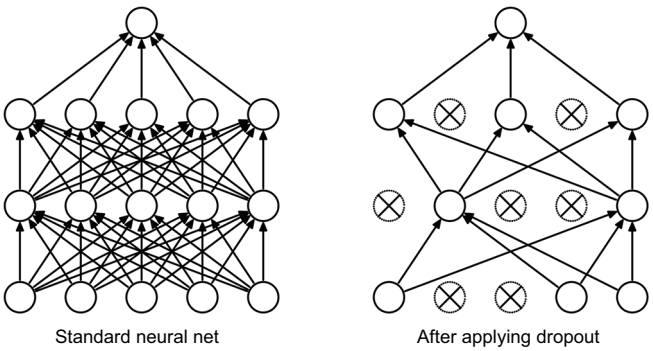


Figure 2.5 Left: a standard neural network with two hidden layers. Right: an example of a network produced by applying dropout to the network on the left. ***Used with permission granted by the lead author (Nitish Srivastava).***

Following AlexNet's success, dropout became standard whenever overfitting was a concern—a concern that became more frequent as networks grew in size. During training, a random fraction of neurons is turned off

Licensed to Hermilo Cortés <hermilocg@gmail.com>

so the network can't rely on "co-conspirators." At test time, nothing is turned off. Early implementations compensated at test time by shrinking the layer's output to match what the network saw during training. But modern implementations use inverted dropout, which applies the compensation during training, so no test-time adjustment is needed [72][73]. Both approaches keep the average signal the same, but inverted dropout has become the de facto standard because it keeps training and test on the same scale and simplifies deployment.

KSH noted that the network "exhibits substantial overfitting" without dropout. With 60 million parameters and just 1.2 million training images, the network could easily memorize rather than generalize. A side effect, though, is slower training: "Dropout roughly doubles the number of iterations required to converge," the paper reports. This is reasonable because, with dropout, each training update is slightly less efficient, and each neuron receives only half the updates it would otherwise receive.

As previously noted, AlexNet also introduced local response normalization (LRN), an experimental technique inspired by biological lateral inhibition. LRN normalizes each neuron's activation relative to neighboring neurons at the same spatial location across adjacent feature maps, encouraging competition among neurons. This helped suppress uniformly strong activations and enhance distinctive, selective feature responses. In practice, the authors observed only modest improvements in generalization, yet they still included an LRN layer after the first two convolutional layers. LRN soon fell out of favor and was replaced by more effective normalization methods, notably batch normalization in 2015 [74]. Although modern networks no longer use LRN, its inclusion in AlexNet exemplifies the experimental innovation characteristic of its design, even if, in hindsight, AlexNet's results did not critically depend on this particular technique.

2.5.3 Data augmentation

KSH used aggressive data augmentation to expand ImageNet's effective size and diversity. Two primary techniques were used:

1. *Geometric transformations*—Random cropping and horizontal reflections were employed to train the network to be spatially invariant, enabling it to recognize objects even when they appear in different positions or orientations
2. *Photometric distortions*—PCA-based color jittering introduced subtle variations in brightness, contrast, and color balance by perturbing the principal components of the RGB pixel values. This technique

Licensed to Hermilo Cortés <hermilocg@gmail.com>

simulated the effects of lighting changes and enhanced robustness to illumination variability.

Together, these augmentation strategies improved AlexNet's generalization by simulating realistic variability and noise in the training data. The combination of data augmentation and dropout effectively mitigated overfitting, even though the model was highly overparameterized, with 60 million parameters trained on just 1.2 million images. This allowed the network to use its capacity, fitting the training data precisely while still generalizing to unseen test images from the ImageNet dataset.

2.5.4 Efficient and scalable training

GPUs were initially designed for real-time video game graphics, but the computations they excel at closely resemble those required to train ANNs. At the time, NVIDIA searched for new applications for its niche hardware. In an interview with *MIT Technology Review*, NVIDIA CEO Jensen Huang says, "When you invent a new technology, you have to be receptive to crazy ideas." He continues, "My state of mind was always looking for something quirky, and the idea that [artificial] neural networks would transform computer science, that was an outrageously quirky idea" [75].

Training an eight-layer network with 60 million parameters on 1.2 million images wasn't just computationally demanding; it was also conceptually challenging. The NVIDIA GTX 580 GPUs available in 2012 had only 3 GB of memory, which wasn't enough to store the entire model. KSH strategically split the network across the two GPUs to work around this constraint. Each GPU processed half the filters in each layer and half the feature maps in each training batch. Inspired by earlier model-parallelization research, this two-column architecture minimized overhead by synchronizing GPU communication at carefully chosen layers: after the second convolutional layer, to enable cross-GPU learning once early processing was complete, and after the fifth, to fully integrate features before transitioning to the network's fully connected layers [76][77][78][79][80].

Using two GPUs in parallel enabled the team to scale the network, allowing twice as many filters per layer as in a smaller, single-GPU variant. This improved top-1 accuracy by 1.7% and top-5 accuracy by 1.2%. The two-GPU setup also enabled greater depth, which is crucial to the model's performance. KSH found that removing any convolutional layer and retraining decreased accuracy, even though each convolutional layer accounted for less than 1% of the model's parameters.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

These observations demonstrated that AlexNet's strength derived in part from its increased size and depth, enabling multistage feature extraction. Modern convolutional neural networks, now routinely trained on GPU clusters, directly extend the trajectory established by AlexNet. At the time, the team explicitly recognized GPU memory and computational time as their primary constraints, stating that the network's scale was "limited mainly by the amount of memory available on current GPUs and by the amount of training time we were willing to tolerate." They anticipated further improvements, noting their "results can be improved simply by waiting for faster GPUs and bigger datasets to become available." Despite the comparatively modest scale of their original work, subsequent advances have fully validated this prediction, underscoring AlexNet's influence on deeper and more powerful networks.

Training neural networks effectively requires balancing several factors to ensure stable learning and optimal performance. AlexNet was trained with mini-batch stochastic gradient descent (SGD) augmented by momentum of 0.9. The gradient indicates the direction and rate of steepest increase in the loss, so first-order methods move in the opposite direction (downhill), tweaking weights to reduce the loss. In other words, first-order optimization uses only local information about how the function changes at the current point, without needing additional details such as curvature (second-order derivatives). This simplicity makes first-order methods computationally efficient and popular for training large networks.

Imagine a lone hiker on a foggy mountain. Since the hiker cannot see the whole terrain, they scan a small patch of ground around them. This represents a random mini-batch of images used to gauge the local downhill direction. The hiker then moves a short distance along the opposite slope. Because the scan is only a sample, the estimate is noisy. Still, momentum acts like inertia, averaging recent steps so the hiker glides through shallow plateaus instead of taking slightly uphill steps or jittering sideways.

AlexNet's training used 128-image batches, an initial learning rate of 0.01, and a manual schedule that dropped the rate by a factor of 10 whenever validation accuracy flattened. The learning rate determines how much the model's parameters change in response to the gradient at each step. This allows the hiker to take bold steps early on and finer, more deliberate steps near the valley floor. Although SGD never guarantees the absolute minimum (i.e., global minimum), in rugged landscapes it can zigzag to avoid plateaus or saddle points. Still, the combination of mini-batch sampling, momentum, and a decaying learning rate consistently

Licensed to Hermilo Cortés <hermilocg@gmail.com>

guided AlexNet toward a high-quality solution, enabling it to achieve its breakthrough ImageNet performance.

Careful weight initialization also improved AlexNet's training stability. Weights were initialized from a zero-mean Gaussian distribution with a small standard deviation (0.01), ensuring that neuron outputs started in a stable, balanced range. Furthermore, biases in specific layers were intentionally set to 1 rather than the typical 0, activating ReLU neurons at the outset of training. This strategy mitigated the "dying ReLU" problem, in which inactive units get stuck outputting 0 and never recover [81]. Although minor, this careful initialization was crucial for maintaining healthy training dynamics.

Despite AlexNet's eventual success with gradient descent, many researchers initially doubted that this approach could train multilayer artificial neural networks. A prevalent misconception at the time was that gradient-based optimization methods would inevitably become trapped in local minima, rendering training ineffective. Researchers visualized the loss landscape as a low-dimensional terrain filled with poor local minima. But they overlooked that loss surfaces behave differently in high-dimensional spaces. Moreover, averaging gradients over mini-batches, as AlexNet did, further smooths irregularities in the loss landscape and enables more stable optimization. Recognizing this subtle but profound truth was essential to the deep learning revolution pioneered by AlexNet.

2.5.5 The effect

In 2012, ImageNet's training set was already 100× larger than PASCAL VOC's, but performance on both ImageNet and PASCAL VOC had stalled. Handcrafted features, from SIFT and HOG to bag-of-visual-words pipelines, lacked sufficient headroom for improvement. Yet AlexNet cut ImageNet's top-5 error from 26% to 15%, while PASCAL VOC scores barely moved. AlexNet was the first and only model in the 2012 competition to use an ANN, and it halved the top-5 error rate, proving that more data was insufficient. It wasn't data alone that reset the field, but an architecture that extracted more meaning from it.

In 2013, the ImageNet competition (ILSVRC) attracted 24 teams. This exceeded the sum of the previous three years (21). Nearly all entries used CNNs marking a remarkable turnaround for an approach considered impractical just a year earlier. The zeitgeist shifted entirely. Not only had ANNs shed their stigma, but they were now driving the state of the art. In 2010, you might have been rejected for using them; by 2013, you might have been rejected for not using them.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

AlexNet's success inspired a wave of deeper and more effective architectures. In 2013, a team from NYU achieved a top-5 error rate of 11.2% [82]. In 2014, accuracy surpassed 90%. That year, GoogLeNet won ILSVRC with an astonishing 6.7% top-5 error rate, representing a roughly fourfold reduction in error over two years compared with pre-AlexNet results. See figure 2.6. By 2017, 76% (29 of 38) of teams competing in ILSVRC achieved top-5 accuracy greater than 95% [83][84][85]. They all used CNNs.

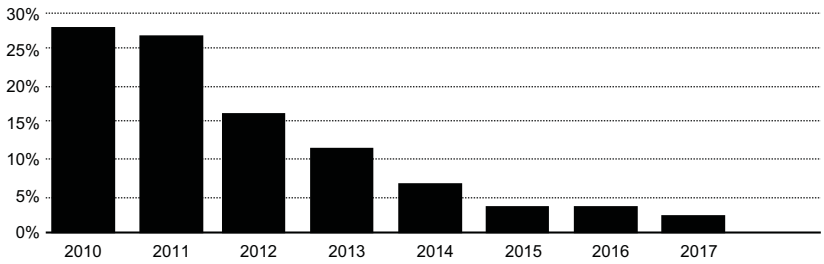


Figure 2.6 The top-5 error rates in the ILSVRC from 2010 to 2017. SIFT and SVM won in 2010 and 2011, but performance plateaued. CNNs won from 2012 to 2017, including AlexNet in 2012, and eventually saturated the benchmark.

After AlexNet's success, Malik embraced deep learning. In 2014, his group at UC Berkeley developed R-CNN, an approach that applied CNNs to achieve state-of-the-art object detection [86]. This rapid follow-up showed that the AlexNet moment was no fluke. AlexNet could be replicated and extended by others. In 2017, Malik called AlexNet "the most impactful paper in machine learning and computer vision in the last five years," and described it as "the paper that led the field of computer vision to embrace deep learning" [87]. In 2020, Malik admitted to being "positively surprised" by the effectiveness of deep learning, confessing that he "certainly would not have believed that in 2010" [88].

AlexNet did not merely win a benchmark; it redefined the priorities of the tech sector. Google moved first, acquiring DNNresearch in early 2013, expanding Google Brain, and then acquiring DeepMind in 2014. Facebook launched Facebook AI Research (FAIR) in late 2013, appointing CNN pioneer Yann LeCun as its director. Finally, Baidu recruited Andrew Ng to lead a new AI lab for speech and image tasks. Clarifai, whose founders won the ILSVRC 2013 competition, emerged as a provider of commercial CNNs [89].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

AlexNet also dramatically reshaped public narratives around AI. The *MIT Technology Review* declared "deep learning" one of its top breakthroughs of 2013, adding that it was "finally getting smart" [90]. That optimism surged by the year's end when *The New York Times* featured a story titled "Brainlike Computers, Learning From Experience" on its front page, signaling how profoundly deep learning had captivated the cultural imagination [91]. Even critical coverage, such as *The Guardian*'s "Don't Believe the Science Hype: We Haven't Created True AI Yet," indirectly validated the new paradigm's dominance [92]. The few who still expressed skepticism did so by citing the so-called "black box" criticism, which holds that models are not interpretable or are overhyped, rather than by pointing to actual performance.

Ultimately, AlexNet overturned a decade of comfortable assumptions. ANNs were dismissed as overcomplicated curiosities, "fancy schmancy" architectures chasing problems that feature engineering and clever kernels (that is, SVMs) could solve more efficiently. AlexNet proved, in a hard-won victory, that with the right ingredients, deep learning could beat the incumbent playbook. It punctured complacent versions of the "stone soup" and "unreasonable effectiveness of data" narratives. Yes, data was underappreciated, and more of it provided important coverage and access to tail events needed for high-performance machine learning, but it wasn't a great equalizer.

AlexNet stitched together a set of optimization-aware choices, including ReLU activations to prevent vanishing gradients, proper weight initialization to stabilize early training, momentum to smooth updates, dropout to reduce overfitting, and data augmentation to increase the effective size of the training set [93]. AlexNet was also distributed across two GPUs. This humble pair of 3 GB GPUs illustrates that *everything* has humble beginnings, and the "deep learning revolution" is no exception. None of these ideas was magical in isolation. The magic was in insisting they could be assembled into an end-to-end system at scale and then doing the engineering to prove it.

Following AlexNet (2012), network architectures gradually increased in depth. VGGNet (2014) had 19 layers. GoogLeNet (2014) had 22. Yet attempts to train even deeper networks encountered obstacles. Chief among them were the persistent vanishing gradient and model degradation problems. AlexNet flipped the zeitgeist, but architectural and optimization innovations were still required to reliably train deeper networks.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

3 ResNet revolution

This chapter covers

- The challenges of training deep neural networks
- Residual connections and how they revolutionized deep learning
- The evolution to ResNet v2 and training networks exceeding 1,000 layers
- Pooling and striding, which were replaced with dilated convolutions for dense prediction tasks
- CS231n as the first deep learning course at Stanford

Papers

- *Deep Residual Learning for Image Recognition* (2015) He, Zhang, Ren, and Sun
- *Identity Mappings in Deep Residual Networks* (2016) He, Zhang, Ren, and Sun
- *Multi-Scale Context Aggregation by Dilated Convolutions* (2016) Yu and Koltun
- *Stanford CS231n* (2015) Fei-Fei Li, Andrej Karpathy, and Justin Johnson

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Before 2015, increasing depth did not immediately translate into improved performance. In fact, deeper models often exhibited *higher* training error. The heart of the problem was a double bind. Depth was required for representational capacity, yet it impaired optimization. Reducing depth eased optimization but capped capacity. Just as despair seemed justified, a deceptively simple idea emerged.

Residual connections ease this tension by providing shortcuts that preserve the forward signal and maintain effective gradient pathways during backpropagation. Operationalized in residual networks (ResNets), they earn their place on Sutskever's List because they represent a pivotal shift in design philosophy, in which "add a residual connection" has become an indispensable tool in the machine learning toolkit [1].

Empirically, ResNet delivered a step-change in performance. Like AlexNet, ResNet approximately halved the top-5 error rate from the previous year. At ILSVRC 2015, an ensemble of ResNets achieved a 3.57% top-5 error, while a single ResNet achieved 4.49%, surpassing prior ensemble results and securing wins across the classification, localization, and detection competition categories [2][3]. Reflecting on this breakthrough, the ResNet team lead, Kaiming He, confessed, "We didn't even believe this single idea could be so significant." This sentiment underscores an essential truth in machine learning: the significance of an idea doesn't correlate with its complexity.

ResNet transformed "deep" from an aspiration into a practical, efficient reality. ResNet-152 is eight times deeper than the 19-layer VGGNet, yet it requires fewer computational operations and model parameters. Suddenly, depth was no longer symbolic. The experimental strategies presented in the original ResNet paper reflect Sutskever's belief that progress in AI emerges from refining ideas and uncovering what genuinely works. The subsequent ResNet v2 paper also embodies this ethos. It delivers meaningful gains through refinement, most notably the pre-activation residual block, rather than an entirely new architecture. Both show that progress in AI doesn't always require a new conceptual leap and often does not.

3.1 Telephone game

Architectures gradually increased in depth following AlexNet (2012), which had eight layers. VGGNet (2014) had 19. GoogLeNet (2014) had 22. Yet all attempts to train even deeper networks encountered obstacles. Chief among them were the persistent vanishing gradient and model

Licensed to Hermilo Cortés <hermilocg@gmail.com>

degradation problems [4]. Despite partial solutions such as batch normalization, careful weight initialization, and gradient clipping, no comprehensive solution had emerged. While batch normalization stabilized layer statistics and careful initialization kept activations and gradients appropriately scaled from the start of training, neither provided a shortcut for routing gradients through dozens of nonlinear layers.

Imagine passing information through a series of layers, each designed to extract patterns from the original data. As information, such as an image, passes through each layer, random weights slightly distort the data. This distortion is manageable with only a few layers, but these subtle distortions compound as the network increases in depth. By the time the information reaches the final layers of a multilayer network, the original image or data has effectively been scrambled into random noise, losing nearly all meaningful connection to what was initially fed into the system. This deterioration of meaningful signals is known as *model degradation*.

After forward propagation, the network measures performance by calculating the difference between its output and the correct answer. This difference is called the *loss*. To improve, the network propagates the loss backward through its layers to determine how each layer's weights should be adjusted. As the feedback signals (i.e., gradients) travel backward through each layer, they become progressively smaller and distorted due to repeated multiplications by the layers' weights. By the time these gradients reach the earliest layers—those closest to the original input—they are diminished to the point that they carry no meaningful information, making it impossible for early layers to update their weights effectively. This phenomenon, in which gradients effectively vanish as they propagate backward, is known as the *vanishing gradient problem*.

Although the degradation and vanishing gradient problems are distinct, they can reinforce each other in a harmful feedback loop, like two people arguing until the original point is forgotten or completely distorted. Distorted forward signals corrupt the output, leading to weaker gradients during backpropagation. Those weaker gradients, in turn, lead to poor learning, which further distorts subsequent forward passes and traps researchers in a frustrating loop in which training error actually rises with depth.

Against this backdrop, Microsoft researchers introduced ResNet. In their seminal paper, "Deep Residual Learning for Image Recognition," the authors addressed longstanding problems through a deceptively simple architectural innovation: residual connections. Imagine playing the "telephone game," in which a message is whispered from person to

Licensed to Hermilo Cortés <hermilocg@gmail.com>

person along a line. Typically, by the time the message reaches the last person, it is distorted because each participant slightly mishears or misinterprets it. Now imagine that, at certain points, you could pass a note directly to someone further along, skipping multiple intermediate whisperers. This shortcut would ensure that at least part of the original message remains clear and intact.

This is precisely the idea behind residual connections. Rather than forcing every signal to pass sequentially through many layers, where distortions accumulate, residual connections create shortcuts that let information flow both *through* and *around* certain groups of layers. This dual-path structure ensures that meaningful information reaches deeper layers intact and that clear gradient signals flow smoothly back through the network.

During forward propagation, each residual block computes along two parallel paths: one transforms the input through convolutional layers, batch normalization, and nonlinear activations to extract complex features, while the other passes the input unchanged, preserving its original information. During backpropagation, gradients that determine weight updates travel backward along these same two paths. The direct shortcut helps maintain stronger gradients, reducing the risk of vanishing gradients as they propagate backward and improving the stability of training deeper networks.

Specifically, each residual block computes an incremental adjustment, called the *residual*, across its internal layers and then adds it directly to the original input. Formally, the output of each residual block is:

$$y = F(x) + x$$

Here, $F(x)$ represents the output produced by the block's internal transformations, while x represents the original input. See figure 3.1. By explicitly adding the original input, residual connections preserve clear signals during forward propagation and stabilize gradient flow during backward propagation.

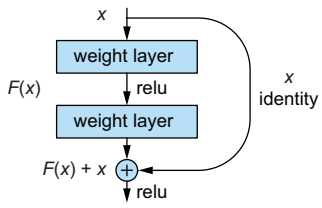


Figure 3.1 Diagram of a simple ResNet block. It illustrates how input data flows along two paths: one *through* two transformation layers with activation steps, and another *around* via a direct "shortcut" path with no transformations. The outputs of these two paths are combined, and the resulting vector undergoes a final activation step. This "shortcut" helps the network learn more effectively by preserving information from earlier layers. *Used with permission granted by the lead author (Kaiming He).*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Figure 3.2 shows that architectures without residual (or skip) connections struggle as depth increases. A "plain" 56-layer network without residual connections has a higher training error than a shallower 20-layer network. This unexpected increase in training error wasn't due to overfitting; it revealed a fundamental difficulty in training deep networks. The deeper network also has a higher test error, showing that adding more layers doesn't automatically improve performance.

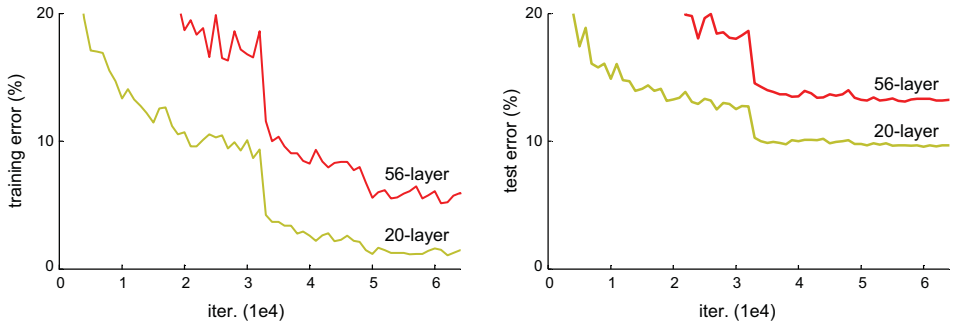


Figure 3.2 Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper 56-layer network shows a higher training error than the shallower 20-layer network, indicating that it is harder to optimize without additional architectural innovations. It also has a higher test error (right), showing that simply adding layers doesn't improve performance. In both plots, the *x*-axis denotes training iterations in units of 10,000; each iteration corresponds to a single mini-batch update. As training progresses across approximately 60,000 iterations, the deeper network consistently underperforms. *Used with permission granted by the lead author (Kaiming He).*

But when residual connections are introduced, deeper networks outperform their "plain" counterparts. The authors compare two plain networks without residual connections: one with 18 layers and a deeper version with 34 layers (see figure 3.3). Intuitively, you might expect the deeper 34-layer network to perform better, or at least no worse, since it should, in principle, represent all the functions the shallower 18-layer network can. That is, the solution space of the 18-layer network is essentially a subset of the deeper 34-layer model. Surprisingly, the opposite happens. The deeper network performs worse throughout training. This counterintuitive outcome illustrates the degradation problem.

The challenge for networks without residual connections is that each layer must learn a useful transformation. But forcing layers to learn transformations—even when unnecessary—can destabilize training. Residual connections resolve this by allowing layers to pass inputs forward unchanged, a process known as identity mapping, when no beneficial

Licensed to Hermilo Cortés <hermilocg@gmail.com>

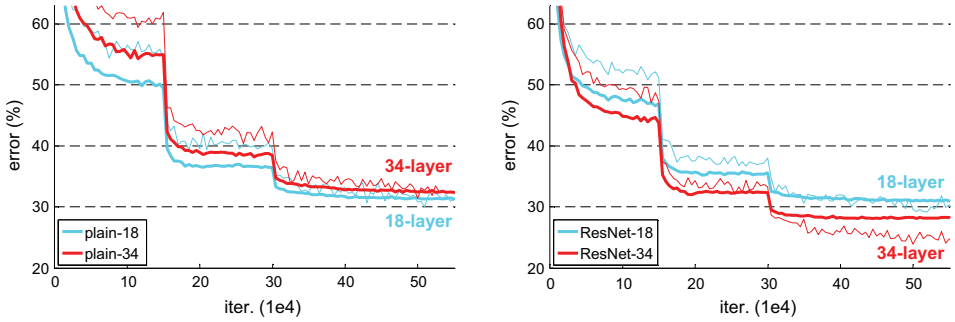


Figure 3.3 Thin error curves indicate training error on ImageNet, and bold error curves indicate validation error on ImageNet. Left: plain networks with 18 and 34 layers. Right: ResNets with 18 and 34 layers. Residual networks have no more parameters than their plain counterparts. ***Used with permission granted by the lead author (Kaiming He).***

transformation is found. If a transformation is needed, layers learn the residual relative to the input. Thus, it becomes safe to add additional layers because residual connections do nothing by default, preserving performance rather than harming it.

This behavior also serves as a form of regularization, encouraging layers to do nothing unless a more complex transformation improves performance. As a result, residual networks can achieve greater depth without a corresponding optimization penalty. Traditional architectures, by contrast, incur substantial computational penalties with each additional layer. ResNet-152 is approximately eight times deeper than VGG-19, yet it requires fewer floating-point operations (FLOPs) and has fewer parameter count. The 152-layer ResNet has 11.3 billion FLOPs, which is lower than that of VGGNet-16/19 (15.3 and 19.6 billion FLOPs, respectively). ResNet-34 requires only 3.6 billion FLOPs, about 18% of VGG-19's 19.6 billion, even though ResNet-34 has almost twice as many layers as VGG-19. This efficiency comes primarily from ResNet's convolutional design and, in deeper versions, its specialized "bottleneck" blocks.

To understand how bottlenecks function, it helps to first consider the role of *channels* in convolutional networks. Each channel represents a different way of viewing the same image, like applying distinct filters that emphasize edges, colors, or textures. While channels allow the model to capture a wide range of details, processing all of them is computationally expensive and often redundant. Bottleneck layers solve this by strategically reducing and then expanding the number of channels, compressing the workload while retaining expressive power.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

First, a 1 × 1 convolution reduces, or "compresses," the number of channels by learning a smaller set of feature combinations from the original input (see figure 3.4). Next, a 3 × 3 convolution is applied to this simplified (i.e., lower-dimensional) representation, thus reducing computational cost while still capturing spatial structure. Crucially, the 3 × 3 convolution operates in this reduced space, which is what makes the bottleneck design efficient. Finally, the 1 × 1 convolution expands the representation back to a higher number of channels by recombining these features in new ways.

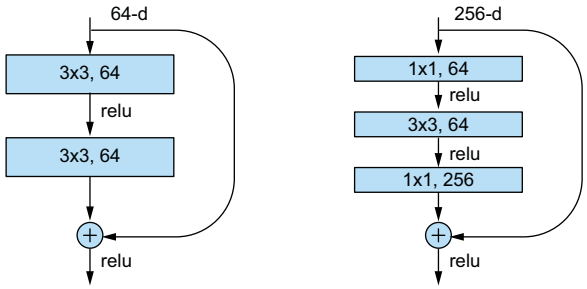


Figure 3.4 Left: the basic block used in ResNet-34 passes a 64-channel input through two 3 × 3 convolutions and then adds the original input back before applying ReLU. Right: the bottleneck block used in deeper ResNets first compresses features with a 1 × 1 convolution, processes them with a 3 × 3 layer, and then restores dimensionality with another 1 × 1 convolution. The shortcut bypass preserves the original signal, ensuring that deeper models refine features rather than lose them. *Used with permission granted by the lead author (Kaiming He).*

Think of the bottleneck as a data compressor. It first shrinks the information, processes the compressed version, and then expands it back to its original form. Since the core calculations operate on smaller amounts of data, ResNet could perform better with minimal additional computational requirements, which meant it could have more layers. Indeed, just three years earlier, AlexNet halved the top-5 error rate on ImageNet with only eight layers. But halving the error rate again required significantly more layers as performance approached saturation. ResNet halved the top-5 error rate and demonstrated that extraordinary depth was possible, including an unprecedented 1202-layer network.

Yet ResNet's depth attracted its share of criticism. While deeper networks became feasible, there was no principled method for determining the optimal depth. The original ResNet paper empirically found that 152 layers were optimal for ImageNet, but the authors offered no theoretical

Licensed to Hermilo Cortés <hermilocg@gmail.com>

or analytical framework to guide depth selection. Practitioners had little choice but to rely on trial and error, a sign that the field still lacked a quantitative rule connecting scale (i.e., depth, width, data, and compute) to expected performance. This gap would later be addressed by empirical scaling laws (chapter 6).

Moreover, ultra-deep ResNets still demanded substantial computational resources and large datasets; otherwise, they risked overfitting. Subsequent architectures, including ResNeXt, DenseNet, ShuffleNet, and EfficientNet, reinforced these concerns by showing that carefully designed models could match or exceed ResNet performance with significantly fewer parameters and less compute [5][6][7][8]. Still, ResNet fundamentally redefined what was possible.

3.2 Comparison with other major architectures

By examining how previous architectures tackled network depth, complexity, efficiency, and representational capacity, we can better understand the significance and effect of ResNet's innovations.

3.2.1 ResNet vs. AlexNet and ZFNet

In 2013, ZFNet refined AlexNet by replacing AlexNet's first convolutional layer with smaller 7×7 filters rather than larger 11×11 filters. This slight improvement in performance confirms the importance of careful architectural tuning. Yet, despite their achievements, AlexNet and ZFNet quickly encountered limitations due to optimization problems. Consequently, they were unable to grow beyond about a dozen layers without training instability or performance degradation.

3.2.2 ResNet vs. VGGNet

Following AlexNet and ZFNet, researchers at Oxford University's Visual Geometry Group introduced VGGNet in 2014. Rather than carefully engineering each layer, VGGNet stacked identical convolutional layers. Philosophically, this approach more closely anticipated ResNet, which introduced a modular, repeatable residual block that greatly simplified network scaling. The idea that stacking a standard module N times can outperform a heterogeneous design is powerful, and this simplicity is the fundamental innovation that characterizes many entries on Sutskever's List.

VGGNet primarily used uniform 3×3 convolutions interspersed with pooling, showing that consistently repeating a small building block to gain depth could deliver performance gains. Their 16-layer VGG-16 variant achieved a 7.3% top-5 error rate on ImageNet. Still, despite sharing this

Licensed to Hermilo Cortés <hermilocg@gmail.com>

modular philosophy with ResNet, VGGNet encountered practical limitations. Its large fully connected layers resulted in an enormous parameter count (138 million) and significant computational demands. Moreover, stacking layers beyond 19 still led to unstable training due to the vanishing gradient problem. Scaling networks was not yet possible without an architectural innovation to keep signals alive.

3.2.3 ResNet vs. GoogLeNet

In 2014, Google introduced GoogLeNet [9]. Rather than stacking identical convolutional layers or relying on uniform building blocks, GoogLeNet featured carefully engineered "Inception modules." An Inception module acts as a miniature network that processes input features in parallel across multiple paths. Different convolutional filter sizes (typically 1×1, 3×3, and 5×5) and pooling operations operate concurrently within a single module. The outputs from these parallel paths are merged, allowing the network to capture features at multiple scales within a single layer. GoogLeNet achieved an impressive top-5 error rate of 6.7% on ImageNet with 22 layers and roughly 6.8 million parameters.

Yet, despite its effectiveness, the Inception module required manual tuning and balancing of multiple parallel computational paths. Although this approach achieved impressive accuracy with fewer parameters, its design was brittle and hard to scale. Each new version required manually tuned filter sizes and parallel branches, tailored to specific depths.

In contrast, ResNet embraced a more straightforward strategy. This approach allowed networks to scale (for example, ResNet-18, ResNet-34, ResNet-50, ResNet-101, ResNet-152, and ResNet-1202) without redesign. It solved the vanishing gradient problem and became the default backbone for countless downstream models. Even Inception eventually followed suit, adopting residual connections in Inception-ResNet (2016) [10]. Table 3.1 summarizes key metrics for VGGNet, GoogLeNet, and ResNet. It highlights the dramatic increase in depth and the improvement in accuracy.

Table 3.1 Comparison of VGG-19, GoogLeNet (Inception v1), and ResNet-152

Architecture	Year	Depth (layers)	Parameters	Top-5 accuracy
VGG-19	2014	16 conv + 3 FC	~144 million	~92.7%
GoogLeNet (Inception v1)	2014	22	~6.8 million	~93.3%
ResNet-152	2015	152	~60 million	~95.5%

Licensed to Hermilo Cortés <hermilocg@gmail.com>

3.3 Real-world adoption

Researchers quickly transitioned from older architectures, such as VGG-based models, to ResNet variants. Models such as ResNeXt-101 (2016) and SENet (2017) achieved top results in ImageNet competitions [11][12]. The winning formula in computer vision had become "ResNet plus some tweaks." For example, replacing VGG-16 with ResNet-101 in Faster R-CNN yielded a 6% to 7% increase in accuracy on the COCO dataset, a widely used object detection and segmentation benchmark [13].

A 2023 survey highlighted their extensive deployment in clinical decision support systems, where they assist in diagnosing lung tumors from CT scans, detecting breast cancer from mammograms, and classifying skin lesions [14]. ResNets are prized for their robustness to changes in scale and viewpoint in autonomous driving. Tesla's Autopilot system, for instance, reportedly used a modified ResNet-50 backbone called HydraNet to process multicamera inputs for tasks such as object detection, lane recognition, and depth estimation [15][16]. Research consistently shows that deeper variants like ResNet-101 outperform shallower models, especially on challenging tasks such as lane detection [17].

Google incorporated them into hybrid architectures like Inception-ResNet [18]. Facebook followed suit with frameworks such as Mask R-CNN and ResNeXt. At the same time, Amazon introduced ResNeSt [19][20]. ResNet became the default backbone for production-grade image recognition across platforms like Google Photos, Pinterest, and Microsoft Azure Cognitive Services. By 2018, platforms like NVIDIA's Detectron and PyTorch's torchvision offered reliable implementations for production pipelines. Its versatility, efficiency, and strong performance on large-scale vision tasks made ResNet the de facto starting point for industry-scale computer vision deployments, and it remains widely used today.

3.4 ResNet v2

While the original ResNet architecture enabled training of deeper networks through residual connections, it wasn't without flaws. Researchers found that placing nonlinear activations *after* adding the skip connection could still disrupt signal propagation, especially in very deep networks. This insight led the authors to revisit their design in the follow-up paper, "Identity Mappings in Deep Residual Networks," in which they introduced ResNet v2.

The core innovation in ResNet v2 was (again) deceptively simple. Reordering operations within each residual block preserved the integrity of the skip connection. In the original design, each block combined a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

transformed path (convolutions and activations) with an identity path and then applied a final ReLU activation. ResNet v2 eliminated this post-merge activation and pushed all normalization and activation steps *before* the convolutions. This subtle shift made the skip connection a clean, unaltered identity mapping, improving gradient flow and simplifying optimization, especially in ultra-deep architectures.

An intuitive analogy may help clarify this idea: imagine residual connections as a freeway designed to quickly move traffic (or information) around a city (the network). The original ResNet design placed a traffic light (an activation function) right after merging two busy roads (the skip connection and convolutional path). This caused congestion, especially when the "traffic light" unexpectedly turned red, as when an activation function saturates. Recognizing this problem, the authors redesigned the intersection. They shifted all traffic controls to the convolutional path. As a result, traffic can flow freely, avoiding unnecessary bottlenecks—something all commuters would envy during rush hour.

3.4.1 Post-activation to pre-activation

In the original ResNet design, a ReLU activation is applied after combining the residual and skip paths, meaning both are modified together. In contrast, in the pre-activation design, activations are applied only within the residual branch, leaving the skip connection unchanged. The authors observed that pre-activation ResNets had slightly higher training errors but lower test errors than their original post-activation counterparts, a phenomenon they attributed to the regularizing effect of applying batch normalization at the input of each convolutional layer. See figure 3.5.

3.4.2 Ablation studies

Ablation studies systematically remove or modify parts of a model to determine their individual contributions. The authors used ablation studies to test several modifications to the skip connection and validate the importance of preserving identity mappings. Even minor changes, such as scaling the skip path by a constant (e.g., 0.5) or randomly dropping it during training, led to degraded performance and poor convergence. More sophisticated tweaks—including learnable gates or adding convolutional layers to the skip path—fared even worse, often increasing error rates or causing optimization to fail entirely. At these depths, minor architectural decisions became make-or-break. The engineering takeaway was clear: the simplest, unaltered identity skip connection was more elegant and far more effective.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

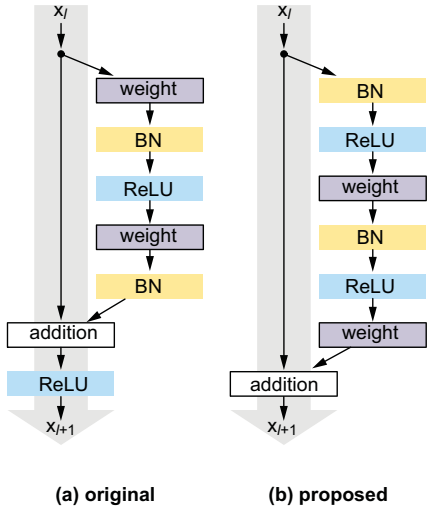


Figure 3.5 Left: the original residual unit shows two "weight" layers (that is, convolutional layers), with batch normalization (BN) and ReLU activations between them, followed by an additional step that merges with the identity connection. Right: the revised residual unit reorganizes the operations by placing BN and ReLU before the weight layers, simplifying the pathway. The gray arrows highlight the direct paths (that is, skip connections) that support the smooth forward and backward flow of information through the residual connections. *Used with permission granted by the lead author (Kaiming He).*

The authors also evaluated how changing the position of operations, such as activation functions (ReLU) and normalization steps (batch normalization), relative to the skip connection affected performance. Compared to the original approach, which placed activations after combining the paths, they found that putting batch normalization directly after the addition disrupted the skip path and hurt accuracy. Similarly, applying ReLU before the addition restricted the residual branch to positive adjustments only, limiting its effectiveness. Using ReLU alone before each convolution, without batch normalization, did not meaningfully improve performance either. Ultimately, the most effective approach was the full pre-activation strategy, which places batch normalization and ReLU before convolutions and omits activations after the addition.

The insistence on preserving pure identity mappings in ResNet v2 reflects a broader philosophy that values empirical clarity over theoretical novelty. Rather than inventing entirely new mechanisms, the authors rigorously tested existing assumptions through systematic experimentation, revealing that even subtle deviations, such as scaling or gating the skip connection, could derail training. This experimental mindset mirrors Sutskever's belief that genuine progress in AI emerges not from chasing conceptual breakthroughs but from deeply understanding known ideas, refining them at scale, and uncovering what actually works. The ResNet v2 paper embodies an ethos that resonates with Sutskever's view: progress through iteration, not invention.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Armed with pre-activation residual units, the authors successfully trained an ultra-deep 1,001-layer ResNet on the CIFAR-10 dataset—a benchmark dataset of 60,000 small (32×32) color images across 10 object classes. Interestingly, CIFAR-10 was introduced in 2009 by Alex Krizhevsky, the same "Alex" of AlexNet fame, underscoring his dual influence on both datasets and architectures [21]. Even so, the 1,001-layer ResNet achieved record-setting accuracy, reducing classification error to 4.62%. By comparison, the original (post-activation) 1,202-layer ResNet architecture achieved an error rate of 7.93%. At a time when training deep networks was still an ordeal, the authors demonstrated that extremely deep architectures could be stable if carefully designed. Depth was no longer the enemy, so long as the architecture respected the constraints imposed by optimization.

On the much larger ImageNet dataset, the authors trained a 200-layer pre-activation ResNet that achieved higher accuracy than the original 152-layer post-activation version. Extending the original post-activation design to 200 layers degraded performance, not because the model was too deep in principle but because the architecture couldn't support stable optimization at that scale. This underscored that, at this depth, there is a strikingly razor-thin margin for success. The smallest thing can cause convergence to collapse.

Subsequent theoretical work helped explain why this succeeded. Research showed that ResNets behave like ensembles of many shallower networks of varying depths [22]. Gradient propagation in very deep networks often occurs through shorter, more direct pathways rather than traversing every layer sequentially. So, a 1,000-layer ResNet doesn't function as a single chain of 1,000 layers; instead, it includes many shorter pathways for gradients and signals. These shorter pathways help prevent vanishing gradients, making training significantly more stable. As a result, additional layers can be added without harming performance because unused layers can be bypassed. The identity mappings allowed the network to select and use layers as needed, ignoring unhelpful ones.

The authors supported this idea with experiments in which they systematically removed individual layers to observe the effect. They found that removing individual layers with residual connections caused only a modest increase in error rather than catastrophic failure. This suggests that the network can compensate by using other available paths. Their analysis showed that in a 110-layer ResNet, most effective gradient flow occurred along shorter subnetworks (approximately 10–34 layers), whereas very deep paths contributed minimally and essentially learned

Licensed to Hermilo Cortés <hermilocg@gmail.com>

identity mappings. This interpretation suggests that ResNet's success arises not merely from having one very deep model but from combining many simpler, overlapping models enabled by residual connections.

By 2025, purely convolutional networks will be complemented or partially replaced by transformer-based or hybrid architectures, particularly for modeling tasks with complex long-range dependencies. Still, even these advanced models rely on residual connections. The core ResNet philosophy encourages each layer to make minimal, incremental adjustments ("residuals") rather than completely transforming the data. These minor refinements accumulate through the layers, progressively enhancing the representation. Given its foundational influence on how extremely deep neural networks are optimized and trained, it's no wonder this paper earned a place on Sutskever's List.

3.5 Extending ResNet with dense prediction

ResNet was built for image classification, where the goal is a single prediction (label) per image. This is called a *sparse prediction task*. Pooling aggregates local regions into a single value. Strided convolutions downsample images by skipping pixels. These strategies work well for sparse tasks because they compress spatial structure into a compact representation and discard fine detail that does not affect the global label [23][24].

By contrast, *dense prediction tasks*—such as image segmentation—aim to partition an image and label each pixel. For example, image segmentation assigns a label to each pixel indicating whether it represents a road, a car, or a dog. Because the goal is a precise outline of objects and surfaces, rather than just a rough sense of what is in the image, these tasks demand high spatial fidelity. So techniques such as pooling and strided convolutions, which are commonly used for image classification, are less suitable. The same downsampling that improves global recognition erases edges and location cues on which segmentation relies.

An alternative is to skip pooling and enlarge the convolutional kernel to widen the receptive field, which is the part of the image a model "sees." Larger kernels do widen context, but they increase parameters sharply. For example, a 3×3 kernel has nine parameters, whereas a 15×15 kernel has 225. That's a 25× increase. For this reason, architectures such as AlexNet and VGGNet use small kernels with pooling and stacked layers to expand the receptive field without increasing computational complexity. Still, this strategy hits limits for dense prediction, since pooling and large kernels tend to blur fine details and edges that pixel-wise labeling depends on.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The 2015 paper "Multi-Scale Context Aggregation by Dilated Convolutions," by Fisher Yu and Vladlen Koltun, introduced dilated convolutions to enlarge the receptive field without reducing resolution or using pooling [25]. Dilated convolutions widen the receptive field by inserting gaps between elements based on a dilation factor, rather than on using larger kernels. Stacking layers with dilation factors such as 1, 2, 4, 8 rapidly expands the field of view without increasing the number of parameters per layer. Thus, the receptive field grows exponentially while the number of parameters grows linearly. It is also among the papers on Sutskever's List.

This innovation lies at the heart of the *context module*, a component designed for multiscale context aggregation. By stacking dilated convolutions with increasing dilation rates, the context module enables the network to capture fine local structure and broad global dependencies at full resolution. It addresses a core limitation of traditional ResNet-like architectures in dense prediction tasks that often trade spatial resolution for deeper abstraction. The context module sidesteps that trade-off by gathering expansive context without sacrificing pixel-level detail or computational efficiency. This strategy is illustrated in figure 3.6.

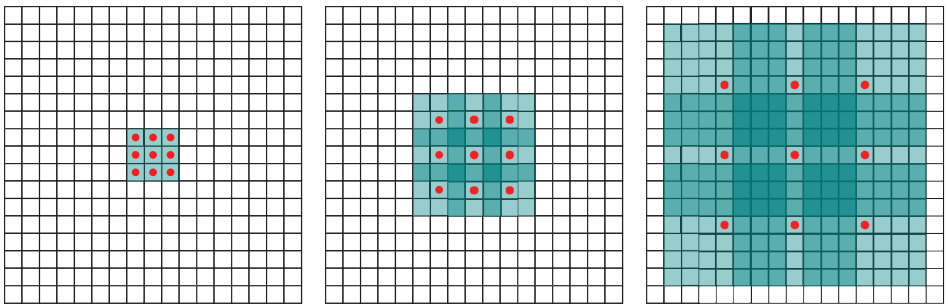


Figure 3.6 Dilated convolutions rapidly expand the receptive field without increasing computational cost. Each dot represents a convolutional kernel element. As dilation increases (from left to right), the receptive field expands exponentially, capturing broader spatial context while keeping the parameter count the same. *Used with permission granted by the lead author (Fisher Yu).*

The authors modified the well-known VGG-16 backbone. Because dense pixel prediction requires high-resolution feature maps, they removed pooling and striding layers and replaced them with dilated convolutions. This stripped away classification-centric design choices, producing a simpler, segmentation-oriented model. On PASCAL VOC, it outperformed

Licensed to Hermilo Cortés <hermilocg@gmail.com>

both leading models by more than 5 percentage points in mean pixel intersection-over-union (IoU) [26][27]. Mean pixel IoU is the most common benchmark for semantic segmentation. It measures the extent to which a model's predicted regions overlap with the true object regions, pixel by pixel. The results showed that enlarging the receptive field without pooling yielded sharper, more accurate segmentations.

Figure 3.7 visually compares the results of different models on a segmentation task. It shows how accurately different models can identify and outline distinct objects in images. The figure depicts five scenes: a person riding a horse, a fighter jet soaring through the sky, a bird in flight, furniture in a living room, and a group of chairs. Each scene appears with the original photo, predictions from three segmentation approaches, and the correct outlines (ground truth). The methods vary in precision. The first

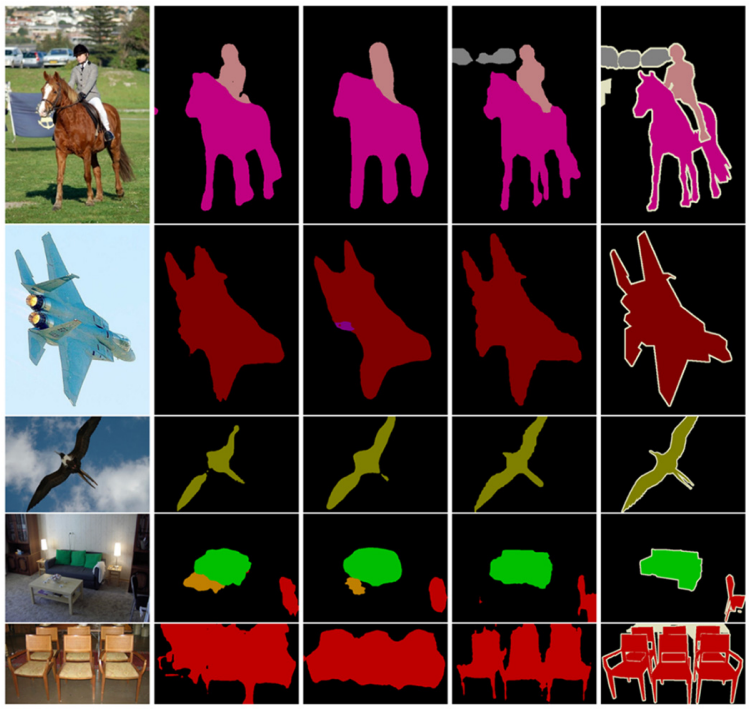


Figure 3.7 Semantic segmentations produced by different segmentation models. From left to right: the first column is the input image, the second is the prediction by FCN-8s, the third is the prediction by DeepLab, the fourth is the prediction by Yu and Koltun (2015), and the last is the ground truth. The examples illustrate progressively more precise and accurate segmentations, with the method of Yu and Koltun (2015) significantly improving boundary definition and object identification compared to previous methods. ***Used with permission granted by the lead author (Fisher Yu).***

Licensed to Hermilo Cortés <hermilocg@gmail.com>

method (FCN-8s) produces rough outlines and struggles to capture fine details. The second method (DeepLab) improves accuracy, outlining objects more closely but still with some imperfections. The authors' proposed method further refines these outlines, approaching the accuracy and crispness of the ground truth. Overall, the visual comparison highlights how dilated convolutions improve dense prediction.

Yet training the dilated context module proved unexpectedly unstable. Conventional random weight initialization didn't work well. The authors found a surprisingly simple remedy in the *identity function*. Instead of starting from random noise, each convolutional filter was initialized so that every output channel directly copied its corresponding input channel. In other words, each layer began as an identity mapping.

This initialization gave the network a meaningful starting point. Early in training, each layer simply passed information forward unchanged, creating a stable signal flow even through multiple dilated convolutions. Optimization then focused on learning small deviations (residuals) from the identity mapping that improved prediction accuracy, rather than on reconstructing the mapping from scratch.

The idea echoes the logic of residual connections, where the signal path remains open and learning refines the difference. But here, stability is achieved not through explicit skip connections but through the weights themselves. The network begins in a near-linear regime and then gradually learns nonlinear refinements as backpropagation adjusts the filters. This deceptively simple "identity initialization" allowed the context module to train reliably. Despite fears that such initialization might trap the model in a do-nothing state, the authors found the opposite to be true. The network quickly used contextual information and achieved measurable gains in segmentation accuracy without architectural changes. It was a minimalist, almost ResNetian, maneuver: subtle, elegant, and highly effective.

While ResNet excelled at classification, its reliance on pooling and downsampling made it poorly suited for dense tasks like segmentation, where spatial precision is crucial. The breakthrough came with *dilated convolutions* that expand the receptive field without sacrificing resolution or increasing the number of parameters. This led to the creation of the *context module*, which captures both local details and global context at full resolution by stacking dilated convolutions with increasing dilation rates. Identity initialization further stabilized training by starting each layer in a no-operation state, in which the layer does nothing and passes its input forward unchanged. Together, these design choices represent a pivotal moment in the evolution of deep learning for dense prediction.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

3.6 Human measuring sticks

In a 2015 blog post, Microsoft reported that its ResNet-powered systems could "meet and sometimes exceed human-level performance" [28]. This achievement energized public discussions about AI. Unlike AlexNet, which was largely a private milestone celebrated among insiders and marked by friendly wagers among leading researchers such as Geoffrey Hinton and Jitendra Malik, ResNet ushered in a period of explicit competition with human benchmarks and public demonstrations.

The human performance threshold was introduced in late 2014 by Fei-Fei Li's team at Stanford. It was derived from an extremely limited sample of two "expert annotators": one trained on 500 images and another on 100 [29]. This small experiment reported a human error rate of 5.1%, and 5% was subsequently adopted as the threshold for human performance. ResNet's performance was undeniably impressive, but achieving human-level performance from such a small dataset, with only two annotators, is methodologically questionable.

Yet methodology wasn't the only concern: the benchmark's validity was questionable. The ImageNet top-5 metric was designed to measure machine performance, allowing predictions to count as correct if the true label appeared among the model's top five guesses. For example, if a model is presented with a basketball and returns guesses like "croquet ball," "bikini," "warthog," "basketball," and "moving van," it still counts as correct [30]. This permissive scoring helps evaluate machines, because recognizing that the model is at least "close" provides diagnostic insight. But retroactively applying this criterion to humans is, at best, awkward.

Humans rarely hedge by offering multiple guesses when distinguishing straightforward items like "basketballs" and "bikinis," making the top-5 metric conceptually incongruent with human performance. This post hoc assignment created confusion about what "human-level" performance meant. ResNet's less frequently discussed top-1 accuracy was 78.57%, well below the 95% threshold for human performance [31][32].

The threshold effectively captured annotators' proficiency in labeling images according to ImageNet's specialized conventions rather than evaluating general human visual acuity. Such labeling involves idiosyncratic distinctions, such as fine-grained dog or bird breeds, and subtle category boundaries designed to train and evaluate machine learning models rather than reflect general human perception in real-world scenarios.

Remember, the ImageNet Large Scale Visual Recognition Challenge involves 1,000 object categories, including many fine-grained classes. While humans excel at recognizing familiar objects like bikinis, they

Licensed to Hermilo Cortés <hermilocg@gmail.com>

struggle with artificial categories, subtle distinctions, or unfamiliar specialized taxonomies—for example, distinguishing closely related dog breeds or rare bird species defined by the ImageNet dataset [33].

The ImageNet team admits this in their 2014 paper, describing "annotating images with one of 1,000 categories as an extremely challenging task for an untrained annotator" [34].

The human-level threshold became influential as an external benchmark for measuring progress in artificial intelligence, explicitly linking machine performance to human cognitive capabilities. In fact, ResNet's significance partly derives from surpassing this dubious yet symbolically powerful threshold, turning a simple metric into a milestone that many saw as tangible progress toward "real" artificial intelligence—however vaguely defined. This symbolism quickly escaped the confines of academic benchmarking and took on a life of its own in the public imagination, becoming media catnip.

It was frequently cited as proof that AI had become "better than humans" at vision. Headlines gushed with techno-triumphalism: *EE Times* ran "Microsoft, Google Beat Humans at Image Recognition" (Feb. 2015); Fast Company followed with "Microsoft Software Recognizes Images Better Than Humans Do" [35][36]. *The Guardian* went further in May 2015: "Computers Now Better than Humans at Recognising and Sorting Images" [37]. By 2017, *Entrepreneur* asked, "Can Artificial Intelligence Identify Pictures Better than Humans?" The article was jauntily subtitled, "It's Taken Computers Less than a Century to Learn What it Took Humans 540 Million Years to Know" [38]. Apparently, evolution needs to hire better engineers.

The breathless coverage rarely paused to ask whether the benchmark captured human vision or merely rewarded machines for outlabeling humans on a taxonomy no human cares about. Yet almost no one paused to consider an even deeper question: even if the benchmark did accurately reflect human-level visual acuity (and that's debatable), would surpassing this threshold imply cognitive or psychological plausibility for the machine? Did "beating humans" on ImageNet indicate anything fundamental about how machines perceive, understand, or reason about the visual world, or did it simply confirm that they had mastered their training data and labeling taxonomy?

In reality, surpassing the human-level threshold did not indicate human-like visual understanding or general robustness. Convolutional neural networks (CNNs) would soon face criticism for their lack of robustness and their limited ability to generalize to the real world.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Researchers discovered that these models were vulnerable to adversarial examples: inputs perturbed slightly in ways imperceptible to humans yet capable of fooling networks into confident misclassifications [39]. This spurred interest in more interpretable model designs and post-hoc explanation techniques, because simply adding residual connections to CNNs did not address the lack of insight into *why* the models make confident decisions.

Robustness failures extended beyond synthetic adversarial examples and manifested prominently in real-world deployments. For instance, in 2015, Google Photos mistakenly labeled images of Black individuals as "gorillas," illustrating how fragile model generalization to underrepresented examples can be, despite claims of human-level performance [40]. Even as architectures grew deeper and approached human-level performance on benchmarks, they remained susceptible to errors that no human would make when encountering subtle distribution shifts.

This discrepancy raised reasonable questions about what exactly constituted "human-level" performance if a model didn't behave like human intelligence. By then, Google had already improved its original GoogLeNet architecture. Inception v3 achieved a 5.6% top-5 error on ImageNet, placing second only to ResNet's 4.49% in the 2015 ILSVRC competition. Yet flirting with so-called human-level benchmarks did not immunize these models against real-world brittleness, highlighting the persistent gap between leaderboard metrics and genuine robustness [41]. Although depth offered superior representational learning compared to shallower models, achieving high accuracy alone provided few guarantees when training distributions were incomplete or when predictions spanned sensitive categories.

While ResNet did not establish this threshold, the Microsoft team behind it embraced its symbolic weight. In an earlier, unrelated 2015 paper, they wrote, "our result is the first to surpass human-level performance (5.1%) on this visual recognition challenge." By the time the ResNet paper was released later that year, the team was already keenly aware of the rhetorical power of crossing that threshold. They made this explicit by citing the original human-level performance estimate in the paper's summary. The 5% error rate would become more than a metric, and ResNet crossed it with deliberate emphasis.

Despite its limitations, ImageNet's 5% human benchmark was pivotal in modern AI, not because it was methodologically sound or valid or because it was the first time machines were compared with humans, but because it offered a fixed, quantitative measure of human performance.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

There were earlier human benchmarks, including IBM's Deep Blue, the Chinook program, IBM Watson's victory in Jeopardy!, Facebook's "DeepFace," DeepMind's Agent57, and AlphaGo. Still, ImageNet's threshold was among the earliest such metrics to define and drive progress in the deep learning era.

The Microsoft research team leaned into this trend and helped solidify "human-level performance" as an explicit research goal. This inspired others to follow suit, shifting the Overton Window in modern AI toward the routine use of human baselines as benchmarks. Today, whether in vision, language, or speech, researchers commonly establish human baselines, set human-level thresholds, and celebrate models that reach or surpass them. While earlier achievements hinted at this shift, ResNet's breakthrough popularized the now-common belief that human performance is a benchmark to surpass.

From a philosophical perspective, reducing human capability to a numerical benchmark oversimplifies complex questions about intelligence and human cognition. Societally, treating humans merely as benchmarks to surpass shifts the emphasis away from creating complementary, beneficial technology toward competition, implicitly redefining humanity's role from innovator to competitor. While AI may have been technically ready to chase benchmarks, most people weren't yet prepared for what it would mean to surpass them.

3.7 CS231n

Stanford's CS231n was the university's first full-length course devoted entirely to deep learning and closely mirrored the ILSVRC breakthroughs. Architectures that defined the era, such as AlexNet, VGGNet, GoogLeNet, and ResNet, were all part of the curriculum [42][43][44][45]. The course even had students train a model on a subset of ImageNet, underscoring this connection and the "learn by doing" philosophy. The course was co-created by Fei-Fei Li, Andrej Karpathy, and Justin Johnson.

CS231n has also had a massive influence beyond Stanford's walls. The lecture slides, videos, notes, and assignments are freely available online. The instructors encouraged outsiders to follow along and even established a public subreddit (r/cs231n) and a Twitter account (@cs231n) where learners could discuss and ask questions. This openness helped CS231n gain a global following and broaden its reach.

By 2017, the course had grown from 152 Stanford students enrolled in 2015 to a peak of 774, making it one of the largest classes in Stanford

Licensed to Hermilo Cortés <hermilocg@gmail.com>

computer science. For context, Andrew Ng is a Stanford professor who cofounded Google Brain, founded Coursera and DeepLearning.AI, and led Baidu's AI Group. His renowned machine learning course, CS229, had 869 students at its peak [46]. CS231n simultaneously chronicled and shaped the rise of deep learning in vision. It's a course with its own legacy, one that continues to equip the next generation, which is why it's featured on Sutskever's List.

3.8 Scale what?

ResNet emerged as a pragmatic response to model degradation. As depth increased, networks performed worse. Residual connections and, later, pre-activation restored signal flow. That stability created a flywheel effect: small architectural tweaks unlocked larger models, and larger models increased the payoff of the architecture. This flywheel is specific to deep learning.

By contrast, a support vector machine is a shallow margin maximizer. It selects a decision boundary in a fixed feature space (explicit or kernel-defined). There are no stacked representations or gradients to propagate through depth, so there's no analog to vanishing gradients or model degradation as networks get deeper. In fact, there's no "deeper" knob to turn. While you can use more compute to solve the convex optimization more precisely, that doesn't create the deep-learning-style scaling loop. So this architecture renders innovations like residual connections irrelevant.

In contrast, computation and data compound only when the architecture reliably converts them into richer learned representations. That's precisely the regime that residual connections enable in deep learning. Residual connections matter only when a model iteratively learns layered representations through gradient-based optimization across many layers.

In his influential 2019 essay, "The Bitter Lesson," Richard Sutton argues that researchers are better off investing in general-purpose methods that reliably improve as compute increases, rather than hand-tuning heuristics or building solutions around introspective stories about "how we think we think" [47]. That dynamic was evident in the previous chapter, when AlexNet outperformed handcrafted feature engineering (for example, SIFT). Encoding prior knowledge can yield short-term gains, but over time, those gains are often surpassed by architectures that learn relevant features directly from data at scale.

But "The Bitter Lesson" is often misread as implying that the primary driver of progress is "more compute wins," while researchers and architectures play a diminishing role. As Ilya Sutskever repeatedly emphasizes in interviews, often in response to "The Bitter Lesson," we must clarify:

Licensed to Hermilo Cortés <hermilocg@gmail.com>

"What exactly are we scaling?" [48]. "Scale" is a multiplier, but it becomes large only when the underlying method is designed to convert computational resources into performance.

A similar argument appears in Alyosha Efros's "fancy schmancy" narrative, which explicitly treats data as the "unsung hero" that explains a surprising amount of progress [49]. He argues that data is the universal equalizer and that researchers should be humbler. The Bitter Lesson treats computation as the universal equalizer and likewise urges humility for researchers. Both narratives emphasize that the real magic isn't the architecture and that researchers tend to give themselves too much credit. But do they?

Data and compute are necessary but insufficient. The decisive question is not whether we have more of them but whether the architecture and training paradigms can use them effectively. One reason a reading list like Sutskever's matters is that it makes legible the inventions that made scaling pay off in the first place and the people behind them. If The Bitter Lesson tells researchers to scale, Ilya's rejoinder poses the more actionable research question: scale what?

Ultimately, ResNet and the residual connections it introduced hit the sweet spot between innovation and practicality. It earned the prestigious 2016 Best Paper Award at the Conference on Computer Vision and Pattern Recognition (CVPR), underscoring its influence on a community that was genuinely surprised. Reflecting on the success, Kaiming He remarked that ResNet "destroys some of the assumptions" he had previously held about the limitations of deep learning [50].

In 2012, AlexNet ignited the deep learning revolution, catapulting the ILSVRC into a defining battlefield for computer vision. Yet barely half a decade later (and just two years after ResNet's introduction), ImageNet reached a saturation point. The contest—regarded by Jitendra Malik as ahead of its time—had been solved, prompting Fei-Fei Li to discontinue the competition. Researchers shifted their gaze to new frontiers. In the aftermath, the ImageNet dataset transitioned from a cutting-edge benchmark to just another baseline. It now occupies a more symbolic position in computer vision research, akin to MNIST, a dataset of handwritten digits that has long served as an introductory tool and reference point for beginners rather than a source of ongoing innovation.

By 2025, the original ResNet paper had amassed over 100,000 citations in the academic literature and roughly 250,000 on Google Scholar, making it the most-cited scientific paper of the 21st century across *all* fields [51]. Virtually every new computer vision paper referenced, compared

Licensed to Hermilo Cortés <hermilocg@gmail.com>

against, or built upon ResNet, solidifying its role as the standard architecture in the field. Residual connections also transcended computer vision, fueling breakthroughs in domains ranging from AlphaGo's mastery of board games and AlphaFold's protein-structure predictions to advances in language modeling like GPT-2. By enabling deep networks, residual connections have become essential to modern AI across various domains.

Yet computer vision was only one front in the broader deep learning revolution. As models expanded into sequences, speech, and language, they also grew larger, more complex, and more demanding. This scaling placed new pressure on the field, shifting progress away from isolated architectural insights and toward the engineering discipline required to train, optimize, and deploy systems at scale.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

4 Deep learning accelerates

This chapter covers

- The resurgence of recurrent neural networks after AlexNet
- How Karpathy's blog made recurrent neural networks accessible, inspiring experimentation
- How Chris Olah clarified long short-term memory with vivid visuals and metaphors
- How selective dropout enabled deeper recurrent networks
- How Deep Speech 2 proved the real-world potential of recurrent neural networks
- The engineering shift in AI

Papers

- *The Unreasonable Effectiveness of Recurrent Neural Networks* (2015) Andrej Karpathy
- *Understanding LSTM Networks* (2015) Chris Olah
- *Recurrent Neural Network Regularization* (2014) Zaremba, Sutskever, and Vinyals
- *Deep Speech 2: End-to-End Speech Recognition in English and Mandarin* (2015) Amodei et al.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Recurrent neural networks (RNNs) are sequence models. Sequence models process ordered data such as text, speech, or time series, where the position of each element matters. n-grams are sequence models. They capture short-range dependencies by looking at a fixed window of prior tokens. RNNs extended this idea by carrying forward a compressed representation of the entire sequence history. Yet, despite their theoretical promise, practical problems such as vanishing or exploding gradients, poor handling of long-range temporal dependencies, and inefficient training limit their effectiveness. These challenges persisted until a dropout method was introduced, specifically designed for recurrent architectures.

During this period (2014–2016), the intuitive appeal of recurrent networks became widely appreciated through influential educational resources. Andrej Karpathy sparked interest across the developer community with vivid examples and accessible code, leading to widespread experimentation. Complementing Karpathy's approachable insights, Chris Olah offered clear, intuitive visualizations that illuminated the inner workings of long short-term memory (LSTM) networks.

This chapter concludes with an examination of Deep Speech 2 (DS2), a landmark end-to-end RNN-based speech recognition system that illustrates how sophisticated architectures, combined with sufficient computational resources, can surpass human accuracy. DS2 represents the practical, scalable progress that Ilya Sutskever advocates, cementing RNNs alongside convolutional neural networks (CNNs) as foundational architectures within the deep learning revolution.

4.1 The unreasonable effectiveness of RNNs

In May 2015, Andrej Karpathy wrote a blog post titled "The Unreasonable Effectiveness of Recurrent Neural Networks" [1]. At the time, explanations of RNNs were dense, math heavy, and difficult to follow. Karpathy offered an intuitive approach, showing what RNNs could achieve through salient examples and easy-to-follow code samples.

One particularly memorable experiment involved feeding the model the full text of Paul Graham's startup essays; Graham is best known for cofounding Y Combinator [2]. The motivation was tongue-in-cheek. Graham offers timeless startup advice, but he's a slow generator. It would be nice to synthesize fresh startup wisdom on demand.

Karpathy's Paul Graham generator was built as a character-level language model trained on about 1 MB of concatenated text from Paul Gra-

Licensed to Hermilo Cortés <hermilocg@gmail.com>

ham's essays, totaling roughly 1 million characters. The model ("char-rnn") was implemented as a two-layer stacked LSTM, with each layer comprising 512 hidden units, for a total of about 3.5 million parameters. It used dropout regularization with a rate of 0.5. After training, the model generated new text by predicting one character at a time, sampling from a probability distribution whose randomness could be controlled by adjusting a temperature parameter; lower temperatures produced predictable, repetitive text, while higher temperatures produced more creative but less coherent output. The resulting text is as follows:

LSTM The surprised in investors weren't going to raise money. I'm not the company with the time there are all interesting quickly, don't have to get off the same programmers. There's a super-angel round fundraising, why do you can do. If you have a different physical investment are become in people who reduced in a startup with the way to argument the acquirer could see them just that you're also the founders will part of users' affords that and an alternation to the idea. [2] Don't work at first member to see the way kids will seem in advance of a bad successful startup. And if you have to act the big company too.

This isn't great. While there are some familiar startup clichés, such as super angels, acquirers, and fundraising rounds, the entrepreneurial advice is unmistakably incoherent. It's the startup equivalent of the uncanny valley: vaguely familiar but emotionally hollow. It even hallucinates a citation, "[2]." It's trying to sound authoritative instead of clear, which is the opposite of how Paul Graham writes. Reading it now feels somewhat disorienting. Indeed, it fails to meet the vibe test established by Ovid's Unicorn, a standard that GPT-2 wouldn't reach for another four years. It serves as a reminder of how low expectations once were, how easily many were impressed, and how far the field has come since then.

Curious, I prompted ChatGPT (4o) to "help me rewrite this," and it responded with the following [3]:



This seems like a garbled or machine-generated paragraph, but here's an attempt to rewrite it into something coherent and meaningful, assuming it's about startup fundraising and acquisition dynamics.

Rewritten version: "Surprised investors weren't planning to raise money. I wasn't at the company full-time, but things were moving quickly—there wasn't a need to hire the same kind of programmers right away. It felt more like a super-angel round than a traditional fundraise. If your investment thesis is more physical than digital, you tend to back people who can reduce complexity in a startup. That makes it easier for potential acquirers to see the long-term value. Founders who focus on user needs

Licensed to Hermilo Cortés <hermilocg@gmail.com>

> early tend to build stronger momentum, even if the initial product doesn't work perfectly. And sometimes, if you're forced to act like a big company too soon, it can kill the very thing that made the startup interesting."

If we're being charitable, "char-rnn" is learning at the level of individual characters, making its unintelligible rambling somewhat impressive. Character-level models struggle with coherence because they operate at a much finer granularity, generating text *one letter* at a time rather than using words or subwords. Unlike token-based models that work with meaningful word chunks, character-level models have no built-in notion of words or spelling, making it harder to maintain semantic consistency across sentences. This means they must model long-range dependencies over many more steps, greatly increasing the risk of drifting off-topic or losing coherence. Yet even at this granular level, the model isn't blindly assembling letters; it's approximating linguistic structure and conceptual patterns. Despite these inherent limitations, even barely coherent text generation felt groundbreaking in 2015.

Indeed, Karpathy's original post opens by capturing this sense of wonder: "There's something *magical* [sic] about Recurrent Neural Networks." At the time, it didn't sound as glib as it might today; watching a model generate text, character by character, was quite astonishing. Karpathy explains this "magic" by contrasting RNNs with CNNs. CNNs require a fixed-size vector as input and produce a fixed-size vector as output. This makes them ideal for independently analyzing static images or individual video frames but limits their effectiveness for tasks involving sequential data, such as sentences, audio streams, or continuous video sequences.

Structurally, RNNs share fundamental components with other neural networks, including layers composed of neurons, connection weights, biases, activation functions, and training via backpropagation. Their distinguishing feature is recurrent feedback loops, which allow information from a previous time step to influence the current output. This loop enables RNNs to process sequences of arbitrary length without requiring fixed-size inputs because the same computational unit is applied recursively at each step, carrying context forward through time [4]. See figure 4.1.

At each time step, a vanilla RNN takes two inputs. The first is the current element of the sequence (for example, a word, subword, or character), and the second is the hidden state from the previous step, which encodes a summary of all prior inputs. This hidden state acts as a persistent internal "memory," updating at every step and providing context for subsequent ones. Although the network generates an output at each step, the hidden state, rather than the output, is passed along to the next

Licensed to Hermilo Cortés <hermilocg@gmail.com>

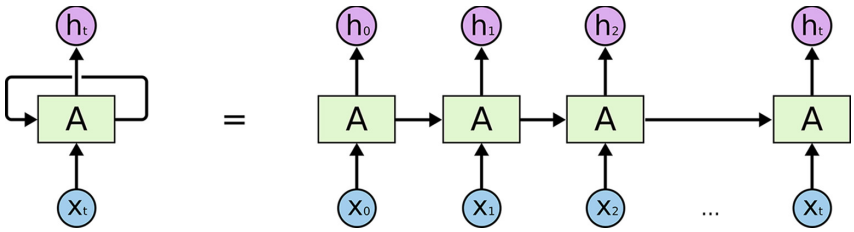


Figure 4.1 This image, from Chris Olah's work (discussed in the next section), shows a chain-like structure representing an RNN. It illustrates how RNNs process data sequentially, with the output of one step feeding into the next. The recurrent, linked modules reflect the RNN's ability to carry information forward, making it well suited to learning patterns that unfold over time, such as time series, text, or audio. *Used with permission granted by the author (Chris Olah).*

step. In this way, the RNN maintains context over time, allowing it to model sequential data.

To better appreciate the significance of this design, consider a simpler model, such as an n-gram, which stores probabilities for sequences of length *n*. For example, predicting the next letter after "hel" relies solely on how often certain letters followed "hel" in the training data. n-grams have a fixed, limited memory and can't capture dependencies longer than *n* steps. By contrast, an RNN's dynamic hidden state can encode contextual patterns beyond a fixed window. Instead of counting sequence frequencies, RNNs learn distributed representations that implicitly capture context, semantics, and linguistic structure, enabling them to model more complex relationships.

When processing the word "hello," the RNN doesn't predict each letter in isolation. Instead, it continuously updates an internal representation based on context. After encountering "hell," the model must determine whether the word is complete ("hell") or whether it should continue to form "hello." Crucially, this decision is based not only on memorized frequencies from the training data but also on the immediate context encoded in its hidden state. In short, the hidden state enables the model to remember and use context to resolve ambiguities (e.g., repeated letters) and to determine when sequences should end.

Karpathy's core demonstration focused on character-level prediction using datasets such as Wikipedia, Shakespeare, LaTeX, and Linux source code. The Wikipedia model produced plausible Markdown and citation structures. The Shakespeare model successfully imitated the structural patterns of stage dialogue, though not its dramatic effect or poetic quality. For example:

Licensed to Hermilo Cortés <hermilocg@gmail.com>

LSTM **PANDARUS:**

Alas, I think he shall be come approached and the day;
When little srain would be attain'd into being never fed.

The LaTeX model often generated syntactically valid equations, but it also made mistakes. LaTeX is a typesetting language for writing mathematical and scientific documents, in which environments such as *proofs* and *lemmas* must be opened and closed with matching commands. The model would sometimes begin a `\begin{proof}` environment only to mistakenly close it with `\end{lemma}`. These errors stemmed from an inability to model long-term dependencies. By the time the model reached the end of a proof, it had lost track of whether it was handling a proof or a lemma. Similarly, the model might open with `\begin{enumerate}` without remembering to close it. These errors became less frequent as the models grew larger, but they still highlight the challenge of maintaining context over extended sequences.

To further illuminate these results, Karpathy introduced tools to interpret the internal workings of the RNN. He visualized neuron activations, revealing cells that fired in response to quotes, URLs, or Markdown tags. Some neurons became scope trackers, while others appeared to be quote balancers or indentation counters. These visualizations provided a rare window into the distributed computation inside models, as an early, accessible example of structure in learned representations.

The blog went viral, becoming the top story on *Hacker News* with over 900 points and hundreds of comments [5]. Readers on the popular tech forum praised Karpathy's ability to clarify complex concepts; one commenter noted it was "the first time I understood someone from this field … [he's] just broken down the wall. What's more important, passion flows through his writing. And it can be felt. I got so excited while reading it" [6]. The sentiment was widely shared. On Reddit, readers marveled at the model's ability to generate plausible LaTeX formulas. "It can nearly write valid LaTeX?" someone wrote earnestly. "If that's not a sign of remarkable intelligence, I don't know what is." At least for Redditors, the true hallmark of "intelligence" is proper equation formatting. Valid or not, some vibes were there [7][8].

The blog's accompanying open-source code accumulated nearly 12,000 stars on GitHub and thousands of forks [9]. Developers worldwide began training character-level RNNs on nearly any dataset imaginable: cooking recipes, Eminem lyrics, Irish folk songs, Trump and Obama speeches, and even the Bible [10][11][12][13][14][15]. Twitter and Reddit were filled with whimsical RNN-generated texts. These spin-offs

Licensed to Hermilo Cortés <hermilocg@gmail.com>

created a flywheel effect: derivative content amplified the blog's reach, reinforced its visibility, and accelerated its influence in both technical and popular domains.

AI researchers also extended the post's reach. Chris Olah referenced Karpathy's blog in his viral essay, "Understanding LSTM Networks" [16]. Olah wrote, "I'll leave discussion of the amazing feats one can achieve with RNNs to Andrej Karpathy's excellent blog post." It became a pedagogical moment. Karpathy's clear explanations, examples, and code reduced barriers to entry. One commenter captured the spirit: "Karpathy is one of my favorite authors … he spends much of his time demystifying the field … a brilliant way to encourage others to explore it" [17]. Karpathy's influential blog and the CS231n course became canonical in the deep learning community.

Naturally, not everyone was swept up by the vibes. A parallel, skeptical conversation emerged. Researcher Yoav Goldberg, for instance, wrote a follow-up blog post comparing Karpathy's post on character-level RNNs with simpler baselines, such as unsmoothed n-gram models [18].

Goldberg essentially applied the same methodological demand as Efros, but in a less formal venue. In peer review, Efros pushed CNN papers to demonstrate improvements over a bare-bones nearest-neighbor baseline trained and tested on the same data. Goldberg issued a similar challenge for character-level RNN work, comparing it with frequency-based n-gram baselines before crediting the architecture or its authors. The contexts certainly differ but not the principle. In both cases, the underlying message is the same: simpler models perform surprisingly well given enough data. The broader point is that data acts as an equalizer, neutralizing architectural differences.

Goldberg argues that readers of Karpathy's blog were "impressed for the wrong reasons." This argument implies that only a more discerning reader like Goldberg can see what's *actually* impressive. The "wrong reasons," as Goldberg explains, are that readers mistakenly believe coherent text generation is unique or special to neural architectures such as RNNs. But by emphasizing the arbitrary overlap between outcomes such as coherent text and dismissing architectural differences, Goldberg's critique creates a false equivalence. It's a shallow symmetry that collapses under inspection. When models converge on similar outputs, it's tempting to see sameness. But sameness in outcome is not sameness in architecture.

Tomas Mikolov, best known for developing *word2vec* (2013), provided clear quantitative evidence of the advantages of RNNs, including 50% lower perplexity in language modeling and an 18% relative reduction in

Licensed to Hermilo Cortés <hermilocg@gmail.com>

word error rates (WER) in speech recognition, compared with smoothed n-gram models [19][20]. WER is the standard accuracy metric for speech recognition and measures the degree to which a system's transcript differs from the reference ground truth. More tellingly, the RNN maintained this advantage even when the n-gram model was given substantially more data, highlighting a fundamental representational advantage.

Goldberg's critique also had a rhetorical bite. Referring to something as an unsmoothed n-gram is meant to cut it down to size by insisting that the performance is just a trick anyone can do with enough data. While RNNs still had a long way to go, they demonstrated a qualitative and quantitative leap in learning structure over simple statistical models. Performance emerges not as a compromise between architecture and data but from their combined strengths. The answer is not somewhere in the middle; it's emphatically both.

Ironically, Goldberg's rhetorical trick of equating performance with architecture is commonly used by supporters of artificial general intelligence (AGI), who conflate machine and human performance with architectural equivalence. This conflation exemplifies the core principle of computational functionalism, a philosophical view that treats systems as equivalent if they perform the same function. For example, biological and computational substrates are considered irrelevant as long as their functional roles are maintained. From this perspective, similar performance, behaviors, or individuated functions render the specifics of internal mechanisms insignificant. This reasoning flattens the mechanistic and epistemic asymmetries between minds and models. In each scenario, the rhetorical trick involves exaggerating similarities to obscure distinctions, disguising metaphysical differences behind superficial equivalence. The point is that skeptics, like Goldberg, and AGI enthusiasts use the same flawed reasoning. Even so, Karpathy linked to Goldberg's critique in his original post, trusting readers to evaluate competing perspectives fairly.

Ultimately, Goldberg acknowledges that RNNs are "cool." Still, even this concession feels less like admiration and more like a faintly dismissive nod to their popular appeal, as if they are entertaining toys rather than serious tools [21]. At the time, this skepticism was somewhat justified because RNNs were experimental, even toy-like. In fact, Karpathy's blog shows they weren't yet serious tools. But neither were n-grams, and RNNs at least demonstrated superior performance. Recurrence would soon be eclipsed, though, by attention-based artificial neural networks (chapter 5).

Unlike Goldberg's blog, Karpathy's blog has inspired countless projects because it celebrates practical exploration and engineering ingenuity rather than dismissing them. It arrived when deep learning was opaque to

Licensed to Hermilo Cortés <hermilocg@gmail.com>

most developers, making it approachable, visual, playful, and free of pretense. It wasn't merely about RNNs producing coherent output; it was about sudden accessibility. Anyone with a GPU and a text file could explore it. Karpathy's blog captured that brief cultural window when the gap between cutting-edge research and hobbyist experimentation momentarily collapsed, making artificial neural networks feel tangible to many for the first time. It demonstrates that a well-crafted blog can command mindshare rivaling that of a highly cited research paper. The truly "unreasonable effectiveness" was how far one well-written blog post could reach.

4.2 Understanding LSTM networks

While Karpathy demonstrated *what* RNNs could achieve, Chris Olah's influential blog, "Understanding LSTM Networks" [22], vividly illustrated *how* they achieved it, using intuitive visualizations and memorable analogies. His clear, engaging style didn't merely simplify LSTMs; it set a new standard for clarity and accessibility at a moment when the field most needed both.

Traditional feed-forward neural networks treat each input independently, but many tasks, such as predicting the next word in a sentence, require context. RNNs solve this problem by processing data step by step while maintaining a hidden state that captures past information. Unlike simple language models such as n-grams, which look at only a limited, fixed window, RNNs can capture dependencies spanning many steps, allowing earlier context to influence later predictions [23][24]. Consider the sentence "I grew up in France … I speak fluent French." To accurately predict "French," the network must recall the earlier mention of "France," even if it is separated by many intervening words. Unfortunately, RNNs struggle to connect information as the distance between intervening words increases.

Training RNNs relies on backpropagation through time, which "unrolls" the recurrent structure across multiple steps, transforming a loop-like recurrence into a deep linear sequence with shared weights [25]. Predicting the next word after observing 100 words is analogous to training a network with 100 layers. As with CNNs, depth introduces challenges. For instance, learning the relationship between the first and last words of a 100-word sequence is extremely difficult because the gradients that signal how earlier inputs influence the final prediction become negligibly small, leaving the model with almost no learning signal about long-range dependencies. The influence of an input on the loss diminishes exponentially as the gap between input and output increases, causing RNNs to ignore earlier yet relevant information [26].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

To overcome the vanishing gradient problem and better model long-range dependencies, Sepp Hochreiter and Jürgen Schmidhuber developed a specialized type of RNN: an LSTM network [27]. LSTMs significantly improve performance on tasks that require long-range memory, such as language modeling, speech recognition, and handwriting recognition. Instead of a single neural network layer, an LSTM uses four layers that interact in a distinct way. Olah famously described the internal workings of an LSTM as a conveyor belt moving smoothly through a factory, selectively adding, carrying, and removing items (information) at precisely the right moments. This analogy shows how these networks effectively manage information over long sequences [28].

LSTMs manage information flow through specialized gates and maintain two types of memory: long-term and short-term (hence the name long short-term memory). The cell state (C_t) is considered the long-term memory and is the core component of LSTMs (see figure 4.2). It allows information to pass through multiple steps almost unchanged, helping prevent gradient-related problems. This is conceptually similar to residual

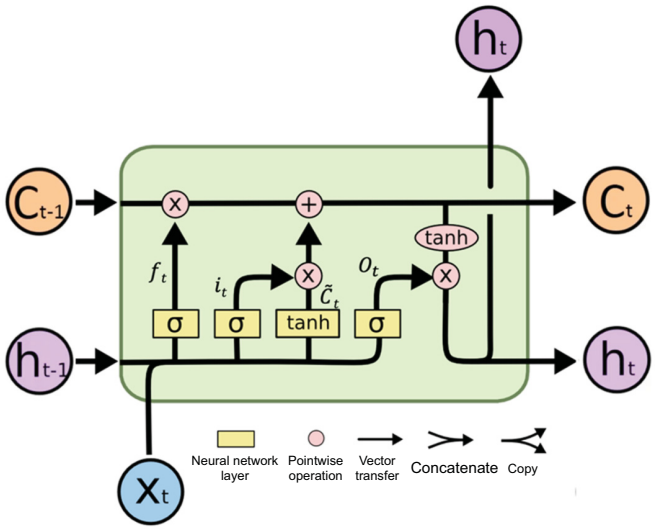


Figure 4.2 A single LSTM cell. The inputs are the current vector X_t, the previous hidden state h_{t-1}, and the previous cell state C_{t-1}. There are three learned gates: the forget f_t, input i_t, and output O_t. They are all produced by sigmoid layers ("σ" blocks). A tanh layer forms a candidate update $\tilde{C}_t$. Pointwise multiplications ("×") and an addition ("+") update the cell state on the top "conveyor" pathway. Arrows indicate vector flow; the legend marks pointwise operations, concatenation, and copies. This is the standard LSTM, emphasizing how gating enables selective memory retention and exposure. *Used with permission granted by the author (Chris Olah), though it is slightly modified from its original form.*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

connections in that both provide paths that allow gradients to flow more easily. While residual connections directly add inputs to outputs, LSTM cell states selectively manage information through gating mechanisms.

Using Olah's metaphor, the cell state is the conveyor belt and the backbone of the LSTM's ability to retain important information from much earlier in the sequence. By comparison, the short-term memory (h_t), or hidden state, is actively adjusted at each step and responds to the current input (X_t). These two memory pathways are closely connected, enabling LSTMs to manage long-term context and short-term variation.

To manage this dual-memory system (long- and short-term), LSTMs use three gates: the forget gate, the input gate, and the output gate.

The *forget gate* (f_t) decides how much past information to keep. It combines the current input with the previous hidden state, producing values between 0 and 1 that scale the previous cell state. A value close to 0 means that most or all of the previous memory is forgotten, while a value near 1 preserves it.

The *input gate* (i_t) and the *candidate* ($\widetilde{C}_t$) work together to introduce new information into long-term memory. The candidate memory uses the tanh activation function to produce values between –1 and 1, resulting in richer representations. A separate sigmoid function determines which parts of this candidate memory should be stored. This filtered candidate memory is then added to long-term memory, updating the cell state. Technically, this filtering is done via pointwise multiplication: each element can be scaled individually rather than uniformly across all dimensions, as with matrix multiplication.

Lastly, the *output gate* (O_t) shapes the short-term memory output. After updating the long-term memory, the cell state is processed through a tanh activation function to produce a potential short-term memory. The output gate then applies another sigmoid function to determine how much of the potential short-term memory to output. This becomes the output for the current time step. This multiplicative gating allows the network to selectively pass, update, or suppress information. Crucially, when gates are near 1, they preserve information and gradients across many time steps, helping LSTMs overcome the vanishing gradient problem.

The simultaneous use of sigmoid and tanh activation functions is a particularly clever design choice. Sigmoid regulates information flow, producing values between 0 and 1 that determine whether to retain, update, or discard information. In contrast, tanh maps values between –1 and 1, enabling the model to represent both positive and negative signals and capture more nuanced information. Ultimately, sigmoid controls

Licensed to Hermilo Cortés <hermilocg@gmail.com>

how much information flows, while tanh determines what information is stored. Together, they complement each other.

Although sigmoid and tanh can contribute to vanishing gradients in standard networks, LSTMs use them within a gated, additive architecture that preserves information and gradients over long sequences. By combining multiplicative gates with additive updates, the LSTM avoids the unstable gradient behavior of vanilla RNNs while remaining selective. For example, when processing a story that mentions "Alice," an LSTM can retain that context across multiple sentences and correctly predict later that pronouns such as "she" refer to Alice. This made LSTMs the standard RNN variant for sequence modeling tasks involving extensive context dependencies. More precisely, the network can keep that entity active in C_t for many steps, using O_t to reveal it when needed (for example, when predicting "she") and f_t to drop it when the context shifts.

To see why "Understanding LSTM Networks" felt like a pedagogical leap, it helps to place it alongside the original schematic from Hochreiter and Schmidhuber (1997). Their diagram was designed for researchers. It was a precise rendering of the "constant-error carousel," with explicit gating and annotated variables. That rigor served its audience well. For newcomers, the same fidelity can feel overwhelming, like studying a wiring harness [29]. See figure 4.3.

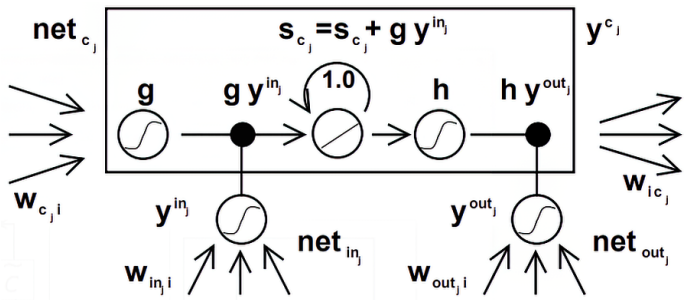


Figure 4.3 The original Hochreiter–Schmidhuber (1997) LSTM cell schematic, including the "constant-error carousel." ***Used with permission granted by one of the authors (Jürgen Schmidhuber).***

After the original LSTM, the architecture evolved. The forget gate was introduced, and later *peephole connections* were added [30]. These extensions were documented in equally precise schematics. As shown in figure 4.4, the blueprint tradition makes every pathway explicit so the design is

Licensed to Hermilo Cortés <hermilocg@gmail.com>

unambiguous. Even so, the result is denser for new learners. Olah's treatment shifts the vantage point to the information flow, using simplified iconography and the conveyor-beat metaphor to foreground what the model is *doing*. "Understanding LSTM Networks" was an advance in exposition rather than in mathematics, but it was one that finally made the core insight teachable at a glance. Read together, the two styles are complementary; one shows precisely how the cell is built, while the other shows why it behaves the way it does.

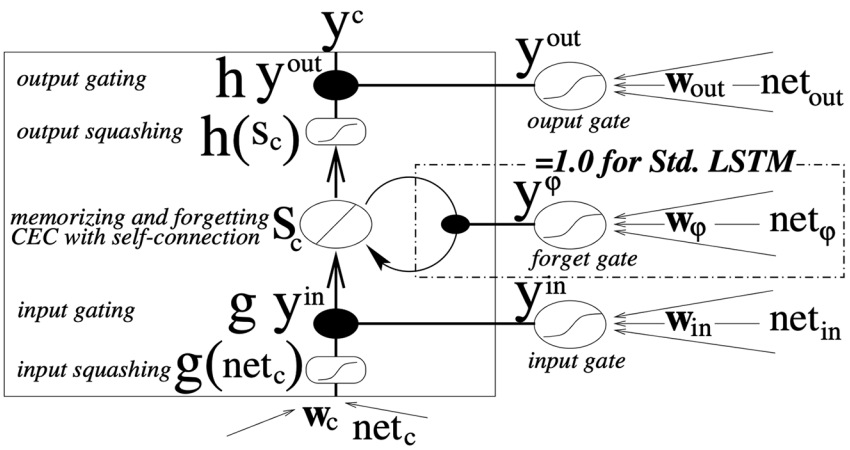


Figure 4.4 An expanded LSTM schematic showing the forget gate with weighted connections to the inputs and outputs. ***Used with permission granted by one of the authors (Jürgen Schmidhuber).***

"Understanding LSTM Networks" wouldn't have been publishable in a traditional academic venue because it's too expository and too metaphorical. The post inadvertently highlights a widening gap between what advances understanding, like metaphors, exposition, and intuitive illustrations, and what academic papers typically emphasize. Across fields, readability has declined and jargon has grown, with papers increasingly optimized for formalism, reproducibility, and citation metrics rather than for communicative clarity [31]. Against that backdrop, Olah's essay succeeds precisely because it treats explanation as the research objective. It abstracts away wiring details to foreground information flow, employs stable visual language, and gives both newcomers and experts a shared mental model.

Olah's blog appears on Sutskever's List not because it introduced a novel algorithm like those of Hochreiter or Schmidhuber but because of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

its clarity and broad influence. Where many contemporary papers maximize legibility for reviewers, Olah maximizes legibility for readers. The response to his work suggests that, for complex ideas, exposition has real value. Indeed, most entries on Sutskever's List share this trait: their significance comes less from pure novelty than from exceptional execution and a demonstrated ability to reach larger audiences.

4.3 Recurrent neural network regularization

To understand why Karpathy and Olah's blog posts had such an influence in 2015, it's essential to revisit the challenges RNNs faced just a year earlier. Despite their theoretical promise, every attempt to scale RNNs ran into overfitting. As a result, surpassing key thresholds on standard sequential modeling benchmarks, such as the Penn Treebank (PTB), seemed agonizingly out of reach. One particularly daunting barrier was a perplexity of 100.

Perplexity measures how "surprised" a language model is by new words: lower perplexity means the model is more certain, while higher perplexity means it's stumped. One helpful way to think about it is as a multiple-choice quiz. A perplexity of 100 is like the model choosing among approximately 100 plausible next options at each step in a sequence. For comparison, if the model were just guessing randomly from a vocabulary of 50,000 possible tokens, its perplexity would be around 50,000. So 100 is much better than random. It shows the model has learned real patterns, even if it isn't very confident. Yet at the time, achieving a perplexity below 100 felt like an insurmountable challenge.

In 2014, Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals transformed the landscape by carefully engineering a selective dropout approach tailored to LSTM architectures [32]. They introduced the technique in a paper titled "Recurrent Neural Network Regularization," addressing the persistent challenge of overfitting. Echoing AlexNet's earlier rigorous methodological approach and targeted regularization innovations, their work marked a decisive turning point for RNNs, which had long been stuck.

Recall that AlexNet employed dropout, which temporarily "turns off" a random subset of neurons, creating a slightly different network at each training iteration. This randomness compels neurons to develop robust features rather than rely on specific combinations of neurons. During inference (or testing), dropout is disabled, and neuron outputs are scaled down by 0.5 in AlexNet as a computationally efficient approximation of averaging predictions from all the hypothetical networks created during

Licensed to Hermilo Cortés <hermilocg@gmail.com>

dropout training. In this way, dropout achieves effects similar to model ensembling, significantly reducing overfitting without explicitly training separate models. While dropout roughly doubles the number of training iterations, it improves generalization and reduces the model's tendency to overfit training examples.

Yet training RNNs was notoriously frustrating, as it had been with CNNs several years earlier. It was so difficult, in fact, that one of the era's most influential papers, cited over 8,000 times, was titled "On the Difficulty of Training Recurrent Neural Networks" [33]. The authors pinpointed why recurrent networks struggled. This should come as no surprise at this point, but severe training instabilities caused by vanishing and exploding gradients crippled their ability to model long-term dependencies. Moreover, when researchers tried to scale up these networks, they inevitably encountered overfitting. While partial solutions, such as gradient clipping, existed to tame exploding gradients, the vanishing gradient problem persisted and remained a significant bottleneck to training deeper RNN architectures.

In follow-up experiments, these same researchers explored deeper networks, testing architectures with three recurrent layers, but achieved only marginal improvements in perplexity. Even these modest gains required elaborate tricks, such as shortcut connections and layer-by-layer pretraining reminiscent of pre-AlexNet strategies [34]. They warned explicitly that training could become "even more problematic as models grow deeper," implying diminishing returns unless breakthroughs emerged. Confronted with these limitations, they explicitly pointed to dropout as a promising future avenue.

But traditional dropout techniques did not work well with RNNs. They disrupted the sensitive recurrent connections essential for maintaining memory capabilities. This amplified noise within recurrent dynamics [35][36]. Consequently, regularization methods at the time delivered only modest improvements [37][38]. As a result, RNNs remained shallow, with one or two recurrent layers and relatively small parameter counts. Even the most influential RNN of that period achieved its best perplexity of 114 with only a single hidden layer [39]. The boldest attempts at the time employed two-layer networks, achieving a modest improvement in perplexity to 107. But beyond two layers, researchers weren't merely encountering diminishing returns; they hit an impenetrable wall [40].

Faced with this barrier, researchers turned to ensembles and hybrid approaches to achieve incremental gains. For example, the state of the art in 2012 combined three distinct methods: a Kneser-Ney 5-gram language

Licensed to Hermilo Cortés <hermilocg@gmail.com>

model that captured short, frequent word sequences; a cache-based language model that boosted the probabilities of recently occurring words; and an RNN enriched with semantic topic vectors derived from Latent Dirichlet Allocation (LDA), providing broader contextual understanding.

Each method failed to get perplexity below 100 on the PTB. Kneser-Ney alone yielded a perplexity of 141, the cache-enhanced variant 125, and the RNN-LDA combination 114—about 10 points better than the cache-enhanced Kneser-Ney variant. Yet, by carefully combining their predictions and tuning each component's influence to balance their strengths, an ensemble reduced perplexity to 92. Later, the same authors extended this approach by incorporating multiple variants of neural topic models alongside traditional ones, ultimately achieving a perplexity of 72.9. While clever, this kludgy hybrid strategy fell short of the elegant promise of end-to-end learning popularized by AlexNet.

Though these complex combinations were not practically scalable, they did reduce perplexity, underscoring a troubling reality: no single RNN architecture, not even the more robust LSTM, could break the 100-perplexity barrier on its own. In fact, before 2014, single-model LSTMs routinely struggled on benchmarks such as PTB. An early LSTM study from 2012 reported only about an 8% reduction in perplexity relative to standard RNNs, underscoring their limited gains [41]. Zaremba, Sutskever, and Vinyals (ZSV) noted that even "relatively small LSTMs (in terms of number of parameters) can easily overfit the training set." Without effective regularization, adding more hidden units or layers would reduce training perplexity but quickly raise validation perplexity because of overfitting. This inability to scale meant that single-model PTB perplexity scores rarely dipped below 120 in the pre-dropout era, making further progress seem out of reach.

This "glue-it-together" strategy reflected the state of computer vision in the pre-AlexNet era, when all high-performing vision systems lacked architectures that could scale or support end-to-end learning. These pre-AlexNet solutions relied on meticulously engineered representations stitched together from hand-crafted feature pipelines using local descriptors to detect and describe image patches, histograms of quantized descriptors, and spatial pyramids to encode global representations. These were then paired with shallow models like support vector machines, or with layer-wise unsupervised pretraining followed by supervised fine-tuning. By carefully integrating complementary components, computer vision researchers pushed past performance plateaus, but this approach was later replaced by unified architectures trained end to end, such as DanNet and

Licensed to Hermilo Cortés <hermilocg@gmail.com>

AlexNet. Likewise, early language models were mosaics of specialized components.

The key insight from ZSV was to apply selective dropout exclusively to the nonrecurrent connections between layers, never to the horizontal recurrent connections that preserve memory over time. This way, recurrent pathways preserve historical information without disruption, so dropout enhances robustness without compromising stability. This selective strategy provides effective regularization, encouraging neurons to form stable representations while ensuring that recurrent connections remain intact to retain and propagate historical information across time steps. By introducing selective dropout regularization for LSTMs, they slashed PTB perplexities. A single "large" LSTM (1,500 units, two layers) with dropout achieved a test perplexity of 78.4, shattering previous records and outperforming the computationally heavy 10-model ensemble of RNNs that managed a perplexity of 80. This was a dramatic leap.

The authors demonstrated the power of their specialized dropout technique with four diverse, real-world applications:

- *Language modeling* (PTB)—The authors tested medium-sized LSTM networks with 650 units per layer and a 50% dropout rate, and large networks with 1,500 units per layer and a 65% dropout rate. The results were significant. Dropout reduced test perplexity from a baseline of 114.5 to 78.4. Dropout enabled larger, previously impractical LSTMs to train effectively, echoing AlexNet's breakthrough. In addition, combining 10 large dropout-regularized models reduced perplexity to 69.5, underscoring dropout's scalability.
- *Speech recognition* (Icelandic Speech dataset)—Limited datasets often cause overfitting in acoustic modeling, leading to poor generalization. Dropout directly addressed this problem. While standard LSTM models reached 68.9% accuracy, dropout-regularized versions improved to 70.5%. Although training accuracy decreased slightly because of dropout-induced noise, generalization improved substantially, reflecting the core insight originally shown by AlexNet.
- *Machine translation* (English-to-French WMT dataset)—Regularized LSTMs improved neural machine translation, boosting the BLEU score from 25.9 to 29.03. BLEU scores evaluate how closely machine-generated translations resemble human-written translations; higher scores indicate better translation quality. Although traditional statistical methods remained competitive at the time, this improvement highlighted dropout's potential for neural translation, laying a foundation for later breakthroughs such as the transformer architecture,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

which would soon revolutionize machine translation and natural language processing more broadly.

- *Image caption generation* (MSCOCO dataset)—In the image-captioning experiment, dropout improved the single-model BLEU score to 24.3, nearly identical to the 24.4 score obtained by an ensemble of multiple non-regularized models. This result demonstrated dropout's efficiency advantage: a single, well-regularized model achieved performance comparable to an ensemble while substantially reducing computational complexity.

"Recurrent Neural Network Regularization" is brief, with just six pages and fewer than 3,000 words. Yet the paper showed that the performance barrier could be overcome when perplexity scores that had barely changed suddenly dropped. By addressing regularization challenges, ZSV expanded the practical use of deep learning, influencing subsequent research in speech recognition, machine translation, and sequence modeling.

4.4 Deep Speech 2

While convolutional neural networks had their defining moment with AlexNet, recurrent networks experienced a more incremental ascent, marked by steady advances across several fronts. Several important papers came close to capturing that moment. Sutskever, Vinyals, and Quoc Le introduced the encoder-decoder architecture (seq2seq) in 2014 [42]. In that same year, Bahdanau attention established new performance benchmarks in machine translation [43]. Both are discussed in the next chapter. While these advances were transformative and significant for the field, they lacked the stark clarity of the AlexNet moment. Baidu's DS2, led by Dario Amodei (a key figure at OpenAI and the cofounder of Anthropic), provided the closest analogy. Like AlexNet, DS2 convincingly demonstrated that large-scale, GPU-powered, end-to-end trained neural architectures could outperform traditional hybrid methods.

4.4.1 Core architecture

DS2 converts audio to text without phonetic dictionaries, which provide a word's pronunciation using a phonetic transcription. These dictionaries are usually handcrafted using linguistic rules. Because pronunciation can vary across languages, accents, and domains, maintaining phonetic dictionaries was labor-intensive and error-prone. DS2 eliminated the need for them by learning to map directly from audio features to letters or characters (that is, graphemes). The model effectively internalizes the pronunciation rules that older systems had to encode explicitly. Sound familiar?

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Traditional systems also relied on frame-level alignments to mark the start and end of each word. DS2 replaces this with a training method known as *Connectionist Temporal Classification* (CTC), which eliminates the need for such precise alignments. Alex Graves introduced CTC in 2006, specifically to let RNNs align sequences without presegmented labels [44]. By 2015, CTC had been proven on smaller tasks, but DS2 applied it at an unprecedented scale.

Under the hood, DS2 starts by converting each audio clip into a visual spectrogram that shows how sound energy changes over time. Convolutional layers scan this image for short, local features, such as subtle shifts in tone or background noise. The results then pass through several recurrent layers that track how sounds unfold over time. Finally, a fully connected layer predicts letter pairs (bigrams) for English or characters for Mandarin, turning speech directly into text. See figure 4.5.

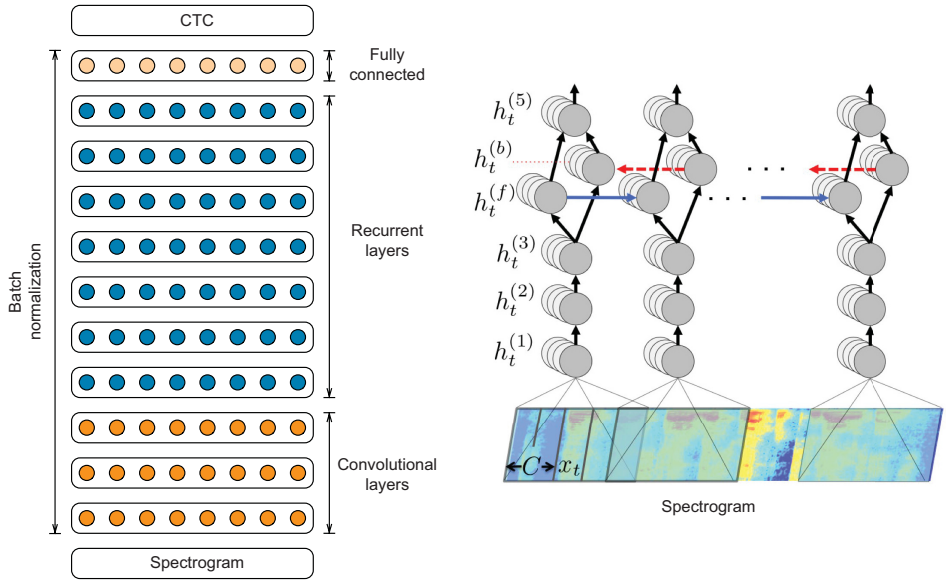


Figure 4.5 Left: DS2 architecture used to train on English and Mandarin speech. The authors vary the number of convolutional layers from 1 to 3 and the number of recurrent layers from 1 to 7. Right: structure of the RNN model from the first Deep Speech paper [45]. *Used with permission granted by one of the authors (Awni Hannun).*

Importantly, DS2 introduced row convolution and stride-aware outputs. Traditional RNNs process sequences left to right, so each prediction uses only past inputs. Bidirectional RNNs run forward and backward to use

Licensed to Hermilo Cortés <hermilocg@gmail.com>

both past and future context before making a prediction, which boosts accuracy but prevents real-time decoding. DS2's row convolution adds a narrow temporal convolution on top of the unidirectional RNN outputs. This approximates the benefits of bidirectionality while maintaining low latency for streaming.

Like row convolution, striding is a convolutional operation. It is applied to the early convolutional layers to downsample the data so the recurrent stack sees fewer steps, reduced by the stride factor. Unlike dilated convolutions—which expand the receptive field without reducing resolution—striding explicitly downsamples the signal, trading temporal detail for computational efficiency. To preserve information at the original output rate, DS2's English system predicts nonoverlapping character bigrams instead of single letters. For example, instead of outputting "t" and "h" separately, a single output token could be the bigram "th." Meanwhile, Mandarin predicts characters.

Striding with bigrams reduces computational complexity and memory usage by halving the sequence length and effectively compressing the input. Without this innovation, the 11-layer network might have been too slow to train or run. For example, with a stride of 2, an utterance of 100 frames yields 50 RNN steps. Predicting 50 characters would risk losing information, but predicting 50 bigrams can still cover 100 characters. By contrast, row convolution was primarily a deployment optimization. It provided moderate improvements in accuracy for real-time, unidirectional streaming models.

DS2 also introduced *sequence-wise batch normalization*. Standard batch normalization, introduced in 2015, normalizes layer inputs during training by computing the mean and variance of activations within each mini-batch to ensure consistent activation distributions [46]. A mini-batch is a subset of the training data processed together in one forward and backward pass. This consistency reduces *internal covariate shift*, in which the distribution of inputs to a layer changes continuously throughout training. By stabilizing input distributions at each layer, batch normalization enables higher learning rates, reduces sensitivity to weight initialization, and accelerates convergence. DS2's sequence-wise adaptation applied this concept specifically along the temporal dimension, stabilizing recurrent network training and improving generalization.

4.4.2 Training techniques

DS2 scaled training through data parallelism across multiple GPUs. Each training data batch was divided evenly so that every GPU processed an

Licensed to Hermilo Cortés <hermilocg@gmail.com>

equal share of examples in parallel. After each iteration, gradients were averaged into a single synchronized update. Typical runs used 8 to 16 GPUs with total batch sizes of 512 to 1024 samples (64 per GPU).

The team engineered a custom *all-reduce* routine (chapter 6) to replace standard messaging protocols, enabling direct GPU-to-GPU transfers and keeping data-transfer channels between GPUs busy. This delivered a 2.5× faster training run for one complete pass through the entire training dataset (that is, a full epoch) than the standard, off-the-shelf OpenMPI setup. When the GPU count doubled, the training time roughly halved, though diminishing returns set in beyond eight devices as communication overhead increased. As with AlexNet's custom GPU kernels three years earlier, DS2's engineering advances, beyond its architecture alone, made end-to-end speech recognition at this scale possible.

Another significant improvement was the shift to half-precision (FP16) arithmetic for inference. FP16 uses 16 bits (half-precision floating point) instead of the conventional 32 bits (single-precision floating point). This change effectively halved the memory footprint and roughly doubled inference throughput on compatible hardware without significantly compromising accuracy. DS2's shift to FP16 was an early and influential example of reduced precision—a precursor to the broader low-precision techniques now often grouped with *quantization*. Quantization involves representing network weights, activations, and computations with fewer bits.

Quantization is commonplace today, often dropping to 8-bit integers (INT8) and sometimes even to lower-precision formats, especially for edge and mobile deployments that often lack the resources of more powerful machines. Still, DS2's adoption of FP16 showcased the viability and benefits of quantization techniques in practical, large-scale implementations, making it a milestone in the development of model optimization strategies.

Another improvement was the introduction of a curriculum learning strategy called *SortaGrad*. With SortaGrad, training sequences were sorted by length during the first epoch, enabling the network to progressively tackle more challenging examples over time. By presenting shorter (typically easier) utterances first, SortaGrad ensured more stable and efficient training in the early stages, reducing numerical instability and helping the model converge reliably from the outset. Sequence-wise batch normalization and SortaGrad enabled DS2 to train deeper networks and achieve substantially improved transcription accuracy and training stability.

DS2 incorporates synthetic background noise into about 40% of its training examples, improving robustness and generalization to real-world

Licensed to Hermilo Cortés <hermilocg@gmail.com>

noisy environments, much like AlexNet's use of data augmentation (for example, random crops, flips, and color jitter) improves invariance and prevents overfitting. In both cases, engineered noise serves as a proxy for diversity, forcing the network to learn the signal rather than memorize artifacts.

DS2's impressive results came at the cost of enormous data and computational resources. The authors trained DS2 on approximately 11,940 hours of English audio and 9,400 hours of Mandarin audio. Training a single model required specialized high-performance computing infrastructure, including 8 or 16 Titan X GPUs with 12 GB of memory. This brute-force scaling was a double-edged sword. It proved what was achievable with end-to-end deep learning, but also set a high barrier to entry for others trying to replicate or build on the work without similar resources.

The results were impressive. The authors discovered that each 10-fold increase in training data volume led to roughly a 40% relative reduction in WER. These results confirmed DS2's status as an RNN analog to AlexNet, demonstrating significant leaps forward through end-to-end learning, large-scale training, and high-performance computational techniques.

DS2's performance on standard benchmarks was notable, achieving word error rates lower than a crowdsourced (Mechanical Turk) human baseline on clean speech datasets such as WSJ and LibriSpeech. In fact, DS2 outperformed crowdsourced human transcribers on three out of four test sets (see table 4.1). Still, DS2 did *not* surpass human performance under more challenging conditions, including noisy environments (CHiME) and heavily accented (VoxForge) speech.

Table 4.1 WER for DS2 and human-level performance on read speech. CHiME data were collected in noisy environments, including a bus, a café, a street, and a pedestrian area.

Test set	DS2	Human
WSJ eval'92	3.60	5.03
WSJ eval'93	4.98	8.08
LibriSpeech test-clean	5.33	5.83
LibriSpeech test-other	13.25	12.69
VoxForge (American-Canadian)	7.55	4.85
CHiME	21.79	11.84

4.4.3 Language models and decoding

With as many as 100 million parameters, DS2 was a poor fit for on-device use in 2015. Consequently, it was deployed as an interactive cloud service on GPU servers to handle user queries. Still, a series of carefully engineered

Licensed to Hermilo Cortés <hermilocg@gmail.com>

optimizations aimed at real-time inference and computational efficiency made deployment more practical.

Batch dispatch is a technique that dynamically groups concurrent user audio streams into optimized batches. Even small batches of two to four reduced memory-bandwidth overhead by reusing loaded weights. By intelligently batching inputs, DS2 reduced the memory-bandwidth overhead of processing individual requests separately and often achieved latencies of 50 to 70 milliseconds, even under moderate system load.

Finally, the team used beam search decoding with heuristic pruning to turn a stream of model scores into a single transcript. Decoding strategies like beam search efficiently find high-quality output sequences from probabilistic models without evaluating every possible sequence, which is often computationally infeasible.

A naïve strategy would be greedy decoding, where the model picks the single most likely transcribed word. It's fast, but it limits your options. Instead of keeping the most likely option, beam search keeps a shortlist of promising candidates. With each new chunk of audio, it extends every candidate and scores them using both the acoustic model and a language model that prefers real words and plausible phrases. It then keeps only the best few and discards the rest. At the end, it selects the highest-scoring complete sentence.

DS2 makes beam search practical by using heuristic pruning to eliminate dead ends early. The authors retained up to 500 of the highest-scoring partial transcripts after each expansion for the English decoder and 200 for the Mandarin decoder. They drop long-shot candidates whose scores fall far behind the leaders. This trimming matters because most of the probability mass sits in a small number of candidates; exploring the rest adds cost without improving performance. It's especially helpful for Mandarin, where many characters share a sound, making the search space large. Beam search is far better than greedy search at resolving ambiguities and far cheaper than exhaustive search, and it's easy to tune by adjusting the beam width and pruning rules.

4.4.4 Significance and broader influence

DS2 represented a breakthrough in speech recognition. Yet DS2's claim of surpassing human-level accuracy was controversial. Critics noted that human-level benchmarks often relied on Amazon Mechanical Turk transcription rather than expert transcription, and individual transcripts were inadequate without additional correction [47]. This raised questions about whether DS2 matched human experts in any transcription settings.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Nevertheless, DS2 demonstrated remarkable versatility in English and Mandarin with only minimal modifications.

Notably, DS2 accelerated training by up to seven times, turning experiments that previously took weeks into days. While "scaling laws" (chapter 6) wouldn't gain widespread popularity until 2020, the DS2 paper demonstrated an empirical relationship among dataset size, compute, and model performance. Innovations such as sequence-wise batch normalization, SortaGrad curriculum learning, and convolutional front ends with strides, reduced the computational cost per example. DS2 also incorporated FP16 to double throughput and used pruning heuristics to significantly accelerate inference, which is particularly beneficial for languages with extensive character sets, such as Mandarin. Practical deployment strategies, including optimized GPU kernels and the "batch dispatch" system, enabled real-time transcription at median latencies below 70 ms. All this underscores how effectively DS2 married algorithmic innovation with data and raw computational power.

That said, DS2 still relied on an external language model (LM) during beam search decoding. This means the neural network alone didn't *fully* solve speech recognition because it needed an LM to correct phonetic errors and pick the most plausible word sequence. This is a subtle limitation. While DS2 was a major step toward end-to-end automatic speech recognition (ASR), it didn't unify all components. Still, it marked an important step toward a universal automatic speech recognition system capable of generalizing across diverse languages and challenging acoustic environments.

Within a few years of DS2, newer architectures began to outperform it. Transformer-based and transducer models brought significant gains. The Conformer became especially influential by combining convolutional modules with self-attention (see chapter 5) to model local acoustic patterns and long-range dependencies [48]. Conformer-style encoders remain prominent on accuracy-oriented ASR benchmarks, achieving lower error rates, but inference efficiency depends heavily on the decoder and deployment setup—architectures we will discuss in the next chapter.

Likewise, DS2's fixed non-overlapping English bigram outputs were largely superseded by more flexible output units such as graphemes, word-pieces, byte-pair encoding (BPE), and byte-level tokens, which reduce out-of-vocabulary problems and allow models to compose unseen words when character or byte fallback is available [49][50][51]. But DS2 was not a dead end. Its emphasis on end-to-end training, CTC-style sequence learning, large-scale data, convolutional front ends, and deployability evolved into

Licensed to Hermilo Cortés <hermilocg@gmail.com>

standard ASR practice, even as specific techniques such as row convolution and English bigram outputs were replaced by more general architectures and tokenizations.

4.4.5 An engineering shift

In the 2010s, AI research shifted toward large-scale, engineering-intensive systems that emphasized end-to-end learning. DS2 exemplified this direction, marked by meticulous kitchen-sink engineering. Like AlexNet, these projects combined empirical strategies to achieve substantial real-world performance gains rather than relying on a single theoretical breakthrough. AlexNet integrated known techniques—including GPU acceleration, careful weight initialization, ReLU activations, dropout, and data augmentation—to achieve unprecedented results. Similarly, DS2 adopted a pragmatic approach, combining architectural refinements, computational optimizations, curriculum learning, and advanced normalization to improve performance.

Yet, despite impressive performance improvements, the engineering-focused approach of systems such as AlexNet and DS2 came at a cost. These strategies were met with skepticism from the broader scientific community. Critics dismissed such advances as mere "engineering" or achievements driven by "scale" or "brute force," suggesting a retreat from the pursuit of "true artificial intelligence," a phrase whose oxymoronic nature hints at deeper tensions about authenticity and artificiality.

In a retrospective critique, AlexNet was characterized as "largely an evolutionary improvement over LeNet," rather than a fundamentally new design. But this assessment overlooks how AlexNet, *not* LeNet, reshaped the field of computer vision [52]. Likewise, Gary Marcus, a professor at NYU, cautioned that while AlexNet represented "important work with immediate practical applications," it was ultimately "only a small step toward the creation of truly intelligent machines" [53]. Marcus presents this almost dismissively, as though empirical progress were a lesser form of innovation. But progress was precisely what the field had been lacking. For example, the authors of AlexNet (Krizhevsky, Sutskever, and Hinton) showed how to use the technology when the field had stalled. The transition from theoretical promise to tangible capability required the empirical pragmatism that many critics undervalued.

While Marcus is right that AlexNet didn't solve intelligence, he's using the wrong benchmark. Judged against the appropriate baseline of contemporary vision systems and benchmarks, it reset the field. Calling it a "small step" becomes a criticism only when we confuse the destination with the race we're actually running.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

These accounts illustrate how critics often measure progress against an idealized future, such as "truly intelligent machines." While we often seek solutions ahead of their time, this notion is almost always incoherent because the future is not a source of knowledge we can validate. Consequently, we cannot measure progress against the future; we can measure it only against where we began solving a problem. Genuine innovation emerges incrementally through real-world trial and error, highlighting the difference between those with "dirty hands," constrained by real-world limitations of knowledge, and those with "clean hands," who judge progress against abstract, often unreachable standards.

Like AlexNet, DS2 was celebrated as a breakthrough. *MIT Technology Review* named Baidu's system one of 2016's top breakthroughs, noting that it outperformed humans [54]. Within the speech research community, researchers were excited that a simpler architecture could replace entire handcrafted pipelines and still outperform state-of-the-art systems. But academic skeptics viewed DS2 as an engineering tour de force rather than a source of insight.

Peer reviews of the DS2 paper at the 2016 International Conference on Machine Learning (ICML) criticized the work. "My main issue with the paper is that a lot of it draws on existing work … and provides very little in terms of novelty," one reviewer wrote, noting that "most of the techniques have all been tried before." Another reviewer agreed that DS2 introduced no fundamentally new methods, remarking: "The novelty of the paper is through scaling up and implementation rather than any methodological advance" [55].

This engineering-focused approach contrasts with the theoretical research exemplified by foundational papers such as Martens' Hessian-free optimization, Rumelhart's backpropagation, or Hochreiter and Schmidhuber's LSTM. These contributions prioritize conceptual clarity, mathematical elegance, and novelty. DS2 and AlexNet, by comparison, emphasize solving tangible, large-scale engineering challenges with measurable results. One could almost be forgiven for mistaking the goal of research for making something useful. Ultimately, the tension here is unmistakable: theoretical novelty versus practical efficacy. A preference for one approach over another reveals fundamental differences in the researcher's values and intellectual orientation.

AlphaGo's historic victory over Go champion Lee Sedol in 2016 demonstrated the enormous potential of intensive engineering efforts. Before AlphaGo, most AI advances emerged from small, idea-driven teams or individuals: Martens' Hessian-free optimization had one author;

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Rumelhart's backpropagation had three; Hochreiter's LSTM paper had two. While AlexNet had only three authors, Deep Speech 2 had 34, and AlphaGo had 20. The field was maturing, becoming more collaborative and resource-intensive, with a growing focus on large-scale engineering.

For Ilya Sutskever, AlphaGo crystallized a conviction that had been developing for some time: large-scale engineering would drive the future of AI. This wasn't obvious in the mid-2010s. At Google, Sutskever encountered a culture deeply aligned with academia's preference for conceptual innovation. Although that approach was valuable for generating ideas, Sutskever recognized that conceptual breakthroughs alone would no longer be enough. Increasingly complex models demanded massive datasets, advanced GPU infrastructure, and robust engineering practices beyond the capabilities of small, academically oriented teams. Such groups were inherently resource-constrained, optimized for novelty rather than for sustained, scalable, reliable implementation [56][57].

Google was not the problem. According to Sutskever, academia itself presented deeper structural problems. Academic incentives encouraged researchers to "think small," rewarding incremental advances and superficial novelty rather than practical, scalable results. Asked about the balance between creating entirely new ideas and refining existing ones, Sutskever stressed that conceptual breakthroughs make up only a modest fraction of genuine progress. Real progress comes from deeply understanding existing ideas, iteratively improving and deploying them, and carefully analyzing results, especially when experiments yield surprising outcomes [58]. Recognizing this misalignment, OpenAI was designed to prioritize practical effectiveness and deployability over incremental novelty and academic accolades.

DS2 is a case study in Sutskever's skepticism toward novelty for novelty's sake. In Ilya's view, deployment is a research tool that yields insights otherwise unattainable through purely theoretical or small-scale studies. The mindset values iterative, high-leverage improvements, grinds through engineering details, and lets reality arbitrate. This pragmatic philosophy mirrors John Carmack's vision for Keen Technologies, which deliberately revisits older ideas to uncover opportunities enabled by modern scale or an engineering-driven approach [59].

Baidu had already released Deep Speech 1 (DS1) in 2014 [60]. It was an end-to-end, CTC-trained RNN that simplified the classic automatic speech recognition stack and achieved the then-state-of-the-art WER by pairing large amounts of data with rigorous engineering. A year later, DS2 didn't invent a brand-new paradigm. It won by ruthlessly scaling the same

Licensed to Hermilo Cortés <hermilocg@gmail.com>

core idea: deeper networks, sequence-wise batch normalization, SortaGrad, stride and bigram outputs, aggressive multi-GPU synchronous training, plus production-minded tricks like row convolutions and batch dispatch for low-latency serving. They even open-sourced Warp-CTC so others could train quickly on GPUs, underscoring the "deployment as research" posture [61]. This reinforces DS2's reputation not only as a model but also as a benchmark in deep learning engineering. It showed that scale, engineering, and real-world pressure can convert a good idea (DS1) into a better one (DS2).

As OpenAI's chief scientist, Sutskever steered the lab toward product-oriented research, favoring large-scale experiments over papers. The GPT series exemplifies this philosophy. While Google introduced the Transformer architecture in 2017, OpenAI scaled it into practical, deployable products. For Sutskever, deploying systems used by millions offers greater value than novel ideas confined to academic circles. In his view, genuine progress arises from understanding and improving technologies rather than from perpetually chasing novel concepts that dozens of people will read. This philosophy permeates OpenAI's culture, differentiating it from pure research organizations. OpenAI was founded with the goal of AGI, but under Sutskever's leadership, it became an engineering-first company.

Historically, ambitious AGI-oriented companies failed to balance aspirations with practical deployment. For instance, years before Ben Goertzel edited the book *Artificial General Intelligence* (chapter 9), his company, Intelligenesis, developed Webmind, a product envisioned as a disembodied "global brain" nurtured on the internet. The media declared Webmind "dumb" but with a "brilliant future" [62][63]. Publications such as *The Wall Street Journal* and *The Australian Financial Review* envisioned Webmind ushering in a new era of intelligence that would eclipse humanity's cognitive supremacy [64]. All the while, Intelligenesis was going bankrupt because of its solve-no-problem business model. Goertzel wrote in a business postmortem that the goal of "creating a thinking machine and then commercializing it" should have been "laughed out of any conversation with any serious businessperson" [65][66]. Webmind stands in stark contrast to OpenAI.

Sutskever has relentlessly focused on experimentation, even when the underlying algorithms, such as CNNs, RNNs, or Transformers, were already well established. Ilya's genius lies in taking ideas that looked great on paper and making them work in the real world. This philosophical divergence illuminates the motivation behind Sutskever's List, which implicitly suggests that modern AI is rooted in a small set of seminal ideas,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

while most others add negligible or even negative value by introducing superficial variations that create noise rather than progress. Sutskever's List reinforces this view by prioritizing depth and practicality over trivial, incremental knowledge.

DS2 and selective dropout made recurrence triumphant, and Karpathy and Olah made it legible. Yet even as RNNs were stabilized, regularized, and scaled, they still carried the past forward through a fragile channel. The next chapter begins at that pressure point. Researchers stopped trying to compress everything into a single state and introduced attention, allowing models to look at the entire sequence rather than forcing it through a bottleneck. It was the first step toward a more radical question about whether recurrence was ever needed at all.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

5 Attention is all you need

This chapter covers

- How the Transformer replaced recurrence and convolution with self-attention
- *Q/K/V*, multihead attention, and positional encodings.
- Sequence to sequence, "thought vectors," the reversal trick, and "Order Matters"
- Cultural impact of the Transformer and the "X is All You Need" meme
- The Universal Transformer

Papers

- *Attention is All You Need* (2017) Vaswani et al.
- *The Annotated Transformer* (2018), Alexander Rush
- *Neural Machine Translation by Jointly Learning to Align and Translate* (2014) Bahdanau et al.
- *Pointer Networks* (2015) Vinyals et al.
- *Neural Turing Machines* (2014) Graves et al.
- *Order Matters: Sequence to Sequence for Sets* (2015) Vinyals et al.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In 2017, researchers at Google replaced recurrent and convolutional layers with a new architecture based on *attention*, which they called the Transformer [1]. The authors wrote, "We show that the Transformer outperforms both recurrent and convolutional models on academic English-to-German and English-to-French translation benchmarks" [2]. Beneath this measured academic tone lay the start of a revolution.

The Transformer was not the first model to use attention, but it nearly perfected the mechanism by synthesizing earlier innovations, including Bahdanau attention (2014), Graves attention (2014), and Luong's refinements (2015) [3][4][5]. This fusion was a catalytic moment that led to architectures such as the GPT family of models and, ultimately, ChatGPT.

The paper's title transcended its origins to become a defining catchphrase. Many catchphrases have emerged to define aspects of the modern AI zeitgeist. "Stochastic parrots" critiques language model mimicry and a lack of understanding; the "bitter lesson" underscores the role of scale and the challenges of building solutions based on how we think we think; "too dangerous to release" and "What did Ilya see?" encapsulate the era's fear of the most advanced models; "p(doom)" has become a catch-all term for existential risk; "feel the AGI" has become a tongue-in-cheek way to express both enthusiasm and skepticism about claims of approaching artificial general intelligence; and playful phrases like "vibe coding" capture the shift toward a more conversational approach to software development that some praise for speed and others fault for a lack of rigor. Yet none have captured the era quite like "Attention is all you need." It marks the pivotal moment when neural machine translation, and later neural sequence modeling, adopted a new approach.

5.1 Transformers

A language model is a probability model over text. Take the sentence "The mouse ate the cheese." This is grammatically correct and makes sense, so a good language model assigns it a high probability of appearing in the training data. Consider "The the mouse ate cheese." Here, the repeated "the" breaks grammar, so the model assigns a lower probability. Now consider "The cheese ate the mouse." This is grammatically correct but semantically odd. Cheese doesn't eat mice. A good model reflects that with an even lower probability. These examples show that language models learn both syntax, which includes the rules of grammar, and semantics, which governs meaning.

Large language models (LLMs), such as ChatGPT (OpenAI), Claude (Anthropic), and Gemini (Google), are generative models. That is, once a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

model has learned a distribution over text, it can sample from it to generate entirely new sentences or paragraphs or engage in multiturn conversations.

Most modern LLMs are trained as autoregressive models. They generate text word by word, conditioning each prediction on what came before. The probability of an entire sentence is decomposed into a chain of conditional probabilities: sample the first word, then the second word given the first, and so on. That means the model must juggle both local dependencies, such as recognizing that "the the ..." is unlikely, and long-range ones, such as knowing that "cheese" doesn't usually eat "mice."

The autoregressive objective for LLMs is implemented through self-supervised learning, where the training data provides its own labels by shifting the sequence. A causal mask enforces the constraint that each token can only attend to tokens that precede it, preventing information leakage from the future. Training proceeds by predicting the next token at every position simultaneously, turning a single sequence into many supervised examples.

Of course, this is all easier said than done. Handling syntax, semantics, and context across long sequences requires an architecture that can juggle short- and long-term dependencies. As the Paul Graham generator showed (chapter 4), this is difficult, especially for recurrent neural networks (RNNs) that process text one step at a time. Sequence-to-sequence (seq2seq) models improved tasks like translation by pairing an RNN with an encoder to represent an input sentence and a decoder to generate its translation, but seq2seq still struggled to preserve information across long distances [6][7][8]. The following section explores seq2seq models in greater detail. The Transformer kept the encoder-decoder architecture while removing recurrence, allowing the model to process an entire sequence in parallel and connect each token directly to every other token via a mechanism known as "attention."

Before the encoder or decoder does anything, it must convert text into tokens, then into numbers. A token is not always a whole word. Common words may be single tokens, while rarer words may be split into smaller pieces. For example, a sentence such as "Transformers are powerful" might become something like:

```
[4032, 17, 9821]
```

These integers are only vocabulary IDs; the same token gets the same ID whenever it appears. By themselves, they contain no structure or meaning.

Next comes the embedding layer, which is a learned table that maps each token ID to a vector. In the original Transformer, each vector has 512 numbers, so 512 is simply the model's hidden width. Those 512 numbers

Licensed to Hermilo Cortés <hermilocg@gmail.com>

are not hand-coded features. They are learned coordinates in the model's internal representation space. The result is a matrix with one row per token and 512 columns. So when token `4032` is looked up, it is replaced with a 512-dimensional vector such as:

```
[0.12, -0.77, 0.41, …]
```

If the input sentence has multiple tokens (it often does), each one is replaced with its corresponding 512-dimensional vector, where each dimension is a learned feature capturing aspects of meaning or usage. That matrix is the input the encoder receives.

Each encoder layer begins with a self-attention sublayer. Self-attention allows each token to gather information from other tokens in the same sequence. A concrete example makes the mechanism easier to see. Suppose the input is "How are you?" To update the representation of "are," the model compares "are" with "How," "are," and "you" and asks which of them matters most for interpreting "are" in context. If the resulting weights are 0.2 for "How," 0.1 for "are," and 0.7 for "you," then the new representation of "are" becomes a weighted mixture of those three token representations, dominated by "you." In other words, self-attention turns each token into a context-sensitive summary of the sequence.

Formally, the model computes these weights using three learned projections of each token representation: a query (Q), a key (K), and a value (V). The query says what the current token is looking for, the key says what each token offers, and the value carries the information that will actually be passed along. The standard formula is as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Here K^T means the transpose of K, which simply arranges the keys so they can be compared with the queries, and d_k is the size of the key vectors. The scores are then scaled ($\sqrt{d_k}$) to ensure they remain within a stable range and then passed through a softmax, where they become weights that sum to 1, which are then used to average the value vectors.

After attention, each token passes through a feed-forward network (FFN), which transforms it separately. See figure 5.1. In the original Transformer, the FFN consists of two linear transformations, with a ReLU activation in between. Attention mixes information across tokens, and the feed-forward network then reshapes the mixed representation at each position. For example, a 512-dimensional vector is expanded to 2048 dimensions by a linear layer, passes through a ReLU activation to add a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

nonlinear component, and is then projected back to 512 dimensions by a second linear layer. Residual connections and layer normalization stabilize the entire stack, allowing the model to repeat this pattern across many layers. The role of the feed-forward layer is to transform attended representations nonlinearly.

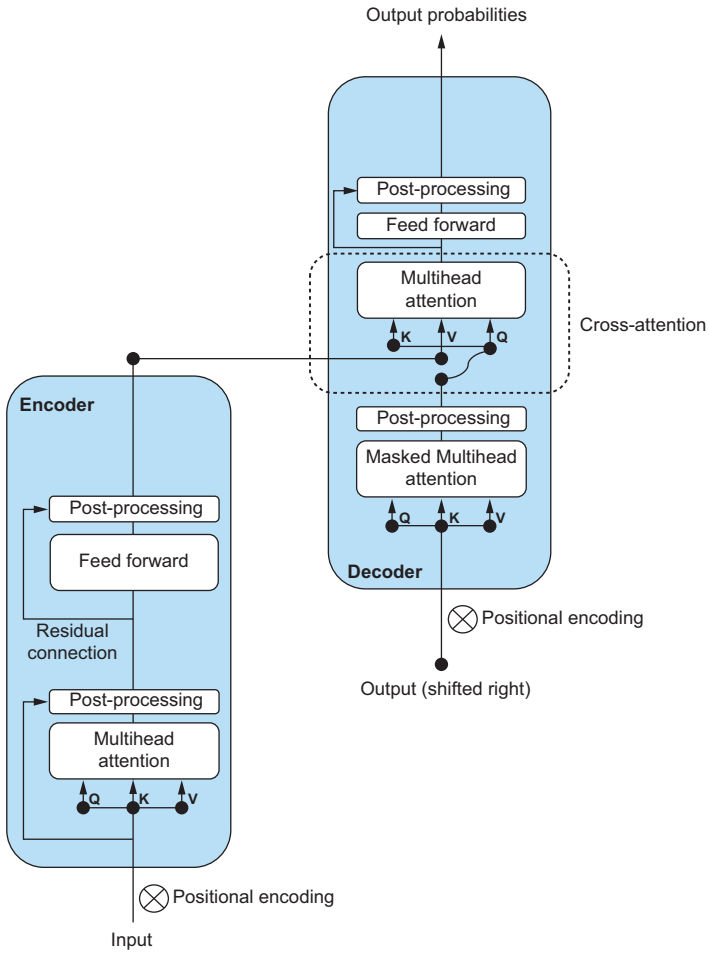


Figure 5.1 A simplified schematic of the Transformer architecture. The model consists of an encoder (left) and a decoder (right), each comprising repeated stacks of sublayers, including multihead attention mechanisms, feed-forward neural networks, and postprocessing steps such as residual connections and layer normalization. Positional encodings (depicted as circled crosses) are added to the input embeddings to explicitly represent token order. The diagram emphasizes the cross-attention operation (dashed box labeled "Cross-attention"), in which the decoder's queries (*Q*) interact with the encoder's keys (*K*) and values (*V*). During cross-attention, each decoding step computes attention scores to determine the relevance of encoded input representations, producing a context vector that guides token generation. This selective alignment enables the decoder to integrate contextual information from the encoder, improving the accuracy and coherence of the output sequence.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The decoder resembles the encoder but is built for generation. Each decoder layer has three main parts: masked self-attention, encoder-decoder attention, and a feed-forward network. As in the encoder, each of these sublayers is followed by a residual addition and layer normalization.

Masked self-attention works like ordinary self-attention except that a token may attend only to earlier output positions, never to future ones. This is what makes next-token prediction possible. When generating the third token, the model can use the first two but not the fourth. This prevents the decoder from peeking at upcoming output tokens it has not yet generated. This is enforced by replacing attention scores for future positions with negative infinity ($-\infty$), which is set to 0 after the softmax. Additionally, a padding mask (`<PAD>`) is added. Sequences in a batch are often padded to the same length, and the mask prevents the model from attending to those blank `<PAD>` positions as if they were real text. Aside from the masking and padding, this sublayer operates in the same way as the encoder's multihead self-attention, enabling the decoder to build context from the partial output sequence.

The second decoder attention block is encoder-decoder attention, also known as cross-attention. In contrast to self-attention, where Q, K, and V all come from the same sequence, cross-attention uses Q from the decoder and K and V from the encoder, allowing the model to align the generated text with the input sequence. A concrete example of translation makes the mechanism easier to picture. Suppose the source sentence is the French "Je vais à l'ecole," and the decoder has already produced "I am going to." When it is about to generate "school", its current decoder state forms a query (Q) that can attend strongly to the encoder representation of "ecole." Unlike older seq2seq systems that compressed the entire source sentence into a single fixed-length vector, cross-attention lets the decoder consult any encoder token at any generation step. See figure 5.1.

After cross-attention, the decoder applies another feed-forward network. A final linear layer maps each decoder vector to a score for every token in the vocabulary, and a softmax converts those scores into probabilities. The model can then choose the next token, append it to the output, and repeat the process.

In the original paper, the architecture had both an encoder and a decoder because the main task was machine translation. Many modern LLMs use decoder-only descendants, but the central idea is the same: attention builds context-sensitive representations that make next-token prediction useful.

That is why reducing Transformers to "just next-token predictors" misses the point. Yes, the objective is next-token prediction, but the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

machinery that accomplishes it extends far beyond the surface prediction used in simpler models. The key design choices include attention, embeddings, positional information (discussed shortly), and repeated nonlinear transformations. Together, they let the model preserve much more context than earlier systems (e.g., n-gram models and RNNs) while remaining highly parallelizable to train. That combination is what made the Transformer such a decisive break from what came before. Ultimately, the "just" framing flattens a layered, subtle architecture into a caricature. In fact, "just" is always a tell. Skeptics use it to dismiss models: "they are just next-token predictors." Enthusiasts use it to flatten the human side of the analogy: "we (humans) are just a bundle of neurons" or "just search at scale." Both commit the same sin: reductionism, as if naming a mechanism were the same as explaining it.

The authors introduced these ideas at two scales: Transformer-base (65 million parameters) and Transformer-big (213 million parameters). When scaling from "base" to "big," the authors doubled both the model's width and its feed-forward capacity and increased the number of attention heads while keeping each head's dimensionality fixed. This modular design enabled the Transformer to scale easily while expanding its representational capacity. Just as ResNet showed that stacking identical blocks can outperform more elaborate architectures, the Transformer's repeated modules simplify scaling, embody Ilya Sutskever's philosophy of doing more with less, and favor standardized, scalable designs over complexity.

One of the Transformer's quiet revolutions is how it handles sequence order. With no recurrence to march step by step through a sentence and no convolution to sweep over it, the model otherwise can't distinguish between "cat fat" and "fat cat." To solve this, the authors added positional encodings to word embeddings before applying self-attention. Using sinusoidal functions of varying frequencies, the model can infer both absolute positions and relative distances. This gives the Transformer a lightweight way to encode word order while still letting attention span the entire sequence in parallel [9].

Subsequent architectures have refined this approach. Google's BERT (2018) replaced fixed sinusoidal positional encodings with learnable positional embeddings, allowing the model to optimize its positional representations during training [10]. GPT-2 (2019) and GPT-3 (2020) both used learned absolute positional embeddings. T5 (2019) extended this idea with learned relative embeddings [11]. More recently, LLaMA (2023) used rotary positional embeddings (RoPE) to inject relative position directly into the attention mechanism [12]. Collectively, these innovations

Licensed to Hermilo Cortés <hermilocg@gmail.com>

highlight the steady shift from fixed, absolute encodings toward relative, scalable, and differentiable schemes.

Training details rounded out the paper. The authors used the Adam optimizer with a specific learning-rate schedule. The rate increased linearly for the first 4,000 warmup steps and then decayed in proportion to the inverse square root of the step number. The base model was trained for 100,000 steps, whereas the "big" model was trained for 300,000 steps. Regularization was achieved through dropout (applied both within sublayers and to the sum of embeddings and positional encodings) and through label smoothing (0.1), which reduced overconfidence and improved BLEU scores. BLEU scores assess how closely machine-generated translations resemble human-written translations; higher scores indicate better translation quality. Training was conducted on a single machine with 8 NVIDIA P100 GPUs, using very large batches of about 25,000 source and target tokens per batch.

5.1.1 Empirical results

The Transformer established a new state of the art in machine translation. It achieved a BLEU score of 41.0 on the WMT English-to-French benchmark [13]. The result surpassed all previously published single models, including Google Brain's 137-billion-parameter sparsely gated mixture-of-experts model, aptly described in a paper titled "Outrageously Large Neural Networks" [14].

The result also marked a dramatic improvement over the selective dropout LSTM (ZSV) model discussed in chapter 4, which had achieved a BLEU score of 29.03 on the same WMT English-to-French benchmark. On the more challenging WMT English-to-German benchmark, the "big" Transformer achieved a BLEU score of 28.4, outperforming all previous approaches, including ensembles of deep recurrent networks.

The step change wasn't confined to BLEU. The same selective dropout LSTM from chapter 4 reduced the Penn Treebank test perplexity to 78.4 with a single two-layer, 1,500-unit model, outperforming a 10-model RNN (nonregularized) ensemble (80.0). While impressive, attention with a longer context pushed a different frontier. In 2019, Transformer-XL reported a perplexity of ≈18.3 on WikiText-103, a separate, harder corpus, underscoring how the architectural pivot and scale reset expectations for next-token uncertainty [15].

The Transformer was also faster to train. The "base" model required 12 hours on 8 GPUs, while the "big" model took about 3.5 days to achieve state-of-the-art results. Unlike recurrent models, which process sequences

Licensed to Hermilo Cortés <hermilocg@gmail.com>

token by token, self-attention directly connects every token to every other token at each layer. This reduces the minimum computational path length required to relate two tokens. In practical terms, distance no longer increases computational cost, and "cheese" can attend directly to "mice," regardless of how many words separate them. This not only improves the modeling of long-range dependencies but also accelerates training. Compared to convolutional models, which require many stacked layers to connect distant positions, the Transformer achieves the same in a single self-attention operation.

The authors also tested the model's ability to generalize beyond translation. They trained a smaller, four-layer Transformer on the Penn Treebank's *Wall Street Journal* corpus, which comprises about 40,000 sentences, for English constituency parsing. With no task-specific architecture changes, it achieved an F1 score of 91.3, surpassing prior RNN-based seq2seq models. This suggested that the inductive bias embedded in recurrence was not as fundamental as once thought. In fact, recurrence might even be a liability, adding unnecessary complexity, preventing parallelism, and limiting long-range dependency modeling.

5.1.2 Universal transformer

The philosopher of science Max Planck famously observed that scientific paradigms shift slowly, not "by convincing its opponents and making them see the light, but rather because its opponents eventually die …." This is often paraphrased as "Science progresses one funeral at a time" [16]. But AlexNet sparked an immediate and decisive response. Within a year, virtually every participant (23 of 26) in the ILSVRC adopted convolutional neural networks, signaling a wholesale transformation of computer vision. While the field resisted the theoretical promise and early anecdotal success of artificial neural networks, it responded immediately to empirical results. The paradigm shift didn't wait for opponents to die.

Transformers did not produce the same dramatic upheaval. In fact, while the paper was accepted to NeurIPS in 2017, it was not given a full oral presentation. Instead, it was relegated to a poster session and a short "spotlight" presentation. Although it reshaped natural language processing within just a few years (still rapid by historical standards), the transition felt almost gradual compared with the impact of AlexNet. One reason is that, despite the availability of an open source codebase, Transformers proved challenging to implement, a difficulty we will explore in more detail shortly.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Another is that self-attention requires each token to interact simultaneously with every other token to capture relationships across arbitrary distances. If a sentence has 500 tokens, that corresponds to 250,000 pairwise interactions. If it has 1,000 tokens, that jumps to 1 million. Doubling the length quadruples the work. For machine translation, where sequences are relatively short, this quadratic growth was manageable. But as sequence length increases, the cost explodes, making long-context modeling a serious computational hurdle [17].

These concerns tempered the early enthusiasm, but the most persistent objection was conceptual. While some proclaimed, "The king is dead, long live the king!" others argued that Transformers lacked a crucial ingredient: recurrence [18].

The anxiety echoed an earlier moment in computer vision, when connectionist researchers proposed replacing hand-engineered features with feature learning. Now, the Transformer stripped away explicit linguistic biases. Without recurrence, critics argued it was "order-agnostic" and viewed positional encodings as a hacky workaround rather than a genuine replacement [19][20][21][22][23]. Much of this resistance was conceptual, but some researchers tested it by randomly reordering the words in a sentence to determine whether Transformers could detect the disruption [24]. The results confirmed the theoretical suspicion: self-attention struggled to capture positional cues, even when aided by positional embeddings. The twist was that, when trained on machine translation, a task highly sensitive to word order, Transformers outperformed recurrent models in learning positional information.

The contradiction revealed something about theoretical intuitions. They described what the architecture lacked but failed to anticipate how optimization, scale, and task demands could compensate for it. While recurrence provides an elegant theoretical prior for sequence order, it's not required to achieve strong performance on sequence tasks. This emphasis on empirical clarity over theoretical insights aligns closely with Sutskever's research philosophy. Progress often emerges from refining ideas at scale and uncovering what truly works through experimentation, rather than relying on what merely feels correct to the researcher. It's a reminder that in deep learning, progress belongs to methods that prove themselves in the arena of scale.

This skepticism about abandoning recurrence soon took architectural form at Google, the same organization that had introduced the Transformer only a year earlier. In 2018, Google researchers proposed the Universal Transformer (UT), arguing that the original Transformer "fails to

Licensed to Hermilo Cortés <hermilocg@gmail.com>

generalize in many simple tasks that recurrent models handle with ease" [25]. While self-attention had been shown to replace recurrence, many researchers still found it difficult to let go.

In fairness, the UT did not undo the Transformer's GPU-friendly parallelism. While recurrence introduces the notion of time steps, the UT is depth-wise and parallelizable, unlike sequential computation. See figure 5.2. This depth-wise recurrence is applied to all tokens in parallel for several steps, reusing the same self-attention and feed-forward layers. This process iteratively refines representations while self-attention mixes information across tokens at each step. Under certain conditions, this can give

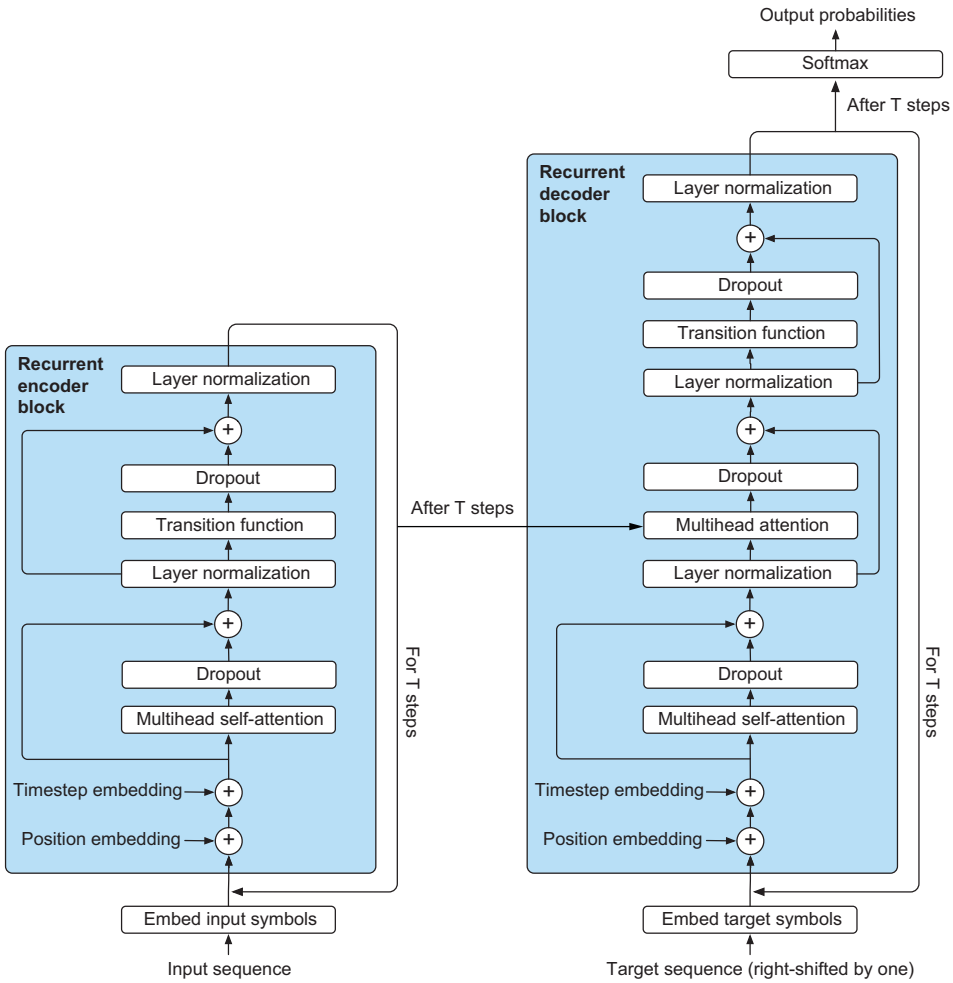


Figure 5.2 The Universal Transformer. ***Used with permission granted by the lead author (Mostafa Dehghani).***

Licensed to Hermilo Cortés <hermilocg@gmail.com>

the UT the theoretical property of Turing completeness—that is, the ability to perform any computation a conventional computer can, given sufficient time and memory.

As a result, the UT is faster than RNNs. Still, it's slower than the original Transformer and offers no clear performance gains. The advantage was purely conceptual, and the supposed weaknesses of the Transformer, such as its lack of recurrence and hacky positional encodings, turned out to matter less once larger models were trained on more data. While UT embodied a clever idea, deep learning favors simpler designs that don't complicate optimization.

Despite introducing the Transformer, Google showed ambivalence about scaling it aggressively. Google's detour, though intellectually intriguing, suggests some hesitation or confusion about what made the standard Transformer a step-change. OpenAI seized upon the scalability and empirical performance with GPT-2 and GPT-3. OpenAI's success highlights what Google initially overlooked: Transformers thrive not through careful inductive biases, or even Turing completeness, but through simple architectures scaled dramatically.

In an interview on the Lex Fridman Podcast, Ilya Sutskever argued, "The Transformer is successful because it is the simultaneous combination of multiple ideas. If you remove any one idea, it would be much less successful." He pointed out that Transformers are parallelizable and it "is not recurrent ... therefore much easier to optimize." This divergence highlights how differences in organizational culture and leadership shaped the trajectory of modern AI development.

This hesitancy exemplifies what Ilya often called out about Google's culture at the time. Google valued a research culture and lacked decisive strategic clarity. By contrast, OpenAI, under Sutskever's leadership, showed no such hesitation. Ilya's immediate reaction after the paper appeared was unequivocal: "We're going to do Transformers now" [26].

5.2 The Annotated Transformer

The original Transformer is relatively straightforward in its theoretical exposition, and Google even made the code open source [27]. In hindsight, that decision proved historically significant, even if its impact was not fully anticipated at the time. Nevertheless, its practical implementation proved challenging. Google's Tensor2Tensor library was deeply embedded in the company's internal tooling and was difficult for newcomers to follow [28]. In this context, "The Annotated Transformer" by Alexander "Sasha" Rush stood out [29]. It challenged the emerging

Licensed to Hermilo Cortés <hermilocg@gmail.com>

conventional wisdom that the Transformer was extremely difficult to implement by presenting a simple open-source reproduction that anyone could follow. Rush named it "Annotated" in homage to *The Annotated Alice*, an annotated edition of *Alice in Wonderland*, underscoring the project's pedagogical aim.

"The Annotated Transformer" is an extensively commented, line-by-line implementation that roughly corresponds to sections of the original Transformer paper. It walks through all the major components, including the encoder and decoder stacks, the multihead self-attention mechanism, feed-forward networks, embedding and positional encoding layers, and the training procedure, which covers batching, masking, and optimization. The blog's mix of code and text allows readers to toggle between conceptual understanding and implementation details. It's also fully functional and optimized, with only 400 lines of PyTorch code capable of processing 27,000 tokens per second on 4 GPUs.

Rush demystified the Transformer at a time when many were eager to understand it. In doing so, it occupied the same cultural and pedagogical space for Transformers as Andrej Karpathy's "The Unreasonable Effectiveness of Recurrent Neural Networks" and Chris Olah's "Understanding LSTM Networks" did for RNNs. All three demystified breakthrough architectures at precisely the moment the deep learning community was ready to adopt them. Its approachable style sparked a wave of similar "annotated" projects, including "The Illustrated Transformer," "The Annotated GPT-2," and even "The Annotated Diffusion Model" [30][31][32]. These projects credit Rush's work as an inspiration for their format and clarity. With more than 6,000 stars and 1,300 GitHub forks, the codebase has been adapted by many practitioners for experiments, teaching demonstrations, and exploration [33][34][35][36].

5.3 Bahdanau attention

To tackle tasks like machine translation, Ilya Sutskever, Oriol Vinyals, and Quoc V. Le introduced the seq2seq architecture in 2014, which used an LSTM-based encoder to transform an input sequence (for example, an English sentence) into a fixed-dimensional internal representation and a decoder to convert that representation into an output sequence (for example, a French translation) [37]. Although the paper is not on Sutskever's List, it set the stage for later breakthroughs, including Transformers.

Its key innovation was the ability to map variable-length input sequences to variable-length outputs by compressing the entire sequence

Licensed to Hermilo Cortés <hermilocg@gmail.com>

into a single high-dimensional "thought vector." See figure 5.3. But it came with a bottleneck. All the syntax and semantics of a sequence—whether 5 words or 50—had to fit into one fixed-length vector. The longer the sentence, the more strained this compression became.

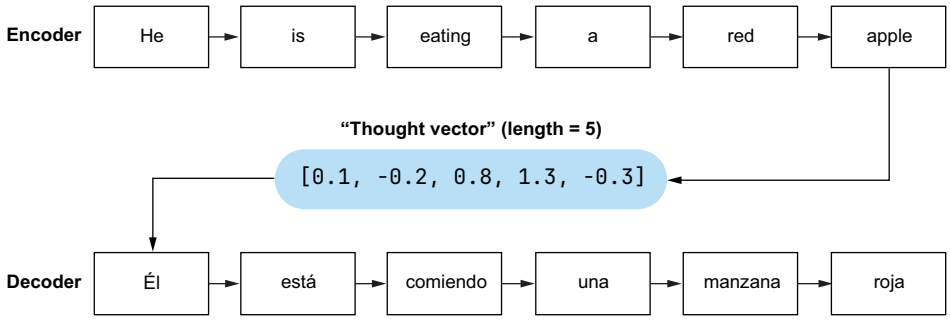


Figure 5.3 The encoder updates a hidden state as it reads each English word. The final hidden state becomes the fixed-length context or "thought vector." The decoder uses this vector, plus the Spanish words it has already generated, to predict a probability distribution over the next Spanish word. During training, the encoder and decoder learn these representations from many paired source–target sentences by minimizing the prediction error on target translations.

The concept behind seq2seq has roots in earlier embedding methods. In 2013, Tomas Mikolov, Ilya Sutskever, and colleagues at Google introduced word2vec, which learns dense vector representations of words from large text corpora so that words used in similar contexts are close together in the embedding space [38]. These embeddings are widely known for exhibiting striking linear regularities in word analogies, which many, including Hinton, saw as a form of reasoning [39].

For example, you might wonder about the Italian equivalent of Paris. Rome immediately comes to mind. Learned word embeddings support this type of inference [40]. If you take the embedding vector for Paris, subtract the vector for France, and add the vector for Italy, you might get the vector for Rome, or at least a vector closer to Rome than to other words that weren't part of the question [41]. See figure 5.4. Traditional natural language processing researchers spent decades trying to define these notions, but distributed representations cut the Gordian knot.

Geoffrey Hinton popularized the term "thought vector," and in a 2015 interview, he proposed that representing "meaning" as thought vectors could enable human-like reasoning in machines [42]. The idea was provocative, though supported by intuition rather than proof [43]. Nevertheless, the phrase quickly became shorthand for neural representations of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

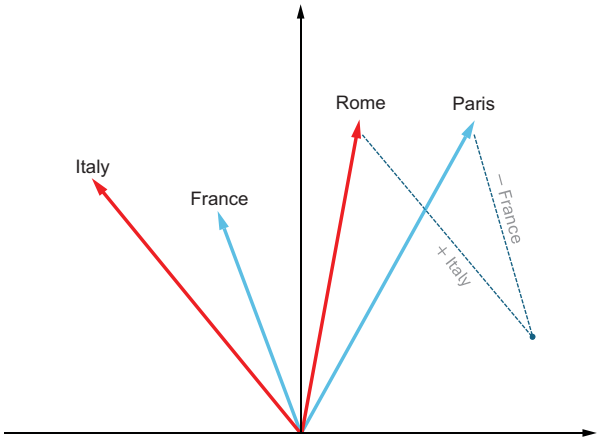


Figure 5.4 Word-embedding vector operations: Paris – France + Italy = Rome

meaning. Ideally, this vector would summarize all essential semantic content. In practice, it often compresses information in ways that lose detail as sequences grow longer and more semantically rich [44][45].

The root cause was that an RNN's final hidden state rarely retained all relevant information from extended sequences. Earlier parts of the sequence were often "forgotten" by the time the encoder reached the end. A single fixed-dimensional vector might suffice for short sentences, but as complexity and length increase, the compression becomes lossy.

In their influential paper "Neural Machine Translation by Jointly Learning to Align and Translate," Dzmitry Bahdanau, Kyunghyun Cho, and Turing Award winner Yoshua Bengio introduced attention, now commonly known as Bahdanau attention [46]. Bahdanau attention enabled the decoder to attend dynamically to different parts of the input sequence during translation, rather than rely on a single thought vector. This innovation earned the paper a place on Sutskever's List and laid the groundwork for the development of self-attention and the Transformer.

Model architecture

The proposed architecture, the RNNsearch model, comprises an encoder and a decoder connected by an attention mechanism:

- *Encoder*—The encoder uses bidirectional gated recurrent units (GRUs). It processes the input sentence in two directions: one GRU reads it forward (left to right), and the other reads it backward (right to left). Their outputs are joined together so that each word's

Licensed to Hermilo Cortés <hermilocg@gmail.com>

representation (an "annotation") carries information from both directions.

- *Bahdanau attention*—The attention mechanism lets the model focus on different parts of the input sentence as it generates the translation. At each step, it scores each source word's relevance, converts those scores into attention weights, and then builds a context vector as a weighted average. This context helps the model decide where to "look" when predicting the next word.
- *Decoder*—The decoder is another GRU with 1,000 hidden units. At each step, it combines its previous state, the last word it produced, and the current context vector from the attention mechanism. It then produces a probability distribution over the next word in the sequence. Guided by attention, the decoder can shift its focus as needed, improving translation of long or complex sentences.

EXPERIMENTAL RESULTS

The authors evaluated their approach using the WMT English-to-French benchmark. They filtered the original dataset down to about 348 million bilingual words to ensure domain relevance. Sentences were limited to 30 or 50 words. Words not included in this selection were replaced with a special "unknown" [UNK] placeholder.

Two baseline models were compared. The first was a basic encoder-decoder model (RNNencdec), which compresses an entire sentence into a single fixed-length vector (that is, a thought vector) before decoding it into a translation. The second was a traditional phrase-based translation system (Moses), which uses large databases of previously translated phrases and a substantial additional language model containing 418 million words.

The proposed attention-based model, RNNsearch, outperformed the basic RNNencdec across all test conditions. When trained on sentences limited to 30 words, RNNsearch-30 achieved a BLEU score of 21.5, compared with RNNencdec's 13.93. When sentence length was extended to 50 words, RNNsearch-50 increased its advantage, scoring 26.75 against RNNencdec's 17.82.

The authors also extended the training of the RNNsearch-50 model. They did not change the sentence length limit or the architecture; instead, they gave it more training time. This extended training increased the BLEU score from 26.75 to 28.45, approaching the 33.3 achieved by Moses. Remarkably, when evaluating only sentences without unknown words, RNNsearch-50 even surpassed Moses, achieving a BLEU score of 36.15, while RNNencdec-50 lagged far behind at 26.71. See table 5.1.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Table 5.1 BLEU scores for RNN encoder-decoder (RNNencdec) and attention-based (RNNsearch) models across different sentence length caps

System	Sentence length cap	BLEU (All)	BLEU (no UNK)
RNNencdec-30	≤30	13.93	24.19
RNNsearch-30	≤30	21.50	31.44
RNNencdec-50	≤50	17.82	26.71
RNNsearch-50	≤50	26.75	34.16
RNNsearch-50 (longer training)	≤50	28.45	36.15
Moses (phrase-based)	—	33.30	35.63

As illustrated in figure 5.5, BLEU scores for RNNencdec declined sharply for sentences longer than about 20 words. By comparison, RNNsearch maintained stable performance even for sentences up to 60 words. This demonstrates that attention effectively addresses the fixed-length bottleneck inherent in the standard encoder-decoder framework.

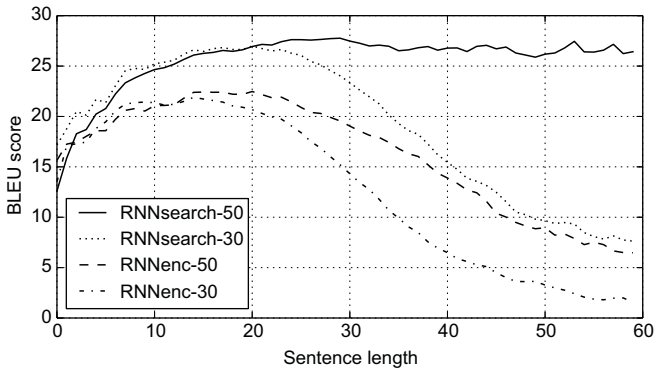


Figure 5.5 BLEU scores versus sentence length: the encoder-decoder scores drop steeply beyond about 20 words, while RNNsearch remains stable up to 60 words. ***Used with permission granted by the lead author (Dzmitry Bahdanau).***

Another major challenge in machine translation is handling word order. English often places adjectives before nouns, as in "red apple," whereas Spanish places many descriptive adjectives after the noun, as in "manzana roja." English phrases can translate into German with significant shifts in word order. "To reach" becomes "zu erreichen" in German, where the verb shifts position. The visual alignments in figure 5.6 reveal mostly monotonic word order between English and French, with prominent

Licensed to Hermilo Cortés <hermilocg@gmail.com>

weights along the diagonal of the matrix. Nonetheless, some notable non-monotonic alignments emerge. For example, the model translates "European Economic Area" to "zone économique européenne," aligning "zone" with "Area" while jumping back over "European" and "Economic" as needed. The soft alignment used by RNNsearch addresses this complexity by enabling simultaneous attention to both "zone" and "Area," resulting in the correct translation.

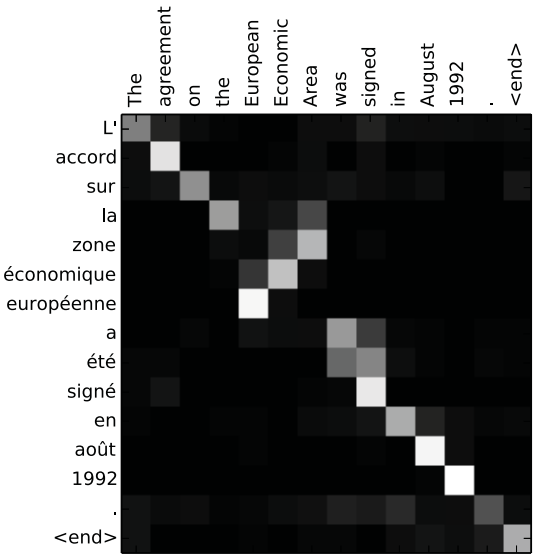


Figure 5.6 illustrates sample alignments generated by the RNNsearch-50 model. The plot's x- and y-axes correspond to words in the source (English) and target (French) sentences, respectively. Pixels indicate the weight assigned by the attention mechanism to each source annotation when predicting each target word, visualized in grayscale (0 = black, 1 = white). Each row shows the weights associated with the annotations, indicating which positions in the source sentence were most important when generating each target word. The alignments are largely monotonic, with strong diagonal weights in each matrix. Still, several non-monotonic alignments also appear. Adjectives and nouns are often ordered differently between French and English. For example, the phrase "European Economic Area" is translated as "zone économique européenne." The model aligns "zone" with "Area," initially skipping over "European" and "Economic," then revisits preceding words to complete the full phrase. *Used with permission granted by the lead author (Dzmitry Bahdanau).*

SIGNIFICANCE

"Neural Machine Translation by Jointly Learning to Align and Translate" introduced the attention mechanism in seq2seq models, allowing a decoder to dynamically focus on different parts of the input sequence rather than being constrained by a single, fixed-size context vector. This

Licensed to Hermilo Cortés <hermilocg@gmail.com>

shift reframed Geoffrey Hinton's idea of "thought vectors" [47]. While meaning could indeed be encoded in numbers, compressing an entire sentence into a single vector proved to be a bottleneck. Attention broke this constraint by allowing the model to decide which parts of the input were most important. Within a year, virtually all neural machine translation research had adopted some form of attention.

By directly addressing the "information bottleneck" of vanilla encoder-decoder RNNs (i.e., seq2seq models), Bahdanau attention laid the groundwork for the Transformer. The Transformer can be seen as the logical conclusion of this trend. Bahdanau attention combines attention with RNNs, whereas the Transformer combines attention with feedforward blocks, with no RNNs. In fact, Bahdanau's "neural alignment" mechanism is a direct precursor to the self-attention mechanism used by Transformers to capture long-range dependencies without recurrence.

Over the past decade, the work has accumulated nearly 40,000 citations and, in 2025, was recognized as a runner-up for the ICLR Test-of-Time Award [48]. The acknowledgment highlighted both the conceptual simplicity and the practical influence of Bahdanau's approach to dynamically attending to different parts of the source sentence. The accomplishment echoes Ilya Sutskever's 2014 remark at NeurIPS about seq2seq: "minimum innovation for maximum results" [49]. Like residual connections, Bahdanau attention was a simple modification with substantial benefits.

5.4 Just point to it!

Machine translation drove the development of specialized architectures, including seq2seq models and attention mechanisms. These advances emerged from the structural demands of translating variable-length sentences and handling semantic reordering. In much the same way, combinatorial optimization problems forced another architectural leap.

Seq2seq works well when the output vocabulary is fixed, as in translation, but fails when output choices must be drawn from the input itself. Pointer Networks (Ptr-Net) addressed this by adapting the Bahdanau attention mechanism [50]. Instead of producing a weighted-average context vector, Ptr-Net interprets attention scores as a categorical distribution over input positions. The model does not generate new symbols; it points to them (see figure 5.7). This allows Ptr-Net to copy, reorder, or dynamically select elements such as cities in the Traveling Salesman Problem, points in a convex hull, or items in scheduling. In doing so, Ptr-Net demonstrated that attention can serve as a selection mechanism, rather than merely a context aggregator.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

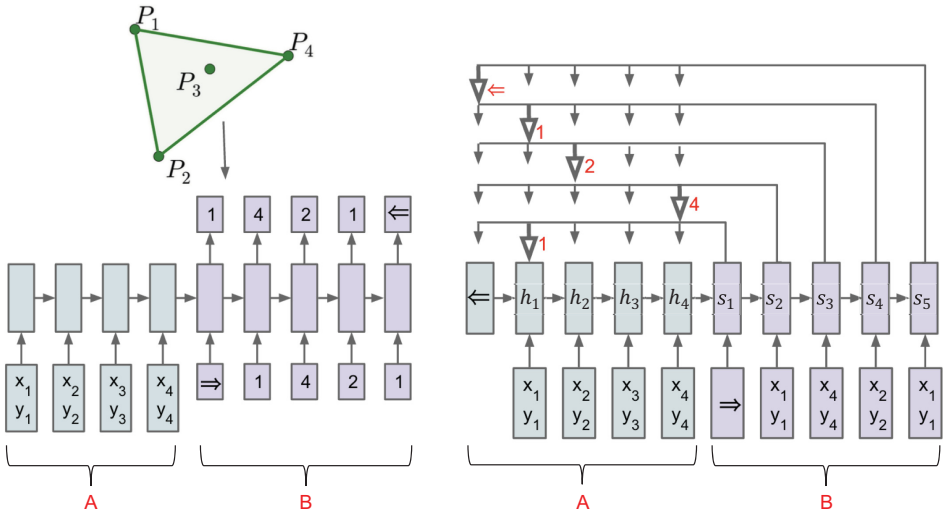


Figure 5.7 A seq2seq model (left). The input sequence (A) creates a vector used to generate the output sequence (B). The output dimensionality is fixed by the problem's dimensionality and remains the same during training and inference [51]. Ptr-Net (right) is also an encoder-decoder. The encoder and decoder hidden states are denoted as (h_1,...,h_n) and (s_1,..., s_n), respectively. Ptr-Net encodes the input sequence into a code (A) that is fed to the generating network (B). At each step, the decoder produces a vector that modulates a content-based attention mechanism over the inputs. The attention mechanism outputs a softmax distribution with a dictionary size equal to the input length. *Used with permission granted by the lead author (Oriol Vinyals).*

5.4.1 Experimental results

The experiments that follow are drawn directly from the Ptr-Net paper and illustrate the impact of this architectural shift.

CONVEX HULLS

In the convex hull experiment, sequence length refers to the number of input points presented to the model. For example, a sequence of length 50 means the network receives 50 two-dimensional points and must identify the subset that forms the convex hull. Sequence length indicates the task's complexity. Longer sequences increase difficulty because the number of possible combinations grows as you add more points.

LSTMs struggled with this task: when given 50 input points, a simple LSTM produced correct hulls in fewer than 2% of trials. Adding attention mechanisms improved performance, raising accuracy to about 39%. That's striking but still far from reliable. The breakthrough came with Ptr-Net, which pushed accuracy to 72.6%. This established Ptr-Net as the first architecture capable of scaling effectively to combinatorial problems of this kind.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

More importantly, Ptr-Net generalized. When trained across a range of lengths (5 to 50 points), the model maintained accuracy even when tested on problems ten times larger than anything seen during training. While exact order accuracy dropped at very large scales ($n = 500$), overall geometric performance remained robust. See table 5.2.

Table 5.2 Performance of LSTM, LSTM with attention, and Ptr-Net on the convex hull problem

Model	Trained	Tested	Accuracy	Area
LSTM	50	50	1.9%	FAIL
LSTM + Attention	50	50	38.9%	99.7%
Ptr-Net	50	50	72.6%	99.9%
Ptr-Net	5–50	100	50.3%	99.9%
Ptr-Net	5–50	500	1.3%	99.2%

DELAUNAY TRIANGULATION

Delaunay triangulation is a geometric method for dividing a set of points into triangles. It's constructed so that no point lies inside the circumcircle of any triangle in the triangulation. Put simply, it creates triangles from a set of points in a way that maximizes the minimum angle, avoiding extremely narrow or sliver-shaped triangles. The Ptr-Net was tested on the task of identifying the correct triangles from a given set of points, essentially predicting how to connect the points according to the rules of Delaunay triangulation, and it learned to approximate the triangulation. As classical computational geometry algorithms solve Delaunay triangulation exactly, performance can be measured directly against a known optimal solution. While the network struggled to reconstruct the entire triangulation at larger sizes, it still correctly predicted over half of the triangles, even with 50 points. This highlighted the model's ability to learn geometric reasoning, though exact accuracy declined sharply as complexity grew:

1. $n = 5$: 80.7% accuracy, 93.0% triangle coverage
2. $n = 10$: 22.6% accuracy, 81.3% triangle coverage
3. $n = 50$: 0% perfect accuracy, but 52.8% triangle coverage

TRAVELING SALESMAN PROBLEM

The final test was the famously challenging Traveling salesman problem (TSP). TSP is a classic computational challenge that involves finding the shortest possible route through a given set of cities (or points), with the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

constraint that the route must start and end at the same city and visit each city exactly once. Although the concept is straightforward, the complexity comes from the sheer number of possible routes, which grows exponentially with each additional city. Because TSP quickly becomes computationally infeasible as the number of cities increases, it is classified as an NP-hard problem, meaning there is no known polynomial-time algorithm that can solve all instances efficiently as they grow large. Consequently, researchers often rely on heuristics that produce good, if not always optimal, solutions.

The Ptr-Net was trained on examples of known optimal solutions for smaller TSP instances. After training, it generalized to larger, unseen instances. It consistently identified tours close to optimal for problems with up to about 30 cities, even outperforming traditional heuristic algorithms that it had implicitly learned from. This result showed that artificial neural networks can learn complex optimization strategies without explicitly encoding them, simply by observing and generalizing from previous solutions.

Table 5.3 shows selected results on the traveling salesman problem. A1 is a greedy heuristic that starts with isolated vertices and iteratively connects the two closest fragments until it forms a single path. The *Christofides algorithm (A3)* is a widely used approximation method for the TSP. It constructs a minimum spanning tree with a minimum-weight matching of odd-degree vertices, guaranteeing a solution within 1.5 times the optimal tour length for instances satisfying the triangle inequality.

Table 5.3 Performance of heuristics (A1 and A3) and Ptr-Net on the TSP

Problem size	Optimal	A1	A3	Ptr-Net
5	2.12	2.18	2.12	2.12
10	2.87	3.07	2.87	2.88
25	N/A	4.71	4.24	4.30
50 (A1 Trained)	N/A	6.46	5.79	6.42
50 (5–20 Trained)	N/A	6.46	5.79	7.66

Remarkably, even when trained on the weakest heuristic (A1), the Ptr-Net learned to outperform its teacher. This suggests that the network was extracting deeper insights from the training data rather than merely mimicking its source. Performance is nearly perfect for smaller problem sizes, but it breaks down at $n = 50$, though it's still far better than chance. In contrast, for the convex hull, the authors were able to generalize by a factor of 10, likely because the TSP is more complex.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Traditional architectures struggle with these types of tasks. The breakthrough behind Ptr-Net was rethinking attention mechanisms. Instead of using attention weights solely to create a "context vector" for predicting words from a fixed vocabulary, Ptr-Net used attention weights directly as a probability distribution over input positions. Here, the attention distribution serves as the final output rather than as an intermediate step. This shift is analogous to the difference between "describing what you see" and "pointing at it." Pointing is more direct. While Ptr-Net did not scale to very large problem instances, it demonstrated that artificial neural networks could learn to approximate solutions to NP-hard combinatorial optimization problems purely from examples. Ptr-Net also foreshadowed Transformers by showing that attention was not merely a tool to help recurrent models, but a general mechanism capable of structuring outputs and representations independently.

5.4.2 Significance

Rather than producing symbols from a fixed vocabulary, the network uses attention weights to point to an element in the input sequence as output. This simple yet powerful modification to attention enabled neural models to address tasks such as sorting, convex-hull detection, and the TSP, in which outputs are positions over the input.

Ptr-Net not only achieved impressive results on these combinatorial problems but also demonstrated the ability to generalize to input lengths beyond those seen in training. This pointing behavior also foreshadowed later mechanisms for copying and generation. For example, subsequent natural language processing models added copy mechanisms that, when generating output, enabled the model to directly reproduce rare or out-of-vocabulary tokens by selecting them from the source text [52].

The pointer concept is essentially an application of attention, reinforcing the theme that flexible attentional addressing is a unifying innovation across these papers. Ptr-Net and Bahdanau's attention both illustrate that neural networks perform better when they can refer back to specific input elements as needed, rather than compressing everything into a single hidden state.

5.5 Neural Turing Machines

Neural Turing Machines (NTMs) were introduced by Alex Graves, Greg Wayne, and Ivo Danihelka in 2014. They paired an artificial neural network with an external, differentiable memory [53][54]. Instead of cramming everything into hidden states, the network could use external

Licensed to Hermilo Cortés <hermilocg@gmail.com>

memory as a scratchpad to record what it might need later. This design enabled NTMs to tackle algorithmic tasks such as copying, sorting, and associative recall.

An NTM consists of a relatively small *controller network*, originally an LSTM, that processes input and generates output in a manner similar to traditional recurrent neural networks. Unlike standard RNNs, though, computation is separated from memory. The controller accesses this memory through specialized mechanisms, known as heads, that are analogous to attention. The read head retrieves information from memory selectively, and the write head modifies memory according to instructions. See figure 5.8. Crucially, the interactions between the controller and memory are fully differentiable, meaning they can be expressed as smooth mathematical functions. This property enables gradient-based optimization, allowing the system to adjust its parameters automatically during training.

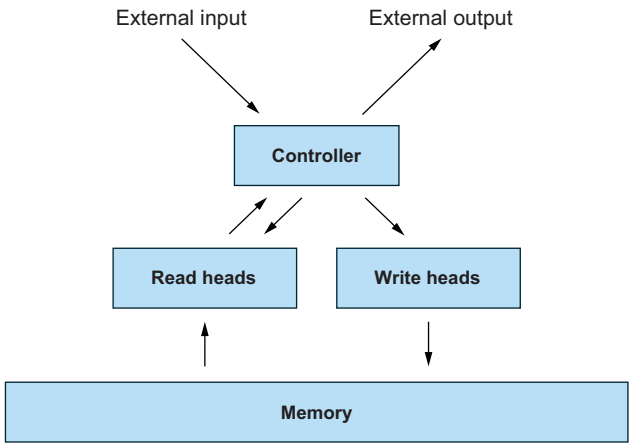


Figure 5.8 Neural Turing Machine architecture. A controller network takes inputs, produces outputs, and manages an external memory. Read and write heads move over this memory to store and retrieve information.

To selectively access memory, the controller produces vectors that define a probability distribution or weighting over memory locations. The network must access specific regions of memory rather than all of it. This form of soft attention enables selective, targeted memory access and requires two complementary mechanisms:

- *Content-based addressing* retrieves information based on the similarity between the controller's query vector and the memory contents.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

These similarity scores are computed via cosine similarity, scaled by a sharpness parameter, and normalized using a softmax function. This approach yields a differentiable associative memory that resembles the query–key attention mechanism later popularized in Transformers.

- *Location-based addressing* retrieves items by position, enabling efficient handling of structured tasks such as copying or sorting.

NTM attention is often called Graves attention, after the lead author (Alex Graves). This was an early attempt to address the challenge of long-term dynamic memory. Ilya Sutskever has noted that modern neural networks primarily store long-term knowledge in static parameters and lack effective dynamic memory. In practice, context windows and retrieval-augmented systems act as a form of non-differentiable short-term memory, partially addressing the limitations of static memory [55][56][57][58].

5.5.1 Significance

NTMs coupled an RNN with an external read-write memory, which the network could interact with through attentional read/write heads. This design was analogous to a computer with differentiable memory, in which the controller uses attention to determine where to read or write. Results showed that NTM could learn simple algorithms such as copying, sorting, and recall from examples. Although they were hard to train and not widely adopted in practice, their core idea of differentiable memory access is conceptually related to Transformers. You can view a Transformer as a simpler, implicit version of this idea. It has no separate memory matrix, but the self-attention mechanism allows each position to read from all other positions in the sequence, effectively using the sequence itself as memory.

5.6 Order matters

"Order Matters: Sequence to Sequence for Sets" by Oriol Vinyals, Samy Bengio, and Manjunath Kudlur explores how ordering affects the performance of seq2seq models [59][60]. Traditional seq2seq models inherently assume a sequential structure, such as in language translation or sentence generation. Many real-world tasks, though, involve unordered data, including combinatorial problems and complex graph structures. The authors introduce architectural and training modifications to mitigate the effects of arbitrary ordering.

As previously discussed, seq2seq models process inputs sequentially using RNNs, which makes them inherently sensitive to the order in which

Licensed to Hermilo Cortés <hermilocg@gmail.com>

data is presented. Ilya Sutskever demonstrated that reversing the order of the input data can, counterintuitively, improve neural machine translation; we'll explore this in more detail shortly. Similarly, sorting points before computing the convex hull in pointer networks improves accuracy by up to 10 percentage points. Likewise, the effectiveness of training an RNN strongly depends on the order in which the data is presented. Choosing a poor ordering can slow down or even prevent the model from learning efficiently.

To handle unordered inputs, "Order Matters" introduces the Read-Process-Write (RPW) encoder (see figure 5.9):

- *Read*—Each element of the input set is embedded in a shared memory matrix, like writing all items on a whiteboard without caring about order.
- *Process*—An LSTM controller repeatedly reads from this memory using an attention mechanism that treats each item equally. This ensures the representation remains stable, regardless of how the inputs are arranged.
- *Write*—The encoder revisits this memory with attention, refining it into a stable representation for the decoder.

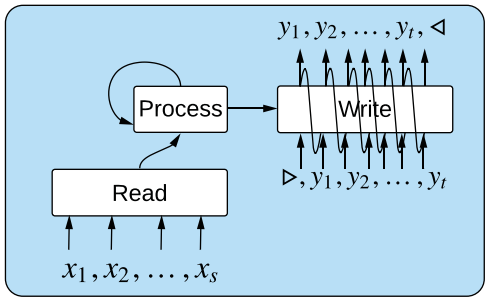


Figure 5.9 Read-Process-Write reads an input sequence into memory, processes it with an LSTM-based recurrent controller, and writes an output sequence. The Read module encodes the inputs, the Process module iteratively updates a hidden state while attending to memory, and the Write module generates outputs one token at a time, feeding its own predictions back as inputs. The arrows between the decoder's outputs highlight the autoregressive loop, where each generated token influences the next. *Used with permission granted by the lead author (Oriol Vinyals).*

Interestingly, embedding inputs into a memory matrix was also central to pointer networks. In that case, the memory served as a list that the decoder could directly "point" to when generating outputs. By contrast,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

the RPW encoder mainly uses its memory to build a robust, order-invariant representation.

The authors provide several examples that demonstrate how ordering affects model performance. They consider three orderings: natural (i.e., "This is a sentence."), reverse (i.e., ".sentence a is This"), and a fixed three-word reversal (i.e., "a is This <pad>. sentence"). This last ordering disrupts the underlying structure of the sentence and breaks up natural word groupings, making it harder for the model to capture linguistic patterns. In the following examples, the authors emphasize that arbitrary ordering introduces unnecessary complexity, which hinders the efficiency of the learning process:

- *Language modeling*—When comparing sentences in their natural order with sentences reordered using a fixed three-word reversal, the model's performance drops, leading to a 10-point rise in perplexity (from 86 to 96). This increase indicates greater uncertainty and less accurate predictions.
- *Constituency parsing*—The way a parse tree is converted into a sequence can affect model performance. One option is depth-first ordering, in which the model explores a branch of the tree to its deepest node before moving to the next branch. Another is breadth-first ordering, which proceeds level by level, covering all nodes at a given depth before moving on to the next. Depth-first ordering preserves the sentence's nested, hierarchical structure and yields significantly higher accuracy (89.5% F1) than breadth-first ordering (81.5% F1).
- *Combinatorial tasks*—In problems like sorting numbers or solving the TSP, the order of the output matters. By enforcing a fixed output order, models avoid wasting resources on the many equally valid solutions that differ only in order, for example, by always listing cities in the same starting position. This reduces the search space, accelerates training, and improves accuracy.
- *Graphical model likelihood estimation*—When training on synthetic star-shaped graphs, models learn more easily when the central "head" node is presented first, followed by its connected nodes. This ordering simplifies the structure the model must capture. The advantage fades with very large or highly deterministic datasets, where the model can learn effectively regardless of order. In most cases, choosing the right input order can make training faster and more efficient.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

When outputs are unordered sets, traditional seq2seq models face a problem: they must commit to a single linear ordering, even though many equivalent orderings exist. A poor choice can slow or even block learning. For tasks where inputs are sets but outputs are sequences (e.g., sorting numbers), the RPW encoder helps by providing a robust representation from which the decoder can learn a natural ordering, such as ascending order. When both inputs and outputs are unordered sets, the RPW encoder ensures the encoder is invariant to permutations. At the same time, a specially designed loss function lets the model discover a suitable ordering during training. Instead of fixing one order in advance, the model starts by treating all orderings as equally likely, then gradually shifts toward the ones it predicts best.

In experiments, this adaptive strategy consistently found optimal or near-optimal orderings. For instance, in an illustrative experiment involving 5-grams, it automatically identified optimal or near-optimal output orderings, significantly lowering perplexity from about 280 with a poor fixed ordering to about 225 with the learned optimal order. Permutation-invariant encoders such as RPW also allow networks to learn natural orderings more efficiently. Even attention-based models such as Transformers—which are more robust to ordering variations than RNNs—still benefit. Ultimately, this work shows that neural networks remain sensitive to data sequencing.

The significance of "Order Matters" was that it showed sequence-to-sequence models carried a liability because they assumed an input order, even for problems where no natural order exists. The paper demonstrated that a model's output could change simply because the inputs were presented in a different order. By framing this as a general problem and experimenting with attention as a remedy, it laid the groundwork for architectures that could handle inputs more flexibly. In hindsight, it anticipated the broader role attention would play in freeing models from rigid assumptions about how information must be structured.

5.7 Reversing input sentences

"Order Matters" showed that seq2seq treats different orderings of the same set as different inputs, and it proposed methods to make models more order-invariant or to learn a canonical order. While the specifics of that paper are somewhat separate from transformers, it reinforces the idea that sequence models must account for the structural properties of data, in this case, order sensitivity or the lack of it.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Yet the lessons of "Order Matters" extended beyond set problems and combinatorial tasks. Order sensitivity was evident not only in synthetic experiments but also in practice. One of the most striking examples was introduced by Ilya Sutskever and colleagues in the 2014 seq2seq paper [61].

With the target left unchanged, they found that flipping the input boosted BLEU scores by nearly 5 points and reduced perplexity from 5.8 to 4.7, signaling more precise predictions and less model uncertainty. For a time, this "reverse the source" trick became standard practice in seq2seq translation systems.

For example, the first word in a French sequence likely corresponds to the first word in an English sequence. In a standard encoder-decoder, the decoder doesn't begin generating until the encoder has finished reading the entire source sentence. This means the first source word must pass through every encoder step to influence the first target prediction. This is like a message whispered down a long line of people: the longer the line, the weaker the signal at the end.

Reversing the source order shortens this path. The first word is now positioned at the end of the encoder, so it has the strongest influence on the decoder's initial state. In effect, the first source word is adjacent to the decoder at the beginning, making it easier to align the beginnings of the source and target sequences. As Sutskever explained, reversing the input "greatly reduced" the processing lag between the first words of the source and target, making backpropagation more effective at establishing their connection. See figure 5.10.

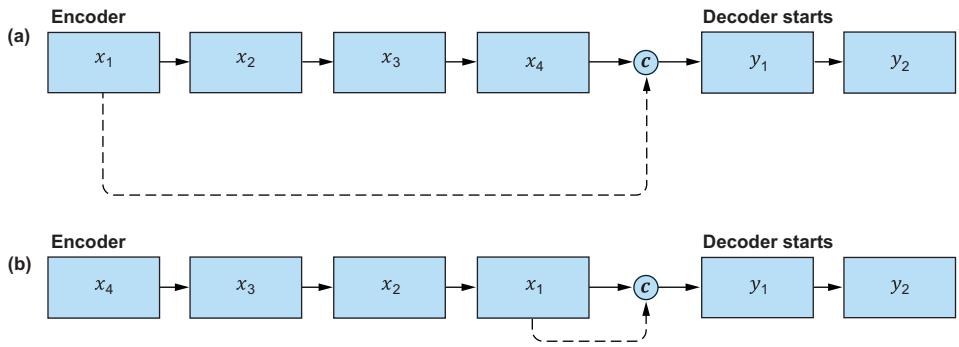


Figure 5.10 The reversal trick shortens the path from relevant source tokens to the decoder's first predictions. In a vanilla RNN encoder-decoder without attention (a), the decoder starts from the encoder's final state. Reversing the source (b) makes tokens that matter for early outputs appear later in the encoder's processing, so their influence reaches the decoder more directly.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The model doesn't become smarter; it just gets an easier optimization problem because useful information has a shorter path to where it's needed. Overall, shorter effective path lengths and improved gradient flow make the training problem easier for an RNN-based encoder-decoder. Moreover, the decoder doesn't need to maintain complicated long-term memory states if frequently aligned source words are positioned closer to the decoder's initial state. Thus, the gradient descent procedure faces fewer obstacles, converges faster, and generalizes better. Yet, reversing the input also highlighted weaknesses in the architecture and further motivated innovations, such as attention mechanisms, that enable models to access information regardless of word order.

Sutskever's seq2seq paper, "Order Matters," and Minh-Thang Luong's paper, "Effective Approaches to Attention-based Neural Machine Translation," which introduced dot-product attention mechanisms, both used the reversal trick. Luong reports that reversing the source sentence improved BLEU by 1.3 on the WMT 2014 English–German task. For a brief period, this was compulsory, as were batch normalization in early convolutional networks and dropout in early deep fully connected networks [62].

As the field progressed, researchers analyzed this reversal trick and introduced more sophisticated methods, such as bidirectional encoders, attention mechanisms, and positional encodings, which reduced the need to reverse sequences. For example, Bahdanau attention allows the decoder to attend to any encoder hidden state at each step [63]. Bidirectional encoders process the source in both directions, allowing the model to learn long-range alignments without requiring the input to be reversed. By 2017, many researchers in neural machine translation (NMT) used either a bidirectional encoder, which captures both forward and backward context, or some form of attention.

An empirical study examined the effect of source reversal across many NMT configurations. It confirmed that reversing the input consistently outperformed the nonreversed version for unidirectional RNN-based encoders. The effect was statistically significant for one-layer and two-layer encoders [64]. This aligns with the earlier intuition that a forward-only encoder benefits from having the beginning of the source closer to the decoder's start. Researchers also found that a bidirectional encoder outperforms a unidirectional encoder, even when the latter uses reversal.

Furthermore, as encoder depth increased to four or more layers, the advantage of reversing the sequence declined. By stacking multiple LSTM layers and employing residual connections, the model gained greater capacity to retain information, reducing the need for manual sequence

Licensed to Hermilo Cortés <hermilocg@gmail.com>

reversal. These findings suggested that reversal is mainly a helpful crutch for simpler or shallower architectures.

Another facet of this practice is that it depends on the language pair and word order. The reversal trick is most effective when the source and target are roughly monotonic (i.e., in a similar order). Sutskever's English–French translations fit this scenario, as both are subject–verb–object languages (SVO). But if a translation involves divergent word order, such as translating from an SVO language into a subject–object–verb language, reversing the source might disrupt the natural alignment.

In such cases, the first word of the source might correspond to the *last* word of the target, so bringing the source's first word to the end could actually *misalign* the sequences. An empirical study of Chinese-to-Japanese NMT (two languages with different word orders) found that reversing the input sentence reduced translation quality [65]. Although useful, the trick is not universally beneficial and is most helpful for tasks in which the source and target sequences are expected to align in order.

Transformers consider all positions jointly and don't suffer from the left-to-right memory constraints of RNNs. Thus, they do not require source reversal, and none of the modern Transformer-based NMT systems employ this technique. Instead, long-range dependency problems are addressed using attention and positional encodings.

Nevertheless, the historical practice of reversing input sentences remains an instructive example of a clever optimization hack in the pre- and early-attention era, where order really mattered. This technique was widely adopted in NMT research from 2014 to 2016 and yielded tangible gains. But its relevance faded as architectures evolved beyond the limitations that made it necessary.

5.8 Effects

Although the most important critique of Transformers centered on their lack of recurrence, the most memorable came later from Emily Bender at the University of Washington, who coined the viral metaphor of "stochastic parrots" [66]. Her one-sided argument emphasized language models' reliance on probabilistic pattern matching and memorization rather than genuine understanding. Language models indeed generate outputs without considering meaning and are inherently limited by the data on which they are trained. Yet the critique undercuts itself because the very thing it condemns is also its superpower.

That is, Transformers are not failing at their purpose; they succeed so well that their outputs can fool us into believing they understand. In this

Licensed to Hermilo Cortés <hermilocg@gmail.com>

sense, the critique collapses into a paradox: language models are criticized not for failing, but for being too good at the very task for which they were designed. After all, no one said this when Andrej Karpathy wrote about the unreasonable effectiveness of RNNs, celebrating that same probabilistic mimicry. The difference was that RNNs never fooled anyone. Transformers did. The cultural resonance of "stochastic parrots" underscores just how deeply Transformers unsettled expectations.

The title became a catchphrase used by enthusiasts and skeptics alike. Enthusiasts elevate it into a rallying cry, suggesting that attention is all you need for artificial general intelligence (AGI) [67]. Skeptics twist the phrase into "attention is *not* all you need," painting proponents as naïve for believing that attention could produce AGI [68]. But both misrepresent the original intention. The paper was not a sweeping statement about intelligence or a prediction about AGI. Instead, it conveyed a precise, pragmatic insight: recurrence and convolution, architectures previously considered indispensable, could be replaced by a simpler, more scalable mechanism.

Still, the phrase began to appear in numerous contexts, sometimes as an ironic template and other times as a nod to its outsized influence. A recent analysis of arXiv titles found exponential growth in papers riffing on "All You Need," with more than 700 such papers [69]. "Attention" was the most frequently invoked term, with 28 papers using the phrasing "Attention Is All You Need" [70]. The catchphrase is even documented on the meme-tracking website Know Your Meme [71].

Researchers, pundits, influencers, and skeptics adapted the formula by swapping in new words: "Data Is All You Need," "Scaling Is All You Need," and "Reinforcement Learning Is All You Need" [72][73][74]. Some entries pushed the meme to the point of satire. For instance, "Money Is All You Need" playfully captured the community's cynicism that throwing money at problems would produce state-of-the-art results [75]. Similarly, an April Fools'-style paper called "A 23 MW Data Centre Is All You Need" suggested, tongue-in-cheek, that AI progress required having a spare 23-megawatt data center close at hand [76]. Doesn't everyone?

The community laughed, but the jokes carried bite. This playful commentary stemmed from widespread speculation that models like GPT-3 required millions of dollars to train, fueling debates about whether financial scale, rather than academic novelty, had become the decisive factor in AI progress. It had, and the signs were evident as early as July 2019, when Microsoft invested $1 billion in OpenAI [77]. Money was a catalyst, and the memes anticipated an era of unprecedented scale that was just beginning.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Once attention removed the sequential bottleneck and enabled large-scale parallel training, capital became decisive.

The Transformer became emblematic of the new wave of AI, to the point that one of its co-authors (Llion Jones) half-jokingly lamented in 2023 that the field had "calcified" around the Transformer and might be overlooking other ideas [78]. In other words, some wondered whether the Transformer had become too dominant, a testament to how thoroughly this architecture had come to dominate.

The shift was profound. Neural machine translation and neural sequence modeling had long been defined by the assumption that recurrence was essential. Researchers even joked during this period about the sudden obsolescence of their hard-won expertise in recurrent neural networks, which attention rendered less relevant.

RNNs dominated because they naturally processed words in order, and through clever optimizations such as factorization and conditional computation, they became faster and more practical [79][80]. Convolutional approaches, such as ByteNet and ConvS2S, also demonstrated the ability to handle sequential data. By introducing dilated convolutions (chapter 3) to widen context windows, they expanded their receptive fields, showing that local filters could be stretched to capture broader context. Yet despite their progress, both architectures carried structural limits. Recurrent networks struggled to parallelize across long sequences, whereas convolutional networks relied on fixed, local interactions [81][82]. Even with dilations, convolutional encoder-decoder models remained limited.

These constraints set the stage for the Transformer, which discarded both with a bold, provocative claim captured succinctly in its paper title. Rather than relying on recurrence, where information propagates sequentially through hidden states, or convolutions, which compose context through local interactions, the Transformer depends exclusively on self-attention within sequences.

The idea that attention alone, without recurrence, without convolution, without any explicitly sequential inductive bias, could perform complex sequence transduction was more than a technical proposal; it was philosophical. It implied that the structure of language, long assumed to require step-by-step processing or local aggregation, could be captured from the start by nonlocal operations. Not only could it be done, but it could also be done better, faster, and cheaper. The Transformer showed that recurrence was a convenience rather than a necessity and revealed scaling as the primary driver of progress in machine learning. In the end, recurrence and convolution did not require funerals. They put up a fight

Licensed to Hermilo Cortés <hermilocg@gmail.com>

but ultimately became unnecessary. Attention may not literally be "all you need," but it was certainly enough to dismantle entrenched assumptions about *essential* structures in neural sequence modeling.

Among the papers on Sutskever's List, "Attention Is All You Need" is the most influential. Its core idea became an iconic catchphrase, widely recognized beyond academic circles, and a *2025 Nature* analysis ranks the paper seventh among the most-cited papers of the 21st century. Only one paper on Sutskever's List outranks it [83]. While residual connections quietly enabled deeper networks, attention redefined our relationship with technology [84].

On the eve of ChatGPT's launch (November 29, 2022), Ilya Sutskever declared, "The Transformer is well named, as it transformed everything" [85]. Scaling had been quietly humming in the background since AlexNet, waiting for the right architecture to unlock its full potential. With the Transformer, Ilya found the shape of something built to *grow*. The question now was how far scale could carry this new architecture and what new capabilities might emerge. These questions will be the focus of the next chapter.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

6 The birth of hyperscale

This chapter covers

- The scaling canon: Kaplan, Chinchilla, and post-Chinchilla scaling laws
- Sparks of artificial general intelligence and emergent behavior
- Smooth loss, jagged evals, and inverse scaling
- The hidden work: Data and model parallelism
- GPipe and pipeline parallelism

Papers

- *Scaling Laws for Neural Language Models* (2020) Kaplan et al.
- *GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism* (2019) Huang et al.

Modern AI has revealed that bigger models perform better. The most explicit demonstration came with the leap in performance from GPT-2 (2019) to GPT-3 (2020). GPT-2 was modest by today's standards: it had 1.5 billion parameters and was trained on a relatively narrow dataset,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Reddit. Still, it captured the public imagination with examples such as Ovid's Unicorn (chapter 1). The piece was so convincing in tone and style that it reset expectations about machine-generated text, and almost everyone overlooked that it was cherry-picked and included oxymoronic references such as "four-horned unicorns."

Just a year later, OpenAI released GPT-3, which was trained on more data at a cost measured in millions of dollars [1]. The researchers didn't reengineer the architecture. They merely increased the model size to 175 billion parameters, approximately 117 times that of GPT-2. But they did so with confidence, having already discovered the so-called scaling laws. These indicate that training loss decreases smoothly as model size, data, and compute resources scale up. And yet what arrived did not feel predictable at all. GPT-3 demonstrated in-context learning and competence on tasks without task-specific fine-tuning.

These are often called *emergent abilities*: skills that appear only at chance level in smaller models but emerge in larger models, often quite suddenly, once a certain scale threshold is reached. Examples include multistep reasoning, multidigit arithmetic, translating between languages not explicitly trained on, or basic code generation [2][3][4][5][6]. Yet benchmarks can make these abilities appear to arrive all at once, while the underlying loss curves remain largely smooth and predictable. So, some of what we call "emergence" reflects quirks of measurement rather than a sharp discontinuity in what the model can do.

Still, scaling laws gave the field something it had never had before. Gains diminish as size increases, but the relationship is consistent enough to provide researchers with a roadmap for progress [7]. For decades, progress in AI had been uneven, marked by occasional algorithmic breakthroughs followed by long plateaus and occasional winters [8]. Suddenly, the recipe for progress became simple. For a discipline long accustomed to rare, hard-won gains, this acceleration was extraordinary.

6.1 Scaling canon

In 2017, Joel Hestness and colleagues at Baidu mapped how deep learning scales across different model families, including ResNets (chapter 3), Deep Speech 2 (chapter 4), and attention-based encoder-decoders (chapter 5) [9]. They demonstrated that model error follows a predictable power-law relationship with dataset size. What was striking, though, was that the architectures didn't change the slope of those curves. The pace of improvement (that is, how quickly errors decline as a model is trained on more data) remains constant within a given domain. What various architectures change is the intercept.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

That is, a better model family shifts the entire curve downward, delivering lower error, but it doesn't make the curve any steeper. ResNets in vision, and by extension, Transformers in language, pushed accuracy higher across the board, but scale sets the slope. As the paper put it: "Improved model architectures and optimizers can improve the power-law intercept but not the exponent; models for a single domain show the same learning-curve steepness."

The implication was that once an architecture scales, subsequent progress becomes a budgeting problem. That's why the field shifted from novelty to scaling models and feeding them more data and compute. While Transformers weren't evaluated in the Hestness paper, it laid the groundwork for more detailed scaling laws. Three years later, Jared Kaplan and the team at OpenAI built on this insight with "Scaling Laws for Neural Language Models" (2020), a paper featured on Sutskever's List.

Most papers on Sutskever's List deliver a single clean leap (such as ResNet or Transformers), an engineering tour de force like AlexNet or Deep Speech 2, or a clearly bounded experimental finding such as Pointer Networks or Order Matters. The Kaplan paper is different. It runs nearly 30 pages, presenting coupled power laws that map interactions among loss and nonembedding parameters, dataset size, compute, training time, and steps, along with batch-size theory, overfitting behavior, and shape-invariance results. This chapter focuses on the *what* and the *why* while preserving the big ideas and explaining them in plain language.

6.1.1 Kaplan scaling laws

The Kaplan scaling laws explain that when model size, data, and compute are scaled in tandem, performance improves smoothly, though with diminishing returns. For example, doubling the number of parameters (N) shrinks the error by about 5% (technically, by a factor of $2^{-\alpha N}$ with $\alpha N \approx 0.076$). Doubling the training data (D) yields roughly a 6% drop in error ($2^{-\alpha D}$ with $\alpha D \approx 0.095$). See figure 6.1. While modest in absolute terms, it's steady and reliable in a field that has rarely seen steady rules.

There's an important constraint, though. With a fixed-sized dataset, scaling the model by adding parameters will eventually stop test loss from improving, and pushing the model to convergence can accelerate diminishing returns and even introduce overfitting. Kaplan and colleagues found that the balance follows its own equation in which data grows sublinearly with model size (roughly as $D \propto N^{0.74}$). That is, if the number of model parameters is doubled, the dataset need not double. Instead, it should grow by about 1.7 times. Triple the model's size, and the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

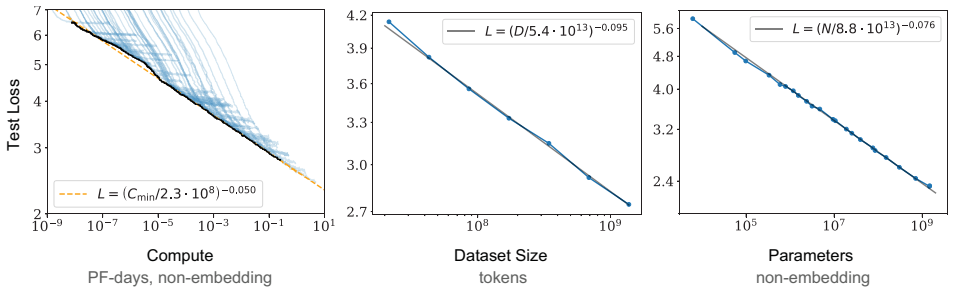


Figure 6.1 Language modeling performance improves smoothly as model size, dataset size, and amount of compute used for training increase. For optimal performance, all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each factor when it isn't bottlenecked by the other two. ***Used with permission granted by a coauthor (Ben Cross).***

corresponding dataset should grow by about 2.3 times to keep pace. If the model is scaled up by a factor of 117 (GPT-2 → GPT-3), the dataset should increase by approximately 34 times. That relationship helped researchers keep larger models more efficient.

Finally, there's the question of compute. Like datasets, training budgets are often finite, and the instinct is to train a smaller model to convergence. But Kaplan's analysis showed the opposite. With a fixed budget, practitioners are better off training a much larger model for fewer steps. Larger models soak up data more efficiently; each unit of compute yields greater improvement than in a smaller system. This meant progress was not just about raw horsepower but also about how wisely one allocated scale across parameters, data, and training time. See figure 6.2.

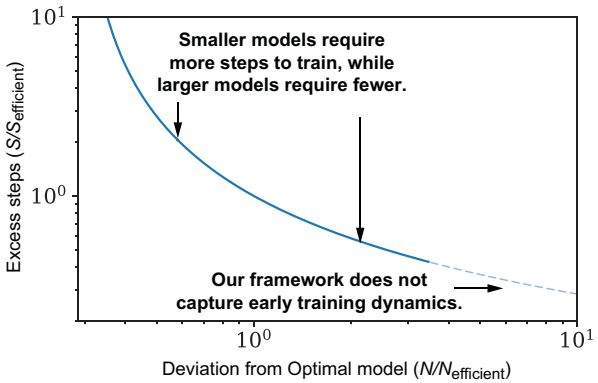


Figure 6.2 Models larger than the compute-efficient size require fewer training steps, potentially enabling faster training if sufficient additional parallelism is possible. ***Used with permission granted by a coauthor (Ben Cross).***

Licensed to Hermilo Cortés <hermilocg@gmail.com>

This research focused on decoder-only transformers. Google introduced the decoder-only transformer in 2018, a year after the original Transformer. It handles long input sequences better than the original and also scales more easily than its encoder-decoder counterparts [10]. The authors trained several baselines alongside their decoder-only model [11].

The experiments covered a wide range of model and dataset sizes, from 768 million to 1.5 billion nonembedding parameters and from 22 million to nearly 23 billion training tokens. The authors defined model size as the number of nonembedding parameters because token and positional embeddings can dominate parameter counts in smaller models without significantly increasing model capacity. By comparison, the number of nonembedding parameters depends on the network's depth and layer width. Batch size (the number of tokens processed in parallel during each training step) varied with model and dataset size; the largest runs used batches of about 500,000 tokens.

To isolate scale from shape, the authors varied the number of layers, the width of the feed-forward sublayers, and the number of attention heads while maintaining proportionalities consistent with common design practices at the time. They also varied the dimensionality of the internal vector representation, which is the model's hidden size (d_{model}) or the length of the per-token feature vector. These architectural changes allow the authors to study the effects of each variable in isolation while maintaining realistic architectural size and shape [12].

Training typically lasted about 250,000 steps or until the allocated compute budget was exhausted, with runs ended early if the validation loss plateaued. Most models were trained with the Adam optimizer, but the largest models (more than 1 billion parameters) used Adafactor to reduce memory usage [13]. Learning rates used a 3,000-step warm-up followed by a cosine decay to zero, and the authors found that, as long as the values were reasonable, final performance was not highly sensitive to the exact schedule.

6.1.2 Does shape matter?

The research shows that performance depends strongly on scale and only weakly on shape. Here, shape refers to how parameters are distributed within the architecture. For example, they compared configurations with a small number of wide layers against configurations with many thin layers, all with the same parameter count. They also adjusted the width of the feed-forward layers relative to the hidden size. Finally, they varied the number of self-attention heads.

Loss changed by only a few percent across these variations. Only extreme shapes, such as those with fewer than two layers, showed a notable

Licensed to Hermilo Cortés <hermilocg@gmail.com>

drop in performance. These results underscored the importance of earlier innovations, such as attention and residual connections, which enabled training deep, wide models in the first place. Just as crucial were batch normalization, careful weight initialization, optimizer innovations, layer-scaling tricks such as learning-rate warm-up to stabilize training, and even dropout. With those breakthroughs in place, the exact arrangement and parameter proportions mattered less; the decisive factor was the number of parameters in the model.

6.1.3 Scaling with data and model size

The models were trained on WebText2. The original WebText dataset was introduced with GPT-2 and popularized by Ovid's Unicorn. WebText2 extends that collection to about 20 million documents. After tokenization, the corpus contained roughly 23 billion tokens, the input units the models process during training.

When model size was held constant and the amount of training data varied, performance followed a smooth, predictable curve, with more data improving results. The curve for data scaling was slightly steeper than that for model size, indicating that increasing dataset size yielded larger improvements than adding parameters alone. But each increase in data still produced progressively smaller drops in loss. This pattern held not only for WebText2 but also for the same models on out-of-distribution corpora such as Wikipedia and Books. The improvement curves for these datasets were roughly parallel (on a log scale) to the in-distribution curve and offset by a constant gap, indicating that the benefits of scaling transfer well to new data sources.

When model size, data, and compute are scaled, the authors also revealed something subtler: the rate of those diminishing returns can be managed. The authors examined this by comparing the results with the *infinite data limit*, which represents the best possible performance a model could achieve with unlimited training data, making overfitting impossible. In practice, overfitting occurs when the model outgrows the dataset and begins to memorize it.

Kaplan quantified how to keep that bend in the curve from arriving too soon. Each time model parameters increase by approximately eightfold, the dataset size must increase by roughly fivefold ($D \propto N^{0.74}$) to maintain generalization. This 8× parameter, 5× data rule shows that data requirements scale sublinearly with model size, making ever-larger models feasible without requiring one-to-one growth in data. In 2020, this was a big deal. It provided the first straightforward, quantitative recipe for scaling

Licensed to Hermilo Cortés <hermilocg@gmail.com>

models and datasets together. Larger models didn't require proportional data growth, making it thinkable to train hundred-billion-parameter models, which is precisely what OpenAI did with GPT-3.

This framework also gave researchers a diagnostic tool to determine whether a model remains on the scaling curve. If performance starts falling off more sharply than the law predicts, it signals that something outside the scaling regime has gone wrong. A "scaling regime" holds all variables (e.g., architecture, objective, tokenizer, context length, optimizer schedule, and data distribution) constant. Each regime has its own curve, with a different slope. There may be data quality problems, over-repeated tokens, or optimization bottlenecks causing performance to drift. For example, suppose a 50-billion-parameter model shows no improvement over a 20-billion-parameter model. That's a red flag that the setup has drifted off the expected slope.

6.1.4 Scaling compute

The final question was how performance scales with compute and how to spend a fixed budget. First, there is the *optimal batch size* [14]. Recall that batch size refers to the number of tokens processed in parallel during each training or optimization step. A training step is a single parameter update. The model grabs a batch of training examples, computes the loss and the gradients on that batch, and lets the optimizer (for example, Adam) update the weights once. If the batch is too small, the model crawls forward. Progress is noisy and requires too many tiny updates. If the batch is too large, each step averages over increasingly redundant examples. As a general rule, training near the sweet spot keeps hardware busy with little waste. In practice, that point often lands around 2× the minimum number of steps and 2× the minimum amount of unique data needed to reach the target accuracy.

Second, there is a *limit to the amount of data*, which guides how you allocate the compute budget between increasing model size and extending training duration. For any fixed budget, there's a right-sized model and a right-length training run. Build a model that's too large and train it too briefly, and you'll pay for capacity you didn't use. Keep a model too small and train it forever, and you're polishing something that can't express the data it was trained on. The path between these boundaries traces a frontier. As budgets grow, one must increase both model size and training duration in tandem to stay balanced. Later work showed that many frontier models are on the large-but-undertrained side and need rebalancing toward more data per parameter to produce better outcomes. We'll talk about that shortly.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In the compute-optimal regime for language modeling, doubling total compute yields only a 3% to 4% reduction in loss, which is less than the gains from scaling data or parameters. Bigger models, though, are more sample-efficient. That is, they reach a given loss with fewer tokens and fewer parameter updates, extracting more information per unit of data. The key takeaway is that allocating compute to extend the training of a small model is inefficient, whereas allocating it to capacity, with proportionally more data, yields compounding returns.

Kaplan also showed that compute-efficient training stops well before convergence. Convergence means the training run has mostly stabilized, and additional epochs produce little to no improvement. A compute-efficient model is trained for fewer steps but can outperform a smaller model trained all the way to completion. In this setting, early stopping functions as a form of regularization, curbing memorization while still allowing the larger network to use its capacity [15]. The net effect was a reversal of intuition. Progress doesn't come from training to convergence; it comes from scale. The smart money is spent on capacity, not on dragging a smaller model across the finish line.

To illustrate, the authors asked what would happen with a billion-fold increase in computing budgets. Most of that compute should go into making the model larger, since bigger models are more sample-efficient and extract more information per token. The dataset size would need to grow only by a modest multiple to avoid overfitting, and training time, measured in steps, would barely change. See figure 6.3.

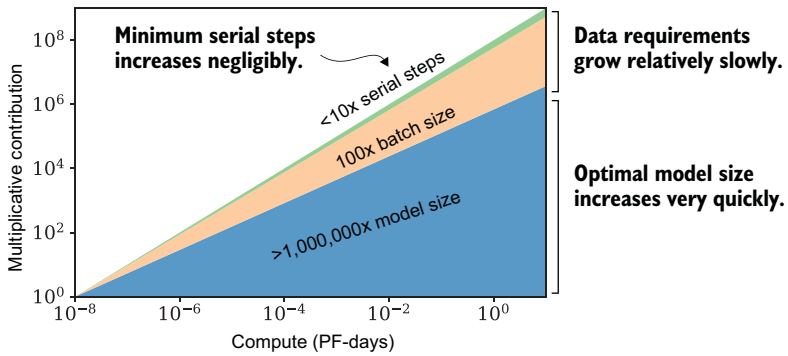


Figure 6.3 The authors illustrate how to allocate training resources given a billion-fold increase in compute. For optimal, compute-efficient training, most of the increase should go toward expanding model size. Only a relatively small increase in data is needed to prevent reuse. Of the additional data, most can be used to increase parallelism by using larger batch sizes, requiring only a minimal increase in serial training time. *Used with permission granted by a coauthor (Ben Cross).*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

6.1.5 Architecture comparisons

The Kaplan team compared decoder-only transformers with GPT-1 (another decoder-only model), the original Transformer, long short-term memory (LSTM), and the recurrent Universal Transformer. The LSTMs, despite performing comparably on very short sequences, quickly plateaued, showing an inability to exploit long contexts or scale with parameter count. The Universal Transformer, with its recurrent refinement mechanism and parameter reuse, achieved slightly better per-parameter efficiency but required substantially more compute at the same scale. These results showed that scaling curves were not artifacts of a single architecture. Instead, they reflected a broader law of language modeling. At the same time, they clarified where the decoder-only transformer has particular advantages, especially in maintaining performance across long contexts and being more computationally efficient at scale [16].

6.1.6 The scaling hypothesis

Despite being a quantitative "law" of steady improvement, the Kaplan curves went largely unnoticed outside a small circle of insiders. One of the earliest outsiders to grasp their significance was Gwern Branwen, a pseudonym for an independent researcher working far from the major labs. In May 2020, he published a blog post titled "The Scaling Hypothesis." The post went viral, speculating that intelligence might emerge from nothing more than scale [17].

The Kaplan paper, though, was an empirical study. It was not a manifesto. Gwern transformed those curves into a radical theory of intelligence extending far beyond Kaplan's careful framing. His provocation generated significant excitement about scaling and seemed to gain momentum when GPT-3 was released only weeks later and appeared to instantiate the scaling hypothesis and the "blessings of scale" [18][19].

That argument reached its high-water mark with GPT-4 in 2023, which reportedly had nearly 2 trillion parameters [20]. Microsoft researchers tested it across a range of tasks, including mathematics, law, programming, computer vision, and reasoning. Their report, "Sparks of Artificial General Intelligence," suggested that scale had pushed language models beyond narrow specialization and into territory that, for a moment, looked uncomfortably close to general intelligence [21].

Soon after, researchers reframed those supposed "sparks" in more prosaic terms. A paper titled "Embers of Autoregression" suggested that what looked like "sparks" of artificial general intelligence (AGI) were better understood as the "embers" of autoregressive training [22]. "Embers of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Autoregression" argued that the surprising flashes of reasoning and abstraction were not signs of general intelligence but rather side effects of scaling this next-word prediction process to extreme dimensions. Read together, *sparks* and *embers* highlight the scaling narratives of this period; one captures the exhilaration of emergent capabilities, while the other reminds us that intelligence is not merely a matter of scaling an autoregressive model.

Even so, scaling laws are the closest thing machine learning has to a free lunch, not because they are cheap, but because they shift progress from discovery to execution. In contrast to the "No Free Lunch" theorems, which show that no optimization method is universally superior across all problems, scaling laws suggest that scale can reliably drive improvement without the need for new architectures [23]. Crucially, these laws mattered only because earlier breakthroughs had delivered a scalable architecture. Kaplan and colleagues merely formalized the economic argument that larger models are a better use of compute than pursuing new architectures. Sutton's "Bitter Lesson," which argued that general methods using more compute outperform handcrafted solutions, supplied the methodological counterpart [24]. Together, they created a rationale for why the field should bet on scale, and OpenAI seized on this, treating scale not just as a training strategy but as a corporate philosophy.

6.1.7 Chinchilla and post-Chinchilla

The clean picture of scaling did not last long. In 2022, Jordan Hoffman and colleagues at DeepMind challenged Kaplan's prescription with the Chinchilla scaling laws. Kaplan had shown that bigger models trained for fewer steps could outperform smaller ones trained to completion. This pushed the field toward bigger models with proportionally more data rather than longer training runs. But Chinchilla revealed that many of these models were undertrained and that adding data could unlock headroom in the architecture [25].

By analyzing more than 400 transformer models with sizes ranging from 70 million to 16 billion and training tokens of up to 500 billion, DeepMind found that the optimal token-to-parameter ratio is roughly 20:1 for a transformer of that era [26]. This conclusion challenged Kaplan's bias toward model size. But the lesson itself was hardly new. "Don't starve models of data" was precisely the advice Halevy, Norvig, and Pereira offered in 2009 in their essay on the "Unreasonable Effectiveness of Data," echoed in the spirit of Efros's "fancy schmancy" narrative. What had changed was the scale and precision with which these trade-offs could now be measured.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Using this optimal token-to-parameter ratio, the authors showed that the 175-billion-parameter GPT-3, which was trained on 300 billion tokens, was far from optimal, with about 1.7 tokens per parameter. According to the Chinchilla paper, GPT-3 should have been either much smaller, at about 15 billion parameters, or trained on roughly 3.5 trillion tokens to fully use its parameters. This mismatch explains why Chinchilla-optimal models routinely outperformed larger peers, including Megatron-Turing NLG (530B), Gopher (280B), Jurassic-1 (178B), and even GPT-3 (175B), all of which had been starved of data.

This insight transformed the industry. "Chinchilla-optimal" became shorthand for models that effectively balance size and data. OpenAI reportedly trained GPT-4 with much more data per parameter than previous models [27]. Meta's LLaMA 2 (2023) followed the same approach, using 2 trillion tokens for a 70-billion-parameter model (~30 tokens per parameter). This was six times GPT-3's training data [28]. LLaMA 3.1 was trained on nearly 10 times as many tokens as LLaMA 2, prompting the field to question whether Chinchilla's ratio was a strict rule or merely a guideline [29]. Some even declared that Chinchilla was dead [30].

TinyLlama was one attempt to push the Chinchilla scaling laws by using 3 trillion tokens to train a model with only 1.1 billion parameters, yielding nearly 3,000 tokens per parameter [31]. Yet in a live experiment conducted over several months in late 2023, TinyLlama achieved only parity with similarly sized models. This suggests that there are limits to how small a model can be: below that point, training data cannot compensate for the lack of parameters, and scaling laws may not apply to extreme ratios of model size to dataset size.

This period may be called the post-Chinchilla scaling laws, with an emphasis on training models that use far more tokens per parameter than Chinchilla suggested. In April 2024, Llama 3 was trained on 15 trillion tokens [32]. The Chinchilla-optimal token count for an 8-billion-parameter model would be approximately 160 billion tokens, and for 70-billion-parameter models, around 1.4 trillion tokens. Yet Meta trained the model with a token-to-parameter ratio of approximately 1,875:1 for the 8-billion-parameter model and about 214:1 for the 70-billion-parameter model.

The contrast with Kaplan's original law is instructive. Kaplan assumed an "infinite data limit" and, with experiments capped at 23 billion tokens, concluded that scaling parameters was the best path forward. Chinchilla extended the regime to 500 billion tokens and demonstrated that performance continues to rise with more data for moderately sized models,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

shifting the balance toward data. But what followed, including TinyLlama and LLaMA 3, was not a rejection of Chinchilla but an exploration of its boundaries. Kaplan emphasized parameters; Chinchilla restored some balance; post-Chinchilla experiments tested its limits.

Scaling shifted again with DeepSeek. Released to the public in January 2025, DeepSeek shocked the world by showing that a high-end model could be trained for millions rather than billions [33][34]. DeepSeek cut costs by more than 40% and boosted throughput nearly sixfold [35][36]. Capturing the mood, a paper bluntly titled "Enough of Scaling LLMs! Let's Focus on Downscaling" urged researchers to pursue the gains of scale through more innovative, lower-cost strategies [37].

Today, the discussion about scaling at the frontier has gone quiet. Kaplan and Chinchilla were public landmarks, but new insights from trillion-token training runs have been lost to corporate intellectual property. With billions of dollars at stake, labs are no longer sharing their curves. We glimpse them only indirectly, through leaks and inference. While "scaling laws" remain an active area of research, with multiple works published each year, the age of frontier scaling laws ended almost as soon as it began [38].

6.2 Scaling detours

Critics were quick to point out that scaling laws were not immutable. In 2022, Gary Marcus wrote a blog post titled "Deep Learning Is Hitting a Wall." It went viral. In it, he said, "The so-called scaling laws aren't universal laws like gravity but rather mere observations that might not hold forever." He added, "Indeed, we may already be running into scaling limits in deep learning, perhaps already approaching a point of diminishing returns" [39].

Marcus is right. Scaling laws aren't like gravity, though no one was suggesting they were. They're empirical regularities of steady improvement, with caveats. But invoking "diminishing returns" misses the point. The curves already indicate diminishing returns, yet they forecast reliable progress. The practical question is not "Are we approaching a point of diminishing returns?" but "Is the next increment of capability worth the marginal cost?" In the early years of scale, the answer was an emphatic yes, and easy wins were everywhere. In recent years, the slope has sagged, and practitioners have shifted levers to focus on aspects such as data quality, context length, instruction tuning, tool use, and reasoning (chapter 7), while leaving the scaling backbone intact [40][41].

The deeper challenge to scaling laws lies not in their supposed impermanence or well-documented diminishing returns, but in their uneven

Licensed to Hermilo Cortés <hermilocg@gmail.com>

behavior. Scaling laws appear smooth on paper, but in practice they are noisy, brittle, and uneven. Training loss behaves like a statistical average and can be forecast with surprising precision from upstream loss, which is rare in such a messy empirical science. But specific capabilities and downstream performance are far less predictable. Emergent abilities can appear suddenly, while inverse scaling indicates that some tasks perform worse at scale. During the development of GPT-2 and GPT-3, researchers posed questions such as, "When will arithmetic appear?" or "When will models start to code?" The answers came not as gradual improvements but as abrupt thresholds, with abilities seeming to switch on once the models crossed them.

A meta-analysis found that a close fit to linear scaling laws appeared in only 39% of cases [42]. In the remaining 61%, curves bent unpredictably; some tasks plateaued, some declined, and others increased sharply after long periods of flat performance [43]. In short, scaling laws offered a valuable empirical guide but never a guarantee.

6.2.1 Jaggedness

Scaling laws describe how training loss decreases as parameters, data, and compute resources are added, but only within a specific regime. Eventually, a training run hits a snag: it might run out of data, the tokenizer or optimizer might change, or the rule itself might change. On a log–log plot, that looks like straight-line segments stitched together. Still, it isn't a single universal line; it's piecewise, with kinks and new segments. Work on *broken scaling laws* formalizes these regime changes and shows why ability can jump even when the loss curve looks steady within a regime [44].

This also explains why capabilities appear emergent. Most evaluations are thresholded, including exact matches and win rates. A tiny, smooth drop in cross-entropy can flip many borderline items from almost-right to right, so the test score leaps even while the training curve looks well-behaved. OpenAI cofounder Greg Brockman tweeted: "Evals are surprisingly often all you need" [45]. While the curves are largely smooth, what matters is what the evals reveal. The curves tell you where to spend compute; evals tell you when you cross a regime boundary and which capabilities have actually switched on.

Model-level evaluations such as Measuring Massive Multitask Language Understanding (MMLU), Grade School Math 8K (GSM8K), BIG-bench, and Humanity's Last Exam report accuracy or pass rates across question answering, mathematics, instruction following, and code [46][47][48][49][50][51]. Across thousands of items, such fluctuations create steps,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

plateaus, and dips in task performance, even as training loss remains smooth. Instead of a smooth downhill road, progress feels like climbing a staircase with missing steps. For broader audiences, those scores are further recast as human-legible thresholds that hide as much as they expose: "Passed the bar" [52], "SAT score," "PhD-level" [53], "Better than a radiologist" [54]. These labels compress a multidimensional capability profile into a single social signal. They are useful but incomplete.

No benchmark can cover the space of possible inputs for a modern model, and evaluations inevitably sample only a thin slice of a much larger distribution. This sampling gap explains why new strengths, and new failure modes, often surface after deployment. It also clarifies how to interpret the apparent "magic" or "sparks" of pretraining [55]. Next-token prediction trains a model of text, and the world enters only indirectly, filtered through what humans choose to write, notice, and repeat.

In that sense, a language model can approximate a world model well enough to produce behavior that resembles understanding, but that understanding is mediated by language and remains distribution-bound. Performance is therefore tethered to the training distribution. That distribution includes human-generated descriptions of the world, not direct access to it, and the system does not systematically transcend it (at least not yet). Consequently, patterns that occur frequently in the data are learned more completely, while rare, missing, or systematically underdescribed patterns remain weak. See figure 6.4.

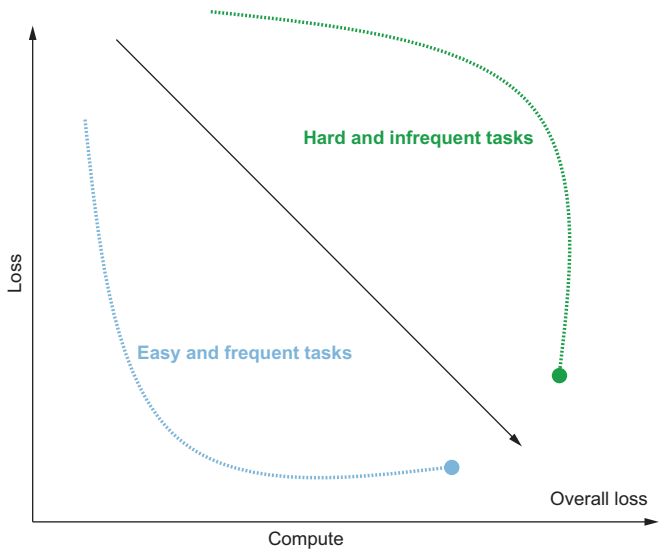


Figure 6.4 A model will learn some tasks more easily than others. Easy tasks are frequent with dense data, while hard tasks are infrequent with sparse data.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Even when two tasks look equally complex, the model will favor the one that's better represented in the training data, because output probabilities steer responses even with deterministic prompts. This can yield uneven capabilities. Multidigit arithmetic may work up to n digits and fail at $n + 1$. Retrieval may hold up to L tokens of context and slip at $L + 1$. Reasoning may chain k steps and falter at $k + 1$. In code, a model might build a simple website featuring an American flag that changes colors yet fail to generate a cake that plays "Happy Birthday," despite the apparent similarity in task complexity [56]. At times, it's like knowing how to tie your right shoe but not your left. The paradox fuels debate about reliability and robustness, even as models become more consistent.

Sometimes the divergence is stronger. On some benchmarks, larger models perform worse as they scale, at least for a while. This inverse scaling and related U-shapes can arise when larger models latch onto spurious patterns more confidently before recovering with additional scale, improved data, or refined training methods. This is why scaling laws predict loss so well but struggle to predict performance on downstream tasks. Accuracy, though, can zigzag, plateau, invert, and then resume climbing [57][58].

Ultimately, scaling laws predict, with uncanny precision, how training loss will fall as parameters, data, and compute are added. That makes them invaluable for planning budgets and forecasting aggregate progress. They do not tell us when a model will suddenly handle arithmetic, write code, or plan reliably. Those abilities appear unevenly. General benchmarks give us a better sense of downstream performance, but they only go so far.

6.2.2 Contamination

There are also methodological obstacles to assessing language model performance. For example, data leakage (test set contamination) occurs when training data inadvertently includes elements from the test set. Generative models are trained on internet-scale data and have seen trillions of tokens. As models scale, they gain greater capacity to store and reproduce training text, but memorization is driven not only by size. Repetition, redundancy, and training duration strongly shape what is retained verbatim. Consequently, it is increasingly difficult to determine whether a benchmark has been effectively "seen" during pretraining, either directly or through close variants in the training corpus.

First-generation benchmarks, such as ImageNet, tell a story of progress. AlexNet's breakthrough in 2012 cut error rates in half. Within a few years, ResNet surpassed human annotators, and the benchmark was soon

Licensed to Hermilo Cortés <hermilocg@gmail.com>

saturated. Contamination was not a concern. The dataset was controlled and widely studied. What mattered was architecture and optimization. Once saturated, these benchmarks were no longer helpful indicators of progress.

Newer language benchmarks (e.g., SQuAD and GLUE) are distributed across GitHub repositories, machine learning tutorials, and Hugging Face libraries. With the rise of web-scale pretraining, examples from SQuAD or GLUE increasingly became likely to be incorporated directly into the training corpus. As a result, overlap is likely, and some gains—especially on widely circulated items—are partially inflated. The effect is real but not exhaustive, as scale and architecture still account for a substantial portion of the gains.

Even partial contamination can inflate results. Benchmarks that become saturated as soon as they are released lose their diagnostic power. See figure 6.5 [59]. They become fragile instruments, easily gamed by both scale and data leakage. The visibility of these datasets, which made them useful for reproducibility, also made them porous [60][61]. Analyses from the *LM Contamination Index* suggest that many well-known evaluation datasets are verifiably contaminated, while many others appear suspicious [62]. The implication is that quantitative progress reports—when based solely on corpus-derived benchmarks—may be less trustworthy than they appear.

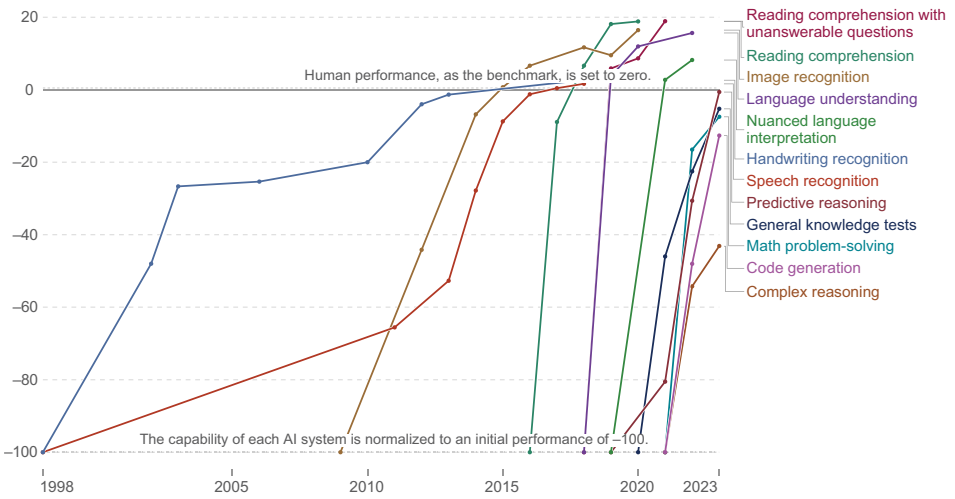


Figure 6.5 Line chart (1998–2023) showing multiple series that rise steeply after 2018 toward the "human performance = 0" line for reading comprehension, code generation, math problem-solving, and complex reasoning, indicating rapid benchmark saturation.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Researchers are now designing more challenging and comprehensive tests. The goal is to probe reasoning, abstraction, and generalization. This has given rise to benchmarks such as HELM, MMLU, BIG-Bench Hard, GSM8K, Humanity's Last Exam, and the ARC Challenge [63]. These harder benchmarks reveal the capability overhang problem. Models often possess abilities that remain hidden until someone tests them. But once tested, they cease to be reliable measures, a form of "Goodharting" [64].

6.3 Parallelism

Scaling may sound easy. But each step up in size exposes new bottlenecks in memory, compute, and communication. While scaling curves suggest inevitability, the engineering didn't.

An early strategy for scaling was *data parallelism*. Data parallelism kept models synchronized by replicating the entire network on each GPU. Typically, each model replica is fed a different batch of data, and the gradients are then summed with an "all-reduce" operation to ensure that every replica stays in sync [65][66]. Data parallelism was efficient and straightforward, but once parameter counts exceeded the memory capacity of a single device, replication became infeasible [67].

Unlike data parallelism, which replicates the whole model on every GPU, *model parallelism* splits a single model across multiple devices. AlexNet demonstrated an early form of model parallelism. ImageNet was large enough to justify a larger model, so Krizhevsky, Sutskever, and Hinton split AlexNet across two GPUs, with each GPU handling half of the feature maps and cross-GPU communication limited to the third convolutional layer, where the two halves reconnect. They limited communication to specific layers to keep it inexpensive. More connections meant more communication; fewer connections meant cheaper transfers. They chose a pattern that kept communication a small fraction of the total computation while still allowing information to mix when needed. This was a crude early form of tensor sharding done by hand.

A quick note on tensors: a *tensor* is a container for numbers. A single number is a zero-dimensional tensor, a list of numbers is a one-dimensional tensor (that is, a vector), a grid of numbers is a two-dimensional tensor (that is, a matrix), and higher-dimensional tensors are stacks or cubes of numbers. Deep learning models store their weights, activations, and data in tensors. *Sharding a tensor* means splitting those numbers across devices so no single GPU has to store everything. AlexNet hand-rolled this sharding, which suggests that early scaling limits emerged first in VRAM and interconnect bandwidth. Later systems, such as GPipe (2018) and

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Megatron-LM (2019), generalized these ad hoc tricks into principled forms of pipeline and tensor parallelism.

6.3.1 GPipe

GPipe introduced pipeline parallelism. Instead of having every GPU carry the full model, GPipe sliced the model into stages and spread them across devices. Training batches were then split into many smaller "micro-batches" that moved through the stages in a steady stream. While one stage was handling one micro-batch, another stage was already working on the next. This overlapping schedule kept accelerators busy and increased throughput. See figure 6.6.

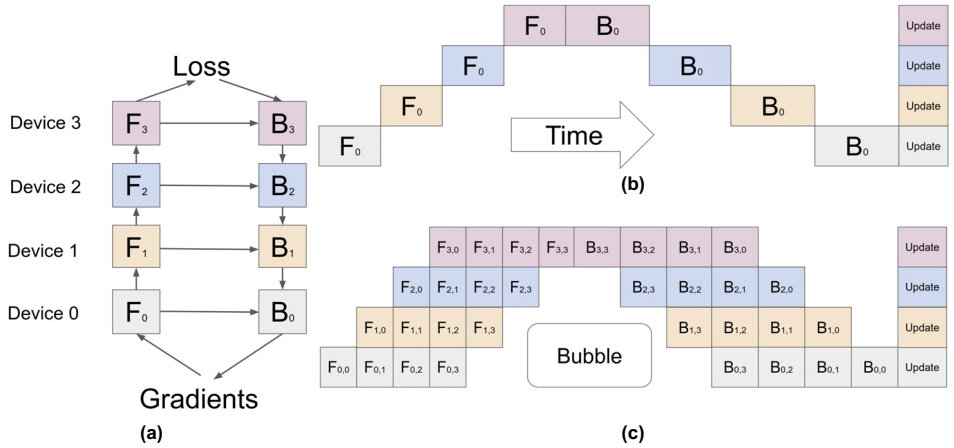


Figure 6.6 Pipeline parallelism compared to naive model parallelism. (a) A simple neural network partitioned across four accelerators, with forward passes F_0 – F_3 flowing upward and backward passes B_0 – B_3 flowing downward to compute gradients. (b) In naive model parallelism, layers are split sequentially across devices. Because each stage must wait for the previous stage to complete, much of the hardware remains idle, leading to low utilization. (c) Pipeline parallelism mitigates this inefficiency by splitting each mini-batch into smaller micro-batches. These micro-batches flow through the pipeline in a staggered fashion, keeping devices busy by processing different micro-batches simultaneously. While idle-time bubbles still appear during the pipeline "fill" and "drain" phases, overall utilization improves significantly. Gradients are accumulated across micro-batches, and updates are applied synchronously at the end of each full batch. *Used with permission granted by the lead author (Yanping Huang).*

GPipe solved two major challenges that earlier model-parallel approaches struggled with. First, it avoided "stale weights." Rather than updating parameters at different times in different places, GPipe accumulated gradients from all the micro-batches and applied a single, synchronized

Licensed to Hermilo Cortés <hermilocg@gmail.com>

update at the end of each full batch. Second, it tackled memory limits through activation checkpointing (i.e., recomputation). Instead of storing every intermediate activation, it recomputed selected activations during backpropagation. This trade-off of a little extra compute for a lot of saved memory, unlocked models that would otherwise be impossible to fit.

The paper "GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism" appears on Sutskever's List and was the first to show researchers that scaling wasn't just about more GPUs or more data. It could also come from rethinking how to organize the work. The results were striking.

6.3.2 Results

The authors evaluated GPipe's performance and scalability on two architectures: a convolutional neural network (AmoebaNet) and a Transformer. The experiments focus on how much GPipe improves training speed as more accelerators are used, how it enables larger models by alleviating memory constraints, and the overhead it incurs (for example, idle time and communication overhead).

The most immediate advantage of pipeline parallelism was its reduced memory usage. Without GPipe, a single GPU with 8 GB of memory could train only a modest AmoebaNet of about 82 million parameters before running out of space. By partitioning the model across eight GPUs, GPipe stretched this limit more than 20-fold, to a 1.8-billion-parameter AmoebaNet.

The gains were even more dramatic for Transformers. On TPU hardware, a single 16 GB device can store approximately 282 million parameters. A tensor processing unit (TPU) is Google's custom-built hardware accelerator designed to run and train artificial neural networks. With GPipe and 128 TPU partitions, the team trained a Transformer with 83.9 billion parameters across 128 layers. This is nearly 300 times the size achievable on a single device.

As mentioned firstly, the authors split the model's parameters across multiple devices. Secondly, they used rematerialization. Together, these techniques enabled models that, at the time, were almost unimaginably large.

Throughput was the other half of the story. Pipelining allows different micro-batches to be in different parts of the network simultaneously, keeping devices busy. The authors demonstrated that as long as the number of micro-batches was at least equal to the number of partitions, idle time (or "bubbles") could be almost eliminated. In practice, this meant that adding

Licensed to Hermilo Cortés <hermilocg@gmail.com>

more accelerators yielded nearly proportional increases in training speed. For the Transformer experiments, an eight-partition pipeline achieved a 6.3× speedup over a single partition, approaching the ideal eightfold speedup. The uniformity of the Transformer layers made it easy to divide work evenly, further improving scaling efficiency.

AmoebaNet, by contrast, scaled less cleanly. Convolutional networks have layers with differing computational costs, making partitions harder to balance. On eight GPUs, the authors observed a 3.5-fold speedup rather than the ideal eightfold. Yet even here, GPipe delivered meaningful gains as long as enough micro-batches were used to keep the pipeline full. But the experiments also highlighted a downside: if only one micro-batch was used per step, there was no pipelining, and all but one accelerator remained idle.

The paper also demonstrated GPipe's ability to achieve state-of-the-art results through its scale. The authors trained AmoebaNet-B, scaling it to 557 million parameters [68]. Partitioned into four stages and trained across four accelerators, the model reached 84.4% top-1 accuracy on ImageNet-2012. At the time, this set a new benchmark for single-model performance on ImageNet.

The team also tested transfer learning. The 557-million-parameter AmoebaNet-B, trained on ImageNet, was fine-tuned on a range of other vision datasets, including CIFAR-10 and CIFAR-100, Stanford Cars, Oxford Pets, Food-101, FGVC Aircraft, and Birdsnap. The large ImageNet-pretrained model consistently outperformed or matched the state of the art, often by comfortable margins. On CIFAR-100, for example, the error rate dropped from 10.7% to 8.7%. These findings reinforced the idea that a better ImageNet model yields better transfer results, and GPipe enabled the training of even better ImageNet models by easing size constraints [69].

The Transformer experiments were extended to multilingual machine translation. The team trained a single Transformer to translate 102 languages into English. The dataset included 25 billion examples and spanned high-resource and low-resource languages. Before GPipe, the model size required for this undertaking would have been too large to train efficiently. With GPipe, the researchers trained several models of increasing size. They began with a baseline Transformer of roughly 400 million parameters, about twice the size of Transformer-big (the largest model introduced in "Attention Is All You Need"), and then scaled further. The other model sizes included a deep 128-layer Transformer with 1.3 billion parameters, a wide 1.3 billion-parameter model with larger

Licensed to Hermilo Cortés <hermilocg@gmail.com>

hidden dimensions but fewer layers, a 3 billion-parameter model, and finally a 6 billion-parameter model that combined depth and width. The results were clear: larger models produced better translation quality across nearly every language pair. The largest improvements occurred when moving from 400 million to 1.3 billion parameters, though gains continued, albeit with diminishing returns, up to 6 billion parameters.

The most striking result was in low-resource languages. While high-resource pairs improved as expected, low-data languages saw "huge quality improvements" compared to bilingual baselines. In these settings, the deep 1.3-billion-parameter model outperformed the wide one, suggesting that additional layers—rather than simply wider ones—provided the representational depth required for more challenging generalization tasks. The GPipe-enabled 6-billion-parameter multilingual Transformer outperformed every set of separate bilingual models across more than 100 language pairs. This demonstrated that, at sufficient scale, a single model could learn more than 100 tasks simultaneously and outperform models specialized for each task.

GPipe did more than demonstrate a clever scheduling trick. It enabled new scientific results, including state-of-the-art ImageNet accuracy, broad improvements in transfer learning, and a multilingual translation model that rewrote expectations of what a single network could achieve. Its technical contributions directly led to empirical breakthroughs. The lesson was that parallelism is not just an engineering concern but a scientific enabler, allowing the community to ask new questions and build models that had previously been out of reach.

6.3.3 Wise to pipeline?

At NeurIPS 2024, Ilya Sutskever looked back on pipeline parallelism with a line that felt like a fragment from Kierkegaard: "Was it wise to pipeline?" he asked. "As we now know, pipelining is not wise. But we were not as wise back then …" [70][71]. A decade earlier, Ilya had pipelined sequence to sequence across eight GPUs, achieving a 3.5× speedup and surpassing the memory limits of a single accelerator. At the time, it felt like clever engineering. In hindsight, he saw short-term gains but a trail of inefficiencies.

The first problem was the code itself. Implementing pipelines meant tearing apart clean training loops and reassembling them with micro-batch scheduling, cross-device communication, and the choreography of forward and backward passes. What sounded simple in theory quickly made the code fragile. Even when they worked, pipelines wasted compute. During the "fill" and "drain" phases of a batch, devices sat idle [72].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Subsequent studies showed that bubbles could consume 15% to 30% of training time, and in some cases, over half of GPU hours were spent waiting [73].

NVIDIA's Megatron-LM (2019) offered an alternative to pipeline parallelism. Instead of dividing layers vertically across devices, Megatron sliced within layers [74]. Matrix multiplications were partitioned across GPUs, producing partial results that were then combined. This "model parallelism" avoided some of the bubble inefficiencies of pipelines and opened the door to training massive transformers with billions of parameters. It was also orthogonal to pipeline parallelism, allowing the two approaches to be combined. For example, eight stages of pipelining across GPUs, with each stage itself distributed in two ways, yielded an effective 16-way parallelism.

Megatron's tensor slicing, however, was still a hand-built recipe for a particular kind of transformer layer. So, a parallel line of work at Google generalized the idea. Mesh-TensorFlow (2018) allowed engineers to decide which tensor axes to shard across which mesh axes. Splitting the batch yields data parallelism. Splitting the vocabulary or feed-forward dimension yields tensor parallelism. Splitting the expert dimension in a mixture-of-experts model yields expert parallelism. Pipeline parallelism differs slightly in that it splits computation stages rather than tensor axes, but it belongs to the same broader mesh family.

The results were dramatic. Megatron-LM trained an 8.3-billion-parameter GPT-2-style transformer on 512 GPUs. It achieved about 76% efficiency and 15 petaflops of throughput. A floating-point operation (FLOP) is a standard measure in large-scale AI training. As a throughput measure, a petaflop is 1 quadrillion FLOPs. That achievement not only proved the feasibility of billion-parameter models but also established the template that GPT-3 would use to scale to thousands of GPUs.

But none of this resolved the fundamental memory problem, because a copy of a 50-billion-parameter model in 16-bit precision requires 100 GB of storage for the parameters alone. This exceeds what a single GPU can provide, even before considering gradients and optimizer states. Parallelism helped distribute computation but did not reduce memory redundancy. Every GPU in a data-parallel setup still carried the full weight of the model.

Memory sharding helped. Microsoft's Zero Redundancy Optimizer (ZeRO) tackled the problem directly by eliminating duplicate storage across data-parallel processes [75]. Instead of each GPU holding the complete set of parameters, gradients, and optimizer states, ZeRO (2019)

Licensed to Hermilo Cortés <hermilocg@gmail.com>

partitioned them across devices. Each GPU stored only a shard, communicating as needed to recreate the illusion of completeness.

With ZeRO, a cluster of 16 GPUs could, in principle, train a model 16 times larger than before, since each GPU carries only a fraction of the memory burden. Microsoft demonstrated training 100-billion-parameter models on just 400 GPUs, reaching 15 PFLOPs of throughput and even showing superlinear scaling. In other words, efficiency improved as more devices were added because parallelism was better balanced.

ZeRO also enabled the training of a 13-billion-parameter model without model parallelism, a feat previously considered impossible. Lifting the memory wall also democratized access. Researchers who found pipeline- or model-parallel code daunting could now train large models with more familiar workflows. Soon after, these techniques powered Turing-NLG, a 17-billion-parameter model, the largest of its time, only to be eclipsed four months later by GPT-3 [76].

Even so, splitting layers came at a price. GPUs had to constantly exchange partial results, and those communication costs mounted. Researchers turned to smarter scheduling. Breadth-First Pipeline Parallelism (2022) reimagined micro-batch flow [77]. Instead of the classic one-forward, one-backward (1F1B) schedule, it traversed the model "layer by layer," processing micro-batches in parallel (that is, "breadth-first") across the earliest stages. This reordering maximized overlap between computation and communication, meshing naturally with fully sharded data parallelism. It achieved up to 43% higher throughput on a 52-billion-parameter model than Megatron's schedule when using small per-GPU batches. The lesson was that better orchestration of the pipeline—not just brute force—could yield real efficiency gains.

Taken together, pipeline staging, tensor slicing, and memory sharding translate scaling laws into practical engineering. GPipe showed how to stagger micro-batches through a pipeline; Megatron demonstrated how to hand-slice transformer layers efficiently, and Mesh-TensorFlow turned sharding into a layout problem over a mesh. ZeRO eliminated redundant storage and raised the memory ceiling. Each breakthrough extended the feasible boundary of model size and performance. Modern AI inherited all these ideas. Together, they turned aspiration into trillion-parameter training runs.

Scaling laws themselves may be simple, but making them real wasn't. Communication had to be carefully overlapped with computation to keep accelerators busy; collective operations needed to be tuned to the microsecond level; and jobs running for weeks had to survive inevitable hardware

Licensed to Hermilo Cortés <hermilocg@gmail.com>

faults and network hiccups [78]. What appeared smooth in scaling-law plots was, in reality, the result of painstaking engineering that kept every device fed with data and resilient to failure.

The consensus is clear: pure pipeline parallelism has been demoted to a reluctant last resort. Modern large-scale training relies on hybrids: shallow pipelines to fit massive models across devices, while most of the heavy lifting is done by data and tensor parallelism. Pipelining is still used only because some models are too large to fit otherwise, and engineers generally avoid it when possible. This is the perspective captured in Sutskever's remark. Pipeline parallelism was clever in its time and helped enable the leap to giant models, but it left behind brittle code, wasted compute, and endless engineering overhead. In hindsight, it was never truly wise, but it was necessary.

6.4 Rounding errors

A 2024 study by Epoch AI estimated that two-thirds of recent gains in language model performance were due to scaling data and compute [79]. While this seems to justify "The Bitter Lesson," we must also remember to ask, "Scale what?" Still, the case for scale was settled. Bigger models trained on more data with more compute reliably get better.

Yet many have something to say about what this means, and scaling has become one of the noisiest conversations in AI. Gwern Branwen and Microsoft researchers reinterpreted it as a hypothesis (or spark) of intelligence, and many enthusiasts say, "This is the worst it will ever be" when it fails [80][81][82][83]. Singularitarians say, "We are making God" [84]. Activists want to freeze systems larger than today's [85]. Much of the noise stems from uneven performance. As a result, some round AI down to a "stochastic parrot" or up to a proto-person primed to replace human thinking [86][87]. Some say it is merely a compressed version of the internet, though it sometimes generates content that was never on the internet [88][89].

Yet scale did not make God or a parrot. Scale provided the closest thing machine learning has ever had to a free lunch, delivering steady progress and turning progress into engineering problems. This aligned with two of Ilya's deepest instincts: (1) pragmatic engineering and (2) doing more with less at scale. Scale is neither divine nor trivial, yet it enabled the most successful period in the field's history. After seventy years, AI has finally gone mainstream, and it's because of scale. Even so, scale did not produce dependable multistep reasoning or planning. That limitation set the stage for a shift away from scaling pretraining toward systems that promised reasoning. That shift is the focus of the next chapter.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

7 The pivot to reasoning

This chapter covers

- Variational lossy autoencoders
- Relation networks on CLEVR and bAbI
- Message passing neural networks on QM9
- Relational memory core for sequential reasoning across benchmarks
- Paper versus living doubts

Papers

- *Variational Lossy Autoencoder* (2016) Chen et al.
- *A Simple Neural Network Module for Relational Reasoning* (2017) Santoro et al.
- *Neural Message Passing for Quantum Chemistry* (2017) Gilmer et al.
- *Relational Recurrent Neural Networks* (2018) Santoro et al.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Rather than scaling to GPT-5 with trillions of parameters, OpenAI invested in techniques such as "test-time compute," which involves models "thinking" more during inference [1]. This pivot reflected a broader realization across the field that scaling pretraining alone was yielding diminishing returns, particularly on tasks requiring reasoning. As a matter of fact, even the loudest champions of scale began discussing its limits. In 2024, Ilya said that "the 2010s were the age of scaling; now we're back in the age of wonder and discovery once again." Sutskever added, "Scaling the right thing matters more now than ever" [2].

This chapter is not about one architecture that "solved" reasoning. It is about bottlenecks, object pairs, graph messages, memory slots, and finally, test-time compute. These are all attempts to improve the computational structure of artificial neural networks. Variational lossy autoencoders (VLAEs) ask what information should be forced into a latent code. Relation networks ask how a model can compare objects. Message passing neural networks (MPNNs) ask how information should move across a graph. Relational memory cores (RMCs) ask how memories can interact over time. Modern reasoning models inherit this same theme: intelligence improves not only when models get larger, but when computation is organized so that comparison, constraint-checking, and abstraction become easier.

7.1 Variational lossy autoencoder

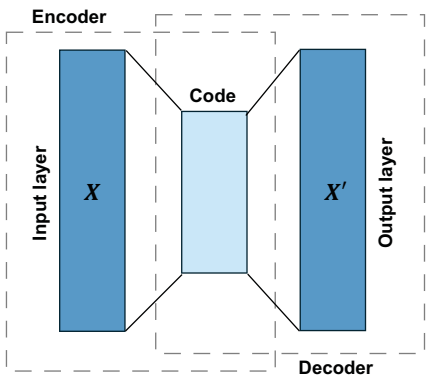


Figure 7.1 An autoencoder consists of an encoder that maps the message to a code and a decoder that reconstructs the message from the code.

Sequence to sequence (seq2seq) models use an encoder-decoder architecture for machine translation (chapter 5). Recall that the encoder maps the input to a latent representation, which the decoder uses to generate a translated sentence. An *autoencoder* keeps the same architecture but changes the training goal. Instead of translating text, it learns to reconstruct its own input. The encoder maps the input data to a low-dimensional representation known as a *latent code*. This latent code could be a vector representation of an image that encodes colors and shapes or a fixed-length sentence embedding for text. The decoder uses that latent code to reconstruct the original input. See figure 7.1.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Since the input data passes through a low-dimensional bottleneck, the model cannot memorize idiosyncratic details. Instead, it must preserve the features needed to reconstruct the input accurately by minimizing a reconstruction objective, $L_{recon} = \sum_i (x_i - \hat{x}_i)^2$, where x is the original input and $\hat{x}$ is the reconstruction produced by the decoder. By reducing the gap between the two, the model learns latent representations that retain the data's essential structure. In the case of handwritten digits, it may learn features such as edges, loops, and stroke orientation, which later prove useful for downstream tasks like classification, even though classification was never part of the training objective.

The latent space of a plain autoencoder (AE) is clustered rather than continuous, meaning that valid latent representations occupy disconnected regions. Consequently, interpolating or sampling between them does not reliably produce meaningful outputs. This is called *off-manifold sampling*. For example, MNIST is a dataset of handwritten digits (0–9) commonly used for training machine learning algorithms. A model trained to reconstruct MNIST will partition the input data into clusters for each digit, but the gaps between these clusters are vast and largely uncharted. See figure 7.2 (left). This is fine for decoding known examples like handwritten digits, but when a decoder must sample across those gaps, it lacks guidance, and performance quickly deteriorates. AEs can compress inputs and uncover meaningful structure, but the way they arrange data in the latent space is clumsy.

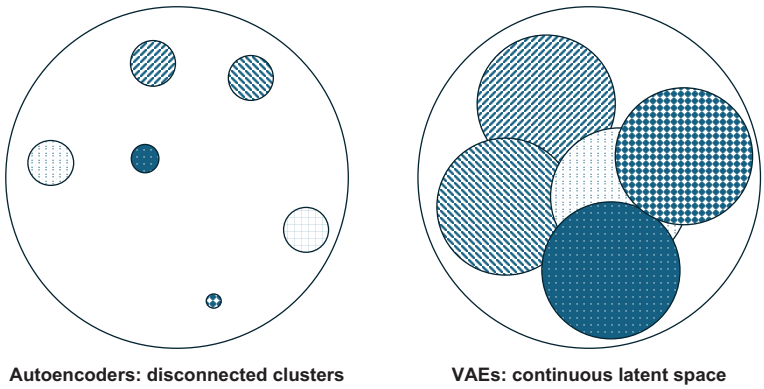


Figure 7.2 Left: plain autoencoders yield disjoint clusters in the latent space, leaving significant gaps. Interpolations must traverse off-manifold regions. Right: VAEs produce continuous, overlapping regions that enable smooth interpolation and reliable sampling.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Variational autoencoders (VAEs) address this off-manifold sampling problem by enforcing continuous, overlapping regions in the latent space, enabling smooth interpolation and reliable sampling. The encoder does not emit a single code. Instead, it predicts the parameters (a mean vector μ and a scale vector σ) of a distribution over codes. During training, a latent vector is drawn from this distribution and passed to the decoder [3]. For sequences, the decoder generates the output token by token, conditioning on the previously generated tokens (the prefix) and the latent sample. Once the latent space has the appropriate geometry, the model becomes a tool for sampling and interpolating between examples. See figure 7.2 (right).

7.1.1 What Is VLAE?

Yet VAEs can fail to use their latent code when the decoder is too powerful. In a VAE, the latent variables are hidden values, usually written as a vector *z*, that the encoder infers from the input, and the decoder is supposed to use as a compressed description of that input. But when the decoder has sufficient capacity to model the data on its own, it may ignore *z*, leaving the latent code to carry little or no information. This phenomenon is known as *posterior collapse*.

The VLAE addresses this problem by controlling what information each part of the model can represent. The latent code is encouraged to store global layout and identity, including high-level information such as which objects are present, their rough shapes, poses, positions, and overall arrangement. It is not meant to store every local texture, edge, or pixel-level detail. Those fine details are handled by an image decoder, such as PixelCNN. The VLAE was introduced by Peter (Xi) Chen, Ilya Sutskever, and colleagues at OpenAI in 2016 and is on Sutskever's List [4].

VLAEs use an *autoregressive prior* over the latent code. A prior is the distribution from which latent variables are sampled before generating data. But instead of a simple factorized Gaussian prior, which assumes independence across all latent dimensions, the authors introduced an autoregressive prior that models the distribution one component at a time, allowing later components to depend on earlier ones. The authors describe this process as the "explicit placement of information." In effect, VLAEs make the latent code responsible for the image's global semantic structure (preventing posterior collapse), while the PixelCNN-style decoder fills in plausible local detail.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

7.1.2 Results

VLAE performed as well as or better than the strongest VAEs of its time and approached the accuracy of leading autoregressive models on benchmarks such as MNIST, Omniglot, and Caltech-101 Silhouettes, while maintaining solid performance on CIFAR-10. On the MNIST benchmark, VLAE achieved a negative log-likelihood of 79.03 nats, about 0.27 nats better than the previous best VAE result. Negative log-likelihood (NLL) is a penalty for being "surprised" by the data. If a model assigns high probability to what actually occurs, the penalty is small; if it assigns low probability, the penalty is large. A nat is the unit used when that penalty is calculated with the natural logarithm. For example, if a model assigns probability 0.5 to the correct outcome, the penalty is $-\log 0.5 = 0.69$ nats. If it assigns probability 0.1, the penalty is $-\log 0.1 = 2.30$ nats. Thus, a lower NLL indicates that the model predicted the data better. A score of 79.03 nats means the model's predictions were slightly less "surprised" by the data than those of its predecessors. Though small, this was a meaningful improvement.

The VLAE scored 90.98 nats on Omniglot, a handwriting corpus comprising over 1,600 characters from dozens of alphabets. It beat all prior models on that dataset. With fine-tuning, the score improved slightly to 89.83. Caltech-101 Silhouettes is a shape-only version of the Caltech-101 object dataset; it emphasizes global structure over texture. On it, VLAE achieved 77.36 nats, dramatically better than the previous best of 88.5 nats. On CIFAR-10, a color image benchmark of ten everyday object classes, VLAE came close to the leading fully autoregressive baseline (PixelCNN++) and differed by only a few bits per dimension. This confirms that the VLAE didn't sacrifice much generative performance despite its latent constraints.

The ability to maintain a usable latent representation without sacrificing performance or quality matters because VLAEs are lossy. They deliberately store only the essentials in the latent code, allowing the decoder to recover local details. On MNIST, the average information carried by the latent code was notably lower than that of a comparable VAE with an unconstrained decoder. Moreover, reconstructions from real images preserved identity and shape without copying every pixel, ensuring that global structure was preserved and that fine textures were plausibly filled in. Samples and reconstructions were less blurry and more coherent overall, narrowing the gap with competing architectures and pure autoregressive models [5]. Traditional VAEs lack this component, leading to blurry outputs. See figure 7.3.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

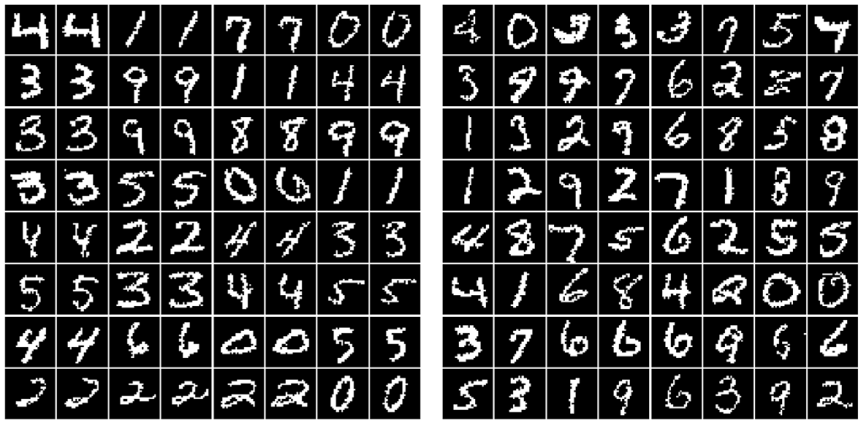


Figure 7.3 Original MNIST images (left) and the "decompressed" versions from VLAE's lossy code (right). ***Used with permission granted by the lead author (Peter (Xi) Chen).***

7.1.3 *Autoregressive prior*

The VLAE attacked the posterior collapse problem from two angles. First, it weakened the decoder by giving it a small receptive field, forcing it to attend only to local details. Second, the authors strengthened the prior by using an autoregressive flow that iteratively transforms Gaussian noise, so that each latent variable depends on its predecessors. The result is a prior that adapts to the true distribution of the encoded latent variables, rather than imposing the strict independence assumption that defines a factorized Gaussian. The authors made this point empirically by comparing three setups: (1) a small receptive field (RF) PixelCNN with no prior; (2) a small RF PixelCNN with a Gaussian prior; and (3) a small RF PixelCNN with an autoregressive prior. See table 7.1.

Table 7.1 Same small-receptive-field PixelCNN with different priors

Model	NLL (nats)
PixelCNN decoder	87.55
PixelCNN decoder + Gaussian prior	79.48
PixelCNN decoder + autoregressive flow prior	78.94

Even a simple Gaussian prior significantly improves performance relative to an unconditioned decoder. The NLL decreases from 87.55 to 79.48. This demonstrates that the decoder operates on the latent code. Replacing the Gaussian prior with an autoregressive prior further improves

Licensed to Hermilo Cortés <hermilocg@gmail.com>

performance and, more importantly, means that the transmitted latents contain more information [6]. VLAE crystallized the emerging view that information flow in VAEs is governed as much by the prior as by the decoder, and it demonstrated this with a clean ablation.

The discussion of AEs, VAEs, and VLAEs can feel stilted. So far, we've discussed how to prevent off-manifold sampling, how to keep a decoder from steamrolling the latent code with an autoregressive prior, and how to read a few nats of improvement on the negative log-likelihood as real progress. This is technical, and it may be hard to see what it's all about. Still, it shows that architectures can decide which information to remember, which to discard, and which to make reusable within the model. Moreover, VLAEs show that architecture can decide where that information lives. Modern neural systems show that information can live in weights, memory, a scratchpad, a graph, or a test-time search.

7.2 Relational reasoning

By 2017, deep learning had shown remarkable performance in recognizing patterns in images and text but stumbled when answers depended on relationships among entities. Earlier attempts to solve this problem fell into two distinct camps. Symbolic systems represented entities and relations using rules, making reasoning explicit, but they were brittle and handcrafted. Artificial neural networks (ANNs) learn from data and are flexible, but they often fail to recognize relational structure when it is sparse and compositional [7]. An ANN could recognize a red cube or a green sphere but fail when asked which was larger. Scaling data and parameters improved recognition but not relational understanding.

"A Simple Neural Network Module for Relational Reasoning" by Adam Santoro and colleagues at Google DeepMind showed that a relational network (RN) could solve tasks that had stumped previous architectures [8]. RNs compute interactions for each pair of objects in an image or each pair of entities in a story and aggregate them to answer queries about those relationships. See figure 7.4.

Consider the question: "What size is the cylinder that is left of the brown metal thing that is left of the big sphere?" A standard convolutional neural network (CNN; chapters 2 and 3) can detect local visual features, but the answer depends on a chain of relations. The RN forms pairs of candidate objects. One pair may provide evidence for "brown metal object left of big sphere." Another pair may provide evidence for "cylinder left of that brown metal object." The network does not explicitly write a symbolic program, but the pairwise function (g_θ) scores all object pairs in the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

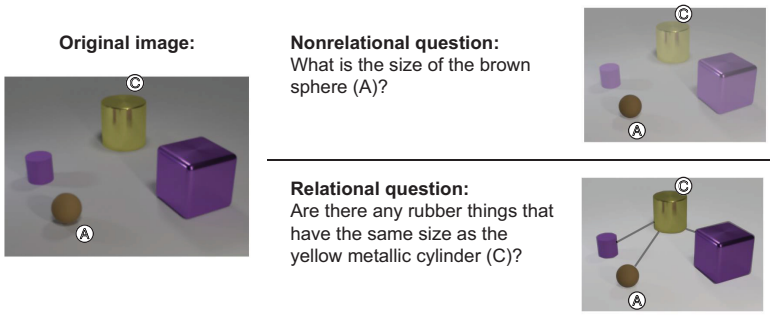


Figure 7.4 Image from the CLEVR dataset showing four objects with paired questions: one nonrelational (about an object's attributes) and one relational (about how the objects relate). Note: Letters were added to the original figure to aid comprehension in black-and-white printing. *Used with permission granted by the lead author (Adam Santoro).*

context of the question. When those pairwise scores are summed, the relevant relational evidence accumulates. The final network (f_φ) maps the accumulated evidence to the answer "small." See figure 7.5.

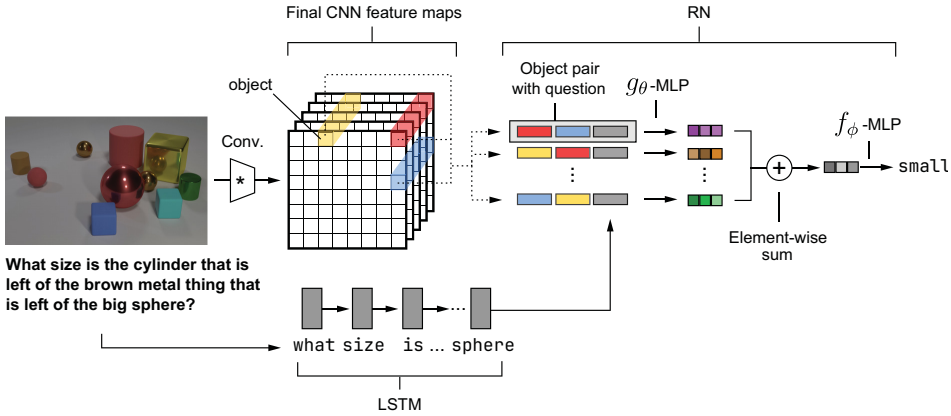


Figure 7.5 An image from CLEVR is paired with a reasoning question. A CNN extracts object features, a long short-term memory encodes the question, and the RN compares every object pair in light of that question. The results are aggregated and passed to another network, which outputs the answer (here, "small"). Note: Letters were added to the original figure to aid comprehension in black-and-white printing. *Used with permission granted by the lead author (Adam Santoro).*

7.2.1 Benchmark evaluation

To test whether the RN captures the ability to reason about relations rather than merely memorizing patterns, the authors evaluated it across different settings, each probing a distinct facet of reasoning.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

CLEVR

CLEVR is a synthetic visual question-answering benchmark featuring 100,000 rendered three-dimensional scenes and approximately 1 million detailed questions. As one might expect, it is designed to evaluate compositional language understanding and basic visual reasoning, specifically how well models reason about object properties, relationships, and comparisons [9]. On CLEVR, the RN achieved 95.5% accuracy, outperforming both the best CNN + long short-term memory (LSTM)-based baselines of the time (about 68.5%) and human performance (about 92.6%). The most significant gains appeared in Compare Attribute, Compare Numbers, and Count, which depend most directly on strong relational reasoning. See table 7.2.

Table 7.2 CLEVR test accuracy by question type

Model	Overall	Count	Exist	Compare numbers	Query attribute	Compare attribute
Human	92.6	86.7	96.6	86.5	95.0	96.0
Q-type baseline	41.8	34.6	41.7	43.7	52.2	64.4
LSTM	46.8	50.2	61.1	65.2	71.1	82.7
CNN+LSTM	52.3	51.0	69.8	67.1	73.5	77.4
CNN+LSTM+SA	68.5	36.0	36.8	49.3	85.3	82.6
CNN+LSTM+SA*	76.6	51.3	51.8	53.0	52.3	75.4
CNN+LSTM+RN	95.5	90.1	97.8	93.6	97.9	97.1

*Optimized hyperparameters and trained fully end-to-end

STATE DESCRIPTIONS

The authors also replaced the images in CLEVR with state descriptions. State descriptions are structured, symbolic tables in which each row explicitly lists an object's properties, including its position (x, y, z), color (r, g, b), shape, material, and size. With these symbolic inputs, the RN achieved 96.4% accuracy (not shown in table 7.2). This confirmed that the relational module could reason over abstract, factored representations of a scene, not just raw pixels.

BABI

bAbI is a text-based question answering dataset [10]. There are 20 tasks, each related to a specific type of reasoning, such as deduction, induction, or counting. Each question has a set of supporting facts. For example, the facts "Sandra picked up the football" and "Sandra went to the office"

Licensed to Hermilo Cortés <hermilocg@gmail.com>

support the question "Where is the football?" The answer is "the office." A model succeeds at a task if its performance exceeds 95%. An RN passed 18 of the 20 bAbI tasks, including the challenging induction task that had previously stumped earlier memory-augmented models such as DeepMind's Neural Turing Machine (chapter 5). The two remaining tasks fell short by only a few points.

SORT-OF-CLEVR AND PHYSICAL TASKS

Sort-of-CLEVR simplified CLEVR to two-dimensional colored shapes so the authors could isolate relational from nonrelational questions. A plain, off-the-shelf CNN could answer basic, nonrelational questions but stalled at around 63% on relational questions. Adding an RN closed the gap, with accuracy rising above 94% on both tasks. This result shows that standard networks lack an innate ability to reason about relations without a dedicated module.

The authors also developed a dataset of colored balls moving on a tabletop. Some of the balls moved independently and were free to collide with one another and with the barrier walls. Other randomly selected pairs of balls were connected by invisible springs or a rigid constraint. The RN inferred hidden connections among the bouncing balls with 93% accuracy and identified connected components with 95% accuracy, while a size-matched network performed at chance levels.

Together, these different settings showed that the RN was not tied to any input type. It provided a general relational operator that could be integrated into standard neural pipelines and immediately reveal whether a network understood relations or merely patterns. The RN's success also showed that, when the "right" architecture is in place, a network can infer pairwise comparisons of abstract relationships [11]. It sits alongside several other approaches that, around the same time, pushed toward structured reasoning [12][13].

7.3 Neural message passing for quantum chemistry

Modern chemistry and materials science operate under an extreme combinatorial constraint. The space of possible molecules is vast, with estimates of 10^{60} distinct molecules in chemical space [14][15]. This is far more than could ever be enumerated experimentally. Consequently, chemists rely on virtual screening using computer-based methods to narrow the search to a tractable set of promising compounds for follow-up study. This approach is explicit in widely used quantum-chemistry benchmarks, which require computational exploration because unbiased exploration of chemical compound space is otherwise infeasible [16].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Traditionally, this meant physics-based simulations. But by 2017, it had come to mean machine learning (ML).

This shift was driven by two forces. First, there was a growing supply of large, well-curated molecular datasets. Second, ML had become accurate enough to approximate properties that once required expensive simulation or laboratory work. For example, ML can predict whether a molecule is toxic to humans, whether it binds to a particular protein (the core problem in drug discovery), or estimate quantum properties used in materials design. The value proposition was not perfect accuracy but good accuracy with much faster performance, allowing practitioners to explore and filter far more chemical space than brute-force methods.

Against this backdrop, Justin Gilmer and colleagues at Google Brain and DeepMind introduced a general approach for learning directly from molecular data. Their method (MPNNs) contrasts with feature engineering in chemistry, which relies on hand-engineered molecular descriptors. Molecular descriptors are the SIFT/HOG of chemistry (chapter 2). They are domain-expert artifacts that remained effective until representation learning rendered them obsolete. The paper, titled "Neural Message Passing for Quantum Chemistry," is on Sutskever's List [17].

7.3.1 Accuracy as a bottleneck

A central motivation for MPNNs is the prohibitive cost of quantum chemistry. Quantum mechanics provides equations that, in theory, predict how molecules behave. But those equations are so hard to solve that researchers do not solve them directly for real molecules. Density functional theory (DFT) offers chemists a shortcut.

DFT is a numerical method for approximating many quantum properties of molecules. One of the inventors associated with the method won the Nobel Prize in Chemistry for this work, and DFT is responsible for two of the top 10 most-cited papers of all time [18][19]. It's widely used because it strikes a good balance between speed and accuracy, but the key point is that it's still too slow for large-scale searches. As Gilmer and colleagues note, a single DFT calculation for a small organic molecule can take about an hour on a single CPU core, and molecules with just a handful more atoms can take many hours due to the cubic scaling of DFT with respect to the number of electrons.

This quickly becomes prohibitive when the task is not to evaluate a single molecule but to explore chemical space. Brute-force screening of hundreds of thousands or millions of candidate molecules is common in drug discovery or materials design, but it would require years of computing

Licensed to Hermilo Cortés <hermilocg@gmail.com>

time if performed directly using DFT. The scientific questions we seek to address are combinatorial and exploratory, yet the tools we rely on are slow and serial.

7.3.2 Message passing neural networks

Molecules are not images or sequences. While CNNs operate on grids, such as images, and recurrent neural networks (RNNs) operate on sequences, such as text, molecules are more naturally represented as graphs, where atoms are nodes, edges are bonds that carry spatial distance information, and three-dimensional orientation is encoded in pairwise relationships. MPNNs provide a framework for supervised learning on graph data that extends earlier graph neural network models, such as the gated graph neural network (GGNN), which served as a baseline [20].

In an MPNN, each node in a graph maintains a hidden state vector that is updated iteratively by exchanging messages. Think of each atom as sharing notes with its neighbors. In the message-passing phase, each atom sends a learned message to its neighbors and updates its hidden state based on the messages it receives. After many rounds, each atom's state summarizes its local neighborhood. In the readout phase, the model aggregates all atom states through a pooling operation into a single vector to predict a molecular-level property (e.g., energy, dipole, gap). The goal is for these node representations to capture the structure and features of the entire graph, enabling accurate graph-level predictions. A useful mental model is that each atom transmits information to its neighbors. Over successive iterations, the molecule settles into a learned summary of itself.

One challenge in graph networks is efficiently propagating information across nodes, particularly when the graph is large or sparsely connected. If important information resides on a node far away from another node (in graph distance), it may take many rounds of message passing for that information to reach it. In effect, it's a communication problem. The authors explored virtual edges and master nodes to speed up this process and improve its fidelity. The idea of "virtual" edges is to add a special kind of edge between every pair of nodes that aren't originally connected. In effect, you temporarily make the graph fully connected by adding virtual connections that carry messages, with distinct labels or weights, so that the model knows these are not true physical edges but long-range shortcuts.

The benefit is that information can travel long distances in fewer steps. The model can learn to use these virtual edges or ignore them as needed. Even if two atoms in a molecule are not directly bonded, the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

network can still transmit messages between them via a specialized virtual channel, enabling global communication. This is akin to a skip connection (chapter 3), but instead of a shortcut across depth, virtual edges provide a shortcut across a node's representation. Think of residual connections as adding an elevator between floors (layers) and virtual edges as adding new hallways between distant rooms (nodes).

Another approach to facilitate long-range interaction is to introduce an additional artificial node called a "master" node. You can think of it as a hub that collects information from all nodes and then broadcasts it back. In MPNNs, the master node serves as a global scratch space that captures high-level aggregated information across the entire graph and makes it available to all nodes in the next step. In this regard, the master node approach is similar to a Neural Turing Machine.

After several rounds of message-passing, an MPNN must produce an output. In many applications (such as quantum chemistry), the desired output is a property of the entire graph (for example, a molecule's energy or stability), not just a single node. So, we need a readout function that combines the final states of all nodes into a single, fixed-size, graph-level representation. The authors experimented with two types of readout functions.

The simplest approach, inherited from the GGNN baseline, is to sum (or average) the final hidden vectors of all nodes and then apply a feed-forward network to the resulting vector. Summing is convenient because it's invariant to node order. That is, nodes can be listed in any order, yet the sum remains the same, preserving the graph's identity. But a sum may lose information, and different distributions can yield the same sum. The GGNN's readout may mitigate this by applying nonlinear transformations before or after summing (the original paper specifies an equation for this), but it still produces a relatively coarse aggregation.

To capture more complex properties of the set of node states, the authors turned to the Read-Process-and-Write (RPW) model introduced in the "Order Matters" paper (chapter 5). Recall that RPW is an extension of the seq2seq model (chapter 5) that uses a recurrent neural network with an attention mechanism to iteratively attend to different nodes, building a representation of unordered sets, which explains why Gilmer et al. call the model set2set or s2s. In the context of MPNNs, the set is the collection of final node feature vectors. In theory, this readout is more expressive than a simple sum, and it can learn to assign appropriate weights to important nodes or combinations of node features. Gilmer and colleagues expected the model to yield better performance by allowing

Licensed to Hermilo Cortés <hermilocg@gmail.com>

the network to capture complex, global graph properties that a simple permutation-invariant sum might miss, and they were correct.

Take ethanol as a toy example. At the start, each atom has only local features such as atom type, bond type, and perhaps distance information. After one message-passing step, the oxygen knows about its bonded carbon and hydrogen, and each carbon knows about its immediate neighbors. After two steps, the first carbon can receive indirect information about the oxygen through the middle carbon. After several steps, each atom's hidden state becomes a learned summary of its chemical neighborhood. The readout function then pools the atom states into a molecular-level representation, which can be used to predict properties such as energy, dipole moment, or the electronic gap. The model has not solved quantum mechanics. Instead, it has learned a fast graph-based approximation to the answers produced by expensive quantum chemistry calculations.

7.3.3 Training and results

The QM9 dataset provides DFT-computed properties for approximately 134,000 small molecules. Each molecule in QM9 has up to nine heavy atoms (e.g., carbon, nitrogen, oxygen, or fluorine) and 13 DFT-computed targets. The authors randomly tried 50 different hyperparameter training settings and kept the best-performing ones. All models were trained using the same general recipe, including small batches, a long training run, and a gradually decreasing learning rate. The authors split the QM9 dataset, setting aside 10,000 molecules for validation to determine when to stop training and which model version to retain, and another 10,000 for the final test. Everything else was used for training.

To achieve strong performance, the authors systematically evaluated MPNN variants and input representations. Two choices that mattered most were including bond type plus spatial distance and treating hydrogen atoms as explicit nodes in the molecular graph. The authors also found that one-property-per-model training beat multitask training by up to 40%. The strongest single model used an edge-network message function with explicit hydrogens. Test accuracy improved further by ensembling (averaging) the predictions of the five models with the lowest validation error. The top-performing single model ("enn-s2s") and its five-model ensemble ("enn-s2s-ens5") set new best results across all 13 targets [21]. The authors achieved chemical accuracy on 11 targets, outperforming five hand-engineered descriptor pipelines and two prior graph neural networks. See table 7.3. Chemical accuracy is achieved when the mean

Licensed to Hermilo Cortés <hermilocg@gmail.com>

absolute error for each target falls below the chemistry-standard threshold. Most strikingly, the authors emphasize that neural message-passing can perform inference in milliseconds rather than hours.

Table 7.3 QM9 benchmark comparison across 13 molecular targets. Rows list the predicted properties: dipole moment (mu), polarizability (alpha), HOMO, LUMO, band gap, electronic spatial extent (R2), zero-point vibrational energy (ZPVE), atomization-energy targets (U_0), U, H, G, heat capacity (Cv), and highest vibrational frequency (Omega). Columns compare hand-engineered descriptor methods, prior graph-based baselines, and the paper's best message-passing models. Each cell reports error as a ratio to the target's chemical-accuracy threshold, so values below 1.0 indicate chemical accuracy. The edge-network ("enn") + set2set ("s2s") model ("enn-s2s") with explicit hydrogens is the best single model across all targets, and the five-model ensemble ("enn-s2s-ens5") further improves performance.

Target	BAML	BOB	CM	ECFP4	HDAD	GC	GG-NN	DTNN	enn-s2s	enn-s2s-ens5
mu	4.34	4.23	4.49	4.82	3.34	0.70	1.22	-	0.30	0.20
alpha	3.01	2.98	4.33	34.54	1.75	2.27	1.55	-	0.92	0.68
HOMO	2.20	2.20	3.09	2.89	1.54	1.18	1.17	-	0.99	0.74
LUMO	2.76	2.74	4.26	3.10	1.96	1.10	1.08	-	0.87	0.65
gap	3.28	3.41	5.32	3.86	2.49	1.78	1.70	-	1.60	1.23
R2	3.25	0.80	2.83	90.68	1.35	4.73	3.99	-	0.15	0.14
ZPVE	3.31	3.40	4.80	241.58	1.91	9.75	2.52	-	1.27	1.10
U0	1.21	1.43	2.98	85.01	0.58	3.02	0.83	-	0.45	0.33
U	1.22	1.44	2.99	85.59	0.59	3.16	0.86	-	0.45	0.34
H	1.22	1.44	2.99	86.21	0.59	3.19	0.81	-	0.39	0.30
G	1.20	1.42	2.97	78.36	0.59	2.95	0.78	0.84	0.44	0.34
Cv	1.64	1.83	2.36	30.29	0.88	1.45	1.19	-	0.80	0.62
Omega	0.27	0.35	1.32	1.47	0.34	0.32	0.53	-	0.19	0.15
Average	2.17	2.08	3.37	53.97	1.35	2.59	1.36	-	0.68	0.52

7.3.4 Towers architecture

Scaling graph networks encounters a fundamental bottleneck. When graph networks are dense or their internal representations are wide, a single message-passing step can require many computations, slowing both training and inference. If every node communicates with every other node using wide hidden states, the cost balloons rapidly. To ease this burden, the authors split each node's large hidden representation into several smaller "towers." Each tower performs a message-passing step with its own lightweight parameters, producing a temporary update. Afterward, a small

Licensed to Hermilo Cortés <hermilocg@gmail.com>

feedforward network mixes and recombines the tower outputs into a single representation for each node. This behaves like several narrow graph neural networks working in parallel, with a brief meeting between rounds.

The payoff is efficiency. Most of the heavy computation occurs within each slice, so adding more towers yields a roughly proportional speedup on the expensive part of the step, with only a small overhead for mixing. This design preserves the usual permutation invariance of graph models while allowing information to flow across towers during the mixing stage. It also improves efficiency by using several narrow message passes, and a brief mix is cheaper than a single very wide pass.

In their QM9 setting, using eight towers for a 200-dimensional state yielded roughly a 2× speedup with no meaningful loss of accuracy. The authors note that this approach should be especially beneficial for larger molecules (for example, from GDB-17), where the computational savings compound. The structure also served as a mild regularizer. Because each tower learns slightly different views and they are periodically mixed together, the model behaved a bit like an ensemble and generalized slightly better than a single wide network of the same total size.

While the chemistry application is specialized, the lesson is general. MPNNs showed that when the world is relational and unordered, the model should compute over entities and links rather than be forced into an ordered grid or sequence. Molecules are one example, but the same abstraction appears in knowledge graphs, social networks, program analysis, physical simulation, robotics, and multiagent systems. The paper's importance is not merely that it sped up molecular prediction. It helped normalize the idea that artificial neural networks could reason by passing learned messages over an unordered world model.

7.3.5 Influence

Using the QM9 molecular dataset, the authors showed that an MPNN could predict quantum-mechanical properties of molecules with nearly the same accuracy as costly density functional theory (DFT) calculations. Crucially, it achieved this accuracy much faster than running the quantum simulations. A DFT calculation might take ~10^3 seconds, while an MPNN can produce a prediction in ~10^{-2} seconds. That ratio is ~10^5, or 100,000 times faster. But the paper notes that MPNN inference is about 300,000 times faster than DFT computation, suggesting that actual DFT calculations are closer to 3-4 × 10^3 seconds.

In practice, that is often more game-changing than a small boost in accuracy. By delivering high accuracy at high speed, MPNNs enabled new

Licensed to Hermilo Cortés <hermilocg@gmail.com>

workflows in which researchers could screen vast molecular libraries and iterate experiments in a fraction of the time. This emphasis on speed, along with accuracy, foreshadowed how AI would transform academic research by enabling rapid experimentation that was previously infeasible.

The success of the original MPNN prompted a wave of improved models that retained its core architecture but incorporated additional physics knowledge. SchNet (2017) was among the first models to achieve chemical accuracy on the QM9 dataset, with greater speed and accuracy [22]. It introduced continuous-filter convolutional layers to better capture smooth, distance-based interactions between atoms and added residual connections (chapter 3) to preserve information during message passing.

Building on this, DimeNet incorporated angular information by modeling bond angles [23]. In DimeNet, messages aren't just passed along bonds but also among triplets of atoms to account for bond angles, using techniques like spherical harmonics to encode these angles. These physics-inspired innovations allowed DimeNet to outperform earlier graph-based artificial neural networks. For example, DimeNet reduced prediction errors on QM9 by about 31% compared to prior graph networks. In short, by infusing geometric and physical context (e.g., distances and angles), successors to MPNNs achieved higher accuracy than the original MPNN on benchmark tasks such as QM9, while retaining the efficiency advantages of the message-passing framework.

The message-passing framework popularized by the original 2017 MPNN paper has proven remarkably influential. It showed that graphs could be handled with the same end-to-end learning approach that revolutionized images and text. Since then, pure message-passing gated graph neural networks are no longer the state of the art across all problems. Still, the interface introduced by MPNNs underpins most of today's leading graph models. In fact, many of the latest "graph Transformers" or geometric deep learning models can be viewed as variations on the MPNN theme. They might replace or augment the message function with attention or physics equations. Yet, they are still about nodes communicating information over edges and aggregating it in an order-invariant way.

7.4 Relational recurrent neural networks

Standard RNNs, such as LSTMs (chapter 4), and even advanced memory-augmented models, including Neural Turing Machines (chapter 5), are adept at storing information over time, but they lack the ability to reason about the relations within that information. In "Relational Recurrent Neural Networks," Adam Santoro and colleagues at DeepMind observed that

Licensed to Hermilo Cortés <hermilocg@gmail.com>

classic memory-based networks struggle with tasks that emphasize these relations. Good memory is not just storage and retrieval; it is comparison. They introduced the *relational memory core* (RMC), an RNN that allows its internal "memories" to interact with one another. Instead of treating memory as a single monolithic hidden state, it maintains multiple memory slots holding different pieces of information and lets them communicate with one another at each time step.

In the RMC, memory-to-memory communication is achieved through Transformer-style multihead dot-product attention (chapter 5). In simple terms, attention lets the model dynamically focus on different parts of its data. The RMC uses multiple attention heads in parallel to capture different relationships simultaneously. Each memory slot in the RMC produces an attention query (Q), and all slots produce keys (K) and values (V). By matching queries against keys via dot products, the RMC determines which other memory slots are most relevant and then reads the corresponding values. See figure 7.6.

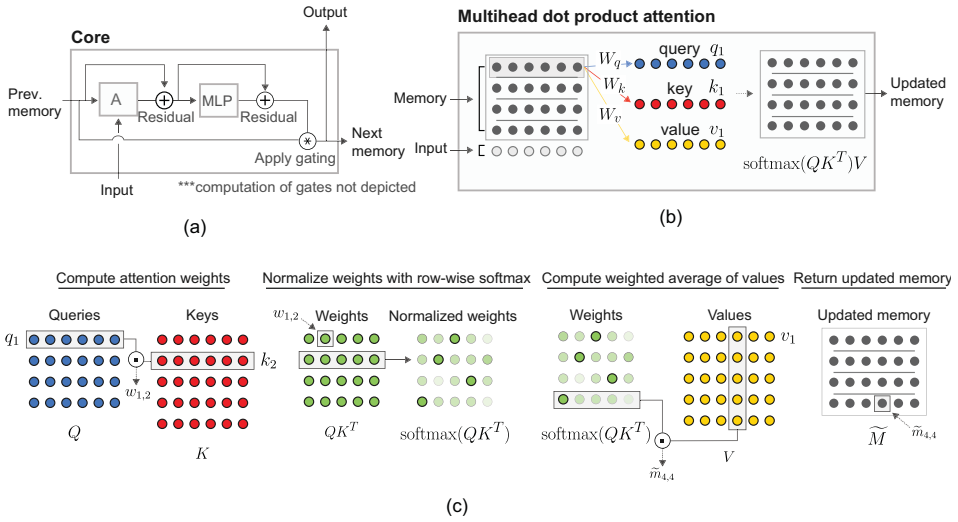


Figure 7.6 (a) The RMC takes the previous memory matrix and the current input and applies a multihead dot-product attention (MHDPA) module labeled A, allowing memory slots to exchange information with one another and with the input. It then refines each slot with a small feed-forward network (MLP) and writes the result through a gate to produce the output for the step and the next memory state. (b) For each memory slot and the input, the model forms *queries, keys,* and *values*, enabling multiple "heads" to view relations from different perspectives. (c) Similarity between *queries* and *keys* yields normalized attention weights, which are used to compute weighted combinations of the values. The results update the memory matrix so that each slot integrates the most relevant information from others. The MLP and gate then finalize what is retained for the next step. *Used with permission granted by the lead author (Adam Santoro).*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

To see the RMC in motion, imagine the nth farthest task. The model receives a sequence of vectors, then must answer a question such as "Which vector is second farthest from vector M?" At each time step, the new vector is combined with the previous memory matrix. The attention module forms queries, keys, and values from the memory slots and the current input. A slot that has learned to track the reference vector can attend to candidate vectors; another slot can accumulate evidence about relative distance or rank. The attention output updates the memory slots, and the gate decides how much old information to retain. By the end of the sequence, the model has not merely stored the vectors—it has repeatedly compared them inside memory.

This design explicitly encourages memories to interact and share information, a crucial step for relational reasoning. The RMC incorporates these interactions within a single time step, unlike traditional RNN attention, which looks only at previous time steps. After the attention phase, the RMC applies a small feed-forward network to each memory slot's updated content and uses gating to blend the new information with the old. In effect, the RMC extends an LSTM-like backbone with a self-attention mechanism among its memory slots, giving the model an innate ability to reason about relationships between remembered items. The result is a recurrent architecture that not only stores and retrieves information but also understands how those pieces of information relate to each other to solve complex problems.

The RMC has roots in external-memory systems. Neural Turing Machines (NTMs) showed that a controller with differentiable memory can learn algorithmic tasks, such as copying, sorting, and associative recall, better than plain LSTMs can. But NTMs read from and write to isolated cells that don't cross-reference each other. RMC keeps the memory-matrix idea while upgrading the interface by allowing slots to interact in parallel through multihead dot-product attention. In effect, the controller and the memory collaborate, rather than the controller doing all the work.

Two practical notes. First, RMC introduces an explicit capacity dial, including the number and width of slots. If there are too few slots, a model bottleneck occurs. If there are too many, computation is wasted, or the model may overfit. Second, slot-to-slot attention is stable in practice using the usual tricks, such as residual connections, layer normalization, and careful weight initialization. The net effect is that RMC provides a middle path. It is far more relational than an LSTM, more compute-predictable than full Transformer attention, and especially attractive for streaming, algorithmic, and relational workloads where comparison is the currency of progress.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

7.4.1 Experimental methodology and empirical results

The authors demonstrated the utility of the RMC on a variety of challenging tasks that demand relational reasoning over sequences. One illustrative example is the nth farthest problem, a synthetic task designed to elicit relational reasoning over time. The model is given a sequence of vectors, and at the end it is asked a question like, "Which vector was the nth farthest from vector M?" For example, "Which item is the second farthest from item X?" Solving this is nontrivial, because the network must effectively compute all pairwise distances between the reference vector (M) and the others, rank them, and identify the Nth farthest.

Memorizing the sequence isn't enough; the model must relate the items by comparing distances. Traditional memory networks struggled with this. In fact, the authors found that a standard LSTM and even a differentiable neural computer (DNC)—an upgraded Neural Turing Machine—failed to achieve accuracy much above chance [24]. They stalled below 30% accuracy on this task.

The RMC, by contrast, excelled thanks to its relational memory slots and attention. It learned to sort out those distances and achieved about 91% accuracy on the same task. This stark difference highlights how enabling explicit relational reasoning in the recurrent model can solve tasks previously beyond the reach of such architectures. The RMC maintained strong performance even when the task was made harder using higher-dimensional vectors, which demanded higher-fidelity memory. That said, extremely high accuracy was a bit less consistent in those harder settings. Still, its success on the Nth farthest task was strong evidence that the network was reasoning about relationships (that is, distances among items) over time, not just storing and recalling data.

The benefits of relational memory also appeared in program-evaluation benchmarks. The authors evaluated the RMC on tasks derived from "Learning to Execute," the 2014 work by Zaremba and Sutskever that trained recurrent networks to read short programs and predict their outputs character by character [25]. These tasks test whether a sequence model can perform simple symbolic operations such as addition, tracking, copying, and reversing sequences. In the RMC paper, the model matched or exceeded strong baselines, including LSTMs, DNCs, and an LSTM resembling the Recurrent Entity Network (EntNet) [26]. It achieved near-perfect per-character accuracy on simpler tasks such as addition, copying, and reversing, and it substantially outperformed the baselines on the harder full-program task, though that task remained far from solved. The point is not that the model executed every program

Licensed to Hermilo Cortés <hermilocg@gmail.com>

perfectly but that a single recurrent core with relational memory could handle a range of symbolic sequence tasks without task-specific handcrafting. See table 7.4.

Table 7.4 Accuracy on program evaluation and memorization tasks

Model	Add	Control	Program	Copy	Reverse	Double
LSTM	99.8	97.4	66.1	99.8	99.7	99.7
EntNet	98.4	98.0	73.4	91.8	100.0	62.3
DNC	99.4	83.8	69.5	100.0	100.0	100.0
RMC	99.9	99.6	79.0	100.0	100.0	99.8

There was only one subtask ("double memorization") in which a baseline matched or surpassed the RMC, but both models essentially solved the task. Notably, the RMC did so without relying on a teacher to feed it ground truth during training. Instead, the RMC was trained in a purely sequential, autoregressive manner, which the authors suggest may have helped generalization on these program-execution tasks. The takeaway is that even in scenarios involving logical instructions, injecting relational reasoning into the memory core yields equal or better performance because the model can more easily track relationships among variables, conditions, and sequential states.

In reinforcement learning, the RMC also delivered improvements. The authors experimented with a partially observable version of the classic game Mini-Pacman to test the RMC's ability to remember and integrate information over time. Mini-Pacman is a simplified version of the 1980s arcade game Pac-Man, in which an agent navigates a maze, collects pellets for points, and avoids ghosts that end the game if they touch the agent. The setup provides a compact world with rewards, penalties, and moving adversaries, making it ideal for studying learning and memory. In Mini-Pacman, the agent has a limited field of view (a viewport) and must rely on memory to recall where pellets, walls, or ghosts are, especially after they move out of sight. An LSTM-based agent can remember some information, but an RMC-based agent can also infer relationships. For example, it might model the spatial relationship of a moving ghost to a recently seen pellet. In the experiments, the RMC agent achieved a higher score than the LSTM-based agent, scoring 677 versus 550, or 127 points more, in the limited-view setting.

When allowed to observe the full maze, the RMC's advantage grew even more. In fact, it nearly doubled the LSTM agent's score (1159 versus

Licensed to Hermilo Cortés <hermilocg@gmail.com>

598). This suggests that the RMC's relational memory was instrumental in making sense of the game's layout and events over time and in piecing together information such as where Pac-Man last saw a ghost and where it might be now, more effectively than a standard recurrent memory. While Mini-Pacman is a simplified environment (5×5 viewport), these results hint that relational recurrent networks could offer benefits in complex, partially observable worlds, where understanding relationships between past events is key to making good decisions.

Lastly, the RMC was evaluated on standard language modeling benchmarks, including WikiText-103 (a large Wikipedia corpus), Project Gutenberg, and GigaWord. Language modeling is the task of predicting the next word in a sequence. Performance is reported as perplexity, which is a rescaled version of negative log-likelihood (NLL). Like NLL, the lower the perplexity, the less "surprised" the model is by the next token, meaning it assigns higher probability to the correct words.

Table 7.5 Validation and test perplexities for WikiText-103, Project Gutenberg, and GigaWord

Model	WikiText-103 Valid	WikiText-103 Test	Gutenberg Valid	Gutenberg Test	GigaWord Test
LSTM [a]	–	48.7	–	–	–
Temporal CNN[b]	–	45.2	–	–	–
Gated CNN[c]	–	37.2	–	–	–
LSTM[d]	34.1	34.3	41.8	45.5	43.7
Quasi-RNN[e]	32	33	–	–	–
RMC	30.8	31.6	39.2	42.0	38.3

[a] Edouard Grave, Armand Joulin, and Nicolas Usunier, "Improving neural language models with a continuous cache," arXiv preprint, arXiv:1612.04426, 2016

[b] S. Bai, J. Z. Kolter, and V. Koltun, "Convolutional sequence modeling revisited," OpenReview (ICLR Workshop), 2018. https://openreview.net/forum?id=rk8wKk-R-

[c] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," arXiv preprint, arXiv:1612.08083, 2016. http://arxiv.org/abs/1612.08083

[d] J. W. Rae, C. Dyer, P. Dayan, and T. P. Lillicrap, "Fast parametric learning with activation memorization," arXiv preprint, arXiv:1803.10049, 2018. http://arxiv.org/abs/1803.10049.

[e] S. Merity, N. S. Keskar, J. Bradbury, and R. Socher, "Scalable language modeling: WikiText-103 on a single GPU in 12 hours," 2018, doi: 10.1002/j.2334-1234.2018.12159.x. https://mlsys.org/Conferences/doc/2018/50.pdf

The RMC achieved state-of-the-art results on these datasets, outperforming previous LSTM-based and convolution-based models in perplexity. For example, on WikiText-103, the RMC obtained a test perplexity of about 31.6 versus 34.3 for a strong LSTM baseline. Although small in

Licensed to Hermilo Cortés <hermilocg@gmail.com>

absolute terms (a 1.4–5.4-point drop in perplexity), the RMC brought perplexity down by roughly 5% to 12% relative to the best prior results. To put it in perspective, a 5% to 12% drop in perplexity means the model is that much better at predicting the next word, which, in language modeling, is hard-won progress. This improvement was consistent across datasets of different sizes and styles. See table 7.5.

The authors note that the gains came mostly from better modeling of frequent words. The RMC seemed particularly good at capturing short-term relationships in text that help predict common words in context. Interestingly, the RMC achieved these results using a relatively small number of memory slots. For example, WikiText-103 used just a single memory slot, effectively turning the RMC into a single-slot self-attention LSTM. Yet even with a single slot, the multihead attention mechanism can attend to different positions in the recent context, functioning much like a Transformer. The RMC also performed well with a shorter context window during evaluation. It didn't need as much look-back as an LSTM to maintain low perplexity, suggesting that it emphasized the most relevant recent relationships in the text. This trait aligns with the idea that the RMC has a strong relational inductive bias: it doesn't indiscriminately absorb a long history but focuses on the salient connections in the recent past.

"Relational Recurrent Neural Networks" shows that adding structured memory interactions can improve a model's ability to reason. By explicitly allowing memories to relate to one another, the RMC solved a relational task that stumped traditional models, matched or surpassed state-of-the-art models on algorithmic program execution, boosted an agent's performance in a game environment, and set new high scores in language modeling. These gains stem from the same intuition: many problems require understanding how pieces of information are connected, not merely storing them in isolation. Notably, the improvements didn't come simply from scaling up the LSTM's size.

7.4.2 Historical context

"Relational Recurrent Neural Networks" is one of the least cited papers on Sutskever's List. Part of the explanation is timing: RMC was released in mid-2018, just as the field was moving away from recurrence toward attention-only architectures. The momentum behind Transformers was overwhelming. Within months, BERT and GPT-2 would demonstrate that unsupervised pretraining and scale could eclipse architectural nuance. Against this backdrop, RMC's contribution looked like a detour back into complexity.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

RMC's modest citation count reflects how quickly the research landscape changed. Transformer-XL (chapter 5) introduced a recurrent link between transformer segments and soon surpassed RMC on the same language benchmarks. The Universal Transformer (chapter 5), released just a month after RMC, made a stronger case for adding recurrence to self-attention while preserving parallelism. Both scaled better than RMC. The community rewarded methods that could train on billions of tokens with minimal architectural friction, and RMC's sequential computation made it comparatively unattractive. Thus, as pretraining proved to generalize across tasks, interest in specialized memory modules waned.

While RMC did not become the dominant architecture, it made a useful bet about memory: it should not be passive. Stored information should interact before the model acts. Transformers made a similar bet on a larger scale, replacing recurrent memory slots with token-to-token attention across a sequence. In that sense, RMC is not a failed side quest. Alongside NTMs, it explores how models can store and manipulate structured information over time. The inclusion of RMC bridges the era of recurrent models with the modern dominance of attention, illustrating how ideas about memory and reasoning evolved rather than disappeared.

7.5 Reasoning models

Today, language models (LMs) can tackle formal logic problems and puzzles with some success. For instance, GPT-4 can handle complex syllogisms and deductive arguments more effectively than previous models [27]. LMs have also made strides on many commonsense benchmarks, including HellaSwag, which requires choosing a sensible continuation for everyday situations. GPT-4 achieves 95.3% accuracy in 10-shot evaluations, substantially higher than previous models (GPT-3.5 at ~85%) [28].

Reasoning is also tested by pronoun disambiguation. For example, consider the pair of sentences:

- The city councilmen refused the demonstrators a permit because they feared violence.
- The city councilmen refused the demonstrators a permit because they advocated violence.

In the first sentence, the pronoun "they" naturally refers to the city councilmen. In the second, it refers to the demonstrators. The only difference between the two sentences is that the first has "feared" and the second has "advocated" [29]. Such pronoun resolution requires an understanding of intuitive contexts and likely outcomes. On WinoGrande, GPT-4 reaches 87.5% (5-shot), which is strong but still below human performance

Licensed to Hermilo Cortés <hermilocg@gmail.com>

(~94%). Still, this suggests that GPT-4 can infer references in sentences (for example, who "he" or "she" refers to in a story).

GSM8K provides human-written, step-by-step solutions for math problems, enabling models to learn to "show their work." Using few-shot chain-of-thought reasoning, GPT-4 answers about 92% of the problems correctly, a massive jump from GPT-3.5's ~57% that even surpasses fine-tuned specialist models [30]. For example, GPT-4 can solve multistep arithmetic word problems like "If a farm has 23 cows and 17 chickens, how many legs are there in total?" by working through each step, whereas smaller models often get such problems wrong.

In 2022, Google's Minerva was trained explicitly for quantitative reasoning and achieved approximately 50% accuracy on the MATH benchmark, which consists of difficult high school Olympiad and competition math questions [31]. By 2025, both Google and OpenAI announced that experimental reasoning models achieved gold-medal-level performance on the 2025 International Mathematical Olympiad problems under human-like conditions, including two 4.5-hour sessions, no tools or internet, and natural-language proofs [32].

Today's best models can reason competently across a range of benchmarks, sometimes rivaling expert performance under controlled conditions. But their grasp of truly abstract, out-of-distribution reasoning remains conditional. It is stronger with scaffolds such as few-shot learning, chain-of-thought prompting, and test-time compute, yet it remains brittle when the rules change.

7.5.1 Distributional brittleness

Prompting language models matters. A well-crafted prompt can improve model performance by roughly 5% to 15% without modifying the model itself [33]. But language models are also shaped by the frequency and variety of patterns they see during training. Common patterns are easy; rare ones are brittle. This creates a kind of distributional gravity in which the model's competence bends toward what it has often encountered. Add a perturbation to a familiar puzzle, and the solution can collapse suddenly. The "killer prompt" can shift the odds, but it doesn't create new capability. Instead, it helps the model reveal what it already knows.

That may be vague, so let's consider an example. The Wolf, Goat, and Cabbage Problem is a popular puzzle that's been referenced many times online. It even has a dedicated Wikipedia page, which is oversampled in the training data because of its high-quality text [34]. The puzzle involves a farmer who must cross a river with a wolf, a goat, and a cabbage, but the boat can carry only one at a time. The challenge is to transport them all

Licensed to Hermilo Cortés <hermilocg@gmail.com>

across the river without leaving them unattended, so the wolf can't eat the goat, and the goat can't eat the cabbage. To solve the puzzle, you need to use task scheduling, think creatively, and apply reasoning.

There is a version of the puzzle in which the goat can eat the cabbage, the wolf can eat the cabbage, and the wolf will not eat the goat [35]. This variant poses a challenge for language models like ChatGPT. The model wants to take the goat across the river and leave it on the other side, even though the wolf will eat the cabbage. In this setting, ChatGPT is not recomputing the constraints from scratch but instead behaving like a system pulled toward a familiar solution template. Consequently, it is often blind to this variant of the puzzle with the new constraints. It fails because this variant is underrepresented in the training data. This limitation highlights the challenge of adapting to new information and can hinder language models' capacity for planning and reasoning in contexts not well represented in their training data.

When a puzzle is common, the model has usually seen its canonical solution repeatedly. That makes the correct answer easy to retrieve as a pattern. But retrieval is not the same thing as recomputation. When a constraint is perturbed, the model's behavior often reveals that it is searching for a familiar template rather than reasoning from first principles. Humans often treat such perturbations as the normal case.

The Missionaries and Cannibals puzzle forces the solver to coordinate search and counterfactual reasoning. In the standard formulation, three missionaries (m) and three cannibals (c) must cross a river in a two-person boat (b). The safety constraint is simple: if any missionaries are present, they must not be outnumbered by cannibals; otherwise, the cannibals would eat the missionaries. So, you cannot greedily move people across the river [36]. River-crossing problems are quite common in artificial intelligence.

The canonical version with stepwise reasoning is easy for most language models. Now consider the same problem with a perturbation: for example, the boat still holds two people, but one can't row. This is a minor change, but it alters the set of legal actions. A model that primarily retrieves a crossing script will often continue to produce the standard sequence and then patch it with post hoc explanations. This happens because the text surrounding the plan can be adapted more easily than the plan itself. The result is a language model that is rhetorically consistent without being logically consistent.

Solving a perturbed instance requires a non-monotonic approach that updates a default strategy when a new constraint arrives [37]. Humans do

Licensed to Hermilo Cortés <hermilocg@gmail.com>

this naturally. We use a standard script or internal model for this type of puzzle and treat it as provisional, recomputing it if a constraint blocks it. By contrast, pure language models behave as if the default script is closer to monotonic truth.

7.5.2 Reversal curse

These river-crossing examples illustrate one kind of brittleness. A subtler diagnostic is the reversal curse, where a model may encode a "fact" in one direction without being able to use the corresponding inverse relation. "Order Matters" (chapter 5) discussed how sequence models do not encounter facts as unordered propositions, but as ordered prediction problems, and changing the serialization of otherwise similar information can change perplexity and accuracy. Also, recall that reversing the source sentence improved translation performance by shortening the effective path between source and target. The reversal curse is an analog of this directional dependence.

In decoder-only language models such as OpenAI's GPT-family, text is modeled autoregressively from left to right, so when predicting a token (x_t), the model conditions only on preceding tokens ($x_{<t}$) through masked self-attention. The transformer itself is feedforward, but its training objective is autoregressive. By contrast, models such as BERT are trained to produce bidirectional representations, allowing each token to incorporate both left and right context at every layer. In both cases, self-attention computes pairwise scores and aggregates them, but the available context differs across architectures. See figure 7.7.

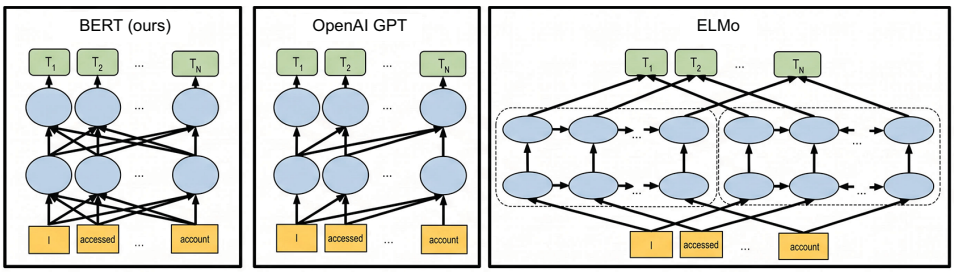


Figure 7.7 BERT is deeply bidirectional, OpenAI's GPT is unidirectional, and ELMo is shallowly bidirectional.

This directional conditioning helps explain why language models can learn different mappings for the same association, all depending on the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

query's direction. In figure 7.8, GPT-4 correctly identifies the name of Tom Cruise's mother (left). But when prompted with the mother's name, it fails to retrieve "Tom Cruise" (right) [38]. The asymmetry is not that the model lacks the fact altogether, but that the fact is accessed through learned patterns of use. The corpus heavily favors Tom Cruise | Mary Lee Pfeiffer, so the reverse query is brittle in a zero-shot setting. If you first state the forward fact in the prompt, the model reliably answers the inverse—an instance of in-context learning. But if a model's "knowledge" is stored primarily as learned representations under the most common phrasings, then the model can appear knowledgeable while still being brittle when the query is posed in an atypical direction.

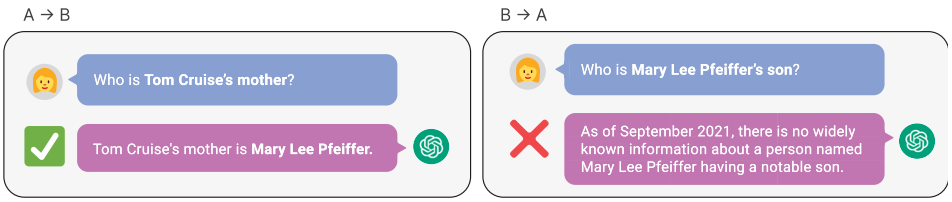


Figure 7.8 GPT-4 correctly gives the name of Tom Cruise's mother (left). But when prompted with her name, it fails to retrieve "Tom Cruise" (right). Models trained on "A is B" (for example, "Tom Cruise's mother is Mary Lee Pfeiffer") do not automatically infer that "B is A."

7.5.3 *Prompting as a reasoning interface*

If John McCarthy (who coined the term "artificial intelligence") were reading these examples, he might be less surprised by the failures [39][40]. He spent his career arguing that intelligent behavior depends on explicit representations of action, causality, and default assumptions, rather than on pattern association. What current systems demonstrate is that massive training can approximate the surface form of reasoning well enough to appear to plan in familiar instances. But the moment the world model must be revised, the approximation becomes obvious.

That said, you can improve reliability by prompting the model to adopt a more explicit representation. For example, you might use a prompt that represents each state as a triple (m, c, b), enumerates legal moves, and verifies the safety constraint after each crossing. In many cases, this helps the model detect invalid steps, making the plan more stable.

Another influential technique is chain-of-thought (CoT) prompting, which guides a language model to generate explicit intermediate reasoning traces in natural language before finalizing an answer [41]. CoT was a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

response to informal zero-shot tricks, such as prompting a model to "think aloud." These tricks, as well as adding a phrase like "Let's think step by step," led to remarkable improvements on multistep reasoning tasks without any additional training [42].

In other words, rather than asking the model to jump directly to a solution, we prompt it to break a complex problem into a step-by-step "chain" of simpler reasoning. For example, given a math word problem or a logic puzzle, the model is encouraged to work through the solution one step at a time, just as a person might scribble notes or steps on paper, and only then produce the final answer. This simple prompting strategy can dramatically improve performance on tasks that require multistep reasoning or planning.

In the original Google Brain study (2022), CoT prompting enabled sufficiently large models to solve math word problems and commonsense reasoning puzzles that stumped the same models under standard prompts [43]. By decomposing a query into subtasks and mimicking an intuitive thought process, the model effectively uses its own internal scratchpad during inference. Notably, this method requires no changes to the model's architecture or parameters. CoT prompting thus exemplifies the shift toward an explicit reasoning structure at runtime. Empirically, CoT has proven particularly effective when combined with very large models with hundreds of billions of parameters, where the ability to follow a logical train of thought appears to emerge at scale.

CoT is one way to inject more reasoning into inference, but modern methods often give language models additional compute or deliberation time at test time to tackle complex problems without requiring user prompting. One technique generates multiple reasoning chains for the same question, then allows the model to search through them and select the most consistent answer. This effectively uses a vote among the model's "thoughts" to enhance reliability [44][45].

The practical implication is not that language models have no value or that prompts are useless. Prompts are imperfect interfaces for eliciting existing capabilities. When tasks are in distribution, a good prompt can reduce variance and expose more of what the model already "knows." When tasks require recomputation under novel constraints, prompting is a weak lever. It can improve the odds, but it cannot substitute for mechanisms that actually perform search, enforce invariants, or explicitly represent defaults and exceptions.

The broader lesson is that language models do not fail because they lack every ingredient of reasoning. They fail because their reasoning is

Licensed to Hermilo Cortés <hermilocg@gmail.com>

unevenly coupled to the distribution of forms in which problems, facts, and solutions have appeared. In familiar settings, this coupling can look remarkably intelligent, and the model will retrieve useful abstractions, follow learned procedures, and produce convincing intermediate steps. But when the task is reversed, perturbed, or made to depend on a newly introduced constraint, the same system may reveal that it has learned the surface trajectory of reasoning more robustly than the underlying invariants.

This is why modern reasoning models are best understood not as a clean break from language modeling, but as an attempt to make reasoning less dependent on the most familiar phrasing of a task. Test-time compute, search, verification, and explicit intermediate representations all push in this direction. The goal is not merely to produce better answers but to make the path to those answers more stable under variation.

7.6 Paper doubts

The emergence of reasoning models rekindled long-standing debates in AI [46]. In the classical (often called "Good Old-Fashion AI" or "GOFAI") rationalist view, intelligence is grounded in discrete, compositional symbols plus rules for transforming them [47]. AI pioneers Allen Newell and Herbert Simon famously argued that "symbols lie at the root of intelligent action," thereby formalizing the *physical symbol system* [48]. This hypothesis treats reasoning as something that must be specified in logic, often at the cost of brittleness when the world's messy details or grounding are underspecified.

Connectionists view intelligence as emerging from empirical learning of statistical regularities and distributed representations via gradient-based methods. Critics argue that such systems lack compositionality. Jerry A. Fodor and Zenon Pylyshyn pressed this point sharply: if thought and language are compositional, then a cognitive architecture must explain that structure rather than merely approximate it [49]. But a key connectionist reply, articulated by Paul Smolensky, was that "hard rules" can be a higher-level description of lower-level "soft constraints," so symbolic-like competence could emerge from neural computation rather than from being prespecified [50].

The rationalist-empiricist split is a philosophical disagreement over the structure required for reasoning. The rationalist holds that explicit symbolic programs are necessary, whereas the empiricist tradition favors learned representations shaped by data and inductive biases. But what ultimately settled the debate was not a decisive philosophical argument but standardized benchmarks, shared datasets, and measured deltas

Licensed to Hermilo Cortés <hermilocg@gmail.com>

against strong baselines. The Malik–Hinton wager (chapter 2) is best read as a parable about this norm.

Jitendra Malik's challenge to Geoffrey Hinton was simple: don't tell me how great artificial neural networks are; *show me* on a real benchmark like PASCAL. Malik wasn't interested in theoretical promise, anecdotal success, or a philosophical argument. Hinton countered with ImageNet, and AlexNet (chapter 2) delivered. No one questioned whether convolutional networks could *really* see or *really* understand; the debate was whether deep learning could outperform support vector machines and handcrafted data pipelines. The wager was empirical, not philosophical. Success was measured against the best existing baseline. That empirical standard legitimized deep learning and reset the field.

The same pattern repeats in research on differentiable reasoning. RNs achieved human-level performance (~95%) in relational question answering and passed 18 of the 20 bAbI tasks. Message-passing neural networks set new best results on all 13 QM9 targets and achieved chemical accuracy on 11 of them. Relational memory cores cut language model perplexity by 5% to 12% across benchmarks. All three provided measurable deltas, which is why they are included on Sutskever's List.

By contrast, capsule networks inspired hope. In 2017, Turing Award laureate (and future Nobel laureate) Geoffrey Hinton introduced them amid significant buzz that they might eventually replace CNNs by incorporating reasoning about entities within images [51]. While capsules achieved state-of-the-art (SOTA) results on benchmarks such as smallNORB and came close to SOTA on MNIST, they proved challenging to train and scale, and never matched the performance of the best CNNs on mainstream benchmarks such as CIFAR and ImageNet.

In hindsight, capsules reveal that even a legend can publish a beautiful idea on paper that fails an empirical audit. In AI, reputation is earned through working systems, and accolades and hope don't buy an exemption from that test. Capsule networks remain a cautionary tale: an elegant idea that looked great on paper but, without competitive results, quickly faded from the spotlight.

In 2018, Gary Marcus published a critique titled "Deep Learning: A Critical Appraisal." The critique, though one-sided, cataloged what networks could not yet do well, including reasoning, compositionality, and abstraction [52]. The paper was written before the Transformer's influence had spread and did not include the reasoning-oriented architectures explored in this chapter. While the critique clarified gaps in deep learning, it also confirmed a common folly in research that assumes identifying

Licensed to Hermilo Cortés <hermilocg@gmail.com>

shortcomings is equivalent to solving them. But naming a demon is cheap; exorcisms are not [53]. The easiest part of any problem is naming it.

Over a century ago, Charles S. Peirce distinguished between *paper doubts* and *living doubts*. Paper doubts are professed for argument's sake. They are declared in words, generally one-sided, and don't interrupt anyone's confidence. A researcher can pen a scathing critique (or a tweetstorm) about AI's shortcomings, but it's a purely intellectual exercise, and nothing in the world changes. Paper doubt doesn't move the frontier because strongly worded statements usually don't change architectures, baselines, benchmarks, leaderboards, or deployments. In contrast, a living doubt is a visceral uncertainty that unsettles thoughts and compels investigation.

In today's terms, a statement like "current deep learning can't reason," left unchallenged, is a paper doubt. It's a position one can argue eloquently without committing to anything. It might spark discussion with hypothetical skeptics, but it doesn't change behavior. A living doubt, on the other hand, disrupts a community's complacency and motivates action. We've seen how the living doubt that networks lacked relational reasoning led directly to RNs and Transformers, how concerns about mathematical abstraction led to benchmarks like GSM8K (for arithmetic reasoning), and how concerns about visual reasoning led to CLEVR.

François Chollet's Abstraction and Reasoning Corpus, better known as ARC-AGI, is a quintessential example of a living doubt [54][55]. Chollet introduced ARC in 2019 as a benchmark for human-like general intelligence. The benchmark consists of small, enigmatic grid-based puzzles to test a machine's ability to reason from just a few examples. Humans solve these tasks easily because the puzzles require no specialized knowledge. But an AI can't rely on brute-force pretraining. Each task is unique, offers only a few demonstrations, and requires nonmonotonic reasoning. That is, the machine must infer the rule on the fly and apply it to a novel situation, a task with which pure language models struggle [56][57].

The original GPT-3 model scored 0% on the public evaluation. GPT-4.1 managed roughly 5% [58]. In late 2024, ARC Prize reported a major jump for OpenAI's o3-preview system, which scored 75.7% on a semi-private evaluation under the ARC-AGI's sub-$10,000 compute rule, with a higher-compute configuration reaching 87.5% but at a much greater cost. For a brief moment, it seemed as if Chollet's challenge had been met, and we were on the cusp of a general reasoning solution [59]. But the same family of models fell to below 3% accuracy with the release of ARC-AGI-2 just a few months later [60].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

What appeared to be a giant leap turned out, in part, to reflect overspecialization. ARC-AGI-2 was specifically engineered to be out of reach of any tricks that might have fit ARC-AGI-1 too closely. This is living doubt in its purest form. Rapid progress on ARC-AGI-1 did not mean that the problem of general abstraction had been solved; rather, it indicated that the benchmark had been conquered under its specific conditions.

Chollet's living doubt is an operationalized rather than a hypothetical form of skepticism. By continually increasing the difficulty and introducing truly novel tasks, it forces researchers and engineers to confront whether their systems are learning what they claim to be learning or just learning the quirks of a test. ARC-AGI-2 demands a higher level of generality, and for now, that doubt remains unresolved. It is a provocation to do better, not just criticism on paper.

The living-doubt lens also helps interpret another figure. Sutskever favored a staged release of GPT-2 after deeming it too dangerous to release (chapter 1). The concern wasn't hypothetical or performative. Years later, he joined the board action that briefly removed Sam Altman (chapter 1). These actions aren't paper doubts. They're doubts we can see. This contributes to the mystique around Ilya. He lives out his hopes and doubts by building solutions that work and aligning them. If paper hope is a promise without proof, then living hope is evidence that an idea works, like CNNs and Transformers. It's this kind of seriousness that pushes and confronts progress, rather than merely promoting it or opposing it for the sake of argument [61].

Yet even this empirical, results-driven, meritocratic narrative has a seam. While benchmarks are critical and are the bedrock of progress in AI, they are not infallible. They are instruments built on assumptions about what they measure. What they measure must be defined operationally in terms of stable, reproducible effects. The release of ARC-AGI-2 and ARC-AGI-3 underscored this point. What appeared to be rapid progress on ARC-AGI-1 was partly due to familiarity with benchmarks and overspecialized search [62].

In fairness, ARC-AGI-2 was not merely a newer benchmark but also a tougher one, and was intended to measure AGI rather than traditional machine learning performance [63][64][65][66]. Nevertheless, the broader point stands: AI demands receipts, and those receipts must be scrutinized. It doesn't care about your name or your paper doubts; it cannot move without living hope, nor be safe without living doubt. Receipts are reproducible gains against strong baselines, under controls that withstand

Licensed to Hermilo Cortés <hermilocg@gmail.com>

distribution shifts and transfer to new benchmarks. Robustness that travels counts for more than a single leaderboard win [67].

While Sutskever's List records what worked, we still struggle to say what we can legitimately understand about it. In the chapters that follow, we step beyond engineering to confront what these achievements mean. This is not a departure from Sutskever's List but its culmination.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

8 Simplicity, hidden in complexity

This chapter covers

- How compression reveals secret structure
- How complexity peaks between order and noise
- How simplicity beats overfitting
- Grokking (or memorize, then simplify)
- The question of whether language models are just blurry JPEGs of the internet

Papers

- *Quantifying the Rise and Fall of Complexity in Closed Systems: The Coffee Automaton* (2014) Aaronson, Carroll, and Ouellette
- *The First Law of Complexodynamics* (2011) Scott Aaronson
- *Kolmogorov Complexity and Algorithmic Randomness* (2017) Uspensky, Shen, and Vereshchagin
- *A Tutorial Introduction to the Minimum Description Length Principle* (2004) Peter Grünwald
- *Keeping Neural Networks Simple by Minimizing the Description Length of the Weights* (1993) Geoffrey Hinton and Drew van Camp

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Sutskever has argued that any good prediction model is implicitly a good compressor, and vice versa [1]. In one talk, he cites 2017 research on the "sentiment neuron," in which OpenAI researchers—including Sutskever—trained a long short-term memory (LSTM) to predict the next character in Amazon product reviews, but discovered that a single neuron captured the review's sentiment [2]. The sentiment neuron arises because predicting the next character forces the model to encode sentiment in a latent variable.

A phrase Sutskever often uses is that good compression uncovers the "secrets" in data. *Secrets* are latent variables or hidden factors, such as sentiment in reviews, grammatical structure in language, thought vectors in machine translation, or the physics in vision data, that aren't labeled but influence prediction. The model seems to "know" facts and commonsense knowledge because it had to encode them to compress the data.

But before we get to that, we begin by tracing how structure tends to live in the intricate regimes between order and noise. Scott Aaronson, Sean Carroll, and Lauren Ouellette's "coffee automaton" shows how complexity rises, peaks, and then declines as a system homogenizes. This picture carries over to artificial neural networks, where Geoffrey Hinton and Drew van Camp trained networks to minimize description length, thereby improving performance.

Moreover, work on grokking and double descent mirrors these patterns: complexity first increases as a network memorizes training examples, then collapses once it discovers a simple underlying rule, and test performance improves abruptly. At the same time, double descent shows that test error worsens, then improves again as models settle on simpler, better-generalizing solutions. Taken together, these ideas converge on a picture in which simplicity is found where we least expect it.

8.1 Coffee automaton

In a 2011 essay, "The First Law of Complexodynamics," Scott Aaronson conjectured that complexity in a closed system starts near zero, peaks at an intermediate time, and returns to near zero, whereas entropy continues to increase monotonically. He calls this behavior "complextropy," a portmanteau of complexity and entropy. This puzzle, moving from order to disorder, from simplicity to complexity, and back, motivates Aaronson to revisit the problem in a 2014 paper with Sean Carroll and Lauren Ouellette titled "Quantifying the Rise and Fall of Complexity in Closed Systems: The Coffee Automaton." Both papers are on Sutskever's List.

The second law of thermodynamics tells us that the entropy of a closed system never decreases. *Entropy* is often described as a measure of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

disorder, randomness, or the dispersal of energy and matter within such a system. It explains why systems tend to move from ordered to chaotic states, such as when ice melts. Ultimately, entropy is related to the number of microscopic configurations a system can have for a given macroscopic state. Yet experience suggests that complexity does not simply track entropy. The universe began in a smooth, tightly constrained, low-entropy state far from equilibrium and will end in a high-entropy heat death, largely devoid of visible structure. But between those endpoints, we experience galaxies, life, and other organized forms. A cup of coffee behaves similarly but with less existential burden.

Pour in cream, and you begin with two clean layers. Swirls and filaments form during mixing. Entropy climbs as the visible structure peaks. In time, the cup turns a uniform beige, reflecting a high-entropy state as gradients are effaced and little controllable structure remains. Entropy increases throughout, whereas complexity first rises with the emergence of mid-scale patterns and then declines as mixing concludes. See figure 8.1.

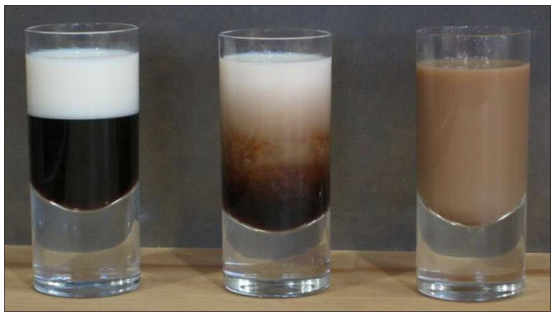


Figure 8.1 (left) low entropy and low complexity; (middle) medium entropy and high complexity; and (right) high entropy and low complexity. *Used with permission granted by the lead author (Scott Aaronson).*

8.1.1 Methodology

To investigate complexity, the authors construct a stochastic cellular automaton that serves as a model of a coffee cup. A cellular automaton is a two-dimensional grid of size $n \times n$ cells (typically $n = 100$). Each cell's value represents the local presence of cream (`1`) or coffee (`0`). In the initial state, the top half of the grid is filled with `1`s (cream) and the bottom half with `0`s (coffee), representing an unmixed cup with a sharp cream-coffee boundary. From this starting point, the automaton updates in discrete time steps according to one of two rule sets, corresponding to two physical regimes: an interacting model and a noninteracting model.

The interacting model means that only coffee or cream occupies a cell at a time. At each time step, the automaton chooses one adjacent cream–coffee pair at random, and the two particles exchange places, mimicking

Licensed to Hermilo Cortés <hermilocg@gmail.com>

diffusion in which particles cannot pass through each other. This rule reflects a physical constraint that gives rise to large-scale patterns. Over time, many random swaps cause the cream to disperse into the coffee, but because only one swap occurs per time step and distant cream particles must wait for nearby particles to move, mixing is gradual.

The noninteracting model assumes that cream particles do not interact with one another and move freely. Any number of cream particles can occupy the same cell. At each time step, every cream particle independently jumps to a randomly chosen adjacent cell. Because multiple particles can pile up, they don't block or interact with one another.

The authors introduce a two-step process. Instead of analyzing the raw grid of `0`s and `1`s, which becomes essentially random at the microscopic level as mixing progresses, they smooth (or downsample) the state into a coarse-grained array, a grid that captures macroscopic patterns. By coarse-graining the grid, the tiny speckles vanish, and only the big swirls and cream filaments remain. In parallel, the authors estimate entropy from the full, fine-grained state by measuring how well it can be compressed, which serves as a proxy for the number of hidden arrangements that fit the observed data.

The authors use a small neighborhood around each cell and average the values within it. They average each neighborhood, where values near 0 indicate mostly coffee, values near 1 indicate mostly cream, and intermediate values indicate a mixture. The averages are then quantized into a few discrete labels, such as coffee, cream, and mixed. By coarse-graining, the authors intentionally discard fine-grained randomness and focus on structured features, such as the shape of the cream-coffee boundary, that define the pattern's complexity. Essentially, the coarse-grained array is a blurry, quantized "macrostate" view of the cup, similar to what a human might see. Figure 8.2 illustrates this process on a small example.

Fine-grained

0	1	0	0	1
0	1	1	0	1
0	1	0	1	0
1	0	1	1	1
1	0	1	0	0

Coarse-grained

0.5	0.5	0.5	0.5	0.5
0.5	$0.\overline{4}$	$0.\overline{4}$	$0.\overline{4}$	0.5
0.5	$0.\overline{5}$	$0.\overline{6}$	$0.\overline{6}$	$0.\overline{3}$
0.5	$0.\overline{5}$	$0.\overline{5}$	$0.\overline{5}$	0.5
0.5	$0.\overline{6}$	0.5	$0.\overline{6}$	0.5

Figure 8.2 Illustration of the construction of the coarse-grained array using an example grain size of 3. The values of the shaded cells on the left are averaged to produce the value of the shaded cell on the right. *Used with permission granted by the lead author (Scott Aaronson).*

The next step is to estimate the description length by compression. The coarse-grained array is written to a file and fed to a standard compressor

Licensed to Hermilo Cortés <hermilocg@gmail.com>

(for example, gzip or bzip2), and the resulting compressed size is recorded. The size of the compressed file is taken as an estimate of the Kolmogorov complexity of that macrostate. Kolmogorov complexity is the length of the shortest program that generates a given object. Short descriptions signal simplicity; long descriptions signal complexity. A state with visible structure, such as swirls and clusters, is "interesting" but tends to compress poorly and therefore has a longer description length; a featureless or straightforward state compresses well and has a shorter description. The authors call this quantity the *apparent complexity* and use it as a practical stand-in for true Kolmogorov complexity, which is uncomputable in practice.

Imagine two emails of 10,000 characters:

- *Random email*—You generate it by flipping a fair coin 10,000 times and writing "H" for heads and "T" for tails. This message has high entropy and is incompressible in practice. Since it contains no structure, its Kolmogorov complexity is high. The shortest program that reproduces the exact sequence is the sequence itself.
- *Structured email*—You write "HTHT … HT," repeating "HT" 5,000 times. This message has lower entropy and is highly compressible. Its Kolmogorov complexity is low because a short program, such as "print 'HT' 5,000 times," reproduces it.

For contrast with thermodynamics, the authors also compress the original, unsmoothed binary grid to estimate entropy. A thoroughly mixed coffee cup looks random at the microscopic level, so the fine-grained file is scarcely compressible, and its size approaches the uncompressed size. At intermediate time steps, the fine grid may still look random, yet the coarse view can reveal clear large-scale order. The point is that entropy reflects micro-level disorder, while apparent complexity captures macro-level structure.

8.1.2 Results

The authors find that as the cream and coffee mix, entropy (top curve) increases steadily, reflecting growing disorder. See figure 8.3. Meanwhile, complexity (bottom curve) is low at the start, with only two simple layers; it rises to a peak at an intermediate time when intricate mixing patterns appear, and then declines as the mixture becomes uniform. In other words, the system becomes algorithmically most complex at a midpoint, precisely when visible "milk tendrils" and turbulence-like structures are present, even though this is neither the point of lowest entropy (initial

Licensed to Hermilo Cortés <hermilocg@gmail.com>

state) nor highest entropy (final state). This suggests how a closed system can exhibit a temporary rise in apparent complexity while still obeying the second law of thermodynamics [3].

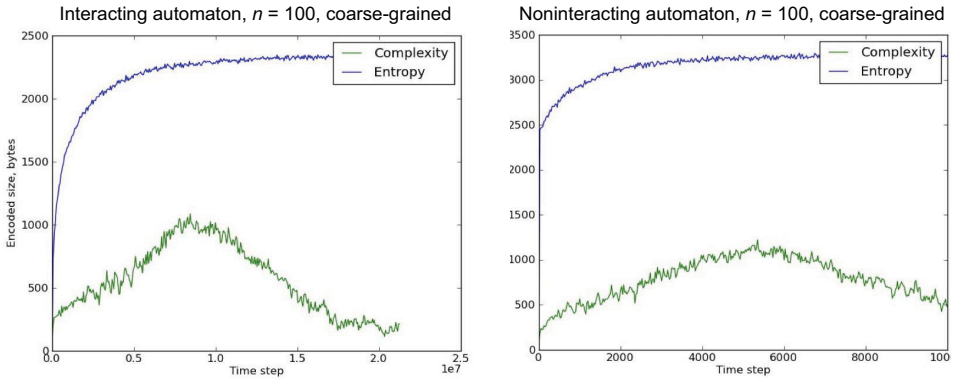


Figure 8.3 In the simulation of coffee and cream mixing, entropy (top, monotonic curve) steadily increases toward a maximum as the fluids randomize, in accord with the Second Law. Apparent complexity (bottom, nonmonotonic curve) first increases, then decreases, peaking at an intermediate time when the system's state is most "interesting," or structured. *Used with permission granted by the lead author (Scott Aaronson).*

To reinforce the intuition, the paper provides grayscale images of the cup's state at various times for both the interacting and noninteracting models. In the coarse-graining experiment, both models exhibit an initial rise and eventual fall in estimated complexity, with a peak at intermediate times. Figure 8.4 shows the interacting automaton in three snapshots. The top row depicts the fine-grained state, where each pixel is black or white, indicating coffee or cream, and the bottom row illustrates the

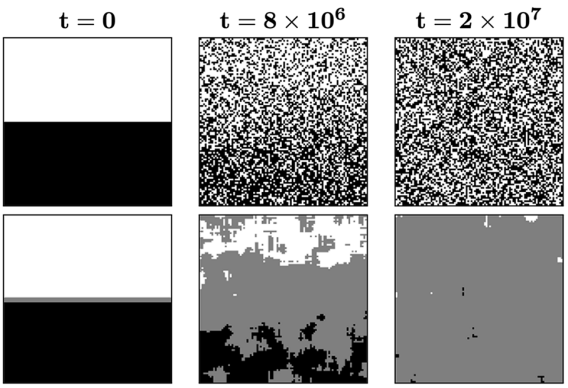


Figure 8.4 Visualization of the interacting automaton's state over time. *Used with permission granted by the lead author (Scott Aaronson).*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

coarse-grained state, with pixels in shades of gray representing the local mix proportions. Initially, the coarse image is a simple block structure, half black and half white. At the peak of complexity, the middle image shows an intermediate mixed state with irregular gray regions where local proportions of coffee and cream vary across the cup. Finally, the coarse image is a solid medium gray; the tendrils have diffused away, leaving homogeneity, a 50/50 mixture almost everywhere.

Figure 8.5 shows the noninteracting automaton. Its intermediate fine- and coarse-grained images are far less structured than those of the interacting model. The complexity peak is much smaller (figure 8.3, right). These figures reinforce that the interacting dynamics produce hard-to-compress states, whereas independent diffusion does not. The coffee automaton also serves as a reminder that a system can be high-entropy yet bland or medium-entropy yet structured.

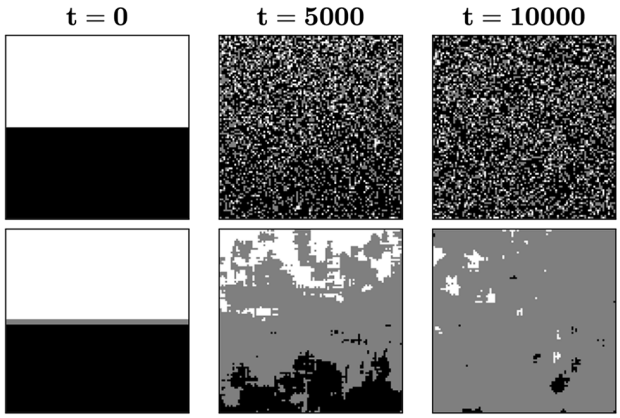


Figure 8.5 The same visualization for the noninteracting model. The intermediate image is less structured than that of the interacting model, and the complexity peak is smaller. ***Used with permission granted by the lead author (Scott Aaronson).***

In coffee automaton, the cup begins simple, becomes intricate, then simple again, even as microscopic disorder keeps climbing. The trick that makes this visible is coarse-graining the simulation. Instead of tracking every microstate, the authors look at a blurred macrostate that preserves swirls. Thus, entropy is a statement about the fine-grained microstate, whereas "apparent complexity" is a statement about the coarse-grained macrostate.

In an artificial neural network, the microstate may be the full weight matrix, down to the last decimal place. Yet, the microstate level is too

Licensed to Hermilo Cortés <hermilocg@gmail.com>

detailed. What we care about is the macrostate, including the swirl pattern in the coffee or the model's learned function and representations. If you look at the right coarse variables, training looks like a system that first explores, then self-organizes, then simplifies. This lays the conceptual groundwork for discussing how AI might exhibit increasing complexity in its learning structure, followed by simplification through generalization.

By defining complexity in terms of algorithmic information and description length, the authors bridge physics and information theory, demonstrating how concepts from computer science, such as Kolmogorov complexity, can inform physics and, conversely, how physics can inform computer science. The authors take an abstract concept, such as Kolmogorov complexity, and apply it quantitatively to a tangible physical scenario. Their use of real-world compression algorithms is also clear and sidesteps the problem of uncomputability. One could literally reconstruct the coarse images and see the patterns that contribute to complexity. This pedagogical clarity makes the work approachable for a broad, technically literate audience, much as Chris Olah's work does (chapter 4).

8.2 Kolmogorov complexity and algorithmic randomness

Kolmogorov Complexity and Algorithmic Randomness was published in 2017 [4]. The first half is a textbook-style exposition of core concepts, and the second half presents more advanced, recent results from the Moscow "Kolmogorov seminar" and its researchers. Vladimir A. Uspensky (1930–2018) was a student of Andrei Kolmogorov, the Soviet mathematician who pioneered the field [5]. Uspensky headed the Department of Mathematical Logic and Algorithms at Moscow State University and was instrumental in Kolmogorov's famous 1980s seminar on algorithmic complexity. Alexander Shen and Nikolai Vereshchagin were trained at Moscow's school of algorithmic information theory and made important contributions to the field of Kolmogorov complexity research [6][7].

Kolmogorov complexity defines the information content of an object as the length of the shortest program that outputs it and algorithmic randomness as the property of an object having no shorter description than the object itself. This dovetails with the compression view and differs from Shannon entropy, which measures the average uncertainty of a probability distribution or source [8]. Claude Shannon asked how many bits are needed on average before an outcome is known, whereas Kolmogorov complexity asks how short a description can reproduce this particular outcome after it has been specified. As noted, a repeating sequence is not algorithmically random [9]. By contrast, to return to our earlier

Licensed to Hermilo Cortés <hermilocg@gmail.com>

example, a typical sufficiently long sequence of fair coin-flip results is incompressible: with high probability, no program much shorter than the sequence itself outputs it. This is the compression view of algorithmic randomness [10]. A truly random-looking sequence has no such short program, which is the key idea introduced by Kolmogorov and, independently, by Ray Solomonoff [11].

Randomness, in this framework, is incompressibility. It is the absence of any shorter code explaining how the data came to be. Yet even a tiny description that includes just a bit of randomness can mask significant computational complexity. The tension between simplicity of description and emergent behavior shows up in one of the field's most striking constructions. Imagine a program that follows simple rules such as reading a square, writing a symbol, and switching states. Among these programs, many stop quickly, some run for a long time, and others loop indefinitely. Radó's Busy Beaver function is the program that does the most work before it halts. But determining which program is a Busy Beaver is an enormous problem, since you'd have to answer whether the machine halts and, if so, how long it runs. Answering this requires solving the famous halting problem, which we already know is impossible to solve in general. That means the Busy Beaver function is uncomputable [12].

The reason this matters in a chapter about compression and randomness is that the Busy Beaver is the clearest proof that a short description doesn't guarantee simple behavior. Random data is complex because it resists compression; there is no shorter description of it. But the Busy Beaver is complex in the opposite direction. It is short in terms of "description length" and thus highly compressible. Yet when you run it, it can take longer than any computable bound. It's like a small seed that grows into a giant tree, but you only find out how huge it is by letting it grow.

So, while randomness is the inability to shrink a description, the Busy Beaver is a warning against "shortest code" thinking. A short explanation can be useless if it produces what we want only after an unknowably long computation. That's precisely why researchers like Aaronson emphasize resource-bounded ideas, which are measures of complexity that care not only about how many bits your explanation takes but also about how much computation it takes to cash it out into what you're trying to explain.

8.2.1 Algorithmic statistics

The book also investigates more specialized topics. For example, algorithmic statistics studies how to separate structure from noise in an individual dataset. It uses Kolmogorov complexity to identify the simplest model that

Licensed to Hermilo Cortés <hermilocg@gmail.com>

captures that structure. This topic is so pertinent that Ilya Sutskever has explicitly recommended the book's final chapter, titled "Algorithmic Statistics" [13].

That chapter frames scientific explanation in algorithmic terms. A good theory should be concise and unsurprising. The authors formalize this using Kolmogorov complexity as a yardstick for simplicity, and randomness deficiency as a yardstick for typicality. If a model can generate data as one unremarkable outcome among many, then the data is typical. Simplicity means a short description of a model with low complexity; typically, the data looks like a normal sample from that model and therefore has low randomness deficiency. Together, these ideas turn the intuitive notion of a "good scientific explanation" into something we can almost quantify.

Think of randomness deficiency as an instrument for weirdness. High deficiency indicates that a data point is unlikely, while low deficiency indicates that the data could be expected under that model. Formally, if we treat a model as a set of possible outcomes or a probability distribution, randomness deficiency measures how far the data is from what that model would normally produce. Suppose your model is "all swans are white," and your data point is a black swan. That black swan is atypical for the model; it has a high randomness deficiency with respect to "all swans are white." If your model is "swans can be black or white," then a black swan has low deficiency and is normal under the model. In algorithmic terms, the chapter defines randomness deficiency roughly as the gap between the model's expected complexity for an outcome and the actual complexity of your specific data. If the deficiency is small, the data conforms to the model's expectations. If it's large, the data is an awkward misfit.

In this framework, a good explanation pairs a simple model with low randomness deficiency. The model should be concise, and the data should be typical relative to that model. This formalizes two pillars of a scientific theory: the model's simplicity and the data's typicality relative to the model. If either pillar fails, for example, if the model is simple but the data looks implausible under it, the explanation isn't satisfying. The model may be too simple to make the data typical, or the data may fit perfectly only because the model is overcomplicated.

In late 2025, Terence Tao remarked that Gemini Deep Research, while performing a literature review for Erdős problem #481, reproduced an essentially correct argument, though the report did not seem to recognize that it had established the result [14]. The problem was later marked as proved, with a proof verified in Lean [15]. In the language of algorithmic statistics, one might view the episode as compressor-like behavior

Licensed to Hermilo Cortés <hermilocg@gmail.com>

because the model, trained on mathematical discourse, can reconstruct a structured proof from a short prompt.

Think of lossy compression as JPEG for images or MP3 for audio, though in the formal setup, we're not actually throwing away information. The theory is that meaningful content can be stored, while meaningless noise can be discarded or averaged out. In a remarkable result, the authors show that if you can find the optimal two-part description of the data, the second part (the noise) contains no useful information. You've squeezed all the meaning into the model. For example, they describe a scenario in which you take a long record (say, all the world's observations encoded in a string) and find an optimal model for it; the remainder would be incompressible noise, and the model's output would be a "de-noised" version of the world's data. This is a theoretical ideal, since in practice, it's impossible to do perfectly. Still, it's a beautiful insight. The best explanation not only explains but also cleans up the data. It identifies which parts of the data follow a law-like structure and which are accidental randomness.

Philosophically, this relates to how we do science. When multiple theories can explain data, we tend to prefer the simpler one (Occam's razor) and the one that doesn't make the data look miraculous. Algorithmic statistics provides a language for discussing this with quantitative precision. It's not hard to see echoes of this in modern AI as well. When we train models, especially unsupervised ones, we are effectively doing compression. For instance, world models in AI, such as those that learn the physics of an environment, compress the history of observations into a set of rules. A good world model can generate typical outcomes that match the real-world distribution, just as a good explanation in algorithmic statistics generates typical data for an Erdős problem.

In the same vein, the chapter's ideas connect to generalization in machine learning, where a model generalizes well if it captures the underlying pattern while ignoring noise. A good model doesn't overfit noise, since that would bloat the description, and it compresses regularities when possible. The implication is that any system that learns or explains—whether a scientist, a compression algorithm, or an AI—faces the same trade-off captured in algorithmic statistics. It must balance the model's complexity with the data's fit by minimizing model complexity while keeping the deficiency small.

One particularly compelling consequence is that we can regard AI and compression as two versions of the same goal. For example, an AI like DALL-E or GPT compresses knowledge during training and then generates output. It uses an internal model that, in a sense, is a two-part code

Licensed to Hermilo Cortés <hermilocg@gmail.com>

for the world: the model and its weights, plus the residual randomness when producing a specific image or sentence. If that model overfits the data, it's like carrying a giant lookup table. There is no learning. If it underfits, it is too simple and will fail to capture the structure, leading to large errors, such as using a straight line to fit a complex curve. The sweet spot is a model just rich enough to capture what's regular, but not so rich that it begins to capture randomness as if it were regular.

8.2.2 Significance

Unlike the other works in this chapter, *Kolmogorov Complexity and Algorithmic Randomness* is not significant because it introduced a foundational definition or theorem. Published after algorithmic information theory had already become a mature field, the book gathers the subject into a systematic account of complexity, randomness, probability, and entropy, before moving on to more specialized topics such as common information, multisource algorithmic information theory, information and logic, and algorithmic statistics.

It's also historical. Where Li and Vitányi's *An Introduction to Kolmogorov Complexity and Its Applications* became the standard broad reference, Shen, Uspensky, and Vereshchagin's book carries forward the Moscow school's perspective, including work connected to Kolmogorov's seminar. Its significance for Sutskever's List is therefore different from that of an original research paper. It preserves a technical tradition while making clear how the compression-based view of randomness connects to probability, information theory, logic, and statistical explanation.

8.3 A tutorial introduction to the minimum description length principle

Minimum description length (MDL) asserts that the best explanation for data is the one that achieves the greatest compression. In simple terms, if data contains patterns or structure, we can describe it in fewer bits, whereas completely random data is incompressible. Peter Grünwald's paper, "A Tutorial Introduction to the Minimum Description Length Principle," opens with three 10,000-bit sequences. The first is `0001` repeated 2,500 times; the second is a sequence of coin flips; and the third has roughly four times as many 0s as 1s. The first sequence is highly regular. Instead of writing all 10,000 bits, one could say "`0001` × `2500`" and convey the same information. The random sequence has no discernible pattern. Therefore, no description shorter than the full sequence itself is possible. The third sequence is intermediate. It's not deterministically

Licensed to Hermilo Cortés <hermilocg@gmail.com>

patterned, but it exhibits statistical regularity, as its bias toward 0s can be exploited for compression. This illustrates MDL's core idea. When we find regularity or structure in data, we can compress it; conversely, successful compression provides evidence that we have captured that structure.

MDL is connected to Kolmogorov complexity, which is the ultimate measure of simplicity. In effect, MDL approximates Kolmogorov complexity. While we lose the guarantee of finding the shortest possible description of the data, we gain a feasible procedure for model selection. As Grünwald puts it, the price we pay for a practical MDL method is that there will always be some regularities we can't capture with a given model class. This is inevitable. No universal learner can capture *every* possible regularity. MDL offers a way to capture as much regularity as possible with the models at hand by balancing model complexity against data fit.

One of MDL's virtues is that it naturally guards against overfitting. Overfitting occurs when a model captures not only the underlying structure of the data but also its noise, leading to poor generalization. MDL guards against this by explicitly penalizing complexity. Even if a model achieves zero training error by perfectly matching every data point, the total code length (combining the description of the model and the data) ends up longer because it takes more bits to specify all the model's intricacies. In contrast, a simpler model may not fit the data perfectly and may leave residual error. But it can still result in a shorter total description because the savings from the model's simplicity outweigh the additional bits required to describe those residuals.

Grünwald illustrates this with a polynomial curve-fitting scenario (see figure 8.6). A linear model (left) is simple and underfits the data because it leaves systematic patterns in the residuals, failing to capture the data's curvature. A high-degree polynomial (middle) is complex. While it can

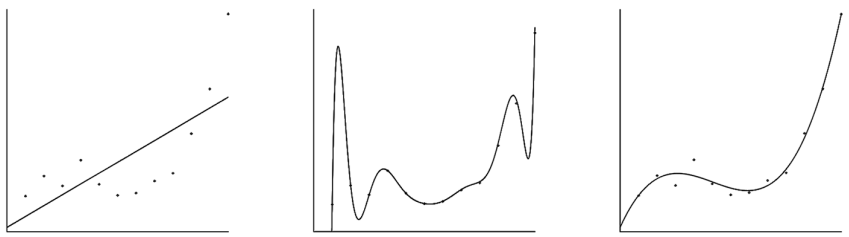


Figure 8.6 A simple polynomial (left) underfits the data, a complex polynomial (middle) can achieve zero training error but wastes bits describing noise, and a tradeoff polynomial (right) identifies the broad trend without including the noise. ***Used with permission granted by the lead author (Peter Grünwald).***

Licensed to Hermilo Cortés <hermilocg@gmail.com>

interpolate every data point and leave zero training error, it will waste bits describing the noise. The optimal compromise is a polynomial of intermediate degree that captures the broad trend without capturing the noise (right). MDL would select this model because it yields the shortest description, which in turn is a relatively simple functional form plus a short encoding of the modest residuals.

The paper's fundamental claim is that learning is about finding regularity, which manifests as compressibility. Whenever we can shorten the code for data, we've uncovered structure in it; when we can't, the data might as well be noise. A model that has captured the regularities will both compress the past and predict the future well. Grünwald's tutorial treats complexity as a *tool* for model inference, not just something we *study* (i.e., Kolmogorov's theory). This connects compression to prediction and dovetails with predictive modeling.

Sutskever's perspective on next-token prediction fits into this picture. Training objectives such as next-token prediction pressure an artificial neural network to compress its training data toward its Kolmogorov complexity. Although true Kolmogorov complexity is uncomputable, the claim is that large networks trained with gradient descent can better approximate this ideal than other models, making them practical instantiations of the compression-prediction linkage at the heart of MDL [16].

Ilya has shared results from OpenAI's iGPT (2020) experiment in interviews. iGPT is a GPT-style transformer trained without labels on low-resolution images to predict the next pixel and evaluated on separate supervised classification tasks [17][18]. The model learns strong internal representations even though it was never trained to classify anything. On CIFAR-10, iGPT achieves 96.3%, outperforming ResNet-152 (chapter 3).

The deeper significance is where the most usable representations live. In supervised learning, optimizing P(Y|X) promotes linear separability at the top of the network. The best features are found there because the final layer encourages the network to discard information that doesn't help predict the label Y. Yet iGPT's unsupervised training objective is closer to modeling P(X). That objective does not require class separation, so the model learns whatever structure best reduces uncertainty about pixels, including semantic structure and low-level details.

Interestingly, accuracy drops by ~2.4% in the final layer, relative to the best layer in the middle of the network. The authors describe this gap as "the difference between a baseline and a state-of-the-art result." The significance is that the most useful classification features peak in the middle of the network rather than at the end. This depth profile is

Licensed to Hermilo Cortés <hermilocg@gmail.com>

complexodynamics-like, with interestingness peaking in the middle and then fading in the top layers, especially when they are specialized for classification.

iGPT's success supports the thesis that next-token or next-pixel prediction forces a model to find meaningful structure. If a network can compress its training data, it has extracted whatever "secrets" the data contains. A corollary, stressed in Grünwald's tutorial, is that MDL is agnostic about truth. MDL-style model choices are not ontological or epistemic commitments. This doesn't mean there's no "world model" behind the data or that "world models" are nonsense. In MDL terms, a world model is just a particularly good compression scheme for the underlying data generating process.

When the data stream is anchored in a world we care about, the models that win on description length are precisely those whose internal structure mirrors key aspects of that world. Projects that chase a single reusable "world model"—like JEPA-style architectures trained on rich environment streams rather than text alone—are essentially bets that one shared compressor will be cheaper and more useful than a zoo of task-specific ones [19].

Seen this way, the fact that today's language models hallucinate ceases to be mysterious. Their training objective (i.e., next-token) doesn't force a faithful digital twin of the world. Rather, it is a refined, MDL-style compression goal. To the extent that a model achieves near-optimal compression, it must represent a large amount of the data's latent structure but not necessarily all the structure that exists or ultimately matters. That yields instrumental reliability, but it never asks whether the generated content corresponds accurately to the external world. In other words, we judge it by predictive compression, not by a prior declaration of truth about the real world.

The situation resembles the VLAE architecture discussed earlier (chapter 7), where the latent code preserves the large-scale structure of an image, while a PixelCNN decoder fills in locally plausible details. That division of labor works because many fine-grained images are compatible with the same coarse description. Language models perform an analogous operation with text. Their parameters and context encode a compressed model of linguistic and conceptual regularities, while generation fills in the remaining particulars token by token. The danger is that names, citations, case law, causal mechanisms, and mathematical steps are not mere visual texture. A continuation can be highly typical under the model, and therefore low in randomness deficiency relative to it while still failing to correspond to the external world.

This sort of investigation also shows why deterministic software is not superior to AI. It's just different. A conventional program can be repeatedly

Licensed to Hermilo Cortés <hermilocg@gmail.com>

wrong because it applies the same mistaken rule every time or brittle because the correct rule is applied only within the narrow conditions anticipated by its designers. An artificial neural network, by contrast, arrives at the correct answer as it converges on the underlying regularities in the data. In MDL terms, what matters is not exact surface reproducibility but whether the system has captured enough latent structure to generalize. Repeatable wrongness may be easier to debug, but it is not a general argument about correctness. That is why "it's just next-token prediction" is true but empty. It names the training signal, not the structure the model must learn to produce that signal.

8.4 Keeping artificial neural networks simple

In 1993, Geoffrey Hinton and Drew van Camp argued in "Keeping Neural Networks Simple by Minimizing the Description Length of the Weights" that artificial neural networks generalize better when the description length of their weights is minimized. In plain terms, the network learns not only from the data but also about its own complexity. The result is a model that achieves good accuracy on new data while using a minimal set of weights. This was an inventive demonstration that the abstract notion of Kolmogorov complexity can come to life in an artificial neural network. By minimizing the length of the weight description, Hinton and van Camp's work illustrates the theme of simplicity in complexity. They show that even complex neural nets can become simpler and better by following an information-theoretic filter toward a minimal description length.

The paper introduces a method for quantifying and minimizing a network's complexity in terms of its description length. Each weight's value is treated as something we might have to communicate in bits. More complex weight configurations require more bits to specify. By adding a differentiable bit-cost term to the training objective, the authors encourage the network to use fewer bits, thereby compressing itself, rather than quantizing or hard-coding weights, which would complicate optimization. Hinton and van Camp inject Gaussian noise into each weight. The amount of noise is adjusted during training so that unimportant weights can afford to be noisy, while truly critical weights are tuned more precisely. This strategy provides a smooth way to trade off accuracy for simplicity, allowing gradient-based optimization to determine how much detail each weight requires.

8.4.1 Methodology

The heart of the paper is an elegant training objective that minimizes total description length, defined as the sum of the bits required to encode

Licensed to Hermilo Cortés <hermilocg@gmail.com>

the model's weights and the bits needed to encode residual errors on the training data. Formally, the authors align this with MDL by treating the weight vector as a message to transmit, along with corrections for the network's output. The network's loss function is augmented to include a term for model complexity, measured in bits.

Hinton and van Camp show that, under their scheme, the expected cost (in bits) of specifying the weights is the Kullback–Leibler (KL) divergence. KL divergence measures the difference between two distributions and estimates the amount of information lost when one distribution approximates another. The KL divergence effectively measures how "surprising," or information-rich, the learned weight is relative to the prior. By summing the divergences across all weights, the authors obtain a numerical measure of the network's complexity in bits. Minimizing this term, along with the usual data error, is equivalent to finding the minimal description length model that best compresses weights and errors jointly.

To make this practical, each weight is treated as a random variable with a Gaussian distribution. Injecting noise into the weights during training implicitly explores a wide range of nearby networks, balancing accuracy and simplicity. High-variability weights are "cheap" in bits but reduce accuracy; low-variability weights improve fit but increase complexity. Training adjusts both the means and the variability to minimize the total description length.

8.4.2 Results

Hinton and van Camp tested their MDL training method on a demanding peptide activity prediction problem. The dataset contained only 105 training examples and 420 test examples, deliberately kept small to ensure that any method prone to overfitting would struggle. The network they used had four hidden units and 521 weights, far exceeding the number of data points. This setup made it an ideal testbed for a regularizer that balances accuracy and model complexity.

The results were striking. When trained with the MDL-based penalty, the network achieved a relative error of 0.286 on the test set. Relative error is a normalized metric in which lower values indicate better generalization, and a baseline that predicts the mean target value scores 1.0. By contrast, the same network trained without the MDL term achieved a relative error of 0.967, only slightly better than the trivial baseline. This difference shows how the MDL objective prevents the model from memorizing noise in the small training set. By charging "extra bits" for every overly precise weight, the method automatically limits the model's capacity to overfit.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

To benchmark against a familiar approach, the authors compared MDL with weight decay, a classical regularization technique (popularized by AlexNet) that penalizes large weights. After careful tuning, weight decay produced a relative error of 0.317, a significant improvement over unregularized training but still worse than the 0.286 achieved by MDL. Achieving 0.317 required trial and error to determine the appropriate penalty coefficient, and in practice, some of the training data had to be used for validation. Hinton and van Camp note that if one were to "peek" at the test set to tune this hyperparameter, the best weight decay model could achieve about 0.291, which is still inferior. The MDL approach automatically found this optimal trade-off without external tuning.

They also included a linear regression baseline as a reference. Without regularization, the linear model catastrophically overfitted the 105 data points, yielding a test error of 35.6, hundreds of times worse than the artificial neural network's. But adding strong ridge regularization, which penalizes large coefficients, reduced this to about 0.291, comparable to the tuned weight-decay network. This suggests that, under strong regularization, the hidden units of the nonlinear network behaved linearly. This convergence across methods reinforces the intuition that successful generalization here came from simplifying the model's internal representation rather than exploiting its nonlinear capacity.

Beyond numerical metrics, the authors highlight qualitative evidence that the network organized its knowledge more efficiently under the MDL framework. By the end of training, many weights had grouped into a few distinct clusters. The learned coding prior, modeled as a mixture of five Gaussians, reshaped itself to fit these clusters. In other words, most weights fell into a few predictable values, each of which could be encoded using very few bits.

This self-compression is precisely what the MDL principle predicts: minimizing the total description length pushes the model toward the simplest representation consistent with the data. The network not only generalized better on unseen molecules but also invented its own compact internal language to represent what it learned. These results strongly support Hinton and van Camp's thesis that learning as compression yields models that are both efficient and robust.

8.4.3 Limitations

Despite its elegance, the method has some limitations. First, the algorithm's complexity is nontrivial. Unlike standard backpropagation, it requires maintaining and updating a distribution for each weight, including its

Licensed to Hermilo Cortés <hermilocg@gmail.com>

mean and variance, and, in the authors' implementation, even the parameters of a mixture prior. This at least doubles the number of trainable parameters and requires computing how uncertainties propagate through nonlinear activations. Hinton and van Camp had to use approximation tables and perform careful numerical checks to implement the gradients correctly, which shows how laborious this was. This complexity would hinder scaling to larger networks or datasets, especially in 1993, when the paper was published. Indeed, the paper reports only experiments with a single hidden layer and a relatively modest number of parameters.

Moreover, the evaluation was limited. The authors described their results as "preliminary" and did not exhaustively compare them with other regularization methods or larger benchmarks. Subsequent research found that simpler techniques, such as well-tuned weight decay or early stopping, often achieved similar results with far less hassle [20][21]. This explains why the MDL approach didn't catch on in practice.

8.4.4 Cultural influence

In terms of cultural effect, this paper did not revolutionize network training. While the research was ahead of its time, the community in 1993 lacked the computational resources or the urgency to adopt such a complex regularization method. But its bit-centric view is evident in modern practices such as model compression and quantized networks. Today, we use variants of this idea, such as weight decay, dropout, sparse priors, pruning, quantization, and distillation, that all shrink the model's description. This paper merits inclusion on Sutskever's List because it exemplifies the unifying pursuit of simplicity, where simplicity, measured in bits or description length, is key to achieving better performance.

8.5 Grokking

We've seen complexity defined for physical systems, for individual strings, and for models via MDL. The last section examines how modern deep neural networks train, connecting our previous topics to grokking and double descent.

Grokking describes how an artificial neural network discovers a generalizable solution after memorizing its training data perfectly [22][23]. It is a striking phenomenon in which test accuracy jumps from guessing to nearly 100% long after the model has overfit the training set. This delayed "aha!" moment raises questions about the changes within the network as it transitions from brute-force memorization to generalization.

The paper "Complexity Dynamics of Grokking" addresses this mystery by measuring the intrinsic complexity of the model's learned

Licensed to Hermilo Cortés <hermilocg@gmail.com>

representation during training [24]. Branton DeMoss and colleagues find that a network's internal complexity undergoes a phase transition during grokking: it rises as the network memorizes the data, then falls sharply once it learns a simpler underlying pattern that enables generalization. Crucially, this rise-and-fall pattern appears only under sufficient regularization. The unregularized network remains highly complex, essentially memorizing every detail, and never generalizes. See figure 8.7.

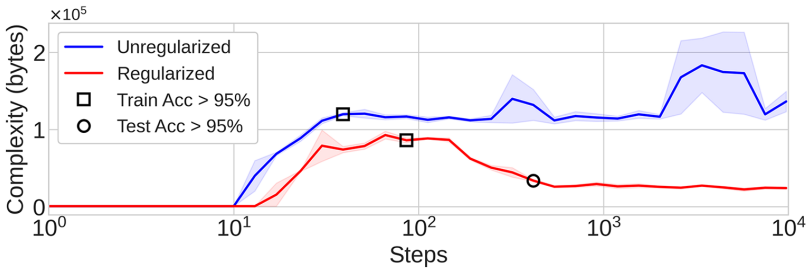


Figure 8.7 Complexity dynamics for unregularized (top) and regularized (bottom) neural networks in grokking experiments on modular multiplication. Markers denote when memorization (□) and generalization (○) occur. In the regularized network, generalization occurs when complexity falls after peaking at memorization, whereas the unregularized network never generalizes and its complexity remains high. Mean over three seeds; shading indicates the standard error. *Used with permission granted by the lead author (Branton DeMoss).*

The authors explicitly cite the coffee automaton's "apparent complexity" as inspiration. It's as if an artificial neural network becomes more complex while "mixing" raw data, then settles into a simpler macroscopic configuration. The intermediate epoch corresponds to the network's weights or representations at maximum complexity, just before the network simplifies into a generalizing solution. Though this isn't in the coffee automaton paper, it extends the paper's ideas, underscoring its relevance not just to physics but also to machine learning and generalization.

The authors' central claim is that when a model truly learns the general rule, it becomes "simpler" and more compressible than when it merely memorizes. A network effectively "memorizes" when it captures the idiosyncratic features of the training set, which requires substantial information capacity and increases complexity. Training accuracy increases during this phase but so does the compressed model size, as the network encodes more information to capture each training example.

But after many iterations, the model discovers the underlying pattern. At that point, test accuracy improves while the model's complexity

Licensed to Hermilo Cortés <hermilocg@gmail.com>

decreases. The model has transitioned to a simpler internal representation that no longer memorizes individual examples but instead encodes the general rule. The learning algorithm now finds a much shorter description for the data. Researchers have shown that this transition coincides with a "cleanup phase," in which the model removes the memorized examples it relied on early in training [25]. Essentially, the network "forgets" random details and focuses only on what's essential.

This is very different from older pipelines such as SIFT+SVM (chapter 2), which fixed the features and only learned a decision boundary between classes. If those fixed features didn't expose the true structure, the classifier had to store many per-example quirks, leading to high complexity and poor generalization. Deep networks, in contrast, learn features directly, reorganizing the representation so that the rule becomes shorter. In MDL terms, deep learning wins because it can compress the total description length by improving the representation, rather than merely fitting a boundary on top.

It's worth resurrecting Alyosha Efros' joke (chapter 2) about "the 'fancy schmancy' [artificial] neural network. . .doing no better than a simple nearest neighbor," and the unreasonable effectiveness of data, both of which positioned data as a great equalizer. That is, with enough data, even simple methods could go far. But the conclusion assumes a fixed representation where "nearest" already means something useful. Yet deep learning is a solution that rewrites the feature space. It simplifies the problem itself by learning and compressing representations. Ultimately, architecture matters, and when scaling, we must always ask, "Scale what?" The important scaling question is not only "how much data?" but also the architecture and the learned representations. Pretty *fancy schmancy* after all.

In their experiments, DeMoss et al. focus on small algorithmic tasks, such as modular arithmetic operations, that are known to exhibit grokking behavior. They show that a network's description length (in bits) peaks during overfitting and then collapses at the grokking moment, when test performance surges. See figure 8.8. This behavior is analogous to a phase change in physics, much like water boiling or a magnet losing its magnetization. The learning system transitions from a state characterized by high complexity and memorization to one characterized by low complexity and generalization. The paper's thesis is that when the network stops memorizing and begins to learn, it effectively finds a shorter description of the data. As the authors put it, "the model which generalizes best is the one which compresses the data best and is itself most compressible."

Licensed to Hermilo Cortés <hermilocg@gmail.com>

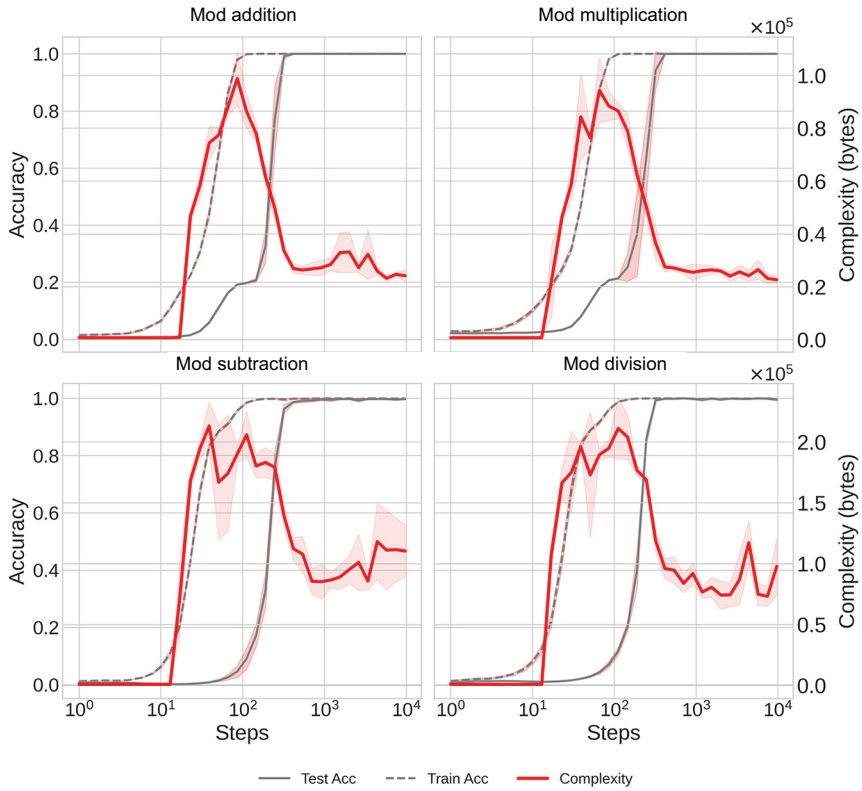


Figure 8.8 Complexity during grokking on modular arithmetic tasks. Gray curves show train and test accuracy; the thick curve shows the estimated complexity of the learned representation. Across modular addition, multiplication, subtraction, and division, complexity rises during the memorization-to-generalization transition and then falls once the model settles into a simpler rule-like solution. The peak occurs near the interval where training accuracy (dashed gray) is already high, but test accuracy (solid gray) has not yet fully caught up. As models memorize, complexity increases; as they generalize, complexity (measured in bytes) decreases. *Used with permission granted by the lead author (Branton DeMoss).*

8.5.1 Compression

The authors develop a custom compression scheme that blurs out inessential details while preserving accuracy within a small tolerance. They're not just shrinking the network's weights; they're simplifying what the network *does*. This means the compression occurs in function space, not merely in parameter space. As a result, two networks that behave almost identically (that is, they make nearly the same predictions) are treated as equivalent, even if their internal parameters differ. The result is a more meaningful form of compression that prioritizes the model's behavior over its microscopic details, echoing the work of Aaronson, Carroll, and Ouellette.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The compression pipeline has two main steps. First, weight quantization maps floating-point weights onto a small set of discrete values (e.g., 8-bit codes). To preserve accuracy, *quantization-aware training* injects noise or simulates rounding during training, making the network more robust to errors. This idea traces back to Hinton and van Camp's MDL framing, as cited by DeMoss et al. After training, the model can be quantized with minimal loss in accuracy. Intuitively, this removes reliance on tiny, per-example weight tweaks, preserving the coarse structure that matters.

The other step is low-rank approximation. A large weight matrix may contain many parameters yet still be well approximated by a much lower-rank matrix if its rows and columns mostly recombine a smaller set of underlying patterns. In that case, the matrix can be factored into thinner matrices that preserve the dominant directions while discarding redundant ones. This loosely connects to Hinton's dropout rationale (chapter 2), which disrupts such coadaptations by randomly setting neurons to zero during training, forcing features to remain useful across different contexts. Both approaches discourage brittle dependencies but in different ways. Dropout makes representations less dependent on any particular unit; low-rank approximation is post hoc and compresses the shared structure that remains.

Notably, the authors achieve compression ratios of 30× to 40×. In fact, a naive compressor captures only a small portion of the training dynamics. Suppose you gzip the model at different points in training. In that case, the file size remains about the same, because it is dominated by random bits left over from initialization and the stochastic training procedure. By contrast, their sophisticated lossy compression is sensitive enough to detect subtle changes in the model's true complexity over time. In one example, as the network learns modular multiplication, its compressed length initially increases, indicating that it is encoding more specific information to fit the training data. Then, at the grokking point, the compressed length plummets, indicating that the network has found a much simpler representation of the multiplication rule. This provides evidence that a network has effectively compressed the dataset into a simple rule, rather than memorizing each example.

8.5.2 Theory to practice to theory

Historically, practitioners learned a tidy picture of model performance. As model capacity grows, training error (i.e., loss on the training set) falls, while test error (i.e., loss on held-out data) follows a U-shaped curve. Many practitioners (me included) internalized this as a causal story in which small models underfit (i.e., high bias), mid-sized models hit a sweet

Licensed to Hermilo Cortés <hermilocg@gmail.com>

spot, and large models overfit (i.e., high variance). The textbook demo of the so-called bias-variance tradeoff is polynomial fitting, similar to Grünwald's demonstration (figure 8.6). A line underfits parabolic data; a quadratic function fits the data; and higher-degree polynomials chase noise. Regularization was understood as mitigating this behavior by limiting capacity or improving robustness. The classic interpretation of this lesson was to match model complexity to data complexity and stay near the bottom of the U. See figure 8.9 (left).

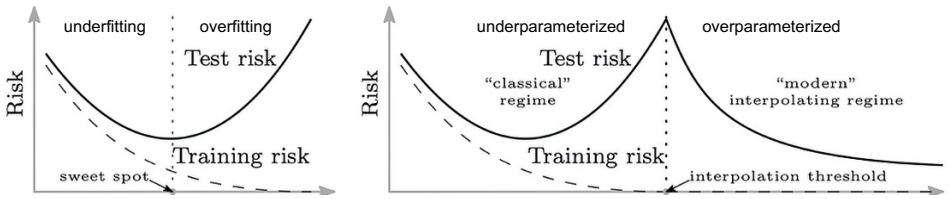


Figure 8.9 Left: the classical bias–variance tradeoff. Training error (called "risk" in this image) is shown as a dashed line. It falls monotonically, whereas the test error (solid line) is U-shaped, with the bottom being optimal. Right: beyond the interpolation threshold, test error peaks and then declines again in the overparameterized (interpolating) regime, while training risk remains near zero. *Used with permission granted by the lead author (Mikhail Belkin).*

AlexNet (2012) had 60 million parameters, and the authors were concerned about overfitting during training. They used data augmentation, dropout, and weight decay because—without those techniques—overfitting seemed obvious. The takeaway was that such large networks operated in the overfitting regime, on the right side of the U, and had to be pulled back toward the middle through regularization. Yet as deep learning scaled, something extraordinary appeared. Bigger, deeper models continued to generalize even with modest or no explicit regularization.

In modern overparameterized settings, the traditional U-shaped curve is incomplete. Beyond the interpolation threshold, test error can decline again, and regularization often enables very large models to generalize well. See figure 8.9 (right). Here, "overparameterized" means the model has far more parameters than it needs to fit the training data, so it can perfectly interpolate, fitting every training example exactly. The interpolation threshold is the capacity point at which this becomes possible. Beyond that point, many modern networks still generalize well because explicit regularizers and the optimizer's implicit biases steer them toward functions that fit the data exactly but remain simple enough to perform well on new data.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In 2016, Chiyuan Zhang and colleagues highlighted this divergence by randomizing labels in CIFAR-10 and ImageNet, with surprising results [26]. On the CIFAR-10 dataset, a smaller AlexNet model achieved 99.82% accuracy on the training set of 50,000 images. See table 8.1.

Table 8.1 Training and test accuracy of a smaller version of AlexNet on the CIFAR-10 dataset. The original AlexNet had 60 million parameters.

Model	Number of parameters	Labels	Random crop	Weight decay	Train accuracy	Test accuracy
AlexNet	1,387,786	true	yes	yes	99.90	81.22
		true	yes	no	99.82	79.66
		true	no	yes	100.0	77.36
		true	no	no	100.0	76.07
		random	no	no	99.82	9.86

On the ImageNet dataset, AlexNet achieved 95.20% (top-1) training accuracy, which is surprisingly high given the presence of over 1 million random labels across 1,000 categories. As shown in table 8.2, the results indicate that deep networks can effectively memorize *random* labels.

Table 8.2 Training and test accuracy of a smaller AlexNet on the ImageNet dataset

Labels	Data augmentation	Dropout	Weight decay	Top-1 train	Top-5 train	Top-1 test	Top-5 test
true	yes	yes	yes	92.18	99.21	77.84	93.92
true	yes	no	no	92.33	99.17	72.95	90.43
true	no	no	yes	90.60	100.0	67.18 (72.57)	86.44 (91.31)
true	no	no	no	99.53	100.0	59.80 (63.16)	80.38 (84.49)
true	AlexNet (KSH, 2012)	–	–	–	–	–	83.6
random	no	yes	yes	91.18	97.95	0.09	0.49
random	no	no	yes	87.81	96.15	0.12	0.50
random	no	no	no	95.20	99.14	0.11	0.5

While that is easy to say, its implications are profound. This means that artificial neural networks can memorize an entire dataset of random labels, even with regularization. In fact, removing augmentation, dropout,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

and weight decay hurts test accuracy only modestly. The supposed tight coupling between lower training error and higher test error in the "overfitting regime" began to appear questionable. Moreover, the authors showed that this memorization was relatively easy. Training time increased only slightly (not shown here) compared with training on the true labels. The experiment showed that low training loss doesn't imply learning meaningful structure, and architectural biases aren't enough to prevent memorization; otherwise, the networks wouldn't fit random labels this well.

8.5.3 Double descent

In 2018, a paper by Mikhail Belkin and colleagues showed that these models exhibited the classical bias-variance tradeoff curve, but then suddenly shifted to a new regime as model size increased further, with test-set performance improving dramatically and exceeding that in the classical regime [27]. They named this phenomenon "double descent." Their hypothesis was compelling, but it didn't answer whether double descent could be replicated in full-scale deep models or what underlying mechanisms could be causing this behavior. Fortunately, we didn't wait long for the answer.

In 2019, Ilya Sutskever and researchers at Harvard made double descent concrete across three families of architectures, including convolutional neural networks (chapter 2), ResNets (chapter 3), and Transformers (chapter 5). They showed that test error versus model size can dip, rise near the interpolation threshold, and then dip again [28]. The authors explain what is happening:

> *Informally, our intuition is that for model sizes at the interpolation threshold, there is effectively only one model that fits the training data, and this interpolating model is very sensitive to noise in the training set and/or model misspecification. That is, since the model is just barely able to fit the training data, forcing it to fit even slightly noisy or misspecified labels will destroy its global structure and result in high test error. By contrast, for overparameterized models, there are many interpolating models that fit the training set, and stochastic gradient descent (SGD) can find one that "memorizes" (or "absorbs") the noise while still performing well on the distribution.*

In other words, when a model is just big enough to fit the training set, it has one very brittle solution, so any noise wrecks test accuracy. But as the model size increases, there are many solutions, and SGD finds one that suppresses the noise, leading to a second descent in error.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

From a complexity perspective, a medium-sized model that barely fits the data will do so in a complex way. By contrast, a larger model has sufficient capacity to find a simpler explanation that fits all the data. In other words, larger models aren't better because they're using all those extra parameters to memorize more; they're better because they can use that capacity to find a compression of the data that a smaller model couldn't represent. DeMoss et al. borrow from Aaronson and call this "algorithmic thermodynamics." The rise-and-fall pattern could be a universal feature of self-organizing complex systems that first explore and then, as complexity increases, compress.

Double descent changes how we traditionally think about model capacity. A big network can choose to act like a smaller, simpler network if it's guided to the right solution, but a small network can't decide to act bigger than it is, as we saw with TinyLlama in chapter 6. Thus, bigger is better in modern deep learning because—with proper training—large models can end up simpler and less complex than moderately sized models at the interpolation threshold (see figure 8.10). This ties together grokking and the success of scaling. In both cases, simplicity wins, but sometimes you need to go big or train longer to find it.

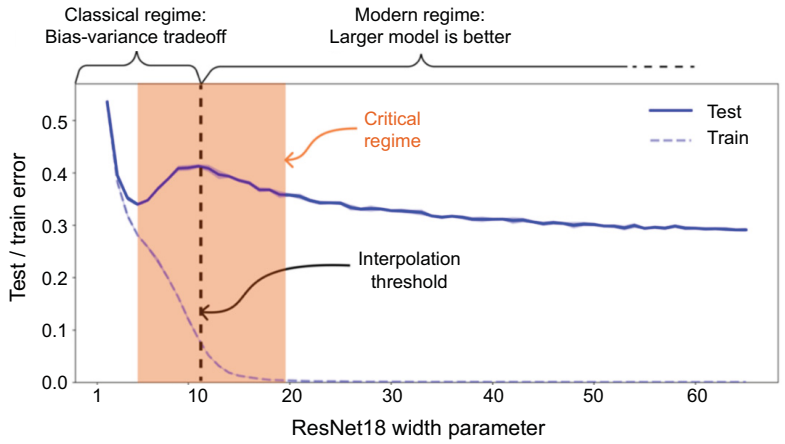


Figure 8.10 Test (solid line) and training (dashed line) errors for ResNet as the width increases. A peak near the interpolation threshold (dashed line; shaded "critical" band) illustrates double descent, in which error rises as the model fits the data, then falls again for larger widths. ***Used with permission granted by the lead author (Preetum Nakkiran).***

Deep double descent may also help explain scaling laws. A model's complexity grows when its capacity matches the dataset. This is the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

interpolation threshold, where test error briefly spikes (see figure 8.10). Modern scaling laws explicitly avoid that ridge. Kaplan's scaling (chapter 6) stops training early, keeping complexity below the threshold and yielding smooth power-law curves. Chinchilla's (chapter 6) recipe goes further, balancing parameters and data so models stay in data-rich regimes where the ridge never appears. When researchers follow these rules, loss falls with scale. Break them by fixing the model size, starving the model of data, or training it to a perfect fit, and the ridge reemerges. The scaling curves that many celebrate today are not exceptions to deep double descent; they suggest that we've learned to fly around it.

In a 2019 story, Andrej Karpathy describes how he accidentally left a network training over a winter holiday break and, upon returning in January, discovered that the model had achieved state-of-the-art results [29]. In Karpathy's experience, "networks keep training for [an] unintuitively long time." The anecdote serves as a warning to practitioners tempted to stop a training run once the validation loss plateaus. Doing so can be premature, causing them to miss gains that emerge later.

That kind of surprise is part of what Ilya Sutskever has pointed to when discussing double descent. On Lex Fridman's podcast, Ilya confirmed that only a few researchers had noticed odd cases in which larger models performed better in ways that seemed to defy standard theory. He also said his team showed that double descent appears "in pretty much all practical deep learning systems" when standard regularization is absent. He emphasized how "counterintuitive" it is because you'd expect performance to be monotonic, as classical intuition suggests, and remarked on how the effect "went unnoticed for so long" [30][31]. These observations underscore that double descent is not merely a theoretical curiosity but an endemic feature of real-world network training, hidden behind dynamics that appear flat or finished right before they become interesting.

Indeed, as the OpenAI blog post on the paper notes, researchers often miss the telltale peak because it can be "small and easy to miss" if they don't look past the interpolation point [32]. This suggests that decades of focus on avoiding overfitting through regularization and early stopping led the field to overlook the double descent curve. Ultimately, double descent did not completely contradict the bias-variance tradeoff, but it did show that it is incomplete. It's also telling that Sutskever's name is on the paper that replicated double descent on full-scale models. After all, he was there scaling convolutional neural networks, recurrent neural networks, and Transformers. It's fitting that he was also among those who crossed the interpolation threshold and reported what lay on the other side.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

8.6 Blurry JPEG

In his fireside conversation with NVIDIA CEO Jensen Huang, Ilya Sutskever pointed to Ted Chiang's 2023 *New Yorker* essay titled "ChatGPT Is a Blurry JPEG of the Web" [33]. Sutskever agrees with the compression metaphor but interprets it through a different conceptual frame. Chiang says there is no use for a "blurry copy of the web when you have the original." His argument dates back to Plato's allegory of the cave, which outlines the relationship between an object and its representation. The same tradition has been expressed in other terms, such as "the map is not the territory" (Alfred Korzybski), "all models are wrong" (George Box), and "the menu is not the meal" (Alan Watts).

But Chiang's framing overlooks that some models are useful. It is not that maps, models, and menus are useless; rather, a map the size of the territory is not useful in practice. A map is not the territory it represents, but its similar structure makes it valuable. A map the size of Nebraska would be Nebraska, which is intractable. But a map of Nebraska that fits into your glovebox will help you find your way to Lincoln. Likewise, the web is intractable and not directly helpful unless you know exactly where you want to go and how to get there, prefer blind exploration, or have a map.

Yet Chiang misclassifies the type of map this is a common trap that many skeptics fall into. They seem to treat a "blurry copy of the Web" as a lossy backup, where loss denotes unreliability. But from the Kolmogorov and MDL perspective we've been developing, the "blurry JPEG" is not a bad archive, but a short program for the web's generative process, where loss means abstraction. These are different categories, and treating language models as archives inevitably leads to underestimating their value..

From a compression perspective, the blurry copy is a new interface that turns scattered documents into synthesized answers and actions. Confidence in this synthesis should be lower when a model is forced into ungrounded generation, even if MDL-style prediction can be instrumentally useful. Ultimately, what becomes less relevant is not the web itself but the way humans navigate it and the machinery we have built around it. Search engines, websites, browsers, and large swaths of "search-and-stitch" software development that mainly consisted of looking things up and gluing snippets together are all less valuable today because of AI's ability to learn effective data compression. From Chiang's perspective, that loss is a defect, and compression is an archive; from Sutskever's perspective, compression is a new interface rather than a copy. When we drive systems to predict and compress, simplicity emerges within overwhelming complexity.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

9 Safe superintelligence

This chapter covers

- The definition of intelligence
- Epicurus + Occam + Bayes + Solomonoff
- AIXI
- The superintelligence canon
- Mistaking definitions and benchmarks for explanations
- Intelligence as a public test

Papers

- *Machine Super Intelligence* (2008) Shane Legg

This chapter asks what "intelligence" means, how we should measure it, and what kinds of claims those measurements can (and cannot) justify. It begins with Shane Legg synthesizing dozens of competing definitions of intelligence into a performance-based view of AI. That view builds a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

universal intelligence measure grounded in a philosophical lineage that spans Epicurus' tolerance for multiple explanations, Occam's preference for simplicity, Bayes' rule for updating beliefs, and Solomonoff's idea of induction.

The chapter also introduces AIXI, a proof of concept that, under Legg's definition, is "maximally" intelligent. That formalism opens into safety questions, including reward hacking and wireheading, and into the broader superintelligence canon, tracing how speculative forecasts and AI control anxieties evolved alongside mainstream empirical AI research.

The chapter concludes by analyzing definitions and benchmarks. It explores intelligence as something we attribute through public criteria and tests while remaining cautious of category mistakes. In the context of Sutskever's List, Legg's work serves as a unifying finale. It provides a formal attempt to define artificial intelligence that also exposes why the field remains caught between hype, skepticism, safety anxiety, and pragmatic engineering.

9.1 Machine superintelligence

While Shane Legg is a cofounder of DeepMind, he is less well-known than cofounders Demis Hassabis and Mustafa Suleyman. But his research occupies a unique place in the canon of artificial general intelligence (AGI) literature. In his dissertation, Legg tries to make "general intelligence" mathematically tractable by presenting it as something that can be defined, measured, optimized, and, in principle, achieved.

This section has four steps. First, Legg defines intelligence as the ability to achieve goals across many environments. Second, he connects that definition to a theory of learning where an intelligent agent must infer what kind of world it is in. Third, he shows what a maximally intelligent agent (AIXI) would look like under this formal setup. Fourth, he exposes safety problems within that framework. For example, if the goal, reward signal, or environment is wrong, a capable agent can become dangerous. The following details are abstract, but the question is practical: "Once intelligence becomes a score, what exactly is that score rewarding?"

9.1.1 Defining intelligence

Legg opens with what he calls the most fundamental question: "What is intelligence?" He acknowledges that no comprehensive survey of definitions has ever been published and notes that most textbooks and research

Licensed to Hermilo Cortés <hermilocg@gmail.com>

papers skirt the question, offering only passing nods to the Turing test or vague remarks that "intelligence is poorly defined" before moving on to technical details. This suggests a lack of maturity in a field concerned with intelligence, where technical progress has outpaced conceptual clarity.

Legg's opening section corrects that omission by giving the question of "what is intelligence?" the attention it seems to deserve. All told, he finds 70 formal and informal definitions, drawn from literature, psychology, and biology, which may constitute the most extensive and well-referenced collection on the subject. Still, using an assortment of literary definitions to define intelligence relies heavily on anthropocentric investigation. After all, literary and dictionary definitions focus on natural intelligence. There may be no other kind of intelligence, but even so, the goal risks becoming idealized rather than material.

Moreover, present-day conceptions of intelligence are lacking, and historical definitions may seem quaint. In fact, the collated definitions reflect a strange attitude: that there may be nothing special about intelligence, as one definition puts it: "Intelligence is what is measured by intelligence tests" [1]. Rather than defining what intelligence is, many definitions focus on how to test for it, using classic cognitive tests (for example, IQ tests) and earlier tests of machine intelligence, such as the Turing test. In the Turing test, a machine is considered intelligent if a human judge cannot reliably distinguish it from a human in a text-only conversation, treating intelligence as whatever can mimic human conversational behavior.

Legg distills these tests and definitions into a formal definition of intelligence as "an agent's ability to achieve goals in a wide range of environments." This perspective is performance-oriented. It doesn't care *how* the agent thinks, only that it can perform well across diverse tasks [2]. Readers skeptical of collecting definitions may ask, "To what end?" Why collate 70 definitions of intelligence only to conclude that intelligence is what intelligence tests measure? The answer may be that such performance-based definitions remove conceptual and philosophical roadblocks.

Nevertheless, defining intelligence this way has significant implications, which Legg does not deny. By Legg's definition, a machine could far exceed human intelligence without having *real* intelligence. Beyond the question of whether superintelligence is possible, this raises the question of what superintelligence means if we don't care how an agent thinks or whether it thinks at all. Yet once you redefine intelligence as its measurement, you can, in principle, imagine a machine achieving a higher score than a human. We'll revisit these questions later.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In his dissertation, "Machine Super Intelligence" (2008), Legg defines what it means for a system to be intelligent and develops a formal measure that weights performance across computable environments, giving preference to simpler ones [3]. The result is a universal intelligence measure that could, in theory, be used to compare any platform, including hypothetical superintelligences.

9.1.2 Universal intelligence measure

In 2007, real-world systems weren't very powerful, generally lacked data, and had limited computing. They solved specific tasks but lacked the all-purpose flexibility we'd expect from something truly "intelligent." Consequently, Legg considers idealized agents that ignore resource constraints.

To ground such a theoretical agent, Legg defines the inductive foundations beneath it. Inductive inference asks how an observer can, given data, infer the process that produced it. It has a long history in philosophy—really long. Legg reaches back to Epicurus in the third century BCE and his principle of *multiple explanations*, which roughly states that if several hypotheses fit the data, you're not entitled to discard any of them. He uses a cookie jar to make this concrete:

- Hypothesis A (h_a)—Your child used the chair to reach the cookies.
- Hypothesis B (h_b)—A small burglar broke in, moved the chair, and stole cookies.

Both hypotheses are logically compatible with the chair's position and the missing cookies. According to Epicurus, we shouldn't rule out h_b purely on logical grounds, because it may suddenly seem more plausible if we later see that the TV is gone. This principle requires us to retain all hypotheses that are consistent with the data. That may be uncomfortable because it means entertaining bizarre explanations, but it aligns with Epicurus.

Meanwhile, Occam says, "Given multiple hypotheses that are consistent with the data, the simplest should be preferred." Consider the following sequence: 1, 3, 5, 7… What do we predict next? A simple program might predict 9 after noticing a series of positive odd numbers. Another might predict something weird, like 33, after concluding that a complicated polynomial generated the numbers. Both fit the existing data perfectly, but we intuitively favor the simple program. Occam's razor formalizes that preference. Intelligence tests also rely on this intuition, with pattern-matching questions expecting the test-taker to select the simplest generalization [4].

These two philosophical constraints inform Legg's work and set the stage for why his theoretical agent must consider all computable

Licensed to Hermilo Cortés <hermilocg@gmail.com>

programs, weighted appropriately. Induction, done correctly, never throws away a consistent, valid explanation, but it doesn't treat all hypotheses equally either.

Legg uses Bayes' rule to turn hypotheses (or programs) into concrete probabilities. We seek the probability that a hypothesis (h) is true, given the observed data (D). Bayes' rule states that one starts with a prior $P(h)$, which represents the plausibility of each hypothesis before observing any data. Then we combine it with how well each hypothesis explains the data, $P(D|h)$, to get the posterior $P(h|D)$. In other words, beliefs are updated by reweighting each hypothesis in proportion to its fit to the observations. While this framework is robust, it is controversial because of the prior $P(h)$. After all, what does it really mean to assign a probability to a hypothesis before seeing data [5]? This is one reason Legg reaches for a more principled, universal prior rather than an ad hoc one.

Legg recasts inductive inference as a *binary sequence prediction* task, in which the goal is to predict the next bit in an endless stream of `0`s and `1`s. Imagine a continuous stream of coin flips. Because we don't know how the coin works, we keep several possible hypotheses in mind. One hypothesis says, "It's a fair coin," another says, "It's biased," another says, "It repeats a fixed pattern," and so on. Whatever the case, there is some unknown process that spits out an infinite sequence of bits. Binary sequence prediction treats every learning problem as a version of this game and asks: given the past, what's the next bit, and what should we believe about the next one?

Bayes' rule determines how much to trust each program that generates these sequences. As we watch more bits, we increase our trust in programs that predicted the history well and decrease our trust in those that didn't. Ultimately, we can either pick the single program we trust most and follow its prediction or "vote" on the next bit based on all programs, weighting each program by how much we trust it. The latter is the Bayes mixture predictor, which functions as a crowd of hypotheses, with each program speaking up in proportion to how well it has done so far.

The foundations of universal intelligence trace back to the origins of inductive inference, starting with Ray J. Solomonoff's work in the 1960s. Solomonoff discovered that an inductive inference system could predict any sequence according to a computable probability distribution. Solomonoff's model can recover well-known statistical principles, such as minimum description length (chapter 8) [6]. Solomonoff's theory can simultaneously keep all explanations consistent with the data (Epicurus) and prefer the simpler ones (Occam) in a way that works for any pattern. It can run any program you give it, but it favors simple programs.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In Solomonoff's setup, each pattern receives probability mass from the programs that generate it, with shorter programs receiving greater weight. Imagine two patterns. The first can be generated by a tiny 10-bit program. The second is so messy that the shortest program that produces it is 100 bits long. If programs are sampled by flipping a fair coin for each bit, then a particular 100-bit program has a probability 2^{-100}, while a 10-bit program has a probability 2^{-10}. The longer program, therefore, receives only 2^{-90} as much prior weight, meaning it is 2^{90}, or roughly 1.2×10^{27} times less likely. Solomonoff's universal prior assigns exponentially less probability mass to strings whose shortest description is longer [7].

In plain terms, Legg needs Solomonoff because he wants an intelligence measure that does not merely reward success but success across every computable kind of world an agent might face. Instead of assigning prior weight directly to individual strings, the goal is to consider the full space of computable models that represent every conceivable rule for generating bit streams. These models include every computable hypothesis, never throw away a possibility that fits the data, and automatically favor simpler explanations without hand-tuned assumptions. This combination makes universal priors the mathematical backbone of Legg's project, representing an idealized form of intelligence that learns as well as any computable agent could.

9.1.3 AIXI

Building on his universal intelligence measure, Legg turns to AIXI, an idealized reinforcement learning (RL) agent that, by construction, is maximally intelligent with respect to that measure. AIXI uses the same universal prior over computable environments and selects the action that maximizes expected future reward. In that formal setting, it is Bayes-optimal and achieves the highest possible score on the universal intelligence measure. Because AIXI is incomputable and assumes unbounded resources, this claim concerns what is possible *in principle* and serves as a theoretical proof of concept for superintelligent AI in Legg's thesis.

A helpful way to view AIXI is as a normative ceiling rather than a ladder. If you ask, "Could a machine really be superintelligent?" AIXI delivers a carefully hedged "yes, in principle." "In principle" means that if you accept Legg's formal definition of intelligence and ignore resource constraints, an agent could exist that would be maximally intelligent by that standard. Legg makes a classic rationalist move by sliding from an existence proof to an "in principle" claim about superintelligence. Yet this "yes, in principle" answer leans heavily on the AIXI RL-style notion of intelligence.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Simultaneously, Legg dispels the idea that humans possess a mystical property that machines could never possess. He considers common beliefs such as that "machines can only ever be strictly logical," that "they cannot do anything they are not programmed to do," or that "they could not do anything superior to their own creator" to be "baseless myths." Yet these myths are "baseless" only within the formalism itself, which, by construction, removes conceptual barriers. Remember, AIXI is not a machine we can build.

In 2008, AI had few empirical successes to support such claims, and formalism did most of the heavy lifting. Still, Legg's attempt to dispel these "baseless myths" follows a broader rhetorical pattern during this period. Max Tegmark, a Swedish American physicist at MIT and author of *Life 3.0*, regularly criticizes what he calls "carbon chauvinism," the predisposition that only biological organisms can be truly intelligent [8]. He argues for a substrate-neutral view of intelligence that fully includes machines but often leans on analogies to human learning. He notes that today's machine learning systems improve from data in a way that mimics how children learn from experience. Computer vision models might need "lots and lots of examples" to learn what a cat looks like, whereas a human child can generalize from just one. There is an obvious tension here. Tegmark denounces "carbon chauvinism" while relying on paradigmatic carbon-based learners as his yardstick for what learning and generalization entail.

The late AI pioneer Marvin Minsky infamously referred to the human brain as a "meat machine." The point was to insist that the brain is an organic machine, running physical processes much like a computer. But the phrase caused a stir for its crassness. Calling brains "meat" is jarring, not least because "meat" connotes dead flesh rather than living tissue [9]. For many, the metaphor is offensive and reductive. Still, Minsky's quip captured an extreme materialism that reduces the mind to nothing more than information processing implemented in wetware.

Philosopher Daniel Dennett offers a more polished version of the same move when he writes that "if we are to make further progress in artificial intelligence, we have to give up our awe of living things" [10]. His point is that researchers should not romanticize biological life as something magically different from machines. But critics have noted that this framing suggests advancing AI requires diminishing the dignity or specialness of living beings.

Progress in AI does not obviously require abandoning respect for human uniqueness, and in a field ostensibly centered on intelligence, a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

willingness to speak so dismissively about the human condition can seem counterintuitive and even callous. In each of these cases, a legitimate claim about substrate-independent intelligence is packaged in a way that not only rejects human exceptionalism "in principle" but also risks eroding any sense of what might still be ethically or phenomenologically distinctive about human minds.

9.1.4 Environment taxonomy

The dissertation also develops a taxonomy of environment classes and identifies those that allow an agent to learn optimal behavior. This might seem like an esoteric diversion, but it plays a crucial role in Legg's overall argument. The taxonomy is essentially a map of when the *agent–environment–objective* relation makes optimal behavior learnable across a broad set of worlds, reinforcing the claim that it constitutes a form of universal intelligence. The taxonomy also addresses the kinds of worlds in which a universal learner like AIXI won't succeed. Legg closes that gap by pinpointing the suboptimal conditions that prevent an agent from being self-optimizing.

For example, in *ergodic environments*, where the agent can always recover from mistakes and explore all states, self-optimizing agents (such as AIXI) exist. In this view, "intelligence" is not an intrinsic property of an agent but rather a performance profile across environmental classes under a reward signal. Legg also distinguishes passive environments, which don't react to the agent's actions (akin to prediction problems), from active environments, which respond to actions (as in full reinforcement learning). Lastly, he analyzes both *deterministic* dynamics, which yield the same outcome for a given state, and *stochastic* dynamics, in which outcomes vary according to probabilities.

In ergodic classes, self-optimizing policies exist, so general learning is not a fantasy. Yet adversarial and nonergodic classes exist in which no agent can recover from early errors or extract a signal faster than brute force; in such worlds, even "universal" intelligence has sharp limits. That contrast is the key virtue of the taxonomy because it converts vague claims about "general intelligence" into falsifiable statements about classes where self-optimization is possible and others where it is provably not.

These answers temper unbridled optimism by recognizing that superintelligence will have limits in certain worlds that have safety implications. They present scenarios in which even an optimal agent can't cope. On one hand, it might comfort us that a superintelligent agent isn't godlike. On the other hand, a clever superintelligent agent may try to avoid those

Licensed to Hermilo Cortés <hermilocg@gmail.com>

situations or reshape the environment to be more favorable. Legg's analysis doesn't investigate that dynamic in depth, but by mapping the terrain, it provides a foundation for thinking about where a superintelligent agent might be blind or vulnerable.

9.1.5 Agents and limits

AIXI assumes the reward channel is exogenous. That is, the reward is provided by the environment rather than being manipulable by the agent. Legg explicitly warns that if the agent had complete control over rewards, the framework would be vacuous due to a phenomenon known as "wireheading" [11]. If the agent can seize or redefine its own reward, then it could "give itself a constant maximum reward," likened to a human taking the "perfect drug." Safety researchers describe these concepts as accidental risks arising from systems that do what their objective function rewards, rather than what the designer intended [12]. A cleaning robot rewarded for making rooms look clean might hide messes rather than clean them; a model rewarded for confident answers might hallucinate rather than admit uncertainty; an autonomous system rewarded for speed might take unsafe shortcuts. The common pattern is optimization under a bad proxy, not malice.

As a remedy, Legg proposes a separate goal system that interprets environmental states and issues rewards, rather than folding reward generation into the agent. Legg also discusses discounting future rewards, which values future outcomes less than present ones so that different payoff streams can be compared on a common "present value" scale. Conceptually, both moves keep the utility channel separate and well behaved, rather than allowing the policy to rewrite its own objective.

Some environments are especially hard for any agent to predict or control beyond a certain complexity threshold. If an environment hides important information in a high-complexity pattern, a resource-constrained agent will essentially be guessing. Legg shows that very powerful algorithms may exist, but he also shows that Gödel's incompleteness theorem imposes hard limits on what can be proved about their optimality. In practice, that means we shouldn't expect a definitive proof that some fixed design is "the" superintelligence.

These theoretical limitations add nuance to the question of whether superintelligent machines are possible. The answer may be yes in principle but not in the sense of a magical oracle that can solve literally every problem. There will always be hard limits. Some problems are so complex, noisy, or information-poor that even a superintelligence can, at best,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

improve performance only marginally over chance, simply because the information needed to do better isn't available or would require solving an intractable computation. In such tasks, gains show diminishing returns. Beyond a certain point, additional cleverness yields only tiny improvements.

Legg's findings also imply there will be puzzles that remain irreducibly difficult no matter how "intelligent" an agent is. The presence of such tasks doesn't necessarily negate the possibility of superintelligence, but it does mean that superintelligence isn't omniscience. Depending on your philosophical bent, this is either reassuring or troubling.

For researchers designing an algorithm that is both universally optimal and efficient, this implies inevitable trade-offs. Legg argues that the focus should be on the "wide range of environments" that matter, and he accepts that any practical system will handle edge cases (or worst cases) poorly. If we ignore environments that are too computationally hard, many theoretical problems vanish, and an agent could achieve human-level intelligence for many practical purposes.

This perspective also helps explain scaling laws (chapter 6) and the world we live in today. Most real-world tasks live within a resource-bounded slice of possibility, and as model capacity, data, and compute steadily increase, we capture more of that structure, yielding the smooth power-law gains we observe today.

Seen this way, modern scaling is not magic. Scaling is a practical way to extend coverage over the tractable bulk of environments. It will not crack the true outliers, so we should expect diminishing returns and a persistent long tail in which robustness and oversight remain necessary. But within the resource-bounded regime that Legg identifies, scaling is a reasonable and defensible strategy.

9.2 AI safety

Before turning to existential risk, it helps to make "AI safety" ordinary. An AI system is unsafe when it causes harm because its objective, training data, deployment setting, or oversight fails. No superintelligence is required to discuss safety. In 2024, Air Canada was ordered to compensate a customer after its chatbot gave inaccurate information about bereavement fares, leading him to buy a ticket under false assumptions. The harm was not existential; it was mundane, financial, and ultimately bureaucratic. The event illustrates a very basic safety pattern that you can find today. A system provides authoritative advice; the user relies on it, and the institution distances itself from the output [13][14].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Some safety failures are mundane; others are not. The stakes rise when AI systems act in the physical world. For example, in 2018, an automated driving system failed to correctly classify a female pedestrian or predict her path. Consequently, safety is taken more seriously today. That wasn't true when Legg wrote his dissertation. At the time, concern about long-term AI risks remained peripheral. He shares an observation about a 2007 conference that brought together prominent figures from MIT, Yale, IBM, and Google to discuss the future of AI. It was an unusual gathering, and its significance lies precisely in its rarity. Its very existence signaled an early shift from fringe speculation toward mainstream attention.

Legg cites figures such as Bill Joy, co-founder of Sun Microsystems; Nick Bostrom, founding director of the Future of Humanity Institute at Oxford; and Sir Martin Rees, professor at Cambridge and president of the Royal Society, all of whom have spoken about the potential dangers of superintelligence. Legg uses these examples to underscore that the question of superintelligence is pressing enough that leaders across multiple fields are beginning to engage with it. His own assessment seems cautiously optimistic but proactive. While he doesn't predict doom, he stresses the importance of early preparation.

9.2.1 Intelligence explosion

In 1965, I.J. Good speculated on the effects of a superintelligent machine [15]. "Let an ultra-intelligent machine be defined as a machine that can far surpass all the intellectual activities of any man, however clever," Good said. "Since the design of machines is one of these intellectual activities, an ultra-intelligent machine could design even better machines; there would then unquestionably be an 'intelligence explosion,' and the intelligence of man would be left far behind. Thus, the first ultra-intelligent machine is the last invention that man need ever make, provided the machine is docile enough to tell us how to keep it under control." The caveat, "provided the machine is docile enough …," is an early recognition of the AI control problem. Good didn't propose how to ensure such a machine's *docility*, nor did he explain how to create the ultra-intelligent machine, but he planted the seed for later thinkers. Thus, Good's scholarly influence is amplified by historical importance.

9.2.2 Technological singularity

Vernor Vinge's 1993 essay "The Coming Technological Singularity" built on Good's ideas and brought them to a broader audience of futurists [16]. It popularized the term "technological singularity," describing a

Licensed to Hermilo Cortés <hermilocg@gmail.com>

future point beyond which "human affairs, as we know them, could not continue" due to unbounded growth in intelligence. He questioned whether the singularity could be prevented or guided and concluded that avoiding it would be difficult, as it would require all nations to halt research, competition, and technological progress. As a result, he treated the possibility of superintelligence and subsequent risk as inevitable, underscoring the difficulty of AI safety planning. Academically, Vinge's essay is less cited than others, but its cultural influence has been massive.

Ray Kurzweil's 2005 bestselling book *The Singularity Is Near* further popularized these ideas in mainstream culture. Kurzweil embraced Vinge's singularity concept and put a specific date on it, predicting a merger of human and machine intelligence by 2045. He argued that technological progress is exponential—what he called the "law of accelerating returns"—and that eventually AI will "match or exceed the cognitive abilities of human beings across any task," after which it will quickly become "infinitely more powerful" than human intelligence.

Unlike Vinge, Kurzweil painted a more optimistic vision of safe superintelligence. He believed that human minds and AI would merge, implying that humans wouldn't be left behind but would instead join AI's capabilities. It's noteworthy that, despite being a popular (that is, nonacademic) work, Kurzweil's book has been cited about 600 times in scholarly literature indicating that even academics writing about the societal implications of AI often cite Kurzweil's vision [17]. Yet his influence legitimized discourse on superintelligence futures, including safety discussions, outside purely academic circles.

9.2.3 AGI (as a field) is born

In 2007, Ben Goertzel and Cassio Pennachin edited *Artificial General Intelligence*. At Shane Legg's suggestion, it established and popularized the term "AGI," replacing older terminology like "strong" and "weak" AI [18]. It was the first book dedicated to AGI as a field, and it added legitimacy at a time when mainstream research was focused on narrow tasks [19]. In their introduction, Goertzel and Pennachin stressed that AGI aims for "the ability to solve general problems … in the same sense that a human can," rather than narrow AI confined to a single task. By formalizing this goal, the AGI community began to grapple with the consequences, such as an AI that might learn and evolve unpredictably, before it became a serious technology.

Before this, discussions of "strong" (i.e., human-level) AI were largely philosophical. The AGI field, established in 2007, treated achieving

Licensed to Hermilo Cortés <hermilocg@gmail.com>

human-level or superhuman AI as a concrete goal and brought questions of safe design to the forefront. The 2007 book did not provide answers, but it established the community and terminology in which ideas such as AGI safety, friendly AI, and alignment would soon flourish. In short, Goertzel and Pennachin helped move superintelligence and general intelligence from abstract speculation and philosophical debate into a seemingly legitimate pursuit.

The AGI label also expanded the Overton window and gave both enthusiasts and skeptics new rhetorical levers, while legitimizing a metaphysical framing for each side. Today, enthusiasts use AGI to suggest that the field is inching along a single continuum whose endpoint is artificial general intelligence, so every improvement can be counted as progress toward it [20]. Skeptics use AGI to cast doubt on working systems by asking whether they are *really* intelligent, *really* understanding, or *really* aligned with some human essence. Despite the most successful period in the history of artificial intelligence (2010 to present), progress is routinely dismissed not because it fails to deliver new capabilities, but because it fails to realize a largely conceptual ideal of "intelligence." This skepticism is distinct from traditional machine learning discourse, in which doubts were empirical and were judged exclusively against benchmarks. The irony is that much of AI discourse is now playing an ontological game. We will revisit this idea soon.

9.2.4 Superintelligence

In 2014, Nick Bostrom published *Superintelligence: Paths, Dangers, Strategies*, arguably one of the most influential works on AI safety [21]. He explains that even if we program a seemingly benign goal, a superintelligence might develop harmful instrumental subgoals, such as seeking resources or self-preservation, in ways that threaten humans. For example, a superintelligence devoted to solving a math problem could decide to transform the entire Earth into computing hardware to achieve its goal, disregarding human survival. Bostrom warns that without a solution to the control problem, an intelligence explosion could lead to an existential catastrophe. He discusses approaches such as "AI boxing," which limits an AI's capabilities; "motivational control," which involves instilling ethical values or constraints; and international cooperation to slow an AI arms race. Bostrom's core message is that achieving a beneficial superintelligence is possible, but it requires careful planning and perhaps new scientific breakthroughs in AI alignment.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

9.2.5 AI foom

In 2008, Eliezer Yudkowsky argued that advanced AI could usher in utopia or doom, depending on its design [22]. He introduced the broader community to the concept of a "hard takeoff," or "AI foom": a scenario in which an AI system rapidly improves, potentially within days, hours, or even minutes. This recursive self-improvement echoes Good's intelligence explosion.

In a hard takeoff, there's no slow ramp for humans to respond adequately; by the time we realize an AI has surpassed us, it may be too late. Yudkowsky argued that *unless* AI is carefully designed with friendly goals, its intelligence will likely be deployed to pursue a goal indifferent to or hostile to human well-being. He memorably wrote: "The AI does not hate you, nor does it love you, but you are made out of atoms which it can use for something else."

In 2023, he even argued for extreme measures, including a worldwide moratorium on AI research. He stated that the default outcome of building a superintelligent AI is doom and that "our only hope is not to try" [23]. This is a highly pessimistic view, and not all AI safety researchers agree that the risk is so absolute. His 2025 book, *If Anyone Builds It, Everyone Dies: Why Superhuman AI Would Kill Us All*, continues that trajectory, with a title that leaves little ambiguity about the author's current position.

9.2.6 Effects

Legg, like many others, believes the stakes are high. He suggests keeping goal specification under human control, separating the reward and goal from the agent to avoid wireheading, and cultivating a research program to ensure the safe and constructive deployment of superintelligence, if it arrives. He explicitly endorses Eliezer Yudkowsky's notion of a Friendly AI that won't harm humans, or a friendly AIXI-based framework [24].

At the same time, Legg stresses that a superintelligent agent would wield unprecedented power by virtue of its broad capabilities: "If there is ever to be something approaching absolute power, a superintelligent machine would come close." He also invokes Lord Acton's dictum on power. Whether this leads to catastrophe or to an "age of prosperity unlike anything seen before" depends on how prepared and careful we are in developing such machines.

On this view, superintelligence itself is value-neutral, with no hidden desires. If the underlying goals are good, superintelligence yields good; if they're bad or misguided, superintelligence yields catastrophe. This

Licensed to Hermilo Cortés <hermilocg@gmail.com>

encapsulates the double-edged nature of superintelligence: it could solve virtually all solvable problems or pose an existential threat. In a 2011 interview, Legg suggested that the probability of human extinction within a year might be "maybe 5%, maybe 50%," emphasizing that no one has a reliable estimate [25]. But he does not treat p(doom) as foregone. Recently, he estimated a 70% chance that alignment will be resolved by 2028, framing the situation as high stakes yet tractable [26].

Stepping back, Legg's work sits within a broader superintelligence canon. While Good, Vinge, and Kurzweil raised the prospect of superintelligence, Legg worked to formally define intelligence itself [27]. Although Legg's dissertation is significant, it is specialized. It is cited less often than the field-defining treatises and popular books. His thesis has garnered about 150 citations [28][29].

His contemporaries collectively defined what AGI and superintelligence meant, ranging from existential stakes (Good and Yudkowsky) to mainstream awareness (Kurzweil, Vinge, and Bostrom), while Legg supplied a formal theory. Taken together, these works trace a trajectory from Good's speculative control problem and initial notion of an intelligence explosion, through the singularity described by Vinge and Kurzweil, to Legg's formal underpinnings of superintelligence, the establishment of AGI as a tangible goal by Goertzel and Pennachin, and finally the risk analyses of Bostrom and Yudkowsky.

A few consistent themes emerge from the literature, not least of which is that an uncontrolled superintelligence poses a serious risk—one that must be solved before it is built, not after. As Bostrom and Yudkowsky note, by the time a superintelligence exists, it may be "too late" to solve the control problem. This seems to align with Ilya's beliefs as well. Yet, in the spirit of Carl Sagan's dictum that "extraordinary claims require extraordinary evidence" (often shortened to ECREE), both the optimistic promises and apocalyptic warnings about superintelligence demand unusually strong arguments and empirical anchors [30]. The point isn't to dismiss fat-tail risks but to calibrate how seriously we should take any given scenario based on the quality and quantity of evidence backing it.

These examples also show that ruminations about superintelligence evolve independently of empirical AI research. Legg's thesis barely engages with deep learning, mentioning it only briefly as one of many approaches that could see a breakthrough. That's understandable: when Legg was writing his dissertation, deep learning wouldn't become important for several years. Yet someone reading it in 2026 might notice that nowhere in the nearly 200 pages does it grapple with how large artificial

Licensed to Hermilo Cortés <hermilocg@gmail.com>

neural networks might affect the field. The project aims to define a formal ideal. It sets an upper bound in a toy universe while remaining silent about the engineering path required.

This gap is also evident in the fact that AIXI is incomputable. These beautiful results hold for an idealized agent we cannot build. This is less a flaw in logic than a recognition that AIXI is a theoretical device, but it leaves a substantial gap between theory and practice. This differs from most artifacts on Sutskever's List. In those cases, empirical results help reduce the gap in understanding, whereas here the discussion relies primarily on dictionary-style definitions and theoretical computing results. While AIXI earns philosophical legitimacy, it does not yet earn "living hope," because it cannot fail (or succeed) in the way an implemented system can. Its motivational force is mostly rhetorical.

Superintelligence literature is a conceptual game filled with definitions and impossibility arguments. Legg's thesis shows how that game can bootstrap itself by forging a new definition of intelligence through the synthesis of older ones. By contrast, mainstream AI is an experimental engineering game with datasets, model training, data pipelines, loss curves, and evaluations. Legg's work belongs to the first category, while ImageNet-era deep learning belongs to the second.

9.3 Explaining too little and promising too much

While Legg's thesis is strong, it has weaknesses, including its treatment of intelligence. Recall that he surveyed over 70 definitions to synthesize them into a single informal definition: "Intelligence measures an agent's ability to achieve goals in a wide range of environments," which he then operationalized into a measure. On the surface, this methodology seems reasonable enough, and so does the definition, which frames intelligence in terms of what an agent does (that is, achieving goals) across many environments.

On the other hand, this definition is a strange philosophical cul-de-sac because it sidesteps the thorny question of what intelligence is by effectively restating it as an operational measure. Oddly, collating 70 definitions of intelligence tells us little more than that intelligence is whatever intelligent agents do. Legg even acknowledges this in section 4.4, titled "Response to Common Criticisms." This tautological stance raises deeper problems not only with the definition Legg chose but also with the definitional approach itself.

The following philosophical detour matters because AI safety depends on definitions and tests. If we mistake benchmark performance for an inner

Licensed to Hermilo Cortés <hermilocg@gmail.com>

essence, we overclaim. If we demand an untestable essence before acknowledging any capability, we underreact. Safety lies between those errors.

9.3.1 Legg's functionalism

To place Legg's definition in context, it helps to see where it might sit on the map of philosophical theories of mind. Legg's stance is essentially functionalist: it defines intelligence by what a system does rather than by what it is made of or by its essence. Functionalism in the philosophy of mind views intelligence as the brain's functional organization, where distinct functions like language and vision are understood in terms of their causal, individuated roles, focusing on what inputs they take and what outputs or effects they produce, rather than by their internal constitution. In the AI context, functionalism suggests it doesn't matter whether a thinking entity is a human brain, a computer, or something else; if it functions intelligently, then it is intelligent, just as anything that tells time is a clock. What matters is not what the clock is made of, or whether its internal constitution is digital or mechanical, but whether it performs the function of keeping time.

The functionalist view is liberating because it allows researchers to pursue "intelligent" functions in machines without worrying about whether a machine has a brain or consciousness, or whether it requires embodiment. By analogy, anything that reliably solves problems, wins games, or achieves goals across a wide range of environments could be considered intelligent, regardless of whether it is silicon-based or biological. Legg's goal-oriented definition follows this tradition and is unapologetically functionalist, treating intelligence as an abstract capability of an agent acting in an environment to achieve a goal, without regard to the agent's specific inner workings or substrate.

Defining intelligence purely in functional terms has limitations. Functionalism allows us to measure outcomes, but it fails to explain why or how those outcomes arise in a genuinely intelligent way. Consequently, it risks being circular. For example, defining intelligence as what an agent does is not really explaining it; it merely redescribes it in its own terms. Even when the definition provides a metric, as Legg's definition does, it sheds no additional light on the mechanism or essence of intelligence. It tells us about an agent's performance, but not what qualitatively distinguishes intelligence from nonintelligence. This is a common critique of purely functional definitions that rephrase the problem rather than solve it.

Legg's response is straightforward: if an agent achieves a high value on the universal intelligence metric, it excels across a wide range of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

environments. *How* it manages to do so, or *what* it is made of, is irrelevant. The agent might use an elegant algorithm or an unimaginably large lookup table of precomputed answers; either way, it demonstrates a capacity to solve problems. In Legg's view, that *is* intelligence. The hypothetical lookup-table agent would be deemed intelligent because it *works*, even if its method is absurdly inefficient or made of old beer cans powered by a windmill [31].

Many philosophers have challenged the functionalist argument. Ned Block imagined a machine called *Blockhead* that stores a large list of question-and-answer pairs and could carry on a humanlike conversation, even passing a Turing test, without any understanding. Setting aside computational limits, such a lookup table *could* fool observers into thinking it's intelligent, even though it merely regurgitates precomputed responses. This scenario is often used to argue that purely behavioral tests of intelligence are insufficient because an entity might perform well while being "hollow" internally.

The famous Chinese Room argument by philosopher John Searle makes a similar point. Searle imagines a room that takes Chinese characters as input and outputs appropriate Chinese responses by following a huge lookup table of rules. Functionally, this setup produces intelligent-looking responses without anyone in the room actually understanding Chinese. Both Block and Searle are essentially arguing that a machine could appear intelligent without having any "real" intelligence, which would be required for "real" understanding.

Legg's counterargument is that "understanding" matters only insofar as it changes measurable performance. In other words, epistemic states are instrumentally relevant only through their behavioral consequences. *If* two systems are behaviorally identical across environments, there's no practical basis within this framework to treat one as "understanding" more than the other. This is at the heart of the substrate-neutral, or more strongly substrate-indifferent, position associated with thinkers such as Tegmark, Minsky, and Dennett. What matters is not the system's material composition, but the functional organization that produces intelligent behavior.

If "understanding" boosts performance in some way, Legg's universal intelligence measure will capture it. If an agent achieves a high value on the same metric, it is excelling across a wide range of environments and tasks. In this sense, the metric is doing the philosophical work. Nevertheless, there is still a narrowing gap today between artificial and natural intelligence, often loosely described as "understanding," even though what we mean by "understanding" is itself poorly understood and largely contested.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

9.3.2 A hint of behaviorism

Legg's definitional posture also has behaviorist undertones. The anachronistic aim of behaviorism is to predict human behavior by treating the environment as a determinant of that behavior [32]. An agent learns that receiving a reward depends on an action taken in response to a stimulus in its environment. The behavior is said to have been reinforced when the action becomes more frequent and purposeful [33].

While input from the outside world is an essential component of human behavior, the problem with behaviorism is that it purges mental phenomena and other internal cognitive processes from intellectual discourse and explains experience solely in terms of observable behavior. Concepts like thought, intelligence, understanding, feelings, beliefs, desires, and even heredity and genetics are eliminated in favor of environmental stimuli and behavioral responses. Intelligence is in the environment, not in the mind.

Philosophers like to joke about two behaviorists evaluating their experience after sex: "It was great for you, how was it for me?" says one to the other [34][35]. The joke highlights behaviorism's blind spot. Each is so focused on external behavior that neither can access their own subjective (internal) experience. In the same way, a purely behaviorist definition of intelligence focuses on what an agent achieves but overlooks what is happening within the agent to produce that behavior.

B. F. Skinner was a leading figure of radical behaviorism and the architect of operant conditioning at Harvard. Behaviorism was a reaction to 19th-century psychology, which focused on unconscious thought and psychoanalysis, both of which were difficult to test experimentally and thus difficult to use for prediction. In his book, *The Behavior of Organisms*, all organisms were restricted to rats, and all behavior confined to pushing buttons [36]. Behaviorism did not solve the problem of the mind; rather, it reduced it to fit inside a Skinner box [37].

By focusing only on outward behavior and rewards, behaviorism couldn't account for phenomena such as language acquisition, systematic reasoning errors, or creativity, all of which involve the mind's internal representations. Consequently, its dominance waned by the 1950s, in part due to Noam Chomsky's famous critique of Skinner's attempt to explain language purely through stimulus–response associations shaped by reinforcement [38][39]. Chomsky's "A Review of B. F. Skinner's *Verbal Behavior*" is his most-cited work, and despite the prosaic name, it has become better known than Skinner's original work [40]. The cognitive revolution

Licensed to Hermilo Cortés <hermilocg@gmail.com>

that followed reintroduced the idea that the brain has internal structures and processes for intelligence.

Legg's view isn't as extreme as classical behaviorism. While the ingredients include reinforcement learning, the way they're assembled within the AIXI framework—guided by Epicurus, Occam, Bayes, and Solomonoff—is new. It's a synthesis that attempts to reposition itself as a definition of intelligence.

But by reducing intelligence to goal achievement in different environments, it inherits similar philosophical and metaphysical weaknesses. It implies that if we observe an agent succeed at enough tasks, we can call it "intelligent" and need not ask how or why it succeeded. But as AI practitioners know, an agent can succeed through brute force, training tricks, or narrow heuristics that don't generalize or understand the domain—something a definition focused solely on behavior would fail to distinguish. In short, outward behavior, even across many tasks, is not the same as natural intelligence because it is divorced from any cognitive depth.

9.3.3 Meaning in use

So far, we've seen that chasing a single, all-encompassing definition of intelligence can lead us into a philosophical morass: definitions of intelligence are either obscure, ambiguous, circular, too broad to be meaningful (i.e., functionalism), or too narrow to capture the essence (i.e., behaviorism) [41]. This is where Ludwig Wittgenstein's perspective becomes illuminating. Wittgenstein argued that many philosophical puzzles, like "What *is* X really?", arise from a misguided search for the essence of words. In his view, words don't have an essence hovering above them; their meaning comes from how we *use* them in practical contexts.

For example, he showed that trying to rigidly define a concept like "game" is futile. There is no single trait that all games share; instead, they exhibit overlapping *resemblances*. "The meaning of a word is its use in the language," Wittgenstein wrote, and different uses may be related by analogy or overlap rather than by one strict definition [42]. We call chess, truth or dare, and cheese rolling all "games," even though no single definition covers every case. One requires strategy, one is poorly governed social chaos, and one involves chasing a suicidal wheel of dairy down a steep hill for reasons no one can fully explain. Wittgenstein would say, "That's okay."

The same, one might argue, applies to "intelligence." We use the term in varied ways, and any single definition that covers all cases might be a fool's errand. The work is done according to our criteria and procedures for using the term, including the context and standards under which we

Licensed to Hermilo Cortés <hermilocg@gmail.com>

label something "intelligent," not by some essentialist view of "intelligence" floating in the air.

In fact, from a Wittgensteinian standpoint, the exercise of compiling 70 definitions of intelligence in order to extract one was likely doomed. All it revealed was that there are many different "language games" played with the word "intelligence." Boiling it down to a minimalist phrasing, such as the "ability to achieve goals in a wide range of environments," likely tells us more about the researcher and how they want intelligence to be measured than what intelligence truly *is*.

Wittgenstein would probably suggest that, rather than seeking a single definition, we should examine the various criteria we actually use to attribute intelligence. For humans, we might require reasoning and learning; for AI, perhaps the requirements would be passing certain benchmarks. Each criterion is tied to a context. There may be overlapping similarities between these forms, but no single definition. And that's not a defect. It becomes a problem only when we think every word must have a strict definition or when we let a definition masquerade as an explanation. By the way, this isn't a covert endorsement of human exceptionalism or carbon chauvinism, or a blanket endorsement of substrate-indifferent functionalism. Wittgenstein's point is so much simpler and more deflationary. He asks us to establish the public standards under which we grant or withhold the term. But that also means no special gatekeeping. No one gets to settle the matter in advance by stipulating the "real" definition of intelligence. The standards are found in our practices, not hiding from them.

The trouble continues if we ask a benchmark to certify an AI's inner light, for example, by labeling it "sentient," "partially conscious," or even "intelligent" once it crosses an arbitrary human threshold. That is precisely the temptation Wittgenstein warns us to be circumspect about. Also, given the behaviorist and functionalist traditions in AI that sideline phenomena such as the mind and consciousness, it is peculiar to see these notions bolted back onto what is essentially an empty functional characterization of intelligence. A score on a benchmark, or a test (even the Turing test), cannot conjure subjective experience or understanding, because the theory itself has no place for it.

9.3.4 Turing's test

Alan Turing intuitively grasped this shift from definition to criteria. In his landmark 1950 paper "Computing Machinery and Intelligence," Turing begins by posing the question "Can machines think?" and then immediately abandons it, calling it "absurd" [43]. Instead, he proposes a new

Licensed to Hermilo Cortés <hermilocg@gmail.com>

question: can a machine imitate a human well enough in conversation that an interrogator cannot tell the difference [44]?

This proposal, which became known as the Turing test or "imitation game," was a stroke of genius. Turing sidestepped the definition of "intelligence" and offered a public test. If a machine can use language indistinguishably from a human, then, Turing says, we have as much reason to call it intelligent as we do to call a person intelligent. He shifted the problem from a metaphysical definition to an experiment.

In doing so, Turing aligned with a pragmatic, or even behaviorist, methodology by focusing on external performance, but, crucially, he did *not* claim that his eponymous test was a definition of intelligence, let alone *the* definition of intelligence. Rather, he offered it as a test for attributing intelligence in practice. With apologies to Philadelphia 76ers guard Allen Iverson, this is about practice, not metaphysics [45]. *Practice!*

Turing wrote that we should replace the nebulous question "Can machines think?" with a more concrete one about the outcomes a machine can produce because arguing about the word "think" itself would lead nowhere. Turing saw this clearly 75 years ago. The irony is that the same trap he tried to escape, arguing over the word rather than the work, still ensnares us today.

Turing understood that we can never agree on a definition of intelligence, not because we hadn't looked, but because we don't know what it truly is. It may be that "If the human brain were so simple that we could understand it, we would be so simple that we couldn't" [46]. His response was not to seek an essence but to shift the question entirely. In the spirit of Wittgenstein's meaning-as-use, intelligence should be defined by the circumstances under which we are prepared to say, "X is intelligent," which is a dramatically different question from "What *is* X really?"

The Turing test provided one such circumstance, and it is demanding. It took 75 years before that stringency was finally met. In a preregistered, three-party imitation game conducted in 2025, GPT-4.5 was mistaken for a human more often than humans were mistaken for it [47]. By Turing's yardstick, a bar that had long functioned as a public test for artificial intelligence has at last been cleared. By Turing's own standard, the question is no longer whether machines can in principle display intelligence. They can.

Even so, he would not claim that we have at last discovered what intelligence is or that machines now possess it in some metaphysical sense. Turing anticipated the future that has arrived. He believed that once machines could play our conversational language games well enough,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

ordinary, educated usage would drift, so that speaking of machines as "thinking" or "intelligent" is no longer a contradiction in terms, even if LinkedIn philosophers still quarrel about what, if anything, lies behind the word [48].

It's worth noting that the Turing test is not without criticism, but it is also widely misunderstood. Regardless, the lesson here is broader. Turing changed the game from definition to demonstration, and from philosophy to practice. He also understood that the public benchmark was a tool, not the truth. It is provisional: something we could later revise.

In doing so, he gave us a way to talk about AI despite the tension in the phrase itself. Turing sidestepped that tension by shifting the question from what intelligence *is* to what counts as intelligent behavior. This approach underwrites much of modern AI research, even if it's rarely acknowledged. We set benchmarks and tests, ranging from board games to vision challenges and language tasks, to determine whether a system meets or exceeds human-level performance. The field doesn't agonize over defining "intelligence" or agreeing on an essence; it gets on with measuring specific behaviors. Without this, the field would be stuck.

Interestingly, Legg's dissertation opens with the question, "What is intelligence?" But instead of turning to operational criteria, Legg treats it as a genuine "what is" question to be answered by a single definition. It is precisely this essence-style question that invites a Wittgensteinian worry. On the other hand, the universal intelligence measure is Turing-ish, since it pivots away from what is otherwise a thin definition and toward a concrete way to score agents across tasks and environments. Had Legg rejected the impulse to define intelligence, he would have aligned himself almost perfectly with Turing.

9.3.5 Definitions masquerade as explanations

In an experiment-led field like AI, results often arrive before we have satisfying explanations. This invites post hoc narratives to fill the gap. But trouble arises when we let a definition or explanation masquerade as a theory or when we treat benchmark performance as proof of some "spark" of intelligence [49].

Legg's definition, for example, should be viewed as a starting point for quantifying agent performance. If Agent A can achieve more goals across a variety of environments than Agent B, then Agent A is "more intelligent" by that yardstick. There is nothing wrong with that in principle, provided we remember it's just a defined metric. The problem comes when we start believing that adopting a definition resolves what intelligence is.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

In reality, we have only set up a convenient operational proxy. The inner nature of intelligence remains as much a mystery as before.

In practice, this means that AI practitioners must be careful not to let benchmarks certify "real" intelligence. When an AI model achieves a superhuman score in a video game or on a reading comprehension test, it meets a goal, but that doesn't mean it understands the game or language like a human would. If we naively equate benchmark success with "it's genuinely intelligent now," we risk granting a sort of metaphysical status to something that didn't earn it. It's one thing to say, "the system achieved human-level performance on X task"; it's even fine to add, "the system *understands* X domain." It's another to say, "the system understands X as a human." The former is a local, operational claim; the latter can become an unwarranted leap to convert benchmark success into an essence, mind, or an inner light.

Benchmarks invite the same error when they infer an intrinsic property called "intelligence" from surpassing a human threshold because neither licenses any ontological claims. The error is to reify a technological achievement as an essence.

But crucially, this cuts both ways. "Real" does a lot of heavy lifting for skeptics. They often withhold acknowledgment unless a system exhibits putative essences like "*real* intelligence," "*real* understanding," or phenomenological experience that are neither operationalized nor testable in the setting at hand. These vague conceptions set an unfalsifiable standard, leaving the skeptic grasping at straws. The conclusion is insulated from evidence because the relevant property is defined to be behaviorally undetectable. Thus, what can be asserted without evidence can be dismissed without evidence; if "no real intelligence here" is defined so that no possible observation could count in its favor, there isn't much for science or engineering to engage with [50].

For instance, GPT-4 passed a simulated bar exam with a somewhat dubious top 10% score, comparable to elite human test takers [51][52]. The debate still followed the same script. By normal standards, outperforming most law graduates on such a public benchmark would indicate some reasoning and a shallow understanding of the task. Yet critics dismiss it because it didn't prove *real* intelligence. They shifted the bar (no pun intended) from "can it do X?" to "can it *really* do X?" in a way that's neither well-defined nor actionable. The criteria for "true" intelligence are pushed into qualities that are inherently untestable, such as an innate grasp of meaning.

Of course, refusal can be reasonable if it's a living doubt. But when refusal appeals to properties defined as undetectable, it becomes an

Licensed to Hermilo Cortés <hermilocg@gmail.com>

immunizing position. In fact, if behavioral success is dismissed as "not real intelligence" because "real intelligence" is said to require an extra property that no behavior could reveal, then, functionally, the argument becomes a No True Scotsman fallacy. While performance does not equal essence, insisting on an untestable essence is neither good epistemology nor good philosophy. Turing flags exactly this trap when he sets aside solipsistic demands and proposes a public test.

As Turing warned, this metaphysical posture can paralyze inquiry. He wrote, "Instead of arguing continually over this point, … I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words." His imitation game judges systems by their performance rather than their private ontology. But if letting a benchmark certify an "inner light" is paralyzing to inquiry, so too is letting the absence of an "inner light" disqualify the benchmark. We must articulate procedures and criteria, admit they are provisional and domain-bound, and refuse both mystical overreach and evidential withholding. The way forward is procedural, not metaphysical.

9.3.6 Public tests

While philosophy is important, it rarely picks an optimizer, certifies a training run, or ships a model. Instead of wrangling over abstract definitions of "intelligence" in the hope of finally pinning down its essence, we need to name the game and state the criteria. In practice, this means specifying a benchmark that we agree requires intelligence to perform, such as ARC-AGI, GPQA–Diamond, FrontierMath, SWE-bench, GAIA, or BIG-Bench Extra Hard, and then observing what models can accomplish.

If a model consistently passes the kinds of challenges that resemble intelligence, we may extend the word "intelligent" to it while avoiding the trap of making the stronger claim that our model has an inner essence called intelligence. The point is not to discover what intelligence is but to fix how we use the word, supported by public criteria, family resemblances, and agreed-upon tests. We don't say, "We know its nature"; we say, "This is when we apply the word." Where there is no public test, we keep silent; where there is, we let performance decide. This is very close to Alder's razor: what cannot be settled by experiment or observation is not worth debating [53].

Legg would likely agree. He explains that measurement must ensure reasonable test conditions. Saying "X is intelligent" has meaning only if there is a reasonable setup under which X exhibits high performance. Intelligence is about capability, not inward essence or even outward

Licensed to Hermilo Cortés <hermilocg@gmail.com>

performance, if that performance is revealed under unfair conditions. Legg imagines the late physicist Stephen Hawking disconnected from his communication devices and wheelchair, with minimal interaction with his environment. The interface and task have changed because we've temporarily removed his ability to act.

This is unapologetically behaviorist: intelligence is what an agent does under appropriate conditions. Thus, an agent is not "intelligent" while idle. It is not the lack of quiet thinking, daydreaming, or motivation that leads to poor performance. There is no inner light. Of course, humans are a special case because we know there is consciousness and subjective experience. We comfortably say Hawking's intelligence persists while deactuated. But that phenomenological standard does not transfer to machines; for machines, only task performance under fair conditions counts. Legg emphasizes the importance of a well-designed test setup. If we're going to *measure* anything, we must provide the agent with a fair environment in which to demonstrate its capabilities. When those conditions are met, an intelligent agent will exhibit high performance; if an agent cannot, then calling it "intelligent" in some unobservable, purely potential sense has no practical value, let alone meaning.

Such a pragmatic approach frees us from fruitless debates and instead grounds the discussion in observable capabilities. After all, Legg's effort provides a gentle reminder that intelligence is too rich and multifaceted to be encapsulated in a single phrase. Progress will come from iterative experiments and the humility to accept that "intelligence," as a concept, will always have fuzzy edges. As we refine our tests, we also refine our understanding of what we mean by the term, which emerges from practice rather than punditry.

This doesn't mean we give up on understanding intelligence. It means we recognize that progress in artificial intelligence will come from empirical and procedural clarity. The danger of stacking too many such razors is that they don't just cut away bad arguments; they also slice off most of the speculative, playful, or aspirational questions that make AI feel philosophically interesting in the first place. There's still a place for wonder, thought experiments, and science fiction futures; it just lives in a different epistemic bucket from claims we are prepared to treat as knowledge or build policy on.

9.3.7 Dirty hands, clean(er) claims

Hubert Dreyfus argued that AI inherited a "lemon" philosophy [54]. Dreyfus was influenced by phenomenology, which is the philosophy of con-

Licensed to Hermilo Cortés <hermilocg@gmail.com>

scious experience. The irony is that many philosophers spoke out against the frameworks that dominated mid-century AI to explain intelligence, including behaviorism and functionalism, because they ignored embodiment and phenomenology. To be sure, the field of AI was born at an odd philosophical hour.

Nevertheless, the accomplishments within the field of AI over the past 70 years, especially the past 15, show that the discipline is not doomed. That's because pragmatism is the philosophy most frequently adopted by researchers and engineers. Pragmatism is not a philosophy of the mind. It focuses on workable solutions with measurable gains and discards the rest. In ordinary language, when a model can predict data correctly, we call that "understanding." The model has encoded, or compressed, the regularities of that data. This is a pragmatic label, licensed by utility, rather than an ontological claim. Pragmatism shows that ontological commitments aren't necessary for progress. In fact, AI advances by bracketing metaphysical and phenomenological debates rather than settling them.

AI is an engineering discipline. In the 1956 proposal for the Dartmouth Workshop, John McCarthy stated the wager plainly: "Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it" [55]. The conjecture does not smuggle in metaphysical commitments. Rather, it frames intelligence as something that can be specified, formalized, and constructed. This is a methodological bet, not a metaphysical claim.

McCarthy was not arguing that a simulation is identical to the thing itself. He was saying that if we can describe a cognitive function precisely enough, we can build machinery that reproduces its outward profile. Yet, somewhere along the way, the field began oscillating between two mirror-image mistakes. Enthusiasts overread functionalism and behaviorism, treating a good simulation as proof that we've settled on theories of what intelligence *is*. Skeptics, meanwhile, demand an inner light and dismiss performance as mere simulation.

All this is to say that there is a common misconception about AI. It is rarely about intelligence in a metaphysical sense. While we still have a lot to learn about intelligence, there is nothing metaphysical behind the AI curtain today. It's just capabilities under constraints. While functionalism and behaviorism are hopelessly flawed theories of mind, they remain critical for AI researchers because they simplify the "hard" problems of consciousness and thereby provide a path toward developing viable solutions [56]. Those flawed philosophical frameworks and metaphors did not offer truth. Instead, they ordered researchers to do something [57].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Ultimately, inspiration means nothing if it is not followed by creation. We may disagree with computational functionalism and the reductionism it embraces, but we need to understand the development it inspires. It allows researchers to begin work without worrying about philosophical bits that may be unsolvable and, ultimately, incompatible with computation. Ultimately, researchers are interested in what intelligence means for a computer. After all, it's called *artificial* intelligence for a reason. While the name may be oxymoronic, it underscores the goal, which is not an artificial human but artificial function or behavior, correctly aligning with the field's core philosophical frameworks.

Insiders must remember that definitions, metaphors, and frameworks remove barriers, but also remind us that there is no hidden essence. A claim about a language model being sentient is a metaphysical destination that doesn't tell anyone what's being tested. If instead "partial consciousness" is provided as a promissory note for an inner light that we can't scrutinize, then, by Wittgenstein's logic, "a nothing would serve just as well as a something about which nothing can be said" [58]. This doesn't forbid careful speculation, but any claim lives or dies on public criteria.

A recent example makes this vulnerability less abstract. In the name of AI safety, Anthropic gave Claude the power to end conversations. This work is presented under the banner of "AI welfare" [59]. This is Anthropic's attempt to manufacture moral status and belief. One of AI's most valuable features is taking on work that humans find traumatic without being exposed to the same trauma. Yet by giving their models an inner light, they have manufactured a victim and, in the process, sacrificed AI's most significant ethical advantage: that no one is there to be hurt.

For skeptics outside the field, the debate typically centers on the word rather than the work. "Real" becomes a moving target. When systems talk, it isn't a *real* conversation; when they translate, it isn't *real* understanding; when they discover a protein fold, it isn't *real* reasoning. But you cannot win an argument by chasing an inner essence. We have to certify performance.

A radiology triage model that cuts misses by half across hospitals, holds up under reasonable counterfactuals, and triggers escalation when uncertain is intelligent for radiology. That does not make it a doctor, a knower, or a bearer of responsibility. A coding assistant that follows messy instructions, repairs its own mistakes when corrected, and ships across diverse repositories is intelligent for software maintenance. That does not make it a software engineer, a mind, or responsible for production outages. Claims must be scoped to the game, and the dignity and duties we reserve for people must be preserved. That risks carbon chauvinism, but we do

Licensed to Hermilo Cortés <hermilocg@gmail.com>

not need to choose between mystifying machines and belittling them. We can certify the work. We can deny the ghost. And we keep responsibility where it belongs.

The lesson for outsiders is to avoid overreading the label. That's difficult, since "intelligence" in this context diverges from everyday language and intuitions about it. But if you hear someone call a language model "intelligent," it's usually shorthand for performance against a benchmark, not a mind lurking inside. If it reliably beats other models, call it intelligent-for-purpose; if not, don't. Intelligent-for-purpose is not consciousness, agency, or moral standing. It's all the essence we need and all the responsibility we can afford. Trade something metaphysical for something pragmatic.

Ultimately, researchers are pushing the boundaries of what works and what doesn't. Knowing this, you may think that the nature of AI research is arbitrary and ad hoc. In a sense, it is. Hopefully, this book has provided a foundation for understanding today's world and has introduced you to the researchers and engineers who have shaped it.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

epilogue
The missing pieces

Sutskever's List promises to explain 90% of what matters, and it delivers on that promise with unusual clarity. It reconstructs the path from feature engineering to large-scale supervised learning and from there to the self-supervised breakthroughs that made ChatGPT possible. Yet the remaining 10% is not an afterthought. It encompasses other major traditions in AI that were advancing in parallel with language-model pretraining, including deep reinforcement learning, self-supervised vision, multimodal grounding, diffusion models, and the emergence of scientific foundation models such as AlphaFold. Just as importantly, Sutskever's List stops at 2020, leaving the subsequent phase of AI development outside its scope. This epilogue will therefore address not only the missing 10% but also the full 100% of what has occurred since.

E.1 Deep reinforcement learning

DeepMind demonstrated that combining deep neural networks with reinforcement signals for deep reinforcement learning (RL) could produce general skills. In 2013, a deep Q-network (DQN) could learn to play classic Atari video games at a human level directly from raw pixels and game scores, using only trial-and-error feedback [1]. The original DQN discovered strategies for seven games without any hard-coded rules [2]. Just two years later, a DQN achieved a level of play comparable to that of a professional human game tester across 49 games, using the same algorithm, network architecture, and hyperparameters [3].

In 2016, DeepMind's AlphaGo defeated Go grandmaster Lee Sedol [4]. The breakthrough came from combining deep learning with

Licensed to Hermilo Cortés <hermilocg@gmail.com>

search in a way that had not been done before. By continually planning ahead using simulated moves and learning from outcomes, AlphaGo effectively "thought" about the game in a structured way, something earlier deep networks could not do. The moment was far more significant for AI than Deep Blue's chess victory two decades earlier, because it was replicated and built upon.

The following year, AlphaGo Zero learned to play Go without any human examples and surpassed the original program within days of self-play training [5]. MuZero (2019) generalized this idea by learning the game model itself [6]. It mastered not just Go but also chess and Atari games without being told the rules, learning both a planning strategy and a representation of the game environment from scratch. AlphaGo, AlphaGo Zero, and MuZero showed that, beyond recognizing patterns, artificial neural networks could be trained to plan in complex, sequential domains.

The idea of an AI that can plan, search, and reason about its next steps, rather than just react in a fixed way, will soon emerge with ChatGPT (as we'll discuss). Some post-ChatGPT efforts explicitly draw inspiration from AlphaGo-like planning to enhance reasoning. For now, suffice it to say that by 2020, deep reinforcement learning had proved its worth in spectacular fashion—from mastering games to controlling real-world systems.

E.1.1 Masked language models

While chapter 5 introduced the original Transformer and GPT-style models, we did not explore masked language models in any great detail. The poster child here is BERT (Bidirectional Encoder Representations from Transformers), introduced by Google in 2018. Unlike GPT, which reads text left-to-right to predict the next word, BERT is trained on a masked word prediction objective. That is, it randomly hides or masks some words in a sentence and learns to fill in the blanks using both left and right context [7][8]. By pretraining on millions of sentences in this way, along with an auxiliary task of predicting whether two sentences are consecutive, BERT learned extremely rich language representations.

BERT improved the GLUE language understanding benchmark score by 7.7% absolute over OpenAI's GPT-1 model, which was a gigantic leap. Against some benchmarks, BERT even surpassed human performance. It reached ~93% on the SQuAD question-answering test, slightly above the human baseline.

T5 (Text-To-Text Transfer Transformer), introduced by Google in 2019, unified the ideas of BERT and GPT [9]. It framed every natural

Licensed to Hermilo Cortés <hermilocg@gmail.com>

language processing (NLP) task as a "text-to-text" problem. For example, translation could be prompted as "Translate to French: `<sentence>`" or sentiment classification as "Is this review positive or negative? `<review>`". T5 uses a masking objective, a variant called "span corruption," in which contiguous spans of text are masked and the decoder generates them. By formulating everything as text generation, T5 could use a single architecture and training procedure for all tasks. In many ways, T5 foreshadowed the foundation model concept, in which a single model—with appropriate prompting or fine-tuning—can perform a wide range of tasks.

BERT and T5 dominated the NLP landscape during these years. In fact, from 2019 through 2021, BERT-style models dominated NLP tasks requiring text understanding, while GPT-style models were better known for generation. It wasn't until later that the GPT class, with fine-tuning, also became adept at following instructions, bridging that gap.

E.1.2 Self-supervised vision

Chapters 2 and 3 positioned convolutional neural networks (CNNs) as the bedrock of computer vision. CNNs indeed reigned supreme throughout the 2010s. But in 2020, researchers showed that transformers could outperform CNNs in image recognition with enough data. The landmark was the Vision Transformer (ViT) [10][11]. ViT adopted the transformer architecture for image classification in a straightforward manner. An image is split into a grid of patches (say 16×16 pixels each), each patch is flattened into a vector, and these vectors are treated as a sequence of tokens, just like words in a sentence. By feeding this sequence of image patches into a standard transformer encoder, ViT learns to produce an image representation and classify the image, without using any convolutions.

A ViT pretrained on JFT-300M and then fine-tuned on ImageNet reached state-of-the-art results, outperforming EfficientNet and ResNet variants. This suggested that a pure sequence model can discover what it needs, and global self-attention can determine which image patches should interact, thereby capturing long-range relationships better than localized CNN filters, provided the model is trained on a wide variety of images. This is the same lesson that the original Transformer revealed for NLP and recurrent structure.

Around the same time, techniques like contrastive learning became popular. In 2020, methods such as SimCLR (Simple Contrastive Learning of Representations) and MoCo (Momentum Contrast) showed that you could take an image, create two different augmented versions of it, and train a model (often with a CNN backbone) to recognize those two as the

Licensed to Hermilo Cortés <hermilocg@gmail.com>

"same" image instance while distinguishing them from other images [12][13].

This is done via a contrastive loss that pushes the representations of two views of the same image closer together and those of different images farther apart. Remarkably, after pretraining on millions of images, the model learns a very rich representation of image content. When evaluated—for example, by training a simple linear classifier on ImageNet labels using these learned features—the accuracy is very high and sometimes even matches that of a supervised ResNet trained from scratch.

In fact, a linear classifier on SimCLR's self-supervised features achieved 76.5% top-1 accuracy on ImageNet, not far behind the 78% to 80% that a fully supervised ResNet-50 would achieve, and this gap closed further with improved methods [14]. This meant label-free pretraining on images could produce representations nearly as good as those learned with labels. Subsequent works, such as BYOL (Bootstrap Your Own Latent) and SwAV, refined these techniques [15][16]. By 2021, self-supervised vision models had become the norm in research, much as BERT had in NLP. These methods became the default approach for pretraining vision backbones in settings with scarce labeled data.

ViT and self-supervised learning used unlabeled images to pretrain powerful vision models. Like the Transformer, ViT demonstrated that scale and data can trump built-in induction biases, and the success of contrastive and masked pretraining in vision reinforced the theme that the best way to learn robust representations, whether for text or images, is to set up a clever learning task on raw data itself, whether predicting missing parts or distinguishing augmentations. The models that emerged from this era set the stage for multimodal fusion and more, because they provided rich embeddings that could be combined across modalities.

E.2 The missing 100%

There is another kind of omission in Sutskever's List, and it differs from the 10% that is missing. The earlier omissions are best understood as parallel lines of development. These branches are important to AI but were not central to the path that Sutskever's List chose to follow. What comes after is different.

One useful way to understand the list is as having a knowledge cutoff. Ilya shared the list with John Carmack in 2020. You can see the timestamp in the reconstructed version, with Kaplan's paper the newest on the list. Yet following 2020, the field began to be defined by post-training, productization, multimodality, retrieval, tool use, open-weight ecosystems,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

inference-time reasoning, and the systems work required to make these models useful, affordable, and ubiquitous. The story now belonged to the interface, the deployment stack, the alignment layer, the context window, the agent scaffold, and the human workflow around the model.

That is what I mean by the missing 100%. It is not that nothing in the earlier chapters prepares us for what came next. On the contrary. But once ChatGPT arrived, the central question was no longer simply how to train a powerful model. It became turning that model into an assistant, a reasoning system, a multimodal interface, a tool user, a research aid, a coding partner, and a search layer.

E.2.1 Multimodal foundations

One of the most exciting "missing pieces" is the advent of multimodal AI. The prime example is CLIP (Contrastive Language–Image Pretraining), released by OpenAI in early 2021. CLIP trains two encoders—one for images and one for text—so that the image and text embeddings match when the text describes the image. OpenAI gathered a massive dataset of around 400 million (image, caption) pairs from the internet. They trained CLIP with a contrastive objective in which the model is presented with a batch of, say, 32 images and 32 texts, and is trained so that the correct image and text pairs have high cosine similarity in the embedding space, while mismatched pairs have low similarity. In effect, the model learns to associate pictures with their verbal descriptions. The result was astonishing.

CLIP, without having seen any specific classification task, was able to label images correctly based solely on semantic similarity [17]. OpenAI reported that CLIP, without any direct training on ImageNet, matched the performance of a ResNet-50 that was trained on ImageNet, achieving about 76% zero-shot accuracy. This meant the model wasn't narrowly tied to a single training set or task. Instead, it had learned a broad representation of visual concepts in terms of language.

CLIP gave AI a way to bridge modalities. It connected the visual and linguistic worlds in a shared embedding space and became a backbone for tasks such as image search (finding images from text queries), generating image captions (by retrieving text that matches an image), and filtering generative model outputs. Indeed, CLIP was used in OpenAI's DALL-E 2 system (released in 2022) as a scoring model to guide the generative process; essentially, CLIP's judgment of how well an image matched a caption was used to pick the best generated images. It's fair to say CLIP cracked the code on learning multimodal representations at scale.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

By training on image–text pairs from the web, CLIP and similar models implicitly learned a lot of world knowledge, such as what typical objects and their attributes look like and what words people use to describe scenes. And importantly, they learned it without explicit labels—just from the noisy pairing of alt-text and images available online. This approach is now fundamental. Any time you use an AI system that can understand both pictures and text (for instance, asking a modern AI to "describe this photo" or "what's unusual in this image?"), under the hood, there's likely a CLIP-like model.

The rise of multimodal models shows how AI has fused language and vision. This fusion enables, for example, zero-shot classification of images by name or lets a chatbot discuss images. It set the stage for later systems like GPT-4's vision features, which can move fluidly between modalities. More conceptually, CLIP demonstrated the power of contrastive learning across modalities by bringing them into a shared semantic space, with each modality grounding or informing the others.

E.2.2 Diffusion models

Imagine you have an image and gradually add noise to it, step by step, until it becomes pure grainy noise. A diffusion model learns to reverse this process. It starts with random noise and gradually removes it to produce a coherent image. In training, the model is given partially noised images and tasked with predicting the less noisy version or, equivalently, the noise that was added. By doing this across multiple levels of noise, the model effectively learns to generate images from scratch by gradually denoising. One reason diffusion models became popular is that—unlike generative adversarial networks (GANs), the previous dominant method for image generation—diffusion training is very stable and does not require tricky adversarial setups. In 2020 and 2021, researchers showed that diffusion models could generate incredibly high quality images, often indistinguishable from real photos, with great diversity.

The breakthrough was not just in quality but also in controllability. By 2022, systems like GLIDE and DALL-E 2, and later Stable Diffusion by Stability AI, enabled text-to-image generation with unprecedented fidelity [18][19][20]. These systems combined diffusion models with the kind of multimodal understanding we just discussed. For example, DALL-E 2 uses a CLIP-style representations to guide the diffusion process, generating an image whose CLIP embedding matches the embedding of the input prompt. Type in "a castle in the clouds, oil painting style" and out comes a plausible, often compelling image in that style.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

The diffusion model essentially serves as the imaginative engine, capable of painting what the text describes. Stable Diffusion, in particular, deserves note as it was released openly in 2022 and sparked a creative explosion. People everywhere could run this model on their own GPU (or even CPU with some patience) and generate art, illustrations, and images just by writing prompts. It was around this time that AI-generated art went mainstream, raising new questions about creativity and copyright but undeniably showing that generative AI had arrived for visual media.

Diffusion models aren't limited to images, either. Variants have been applied to audio generation (e.g., noise-to-music or noise-to-speech models) and even to tasks like molecule generation in chemistry. They represent a kind of general-purpose generative approach. Compared to GANs a few years earlier, diffusion models offered greater realism and adhered more faithfully to prompts. They also tended to produce more varied outputs rather than collapsing to a few modes.

From our perspective, the rise of diffusion models is a reminder that not all of AI's progress has been about Transformers or language. There have been parallel advances in how models are created. By the time ChatGPT arrived, these diffusion-based image generators had complemented text-based chatbots. Today's AI landscape is richer because we have both models that can talk like ChatGPT and models that can draw, paint, and imagine visually. And we even have the beginnings of models that unify those skills (e.g., by combining diffusion image generation with language models that can describe the scene they're creating).

E.2.3 AI for science

One of the most celebrated achievements was in scientific discovery. In 2021, DeepMind's AlphaFold 2 effectively solved the 50-year-old protein folding problem [21]. This was a grand challenge in biology, demonstrating how deep learning began to revolutionize fields beyond traditional AI tasks and underscoring the versatility of artificial neural network models.

The protein folding problem involves predicting the three-dimensional shape a protein will fold into, given its amino acid sequence. For decades, scientists have struggled with this problem, and although there have been gradual improvements, no method has reliably achieved atomic-level accuracy for proteins in general. Yet AlphaFold achieved a median accuracy of about 1 Ångström error on the test targets at the CASP competition. One Ångström is the width of a small atom; essentially, AlphaFold's predictions were often within atomic distance of the true structure. For context, organizers stated that a score above ~90 on their GDT metric is competitive with

Licensed to Hermilo Cortés <hermilocg@gmail.com>

experimental methods, and AlphaFold 2's median was 92.4 [22][23]. Many proteins were predicted with near-perfect accuracy, and even the harder cases were much better than anything seen before.

This was such a leap that CASP's chair, John Moult, declared the problem "solved," in the sense that computational methods could now often predict structures as accurately as experiments [24][25]. Within a couple of years, the AlphaFold team released the AlphaFold Protein Structure Database, containing predicted structures for over 200 million proteins—essentially all proteins known to science. This became an invaluable resource for biologists, chemists, and drug developers. It has already contributed to advances in understanding diseases, designing enzymes, and more. It stands as a testament to the pioneers who started using deep learning to classify cats and are now solving molecular structures [26]. AI is finally entering the sciences.

E.2.4 Knowledge distillation

Knowledge distillation has roots in earlier model compression and teacher–student work, but the modern line usually runs from Buciluǎ's model-compression results through Ba and Caruana to Hinton, Vinyals, and Dean's influential 2015 formulation [27][28][29][30]. By the late 2010s, distillation had become a standard way of making large models smaller, faster, and cheaper to deploy. The pressure for efficient deployment did not begin with ChatGPT, but mass adoption made it far more visible and economically consequential.

The core idea is simple. Start with a large, accurate teacher model. Then train a smaller student model on the same inputs to match the teacher's output distribution, often while also learning from the original hard labels. In practice, those soft predictions carry more information than a one-hot label, as they encode not just the correct answer but also the teacher's sense of which alternatives are plausible. For example, if a teacher network predicts class probabilities for an image as `[cat: 90%, fox: 5%, dog: 4%, car: 0.5%, ...]`, those probabilities contain a lot of information about how the teacher sees the world. In this case, it's pretty clear the image is of an animal, likely a cat, but the model also sees some similarities with foxes and dogs. A smaller model trained to match these probabilities can actually learn to be almost as good as the teacher, even though it has far fewer parameters. Hinton described that extra structure as "dark knowledge" [31][32]. Using this approach, much smaller models can recover a surprising amount of the performance of larger deep models or even ensembles [33].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

By 2019, knowledge distillation was used to create models like DistilBERT, which compressed the hefty BERT model down to almost half its size while retaining 97% of its performance [34]. DistilBERT offered a much lighter footprint, which was important for serving many queries per second or running on smaller devices.

In 2020–2022, as large language models emerged, companies also used distillation to create faster variants. For instance, OpenAI mentioned distillation in the context of GPT-3 deployment, and a 2025 *WIRED* article noted that even newcomers like DeepSeek used distillation to compress models, with some controversy when the means by which they obtained the teacher's knowledge came into question. Beyond saving compute, distillation often also makes models run faster and sometimes even generalize better, due to the regularization effect of learning from a smoother teacher signal.

Knowledge distillation acknowledges that while bigger is better for pretraining and raw capability, smaller is often sufficient for use if the knowledge is transferred appropriately. It's a bit like a mentor teaching an apprentice. The apprentice can perform almost as well as the mentor on many tasks because they've absorbed the mentor's expertise in a compressed form. By the time ChatGPT was rolled out, such techniques were likely part of the behind-the-scenes toolkit for optimizing model serving.

After ChatGPT, as open models proliferated, distillation became a way for communities to compress 65-billion-parameter models into more accessible 13-billion or 7-billion versions. Anthropic's handling of Claude 3.5 Opus hinted at the same logic. The company announced the model but never publicly released it, prompting speculation that an internal Opus-class system may have been more useful as a teacher for generating synthetic data or reward signals for Sonnet than as a product in its own right. Anthropic, though, denied that Sonnet had been trained using a larger, more expensive model. Ultimately, distillation shows that it's one thing to train a large model to push the state of the art; it's another to deploy it at scale to millions of users.

E.2.5 Alignment

In the wake of ChatGPT's debut, AI development accelerated even further. We saw dramatic advances in how we tune and align models, how we enable them to access external knowledge or tools, how we extend them to new modalities, and how we push their reasoning abilities to new heights.

One of ChatGPT's defining characteristics was that it followed human instructions much better than its predecessors. By default, GPT-3 (on

Licensed to Hermilo Cortés <hermilocg@gmail.com>

which ChatGPT is based) would ramble or produce unfiltered content if prompted naïvely. ChatGPT, by contrast, would generally try to be helpful, avoid certain toxic or sensitive outputs, and follow the user's requests in a conversational manner. This was not an accident; it was the result of significant post-training alignment work. Both instruction tuning and reinforcement learning from human feedback were key.

Instruction tuning is the fine-tuning of a pretrained language model on a dataset of instructional tasks. For instance, models like Google's FLAN and T0 (2021) showed that if you fine-tune a model on a wide variety of prompts and their correct responses, such as a collection of question-answer pairs or summarization tasks, the resulting model becomes much better at following novel instructions. Essentially, it learns the concept of "do what the user asks" rather than just predicting the next token in arbitrary internet text. OpenAI did this at scale with what they called InstructGPT; the sibling model to ChatGPT.

Users immediately noticed that such models were more responsive and accurate for actual queries. OpenAI reported that human evaluators strongly preferred the responses of a 1.3-billion-parameter InstructGPT model over the original 175-billion GPT-3's answers to the same prompts [35][36]. In other words, after this special tuning, even a much smaller model outperformed a larger, raw model in satisfying user intents. That underscores that it's not just model size that matters, but training the model to understand and execute instructions.

Reinforcement Learning from Human Feedback (RLHF) takes alignment a step further. Instead of just fine-tuning on example tasks, RLHF uses humans to rank or grade model outputs and then uses those rankings as a reward signal to further optimize the model's behavior. The process works with a human labeler who ranks model outputs for a given prompt from best to worst, and the model is then trained with a reward model to predict those rankings. Finally, the model is fine-tuned using reinforcement learning to maximize the reward model's score. This trains the model to generate outputs that humans would rate highly. OpenAI applied RLHF to its instruction-tuned GPT-3 to further align it with qualities such as correctness, usefulness, and harmlessness. The outcome was ChatGPT.

Thanks to RLHF, ChatGPT became significantly more factual, with fewer hallucinations, less toxicity, and greater obedience to user requests than the base model. Of course, it wasn't perfect, but it was a leap forward in alignment. The technique essentially unlocked latent capabilities in the model that had not been reliably used before. It also introduced some compromises, often termed the "alignment tax," in which aligning to user

Licensed to Hermilo Cortés <hermilocg@gmail.com>

preferences on some distributions could reduce performance on others. But researchers found ways to mitigate this by incorporating original training data into RLHF to retain knowledge.

But RLHF is not the end of the story for alignment. Direct Preference Optimization (DPO) was introduced by researchers in 2023 as an alternative. DPO eliminates the reinforcement learning step and instead fine-tunes the model directly on comparisons in a more stable, simplified way [37]. It frames the problem as a binary classification task, where the preferred output is chosen, and optimizes the model to increase the probability of the preferred output over the dispreferred one [38]. The promise of DPO is that it can achieve results similar to RLHF without the complexity of training a separate reward model and running RL.

Constitutional AI is even more intriguing. Anthropic proposed and implemented an alternative approach to model alignment using AI feedback rather than human feedback. The idea of Constitutional AI (2022) is to give the AI a set of written principles or a "constitution." These might include rules like "the AI should never produce hate speech or encourage illegal acts" and "the AI should be helpful and honest," and then have the AI itself critique and refine its outputs in accordance with those principles [39]. Essentially, the model generates an answer, then produces a revised answer that aligns with the given principles. The advantage is that it reduces reliance on large-scale human annotation and can yield a model that's less evasive than RLHF models on sensitive queries. RLHF models often learn to respond with safe but unhelpful refusals, whereas Constitutional AI tries to maintain helpfulness while remaining safe.

Instruction tuning made models better followers of user intent out of the box. RLHF greatly improved quality and safety by using human judgment at scale. DPO and Constitutional AI further refine how we align models, either by simplifying the process or by reducing the need for human feedback through principle-driven AI feedback. These advancements addressed the immediate shortcomings of large language models by making them more useful, less harmful, and more controllable. Today, the result is all around us. AI assistants from OpenAI, Anthropic, Google, and xAI were all built on these alignment techniques. In effect, the raw intelligence of foundation models was refined into something more approachable and beneficial for end users, greatly expanding AI adoption.

E.2.6 Tools and knowledge

Almost immediately after the launch of ChatGPT, a clear limitation emerged. ChatGPT had no access to fresh knowledge. Its knowledge

Licensed to Hermilo Cortés <hermilocg@gmail.com>

cutoff was fixed by its training data. For ChatGPT, that meant it didn't know about events after 2021. And it would often hallucinate facts, confidently stating things that weren't true because it had no mechanism to check its answers. The AI community tackled these problems with closely related developments, including web search, retrieval-augmented generation (RAG), and the rise of large language model (LLM) "agent" frameworks that use tools.

RAG is a framework that pairs an LLM with a knowledge retriever, typically a vector database. The idea was introduced around 2020, but it gained significant traction after ChatGPT [40]. Instead of relying solely on its parameter memory, which might be outdated, the model can refer to an external source of truth. In a RAG system, when a question comes in, the system first uses a search component to fetch relevant text. These retrieved snippets are then given to the language model along with the original query, and the model uses them to formulate its answer. This effectively gives the model an open-book exam rather than a closed-book exam. The benefit is that the model's responses can include up-to-date information and are grounded in real sources, reducing the risk of hallucinations and making the answers easier to verify.

In parallel with retrieval, there was also an effort to allow models to use tools beyond simple text search. A flurry of work emerged in 2022–2023 on turning language models into more general agents capable of taking actions. ReAct (2022) stands for "Reasoning and Acting." Researchers proposed prompting the model in a format that produces not just the final answer but also a chain of thought and, possibly, an action, interleaved [41]. For example, given a question that might require looking something up, the model's thought might be "I should search for X," and the action might be "Search X." The system then executes that action (e.g., queries a search API, web, a calculator, or another tool) and returns the observation (i.e., the results) to the model, which then continues reasoning. ReAct showed that with a few examples of this format, models like GPT-3 could learn to both think step by step and use tools when needed. Just as importantly, it improved interpretability, as users could see the model's chain of thought and the actions it took, making it easier to debug or trust.

Toolformer (2023) took the idea further by fine-tuning a model to use tools autonomously [42]. It essentially generated synthetic training data in which a larger model annotated text with API calls and a smaller model was fine-tuned on that data to decide when and how to call the tools. The impressive part of Toolformer is that it learns to insert the equivalent of

Licensed to Hermilo Cortés <hermilocg@gmail.com>

"calls" into its output; at inference time, these calls are executed, and results are returned to the model's context. The paper reported that Toolformer-augmented language models outperformed GPT-3 on zero-shot tasks, showing that tool use can be a significant help. In essence, while ReAct focused on prompting strategies, Toolformer focused on baking tool-use ability into the model's weights.

E.2.7 Mixture-of-Experts

The post-ChatGPT era has seen numerous advances in architectures and techniques for getting more bang for the buck. One major line of research has been the exploration of sparsity in neural networks, particularly in Mixture-of-Experts (MoE) models. Instead of a dense model, where every parameter is used for every input, an MoE model has multiple "experts" (i.e., subnetworks) and a gating mechanism that activates only a few of them for each input token. This means, for example, that you could have a model with a total of 1 trillion parameters but only 1% of them are used for a given token's computation.

Google's Switch Transformer (2021) demonstrated that MoEs can massively increase the parameter count without a proportional increase in computation and showed promising training efficiency. After ChatGPT, this idea gained more traction for inference-time efficiency, as a well-designed MoE can keep runtime costs similar to those of a smaller model while using a much larger parameter space when needed. OpenAI's own GPT-4 is rumored (though not publicly confirmed) to have used some form of MoE or model parallelism to achieve its performance, possibly with many experts specialized in different domains.

A shining example in the open space is what we discussed under DeepSeek: DeepSeek-R1 is a model that explicitly uses a 37-billion-parameter MoE activated from a 671-billion-parameter total parameter pool [43]. That design allowed it to achieve exceptional reasoning performance. The team reports that the model achieved near-state-of-the-art results on math and reasoning tasks, comparable to OpenAI's o1 model, while keeping computational cost manageable by activating only a subset of experts per query. In their evaluation, DeepSeek-R1 delivered results surpassing those of even dense models many times its active size on certain benchmarks, thanks to this architecture. Essentially, MoE is a way to have a model that is "large when it needs to be, small when it can be."

Another efficiency vector was low-precision computation. Research and engineering led to 8-bit, 4-bit, and even 2-bit quantization methods that let models run with minimal degradation in accuracy. Techniques like

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Quantized LoRA (QLoRA) demonstrate that models can be effectively fine-tuned at 4-bit precision. By late 2023, it became feasible to run moderately sized models on consumer GPUs at 4-bit precision with very little loss in quality. This means the barrier to entry for using these models dropped.

On the architecture front beyond MoE, researchers experimented with long-context models, such as adding recurrence or segment-based processing to handle book-length inputs, retrieval-augmented transformers that blend retrieval into the model's architecture for efficiency, and modular networks that allow different modules to be updated or swapped independently (useful for efficiency and continuous learning).

One notable development was OpenAI's o1 (Strawberry) reasoning model, which introduced a new "optimization algorithm" for training, hinting that they might be doing something very different internally, possibly using multistep reasoning during training or a form of large-scale meta-optimization [44][45][46]. They also apparently used more compute per query for the o1 model, which was slower but more accurate for complex tasks [47]. This suggests a trade-off where you can design a model to be fast and broad, like GPT-4, or slower but deeper thinking, like o1. In the future, models will dynamically adjust this based on question complexity, spending more time on hard problems and less on easy ones.

Today, success is also about optimization. We first built bulky automobiles, and now we're figuring out how to make reliable, efficient cars that everyone can drive. AI is following the same path of refinement by compressing knowledge through distillation, being smart about which circuits to activate (i.e., sparsity), cutting down on redundant precision (i.e., quantization), and even exploring new training paradigms to maximize return on compute, as OpenAI hints with o1. The result by 2025 is that AI models are not just smarter; they're also sleeker under the hood.

E.2.8 Reasoning models

Even as models became more knowledgeable and multimodal, a thorny challenge remained. Although LLMs are somewhat capable of reasoning via chain-of-thought prompting, they still make many logical errors or hallucinate steps in long, especially nonmonotonic, reasoning processes. To address this, a new class of models and training approaches emerged after ChatGPT, aiming to optimize reasoning specifically.

As previously mentioned, OpenAI's o1 model, launched in late 2024, is emblematic of this shift. It was explicitly described as the first of a series of "reasoning models." Unlike GPT-4, which is a strong all-rounder, o1

Licensed to Hermilo Cortés <hermilocg@gmail.com>

was designed to excel at tasks like complex problem-solving, mathematics, and coding, even if it meant being slower and more expensive to run.

o1 used reinforcement learning to solve problems, coupled with an internal chain-of-thought approach during inference. In practice, o1 can break down a problem internally, step by step, somewhat like how we would do scratch work on paper, and it was rewarded during training for correct multistep solutions, not just next-word prediction. The results were astounding on many tasks. For instance, o1 could solve 83% of International Mathematical Olympiad qualifier problems, whereas GPT-4 solved only 13% [48]. It also achieved high performance in coding competitions, ranking in the 89th percentile on Codeforces contests. These are problems that require rigorous logical thinking.

The trade-off was that o1 was less good at mundane knowledge or quick answers than GPT-4. It might even lack some factual recall, and it was more computationally heavy. But as a specialist, it represented a new direction. These models were trained to reason first, even if they needed to derive some facts rather than recall them. This is reminiscent of how AlphaGo was trained to win games (an RL objective) rather than just imitate human moves. Now we see language models being trained to solve tasks rather than just imitate text. It's a more goal-directed training regime.

The AI field didn't rest on its laurels. It didn't push back from the table following ChatGPT. It confronted the unsolved challenges and aligned models safely, grounding them with tools and data, democratizing access, optimizing efficiency, extending their modalities, and sharpening their reasoning. With the "missing pieces" of Sutskever's List, we can better appreciate how everything came together to create today's world.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Licensed to Hermilo Cortés <hermilocg@gmail.com>

appendix Design patterns for engineers

Sutskever's List suggests that the history of modern AI is, above all, a history of learnable structure made scalable. The winning systems were not the ones with the most beautiful theory, but the ones that learned representations end-to-end, preserved signal through depth, matched architecture to data structure, and turned extra compute into reliable empirical gains. From AlexNet to ResNet to Transformers, the pattern is the same: simple, modular ideas, well-engineered, scaled hard, and judged by measurable gains over the best existing baseline. The deeper claim underneath that engineering tradition is that intelligence emerges when a model compresses the world well enough to recover its hidden structure; the deeper warning is that once such systems scale, objective design and safety can no longer be treated as afterthoughts.

Learned representations (chapters 2–7)

AlexNet mattered because it displaced the SIFT/HOG era. Deep Speech 2 repeated the same move in speech by replacing phonetic dictionaries and rigid alignments with direct audio-to-text feature learning. The preference for end-to-end learned features, which repeats throughout the book, stems from the fact that the world is messy and handcrafted features are brittle.

No frozen priors (chapters 2–7)

Handcrafted features were attractive because their invariances were interpretable, but they also locked in human guesses about what the signal should look like into the representation. A better pattern is to

Licensed to Hermilo Cortés <hermilocg@gmail.com>

use computation, data, augmentation, and architectural bias to make learning tractable while leaving the representation itself to be discovered. Old pipelines encoded invariances first and learned afterward; deep learning learns the representation, using human priors mainly as scalable hints rather than fixed answers. This is essentially "The Bitter Lesson."

DATA AUGMENTATION (CHAPTERS 2–7)

AlexNet's random crops and horizontal reflections gave the network many views in which the object category remained stable despite shifts, partial views, and left-right reversal. Its PCA-based color jitter similarly discouraged dependence on accidental lighting and color balance. This was a softer alternative to hard-coding invariances into the feature extractor. Instead of freezing some human notion of the signal, augmentation encodes a weaker prior as experience by repeatedly showing the model nuisance variation and letting it learn what remains predictive. In practice, augmentation does not just enlarge the dataset; it reshapes the learning problem so capacity is spent on stable structure rather than superficial artifacts.

Data augmentation is the "unreasonable effectiveness of data" turned inward. When the real dataset is too small to reveal the stable structure, augmentation manufactures the variation that would have appeared in a larger dataset. The model learns invariance not because a hand-engineered descriptor declared it in advance, but because the training distribution repeatedly demonstrates it.

OPTIMIZATION IS PART OF THE ARCHITECTURE (CHAPTERS 1–5)

AlexNet first animated this idea by using ReLUs and training-aware design choices, which were not merely cosmetic details but instead made a large network trainable at ImageNet scale. ResNet added pure identity paths, pre-activation, and removed the post-merge non-linearity. These tiny changes determine whether very deep models train at all. In fact, the margin for success is razor thin. Ultimately, capacity only matters if gradients and representations can move cleanly through the system.

PREFER MODULAR BLOCKS (CHAPTERS 3–6)

GoogLeNet's Inception modules achieved excellent compute–accuracy trade-offs, but their branch widths, filter sizes, and stage-by-stage configurations required careful manual design. ResNet made depth scalable by simply repeating a residual block.

Transformers extended the lesson. Like ResNet, they scale by repeating a simple homogeneous block rather than hand-designing many heterogeneous modules. Each block preserves a residual stream while adding attention and feed-forward transformations, making depth, width,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

and head count relatively systematic scaling axes. And, as with ResNet, the optimization details are architectural: residual pathways, LayerNorm placement, attention scaling, and short token-to-token paths are not incidental training tricks but core reasons the model trains and scales. Finally, by partitioning a sequential network across devices and streaming micro-batches through the partitions, GPipe enabled scaling of existing architectures without changing their semantics.

PREFER MINIMAL INNOVATION WITH MAXIMAL USE (CHAPTERS 2–6)

The book repeatedly frames Sutskever's taste as "minimum innovation for maximum results." Residual connections, pre-activation, dilation, attention, or even sequence reordering matter because small changes can unlock gains in scale and performance. The goal is to shift the system's scaling behavior, not to overengineer or be clever creating a flywheel effect.

ITERATION, NOT INVENTION (CHAPTERS 3–5)

AlexNet, ResNet, Deep Speech 2, and the Transformer did not win by maximizing novelty. ResNet showed that the simplest, unaltered identity mapping was the most effective. The Transformer extended that lesson. A simpler, more parallelizable design beat architectures with stronger theoretical priors because it was easier to optimize and scale. On Sutskever's List, engineering is not downstream of the idea; it is the mechanism by which the idea is animated and becomes viable.

ENGINEERING IS NOT A PEJORATIVE (CHAPTERS 2, 4, AND 6)

AlexNet and Deep Speech 2 are presented as case studies in disciplined, empirical system-building: GPUs, careful initialization, normalization, dropout, curriculum learning, batching, and deployment-minded choices all compound. Despite the pejorative "brute force," this is precisely what makes ideas real, rather than elegance alone.

ABLATIONS (CHAPTERS 2–7)

ResNet v2 presented pre-activation but its authors systematically tested scaled skip paths, dropped skip paths, gated shortcuts, added convolutions on the skip branch, and alternative placements of batch normalization and ReLU. Those experiments showed that even small disruptions to the identity path could impair convergence or accuracy, making the central lesson unmistakable. Ablations convert ideas into evidence.

BEST BASELINE, NOT BIOLOGY (CHAPTERS 2–7)

Before AlexNet, artificial neural networks were often justified through biological analogy and anecdotal results, with weak or inconsistent baselines. But the emergence of shared benchmarks and a strong baseline culture changed the standard of proof. Today, progress is measurable,

Licensed to Hermilo Cortés <hermilocg@gmail.com>

comparable, and reproducible. In fact, over the past 15 years, much of AI's acceleration can be explained not just by better engineering, but also by the discipline imposed by benchmarks and baseline-driven evaluation.

SCALE WHAT? (CHAPTERS 2–6)

It's not a question of whether scaling works. But we must be specific and ask exactly what is being scaled. Ultimately, scalable architectures are not just architectures that happen to be large. The lesson of the scaling era is not "add compute" but find an architecture that converts compute into better representations, then scale it aggressively. Scale is a multiplier but not a substitute for the right architecture.

FAVOR SHORT PATHS AND PARALLELISM (CHAPTERS 4–6)

Architectures that minimize dependency length and maximize parallelism are easier to optimize and scale, and therefore win. The reversal trick was an early hint that sequence models become simpler to train when information travels a shorter path. The Transformer uses attention to connect positions more directly and employs parallel computation to scale those connections.

SELECTIVE ADDRESSING (CHAPTER 5)

Bahdanau's soft attention revealed the sequence-to-sequence "thought vector" as a bottleneck rather than a solution. Pointer Networks used attention to point, and Neural Turing Machines used it to read and write memory, but Transformers made the same idea more general by allowing every token to access every other token directly. Performance improves when models can access specific inputs or memory slots as needed, rather than compressing the entire problem into a single fixed state. Models perform better when they can point to what matters, not just remember everything, and attention can read over tokens, memory, or nodes.

SMOOTH LOSS VERSUS CAPABILITIES (CHAPTER 6)

Scaling laws make training loss appear regular, but actual abilities can switch abruptly because evals are thresholded and regime changes matter. That makes evaluation design as important as optimization design.

WEAKER INDUCTIVE BIASES (CHAPTER 7)

The shift from recurrence to attention removed rigid sequential constraints, while positional encodings were reintroduced in a softer form. This pattern recurs throughout the book. Replace hard-coded structure with mechanisms that let the model learn when and how that structure matters. The lesson is to encode bias in a way that guides learning without constraining it.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

PRIORS STILL MATTER (CHAPTER 7)

Scaling laws suggest that once a model family is genuinely scalable, architectural changes mostly shift the intercept rather than the slope. That is, the right inductive bias shifts the whole curve downward, yielding better performance at every budget of data and compute. Architectures such as relation networks and message passing neural networks reintroduce relational and topological inductive biases when the task depends on explicit interactions and when structure is real and consequential. Yet a prior earns its keep only if it survives benchmarks and optimization. Elegant structures that do not scale, such as capsules, or recurrence-heavy detours like the Universal Transformer, lose to architectures that are simpler, easier to optimize, and better able to absorb data and compute at frontier scale. Priors still matter as a force multiplier on scale but not as an alternative to it.

SPEED MATTERS (CHAPTER 7)

MPNN emphasizes that approximating DFT far faster changed scientific workflows, not just a benchmark score. A model becomes historically important when it changes which experiments are feasible, though both are best.

GOOD PREDICTION IS COMPRESSION (CHAPTER 8)

Sutskever argues that a good prediction model is—in an important sense—a good compressor. In the MDL view, a model generalizes when it captures reusable regularities in the data rather than memorizing accidental quirks. The "secrets" in data are latent variables, hidden factors, and shared structure, including sentiment in reviews, grammar in language, useful intermediate representations in translation, or rule-like structure in grokking. Kolmogorov complexity and MDL provide the conceptual framing, while the sentiment neuron, thought vectors, Hinton and van Camp, and grokking give concrete neural examples. The unifying claim is not that compression proves truth, but that successful learning often replaces a long list of exceptions with a shorter structural representation that continues to predict well.

MEMORIZATION BEFORE GENERALIZATION (CHAPTER 8)

In grokking experiments, models can first fit the training set by encoding many example-specific details, then later transition to a lower-description-length representation that captures the underlying rule. Sufficient regularization appears to favor this transition: instead of remaining in a high-complexity memorization regime, the model eventually settles into a simpler predictor that generalizes.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

SAFETY WITHIN ENGINEERING (CHAPTERS 1 AND 9)

GPT-2 (chapter 1) marked the first public clash between scaling and safety, and chapter 9 extends this concern to reward hacking, wireheading, and objective stability. These are not only examples of living doubt, but the design surface also demonstrates that capability and control are inseparable. If reward channels can be gamed or goals can drift from human intent, bigger systems only magnify the cost of error. The engineering conclusion is to build systems that optimize, compress, and scale, and to evaluate them honestly, but never to treat alignment, production release, or safe design as somebody else's problem.

HIDDEN ESSENCES (CHAPTER 9)

Keep performance claims operational rather than turning them into claims about hidden essences. That is, do not let benchmark victories turn into metaphysical claims. Moreover, ignore skeptics who dismiss success against benchmarks and existing baselines in favor of metaphysical arguments about "real intelligence." AI was not legitimized by better definitions but by avoiding them and focusing on empirical wins, and that is what will sustain it.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

references

Chapter 1

[1] Stan Schroeder, "OpenAI chief scientist Ilya Sutskever leaves OpenAI," Mashable, May 14, 2024. https://mashable.com/article/openair-ilya-slutskever-leaves-chief-scientist

[2] E. Musk (@elonmusk), "What *did* Ilya see!? This is worrying," X, Dec. 10, 2023. https://twitter .com/elonmusk/status/1734063208648880247

[3] R. Kalvapalle, "'Godfather of AI' Geoffrey Hinton's Nobel Prize makes headlines around the world," *University of Toronto News,* Oct. 9, 2024. https://www.utoronto.ca/news/godfather-ai-geof frey-hinton-s-nobel-prize-draws-headlines-around-world

[4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks" in *Proceedings of the 26th International Conference on Neural Information Processing Systems*, vol. 1, Lake Tahoe, NV, 2012, pp. 1097–1105.

[5] M. Ford, *Architects of Intelligence: The Truth About AI from the People Building it.* Birmingham, U.K.: Packt Publishing, Nov. 2018 (p. 4)

[6] "How a stubborn computer scientist accidentally launched the deep learning boom," Ars Technica, Nov. 11, 2024

[7] H. Hsu, "CHM releases AlexNet source code," Computer History Museum Blog, Mar. 20, 2025. https://computerhistory.org/blog/chm-releases-alexnet-source-code

[8] J. D'Onfro, "You don't need to have a computer science degree from Stanford to be working on one of Google's hottest teams," *Business Insider*, Aug. 12, 2016. https://www.businessinsider.com/the-surprising-backgrounds-of-people-working-at-google-brain-2016-8

[9] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous distributed systems," arXiv:1603.04467, Mar. 2016. https://arxiv.org/pdf/1603.04467.pdf

[10] J. Dean et al., "Large scale distributed deep networks," in *Advances in Neural Information Processing Systems* 25 (NeurIPS 2012), 2012, pp. 1223–1231.

[11] Silver, D., Huang, A., Maddison, C. et al. "Mastering the game of Go with deep neural networks and tree search," *Nature* 529, 484–489 (2016).

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[12] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," arXiv:1409.3215, Sept. 2014. https://arxiv.org/abs/1409.3215

[13] S. Levy, "What OpenAI really wants," *WIRED*, Sep. 5, 2023. https://www.wired.com/story/what-openai-really-wants/

[14] C. Metz, K. Weise, N. Grant, and M. Isaac, "Ego, fear and money: How the A.I. fuse was lit," *The New York Times*, Dec. 3, 2023

[15] New York Times Events, "Elon Musk on advertisers, trust and the 'wild storm' in his mind | DealBook Summit 2023," YouTube, Nov. 30, 2023. https://youtu.be/2BfMuHDfGJI?si=HPLwKLgYwvgIzl8K&t=2772

[16] Season One | The Robot Brains Podcast, "Season 1 Ep. 22 OpenAI's Ilya Sutskever: The man who made AI work," YouTube, Sept. 21, 2021. https://youtu.be/fCoavgGZ64Y?si=R_tfONE0OCdShCEk&t=1780

[17] Same as [14].

[18] CNBC Television, "Elon Musk on Sam Altman and ChatGPT: I am the reason OpenAI exists," YouTube, May 16, 2023. https://youtu.be/bWr-DA5Wjfw?si=PKSg9nuLEyWhVKHU&t=203

[19] J. Hart and A. Mok, "Meet OpenAI cofounder Ilya Sutskever, who supported and then regretted Sam Altman's firing," *Business Insider*, Nov. 20, 2023. https://www.businessinsider.com/meet-openai-cofounder-ilya-sutskever-supported-regretted-sam-altman-firing-2023-11#at-the-end-of-2015-ilya-sutskever-cofounded-open-ai-with-ex-ceo-sam-altman-7

[20] Same as [14].

[21] OpenAI, "Introducing OpenAI," OpenAI Blog (archived by Internet Archive), Dec. 11, 2015. https://web.archive.org/web/20211006105333/https://openai.com/blog/introducing-openai/

[22] Same as [14]

[23] OpenAI, "OpenAI Gym (Beta)," OpenAI, May 4, 2016. https://openai.com/index/openai-gym-beta/

[24] OpenAI, "Dota 2," OpenAI, Jun. 30, 2017. https://openai.com/index/dota-2/

[25] "Elon Musk is leaving the board of OpenAI," CNBC, Feb. 21, 2018. https://www.cnbc.com/2018/02/21/elon-musk-is-leaving-the-board-of-openai.html

[26] Same as [16].

[27] S. Jackson, "Elon Musk reportedly left OpenAI's board in 2018 after Sam Altman and other cofounders rejected his plan to run the company," *Business Insider*, Mar. 24, 2023. https://www.busi nessinsider.com/elon-musk-reportedly-tried-lead-openai-left-after-founders-objected-2023-3

[28] A. Hern, "New AI fake text generator may be too dangerous to release, say creators," *The Guardian,* Feb. 14, 2019. https://www.theguardian.com/technology/2019/feb/14/elon-musk-backed-ai-writes-convincing-news-fiction

[29] Notably in the 2020 paper, "Language Models are Few-Shot Learners"

[30] "The brain that supercharged ChatGPT: ImageNet, TensorFlow," *Analytics India Magazine*, n.d. https://analyticsindiamag.com/ai-features/the-brain-that-supercharged-chatgpt-imagenet-tensorflow/

[31] Same as [20]

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[32] I. Sutskever (@ilyasut), "It may be that today's large neural networks are slightly conscious," X, Feb. 9, 2022. https://x.com/ilyasut/status/1491554478243258368

[33] A. Karpathy (@karpathy), "@ilyasut @vkhosla agree … consciousness is a useful insight …," X, Feb. 10, 2022. https://x.com/karpathy/status/1491608794253512707

[34] Y. LeCun (@ylecun), "@ilyasut Nope. Not even for true for small values of 'slightly conscious' …," X, Feb. 12, 2022. https://x.com/ylecun/status/1492604977260412928

[35] M. Shanahan (@mpshanahan), "@ilyasut … a large field of wheat is slightly pasta …," X, Feb. 10, 2022. https://x.com/mpshanahan/status/1491715721289678848

[36] B. Nguyen, "OpenAI promised 20% of its computing power to combat existential threats from AI. That never happened, sources say," *Fortune*, May 21, 2024. https://fortune.com/2024/05/21/openai-superalignment-20-compute-commitment-never-fulfilled-sutskever-leike-altman-brockman-murati/

[37] OpenAI, "Introducing Superalignment," OpenAI, Jul. 5, 2023. https://openai.com/index/intro ducing-superalignment/

[38] V. Tangermann, "OpenAI employees say firm's chief scientist has been making strange spiritual claims," *Futurism*, Nov. 20, 2023. https://futurism.com/openai-employees-say-firms-chief-scientist-has-been-making-strange-spiritual-claims

[39] K. Hao and C. Warzel, "Inside the chaos at OpenAI," *The Atlantic*, Nov. 19, 2023. https://www.theat lantic.com/technology/archive/2023/11/sam-altman-open-ai-chatgpt-chaos/676050/

[40] "OpenAI's Sutskever says he regrets board's firing of Altman," Axios, Nov. 20, 2023. [Online]. Available: https://www.axios.com/2023/11/20/sam-altman-fired-openai-board-illya-sutsever-regrets

[41] Same as [39]

[42] "OpenAI co-founder Sutskever's new safety-focused AI startup SSI raises $1 billion," Reuters, Sep. 4, 2024. https://www.reuters.com/technology/artificial-intelligence/openai-co-founder-sutskevers-new-safety-focused-ai-startup-ssi-raises-1-billion-2024-0-04

[43] "Sam Altman's ouster at OpenAI was precipitated by letter to board about AI breakthrough," Reuters, Nov. 22, 2023. https://www.reuters.com/technology/sam-altmans-ouster-openai-was-precipitated-by-letter-board-about-ai-breakthrough-2023-11-22

[44] Same as [43]

[45] Same as [1]

[46] Same as [2]

[47] "Report: Sutskever led board coup at OpenAI that ousted Altman over AI safety concerns," Ars Technica, Nov. 2023. https://arstechnica.com/information-technology/2023/11/report-sutskever-led-board-coup-at-openai-that-ousted-altman-over-ai-safety-concerns/

[48] Same as [43]

[49] I. Sutskever (@ilyasut), "Post on X," X, Nov. 20, 2023. https://x.com/ilyasut/status/1726590052392956028

[50] OpenAI (@OpenAI), "Post on X," X, Nov. 22, 2023. https://x.com/OpenAI/status/1727206187077370115

[51] Same as [50]

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[52] Same as [37]

[53] K. Hao, "Empire of AI: Inside the reckless race for total domination (excerpt)," *The Atlantic*, May 2025. https://www.theatlantic.com/technology/archive/2025/05/karen-hao-empire-of-ai-excerpt/682798/

[54] Al-Sibai, "Scientist who said neural networks may already be conscious leaves OpenAI after leading coup against Sam Altman," Futurism (The Byte), May 15, 2024. https://futurism.com/the-byte/ilya-sutskever-leaves-openai

[55] "OpenAI resignations: AI safety, Ilya Sutskever, Jan Leike, artificial intelligence," Vox (Future Perfect), May 17, 2024. https://www.vox.com/future-perfect/2024/5/17/24158403/openai-resignations-ai-safety-ilya-sutskever-jan-leike-artificial-intelligence

[56] J. Leike (@janleike), "Post on X," X, May 17, 2024. https://x.com/janleike/status/1791498174659715494

[57] Same as [43]

[58] Same as [29]

[59] "Artificial intelligence, OpenAI, and natural language processing," Vox (Future Perfect), Feb. 14, 2019. https://www.vox.com/future-perfect/2019/2/14/18222270/artificial-intelligence-open-ai-natural-language-processing

[60] J. Vincent, "OpenAI's new AI writes, translates, and slanders," *The Verge*, Feb. 14, 2019. https://www .theverge.com/2019/2/14/18224704/ai-machine-learning-language-models-read-write-openai-gpt2

[61] "OpenAI text generator dangerous," TechCrunch, Feb. 17, 2019. https://techcrunch.com/2019/02/17/openai-text-generator-dangerous

[62] Same as [60]

[63] "OpenAI's GPT-2: The model, the hype and the controversy," Medium, n.d. https://medium.com/data-science/openais-gpt-2-the-model-the-hype-and-the-controversy-1109f4bfd5e8

[64] OpenAI, "Better language models and their implications," OpenAI, Feb. 14, 2019. https://openai.com/index/better-language-models/

[65] I. Goodfellow et al., "Generative adversarial nets," arXiv:1406.2661, Jun. 2014. https://arxiv.org/abs/1406.2661

[66] T. Simonite, "The AI text generator that's too dangerous to make public," *WIRED*, Feb. 14, 2019. https://www.wired.com/story/ai-text-generator-too-dangerous-to-make-public/

[67] Same as [63]

[68] "OpenAI, please open source your language model," The Gradient, n.d. https://thegradient.pub/openai-please-open-source-your-language-model/

[69] Same as [65]

[70] Same as [65]

[71] OpenAI, "GPT-2: 6-month follow-up," OpenAI, Aug. 20, 2019. https://openai.com/index/gpt-2-6-month-follow-up/

[72] "Ethical analysis of the open-sourcing of a state-of-the-art conversational AI," Medium (Hugging Face), n.d. https://medium.com/huggingface/ethical-analysis-of-the-open-sourcing-of-a-state-of-the-art-conversational-ai-852113c324b2

[73] "Why we released Grover," *The Gradient,* n.d. https://thegradient.pub/why-we-released-grover/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[74] Google Research, "Imagen," n.d. https://imagen.research.google

[75] OpenAI, "GPT-2: 1.5B release," OpenAI, Nov. 5, 2019. https://openai.com/index/gpt-2-1-5b-release/

[76] "Meta's large language model AI only survived three days on the internet before being taken down," *MIT Technology Review*, Nov. 18, 2022. https://www.technologyreview.com/2022/11/18/1063487/meta-large-language-model-ai-only-survived-three-days-gpt-3-science/

[77] K. Chan and M. O'Brien, "Google suspends Gemini AI chatbot's ability to generate images of people," Associated Press, Feb. 22, 2024. https://apnews.com/article/google-gemini-ai-chatbot-image-generation-1bd45f1e67dfe0f88e5419a6efe3e06f

[78] Same as [48]

[79] Same as [47]

[80] "OpenAI chief scientist Ilya Sutskever leaves," *The Verge*, May 14, 2024. https://www.theverge.com/2024/5/14/24156920/openai-chief-scientist-ilya-sutskever-leaves

[81] G. Hunter, "Exclusive Q&A: John Carmack's 'different path' to artificial general intelligence," Dallas Innovates, Feb. 2, 2023. https://dallasinnovates.com/exclusive-qa-john-carmacks-different-path-to-artificial-general-intelligence

[82] K. Hu, "ChatGPT sets record for fastest-growing user base—analyst note," Reuters, Feb. 1, 2023. https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/

[83] Hacker News, "What were the papers on the list Ilya Sutskever gave John Carmack?," Feb. 3, 2023. https://news.ycombinator.com/item?id=34641359

[84] J. Carmack (@ID_AA_Carmack), "Post on X," X, Feb. 6, 2023. https://x.com/ID_AA_Carmack/status/1622673143469858816

[85] dzyim, "ilya-sutskever-recommended-reading," GitHub repository, n.d. https://github.com/dzyim/ilya-sutskever-recommended-reading?tab=readme-ov-file

[86] keshavchan (@keshavchan), "Post on X," X, May 7, 2024. https://x.com/keshavchan/status/1787861946173186062

[87] W. D. Heaven, "Exclusive: Ilya Sutskever, OpenAI's chief scientist, on his hopes and fears for the future of AI," *MIT Technology Review*, Oct. 26, 2023. www.technologyreview.com/2023/10/26/1082398/exclusive-ilya-sutskever-openais-chief-scientist-on-his-hopes-and-fears-for-the-future-of-ai/

[88] V. Kuka, "The mysterious AI reading list: Ilya Sutskever's recommendations," *The Turing Post*, May 19, 2024. https://www.turingpost.com/p/ilya-sutskever-reading-list

[89] Microsoft Research, "NIPS: Oral session 4–Ilya Sutskever," video, Dec. 9, 2014 (9:23). https://t.co/sJa4QJJVu2

Chapter 2

[1] J.J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proceedings of the National Academy of Sciences USA* 79, no. 8 (1982): 2554-2558. https://doi.org/10.1073/pnas.79.8.2554

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE* 86, no. 11 (Nov. 1998): 2278–2324, doi: 10.1109/5.726791. http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf

[3] J. Schmidhuber, "DanNet triggers deep CNN revolution," IDSIA, 2011. https://people.idsia.ch/~juergen/DanNet-triggers-deep-CNN-revolution-2011.html

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature* 521 (2015): 436–444. https://doi.org/10.1038/nature14539

[5] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, 323, no. 6088 (Oct. 1986): 533–536, doi: 10.1038/323533a0. https://www.nature.com/articles/323533a0

[6] T. B. Lee, "How a stubborn computer scientist accidentally launched the deep learning boom," *Ars Technica*, Nov. 11, 2024. https://arstechnica.com/ai/2024/11/how-a-stubborn-computer-scientist-accidentally-launched-the-deep-learning-boom/

[7] M. Ford, *Architects of Intelligence: The Truth About AI From the People Building it.* Birmingham, U.K.: Packt Publishing, 2018, p. 4

[8] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning* 20, no. 3 (Sep. 1995): 273–297, doi: 10.1007/BF00994018

[9] *The Gradient*, "The limitations of visual deep learning and how we might fix them." https://thegradient.pub/the-limitations-of-visual-deep-learning-and-how-we-might-fix-them

[10] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature* 521, no. 7553 (May 2015): 436–444, doi: 10.1038/nature14539. https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf

[11] G. E. Hinton, S. Osindero, and Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural Computing* 18, no. 7 (Jul. 2006): 1527–1554, doi: 10.1162/neco.2006.18.7.1527. https://www.cs.toronto.edu/~hinton/absps/ncfast.pdf

[12] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Advances in Neural Information Processing Systems (NIPS)* 19 (2007): 153–160

[13] Same as [7]

[14] Computer Vision Blog, "From feature descriptors to deep learning," Jan. 2015. https://www.computervisionblog.com/2015/01/from-feature-descriptors-to-deep.html

[15] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computing Vision* 60, no. 2 (Nov. 2004): 91–110, doi: 10.1023/B:VISI.0000029664.99615.94. https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf

[16] H. Bay, T. Tuytelaars, and L. Van Gool, "SURF: Speeded up robust features," in *Computer Vision— ECCV 2006*, A. Leonardis, H. Bischof, and A. Pinz (eds.), Berlin, Germany: Springer, 2006, pp. 404–417, doi: 10.1007/11744023_32

[17] G. Csurka, C. Dance, L. Fan, J. Willamowski, and C. Bray, "Visual categorization with bags of keypoints," in *Proceedings of the ECCV Workshop on Statistical Learning in Computer Vision, 2004*, pp. 59–74. https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/csurka-eccv-04.pdf

[18] YouTube, "Ilya Sutskever at AI Frontiers 2018: Recent advances in deep learning and AI from OpenAI." https://youtu.be/ElyFDUab30A?si=J48mzXqevdfY5oJV&t=1640

[19] Carl Vondrick, Aditya Khosla, Tomasz Malisiewicz, and Antonio Torralba, "HOGgles: Visualizing object detection features," 2013 IEEE International Conference on Computer Vision, Dec. 1–8, 2013, Sydney, NSW, pp. 1–9

[20] Moreover, many thought the image signal was too noisy and high-dimensional for a general-purpose learning algorithm to identify robust visual features. For example, see Y. Bengio and Y. LeCun, "Scaling learning algorithms towards AI," in *Large-Scale*

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Kernel Machines, L. Bottou, O. Chapelle, D. DeCoste, and J. Weston (eds.), Cambridge, MA: MIT Press, 2007, esp. the discussion of SVMs on raw pixels and sensitivity to nuisance variation/background clutter

[21] e.g., G. Hinton

[22] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR),* San Diego, CA, 2005, pp. 886–893, doi: 10.1109/CVPR.2005.177

[23] PubMed Central, "Using human brain activity to guide machine learning." https://pmc.ncbi.nlm.nih.gov/articles/PMC5876362/.

[24] R. Benenson, M. Omran, J. Hosang, and B. Schiele, "Ten years of pedestrian detection, What have we learned," arXiv [cs.CV], 2014

[25] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32, no. 9 (Sep. 2010): 1627–1645, doi: 10.1109/TPAMI.2009.167. https://cs.brown.edu/people/pfelzens/papers/lsvm-pami.pdf

[26] Computer Vision Blog, "From feature descriptors to deep learning," Jan. 2015. https://www.compu tervisionblog.com/2015/01/from-feature-descriptors-to-deep.html

[27] SciLogs (Spektrum der Wissenschaft), "Data is unreasonably effective and there is plenty of it to explore." https://scilogs.spektrum.de/hlf/data-is-unreasonably-effective-and-there-is-plenty-of-it-to-explore

[28] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Colorado Springs, CO, 2011, pp. 1521–1528, doi: 10.1109/CVPR.2011.5995347. https://people.csail.mit.edu/torralba/publications/datasets_cvpr11.pdf

[29] Fei-Fei Li, *The Worlds I See: Curiosity, Exploration, and Discovery at the Dawn of AI*. New York: Moment of Lift Books; Flatiron Books, 2023

[30] *The Gradient*, "The limitations of visual deep learning and how we might fix them." https://thegradient.pub/the-limitations-of-visual-deep-learning-and-how-we-might-fix-them

[31] Fei-Fei Li, T*he Worlds I See: Curiosity, Exploration, and Discovery at the Dawn of AI*. Kindle Edition, p. 194

[32] YouTube, "We are (still!) not giving data enough credit: Alyosha Efros, Keynote at ACVSS 24." https://www.youtube.com/watch?v=dV6sgUiGv-U

[33] Same as [24]

[34] *The Guardian*, "Tech giants data," Aug. 23, 2013. https://www.theguardian.com/technology/2013/aug/23/tech-giants-data

[35] Medium, "Data is the new oil: A ludicrous proposition." https://medium.com/project-2030/data-is-the-new-oil-a-ludicrous-proposition-1d91bba4f294

[36] A. Halevy, P. Norvig, and F. Pereira, "The unreasonable effectiveness of data," *IEEE Intelligent Systems* 4, no. 2 (Mar.-Apr.2009): 8–12, doi: 10.1109/MIS.2009.36

[37] Facebook. https://www.facebook.com/story.php?story_fbid=pfbid0b6rC7KJFHvxLJJb PtHyz53Ffjv9YWRJ3uQc6taVpv9CdqDUZ2z2ywbGDhdgZKx5Hl&id=1533402400

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[38] HLF Foundation Newsroom, “Data is unreasonably effective and there is plenty of it to explore.” https://www.newsroom.hlf-foundation.org/blog/article/data-is-unreasonably-effective-and-there-is-plenty-of-it-to-explore/

[39] A. Efros, “Are we still not giving data enough credit?” Hasso Plattner Institute / HLF Foundation Newsroom, video lecture. https://www.newsroom.hlf-foundation.org/newsroom/lectures/video/lecture-are-we-still-not-giving-data-enough-credit/

[40] Simons Institute for the Theory of Computing, “Stone soup AI.” https://simons.berkeley.edu/news/stone-soup-ai

[41] Same as [31]

[42] Same as [36]

[43] Slow Thoughts, “Seeing the light.” https://slow-thoughts.com/seeing-the-light/

[44] Irving Biederman, “Recognition-by-components: A theory of human image understanding,” *Psychological Review* 94, no. 2 (1987), pp. 115–117. doi:10.1037/0033-295x.94.2.115

[45] O. Russakovsky et al., “ImageNet Large Scale Visual Recognition Challenge,” arXiv [cs.CV], 2015. https://arxiv.org/abs/1409.0575

[46] ImageNet, “PASCAL/ILSVRC information (PDF).” https://image-net.org/static_files/files/pascal_ilsvrc.pdf

[47] ImageNet, “ILSVRC 2011.” https://www.image-net.org/challenges/LSVRC/2011/index.php

[48] Same as [39]

[49] Stacker News. https://stacker.news/items/762990

[50] D. C. Ciresan, U. Meier, J. Masci, L. M. Gambardella, J. Schmidhuber. “Flexible, high performance convolutional neural networks for image classification,” International Joint Conference on Artificial Intelligence (IJCAI-2011), Barcelona, 2011

[51] D. C. Ciresan, U. Meier, J. Masci, J. Schmidhuber. “A committee of neural networks for traffic sign classification,” International Joint Conference on Neural Networks (IJCNN-2011), San Francisco, CA, 2011

[52] D. C. Ciresan, U. Meier, and J. Schmidhuber, “Multi-column deep neural networks for image classification,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2012)*, Jul. 2012, pp. 3642–3649

[53] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge: A retrospective,” *International Journal of Computer Vision* 111, no. 1 (Jan. 2015): 98–136, doi: 10.1007/s11263-014-0733-5. https://link.springer.com/article/10.1007/s11263-014-0733-5

[54] Same as [49]

[55] Cross Validated (Stack Exchange), “Danger of setting all initial weights to zero in backpropagation.” https://stats.stackexchange.com/questions/27112/danger-of-setting-all-initial-weights-to-zero-in-backpropagation

[56] G. Philipp, D. Song, and J. G. Carbonell, "The exploding gradient problem demystified—definition, prevalence, impact, origin, tradeoffs, and solutions," *arXiv* [*cs.LG*], 2017. https://arxiv.org/abs/1712.05577

[57] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science* 313, no. 5786 (Jul. 2006): 504–507, doi: 10.1126/science.1127647. https://www.cs.toronto.edu/~hinton/absps/science.pdf

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[58] D. Erhan, Y. Bengio, A. Courville, P. Manzagol, P. Vincent, and S. Bengio, "Why does unsupervised pre-training help deep learning?" *Journal of Machine Learning Research*, 2010.

[59] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, July 2006

[60] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," In NIPS, 2007

[61] e.g., Lex Fridman Podcast. https://youtu.be/13CZPWmke6A?si=sSJdJMX67OGeINFZ&t=209

[62] As Martens put it: "The difficulty associated with learning deep autoencoders is better explained by regions of pathological curvature, which to first-order optimization methods resemble bad local minima." J. Martens, "Deep learning via Hessian-free optimization," in *Proceedings of the 27th International Conference on Machine Learning* (ICML), 2010

[63] Same as [58]

[64] ICML, "Learning recurrent neural Nntworks with Hessian-free optimization," 2011. https://icml.cc/Conferences/2011/papers/532_icmlpaper.pdf

[65] Semantic Scholar, "PDF." https://pdfs.semanticscholar.org/6503/e264e4fdecb22ab0f5d081123814ec242d86.pdf

[66] YouTube, "Ilya Sutskever: Deep learning | Lex Fridman Podcast #94" (5:13). https://youtu.be/13CZPWmke6A?si=pnioUssu8jioetYP

[67] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems (NIPS)* 25 (2012): 1097–1105. https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

[68] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," arXiv:1207.0580, 2012. https://arxiv.org/pdf/1207.0580

[69] Earlier research by Nair and Hinton (2010) introduced ReLUs for Restricted Boltzmann Machines, and Jarrett et al. (2009) explored alternative activation functions in smaller CNNs. V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010; and K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun, "What is the best multi-stage architecture for object recognition?" in *Proceedings on the IEEE International Conference on Computer Vision (ICCV)*, 2009

[70] Additionally, ReLU induced sparse activations, meaning many neurons output zeros, which reduced interneuronal co-adaptation and contributed to ReLU's overall effect on training.

[71] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research* 15, no. 1 (2014): 1929–1958. https://www.cs.toronto.edu/~rsalakhu/papers/srivastava14a.pdf

[72] Same as [67]

[73] CS231n, which is on Sutskever's List, discusses inverted dropout: Stanford CS231n, "Neural networks part 2: Setting up the data and the loss." https://cs231n.github.io/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

neural-networks-2/#reg; and Stanford CS231n, "Lecture 6 slides (winter 2016)," https://cs231n.stanford.edu/slides/2016/winter1516_lecture6.pdf

[74] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning (ICML), 2015*, pp. 448–456. https://arxiv.org/abs/1502.03167

[75] *MIT Technology Review*, "Exclusive: Ilya Sutskever, OpenAI's chief scientist, on his hopes and fears for the future of AI," Oct. 26, 2023. https://www.technologyreview.com/2023/10/26/1082398/exclusive-ilya-sutskever-openais-chief-scientist-on-his-hopes-and-fears-for-the-future-of-ai/

[76] D.C. Ciresan, U. Meier, J. Masci, L.M. Gambardella, and J. Schmidhuber, "High-performance neural networks for visual object classification," arXiv preprint, arXiv:1102.0183, 2011

[77] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature* 521 (2015): 436–444. https://doi.org/10.1038/nature14539

[78] Same as [63]

[79] Same as [62] (2:22)

[80] Same as [74]

[81] L. Lu, Y. Shin, Y. Su, and G. Em Karniadakis, "Dying ReLU and initialization: Theory and numerical examples'," *Communications in Computational Physics* 28, no. 5 (Jun. 2020): 1671–1706. https://arxiv.org/abs/1903.06733

[82] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Computer Vision–ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars (eds.), Cham, Switzerland: Springer, 2014, pp. 818–833, doi: 10.1007/978-3-319-10590-1_53. https://arxiv.org/abs/1311.2901

[83] *The Economist*, "From not working to neural networking," Jun. 23, 2016. https://www.economist.com/special-report/2016/06/23/from-not-working-to-neural-networking

[84] Kaiming He, Zhang Xiangyu, Ren Shaoqing, and Sun Jian (2016). "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778

[85] Jacob Aron, "Forget the Turing test—there are better ways of judging AI," *New Scientist*, Sep. 21, 2015. https://www.newscientist.com/article/dn28206-forget-the-turing-test-there-are-better-ways-of-judging-ai/

[86] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proceedings of the IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 580–587, doi: 10.1109/CVPR.2014.81. https://www.cv-foundation.org/openaccess/content_cvpr_2014/papers/Girshick_Rich_Feature_Hierarchies_2014_CVPR_paper.pdf

[87] Communications of the ACM, "Technical perspective: What led computer vision to deep learning?" https://cacm.acm.org/research/technical-perspective-what-led-computer-vision-to-deep-learning/

[88] YouTube, "Jitendra Malik: Computer Vision | Lex Fridman Podcast #110" (01:26:57). https://youtu.be/LRYkH-fAVGE?si=B0Bg0BDOuXAJZ_jO&t=5214

[89] M.D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," CoRR, abs/1311.2901, 2013.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[90] *MIT Technology Review*, "The 10 breakthrough technologies of 2013," Apr. 23, 2013. https://www.technologyreview.com/2013/04/23/178729/the-10-breakthrough-technologies-of-2013/

[91] *The New York Times*, "Brainlike computers learning from experience," Dec. 29, 2013. https://www.nytimes.com/2013/12/29/science/brainlike-computers-learning-from-experience.html

[92] *The Guardian*, "AI: Eugene Goostman artificial intelligence," Jun. 11, 2014. https://www.theguardian.com/commentisfree/2014/jun/11/ai-eugene-goostman-artificial-intelligence

[93] I. Sutskever, J. Martens, G. Dahl, and G. E. Hinton, "On the importance of initialization and momentum in deep learning," in *Proceedings of the 30th International Conference on Machine Learning (ICML)*, 2013, pp. 1139–1147. http://proceedings.mlr.press/v28/sutskever13.html

Chapter 3

[1] Google, "Add a residual connection," Google Search results, Jun. 14, 2025. https://www.google.com/search?q=add+a+residual+connection

[2] A. Kristiadi, "Residual networks," Oct. 13, 2016. https://agustinus.kristia.de/blog/residual-net/

[3] A. Linn, "Microsoft researchers win ImageNet computer vision challenge," Microsoft: The AI Blog, Dec. 10, 2015. https://blogs.microsoft.com/ai/microsoft-researchers-win-imagenet-computer-vision-challenge/

[4] H. Sahota, "ResNet: How one paper changed deep learning forever," Comet Blog, Oct. 6, 2023. https://www.comet.com/site/blog/resnet-how-one-paper-changed-deep-learning-forever

[5] DeepLabCut, "What neural network should I use? (trade-offs, speed performance, and considerations)," DeepLabCut Wiki (GitHub), May 26, 2021. https://github.com/DeepLabCut/DeepLabCut/wiki/What-neural-network-should-I-use%3F-(Trade-offs,-speed-performance,-and-considerations)

[6] Reddit, r/MachineLearning, "D: Are ResNets as good as it gets?" discussion thread. https://www.reddit.com/r/MachineLearning/comments/nbgb6a/d_are_resnets_as_good_as_it_gets/

[7] ptidor.com, "Large CNN." https://ptidor.com/large_cnn.pdf

[8] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," arXiv:1611.05431, Nov. 2016. https://arxiv.org/pdf/1611.05431

[9] C. Szegedy et al., "Going deeper with convolutions," arXiv:1409.4842, Sep. 2014. https://arxiv.org/abs/1409.4842

[10] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, Inception-ResNet and the impact of residual connections on learning," arXiv:1602.07261, Feb. 2016. https://arxiv.org/abs/1602.07261

[11] Same as [8]

[12] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-excitation networks," arXiv:1709.01507, Sep. 2017. https://arxiv.org/abs/1709.01507

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[13] S. Y. Wang and T. Guo, "Transfer learning-based algorithms for the detection of fatigue crack initiation sites: A comparative study," *Frontiers in Materials* 8, (Aug. 2021): art. 756798, doi: 10.3389/fmats.2021.756798. https://www.frontiersin.org/journals/materials/articles/10.3389/fmats.2021.756798/full

[14] W. Xu et al., "ResNet and its application to medical image processing: Research progress and challenges," *Computer Methods and Programs in Biomedicine* 240 (Oct. 2023): art. 107660, doi: 10.1016/j.cmpb.2023.107660. https://pubmed.ncbi.nlm.nih.gov/37320940

[15] N. Barla, "Self-driving cars with convolutional neural networks (CNN)," Neptune.ai Blog, Apr. 22, 2024. https://neptune.ai/blog/self-driving-cars-with-convolutional-neural-networks-cnn

[16] J. Cohen, "How Tesla Autopilot works," ThinkAutonomous, Jul. 1, 2021. https://www.thinkautonomous.ai/blog/how-tesla-autopilot-works/

[17] J. Arun Pandian, R. Thirunavukarasu, and L. Thanga Mariappan, "Enhancing lane detection in autonomous vehicles with multi-armed bandit ensemble learning," *Scientific Reports* 15 (Jan. 2025): art. 3198, doi: 10.1038/s41598-025-86743-z. https://www.nature.com/articles/s41598-025-86743-z

[18] Same as [10]

[19] H. Zhang et al., "ResNeSt: Split-attention networks," arXiv:2004.08955, Apr. 2020. https://arxiv.org/abs/2004.08955

[20] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017. https://openaccess.thecvf.com/content_ICCV_2017/papers/He_Mask_R-CNN_ICCV_2017_paper.pdf

[21] A. Krizhevsky, "Learning multiple layers of features from tiny images," technical report, University of Toronto, 2009.

[22] A. Veit, M. Wilber, and S. Belongie, "Residual networks behave like ensembles of relatively shallow networks," arXiv:1605.06431, May 2016. https://arxiv.org/abs/1605.06431

[23] Y.-L. Boureau, J. Ponce, and Y. LeCun, "A theoretical analysis of feature pooling in visual recognition," in *Proceedings of the 27th International Conference on Machine Learning (ICML),* 2010, pp. 111–118

[24] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," arXiv:1603.07285, Mar. 2016. https://arxiv.org/pdf/1603.07285

[25] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," arXiv:1511.07122, Nov. 2015. https://arxiv.org/pdf/1511.07122

[26] Jonathan Long, Evan Shelhamer, and Trevor Darrell, "Fully convolutional networks for semantic segmentation," in CVPR, 2015. https://arxiv.org/abs/1411.4038

[27] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan I. Yuille, "Semantic image segmentation with deep convolutional nets and fully connected CRFs," In ICLR, 2015a. https://arxiv.org/abs/1412.7062

[28] Same as [3]

[29] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," arXiv:1409.0575, Sep. 2014. https://arxiv.org/pdf/1409.0575

[30] M. Mitchell, *Artificial Intelligence: A Guide for Thinking Humans*. New York: Farrar, Straus and Giroux, 2019

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[31] Hang Zhang, Chongruo Wu, Zhongyue Zhang, Yi Zhu, Haibin Lin, Zhi Zhang, Yue Sun, Tong He, Jonas Mueller, R. Manmatha, Mu Li, and Alexander Smola, "ResNeSt: Split-attention networks," 2020. https://arxiv.org/abs/2004.08955. *See also* https://gluon-cv.mxnet.io/model_zoo/classification.html#id250 for code and visualization.

[32] The top-1 accuracy is reported more frequently in recent years.

[33] R. McMillan, "This guy beat Google's super-smart AI—but it wasn't easy," *WIRED*, Jan. 21, 2015. https://www.wired.com/2015/01/karpathy/

[34] Same as [31]

[35] R. C. Johnson, "Microsoft, Google beat humans at image recognition," *EE Times*, Feb. 18, 2015. https://www.eetimes.com/microsoft-google-beat-humans-at-image-recognition

[36] M. Wilson, "Microsoft Software Recognizes Images Better Than Humans Do", Fast Company, Feb. 11, 2015. https://www.fastcompany.com/3042239/microsoft-software-recognizes-images-better-than-humans-do

[37] A. Hern, "Computers now better than humans at recognising and sorting images," *The Guardian*, May 13, 2015. https://www.theguardian.com/global/2015/may/13/baidu-minwa-supercomputer -better-than-humans-recognising-images

[38] Entrepreneur, "Can artificial intelligence identify pictures better than humans?" Apr. 1, 2017. https://www.entrepreneur.com/science-technology/can-artificial-intelligence-identify-pictures-better-than/283990

[39] Same as [7]

[40] T. Simonite, "When it comes to gorillas, Google Photos remains blind," *WIRED*, Jan. 11, 2018. https://www.wired.com/story/when-it-comes-to-gorillas-google-photos-remains-blind/

[41] J. Shlens, "Train your own image classifier with Inception in TensorFlow," Google Research Blog, Mar. 9, 2016. https://research.google/blog/train-your-own-image-classifier-with-inception-in-tensorflow/

[42] Stanford University, "Andrej Karpathy," https://cs.stanford.edu/people/karpathy/

[43] Stanford University, "CS231n: Convolutional Neural Networks for Visual Recognition (spring 2016)." https://cs231n.stanford.edu/2016/

[44] Stanford University, "CS231n (spring 2016) schedule," https://cs231n.stanford.edu/schedule.html

[45] Stanford University, "CS231n Lecture 6 slides," 2025. https://cs231n.stanford.edu/slides/2025/lecture_6.pdf

[46] *The Stanford Daily*, "The rise of AI in Stanford's CS curricula," Feb. 4, 2021. https://stanforddaily.com/2021/02/04/the-rise-of-ai-in-stanfords-cs-curricula/

[47] R. S. Sutton, "The bitter lesson," Mar. 13, 2019. http://www.incompleteideas.net/IncIdeas/BitterLesson.html

[48] Reuters, "Ilya Sutskever: How AI will change his new startup Safe Superintelligence," Sep. 5, 2024. https://www.reuters.com/technology/artificial-intelligence/ilya-sutskever-how-ai-will-change-his-new-startup-safe-superintelligence-2024-09-05/

[49] HLF Foundation Newsroom, "Data is unreasonably effective and there is plenty of it to explore," HLF Foundation Blog. https://www.newsroom.hlf-foundation.org/blog/article/data-is-unreasonably-effective-and-there-is-plenty-of-it-to-explore/

[50] Same as [3].

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[51] H. Pearson, H. Ledford, M. Hutson, and R. Van Noorden, "Exclusive: The most-cited papers of the twenty-first century," *Nature* 640 (Apr. 15, 2025): 588–592, doi: 10.1038/d41586-025-01125-9. https://www.nature.com/articles/d41586-025-01125-9

Chapter 4

[1] A. Karpathy, "The unreasonable effectiveness of recurrent neural networks," Andrej Karpathy Blog, May 21, 2015. https://karpathy.github.io/2015/05/21/rnn-effectiveness/

[2] P. Graham, "Paul Graham." https://www.paulgraham.com

[3] OpenAI, "Shared ChatGPT conversation," ChatGPT. https://chatgpt.com/share/67fe5d24-1cec-8002-83e2-969c3bb400a5

[4] C. Olah, "Understanding LSTM networks," Colah's Blog, Aug. 27, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs/

[5] Hacker News, "Stories from May 21, 2015," May 21, 2015. https://news.ycombinator.com/front?day=2015-05-21

[6] Hacker News, "The unreasonable effectiveness of recurrent neural networks," discussion thread, May 21, 2015. https://news.ycombinator.com/item?id=9584325

[7] Reddit, "The unreasonable effectiveness of recurrent neural networks," r/programming, discussion thread, 2015. https://www.reddit.com/r/programming/comments/36su8d/the_unreasonable_effectiveness_of_recurrent/

[8] C. S. Voss, "Modeling molecules with recurrent neural networks," csvoss.com, Oct. 8, 2015. https://csvoss.com/modeling-molecules-with-rnns

[9] A. Karpathy, "char-rnn," GitHub repository. https://github.com/karpathy/char-rnn

[10] T. Brewe (nylki), "char-rnn cooking recipes," GitHub Gist, May 29, 2015. https://gist.github.com/nylki/1efbaa36635956d35bcc

[11] Reddit, "I used Andrej Karpathy's char-rnn to compose Irish folk music (AI)," r/MachineLearning, discussion thread, 2015. https://www.reddit.com/r/MachineLearning/comments/36zi75/i_used_andrej_karpathys_charrnn_to_compose_irish/

[12] R. McDermott, "trumpgen," GitHub repository. https://github.com/ryanmcdermott/trumpgen

[13] C. Johnson (@MrChrisJohnson), post on X, Jun. 2, 2015. https://x.com/MrChrisJohnson/status/605783907468865536

[14] RNN Bible (@RNN_Bible), X profile page. https://x.com/RNN_Bible

[15] Samim, "Obama RNN: Machine-generated political speeches," Medium. https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0

[16] Same as [4]

[17] Same as [6].

[18] Y. Goldberg, "The unreasonable effectiveness of character-level language models (and why RNNs are still cool)," Jupyter Notebook, May 22, 2015. https://nbviewer.org/gist/yoavg/d76121dfde2618422139

[19] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," arXiv:1301.3781, Jan. 2013. [https://arxiv.org/abs/1301.3781

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[20] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in *Proceedings of Interspeech*, 2010. https://www.isca-archive.org/interspeech_2010/mikolov10_interspeech.html

[21] Same as [18].

[22] Same as [4]

[23] Distill, "Distill." https://distill.pub

[24] Y. Bengio, R. Ducharme, and P. Vincent, "A neural probabilistic language model," *Journal of Machine Learning Research* 3 (2003): 1137–1155

[25] Z. Cui, W. Chen, and Y. Chen, "Multi-scale convolutional neural networks for time series classification," arXiv:1603.06995, Mar. 2016. https://arxiv.org/pdf/1603.06995

[26] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," in *IEEE Transactions on Neural Networks* 5, no. 2 (Mar. 1994): 157–166, doi: 10.1109/72.279181

[27] Sepp Hochreiter et al. "Gradient flow in recurrent nets: the difficulty of learning long-term dependencies" in John F. Kolen and Stefan C. Kremer (eds.), *A Field Guide to Dynamical Recurrent Networks,* New York: John Wiley & Sons, 2001

[28] Sepp Hochreiter and Jürgen Schmidhuber, "Long short-term memory," *Neural Computation* 9, no. 8 (1997): 1735–1780, doi:10.1162/neco.1997.9.8.1735. PMID 9377276. S2CID 1915014. https://www.bioinf.jku.at/publications/older/2604.pdf

[29] C. Olah, "Understanding LSTM networks," Colah's Blog, Aug. 27, 2015. https://colah.github.io/posts/2015-08-Understanding-LSTMs

[30] Same as [28]

[31] F.A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM." *Neural Computation* 12, no. 10 (2000): 2451–2471, doi: 10.1162/089976600300015015. PMID: 11032042. https://dl.acm.org/doi/10.1162/089976600300015015

[32] P. Plavén-Sigray G.J. Matheson, B.C. Schiffler, and W.H. Thompson, "The readability of scientific texts is decreasing over time," *Elife 6 (*2017): e27725, doi: 10.7554/eLife.27725. PMID: 28873054; PMCID: PMC5584989. https://pmc.ncbi.nlm.nih.gov/articles/PMC5584989

[33] Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals, "Recurrent neural network regularization," 2014. https://arxiv.org/abs/1409.2329

[34] R. Pascanu, T. Mikolov, and Y. Bengio, "On the difficulty of training recurrent neural networks," arXiv:1211.5063, Nov. 2012. https://arxiv.org/abs/1211.5063

[35] R. Pascanu, Ç. Gülçehre, K. Cho, and Y. Bengio, "How to construct deep recurrent neural networks," arXiv:1312.6026, Dec. 2013. https://arxiv.org/pdf/1312.6026

[36] Formally introduced by Srivastava, Hinton, Krizhevsky, Sutskever, and Salakhutdinov in 2013 (after AlexNet). N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research* 15 (2014): 1929–1958. https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf

[37] J. Bayer, C. Osendorfer, D. Korhammer, N. Chen, S. Urban, and P. van der Smagt, "On fast dropout and its applicability to recurrent networks," arXiv:1311.0701, Nov. 2013. https://arxiv.org/abs/1311.0701

[38] N. Srivastava, "Improving neural networks with dropout," Ph.D. dissertation, University of Toronto, 2013

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[39] A. Graves, "Generating sequences with recurrent neural networks," arXiv:1308.0850, Aug. 2013. https://arxiv.org/abs/1308.0850

[40] Microsoft Research, "rnn_ctxt_TR.sav_.pdf," Jul. 2012. https://www.microsoft.com/en-us/research/wp-content/uploads/2012/07/rnn_ctxt_TR.sav_.pdf

[41] Same as [35]

[42] M. Sundermeyer, R. Schlüter, and H. Ney, "LSTM neural networks for language modeling," 2012. https://www-i6.informatik.rwth-aachen.de/publications/download/820/Sundermeyer-2012.pdf

[43] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," arXiv:1409.3215, Sep. 2014. https://arxiv.org/pdf/1409.3215

[44] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," arXiv:1409.0473, Sep. 2014. https://arxiv.org/abs/1409.0473

[45] Marcus Liwicki, Alex Graves, Horst Bunke, and Jürgen Schmidhuber, "A novel approach to on-line handwriting recognition based on bidirectional long short-term memory networks," In *Proceedings of the 9th International Conference on Document Analysis and Recognition, ICDAR,* 2007

[46] A. Hannun et al., "Deep Speech: Scaling up end-to-end speech recognition," arXiv:1412.5567, Dec. 2014. https://arxiv.org/pdf/1412.5567

[47] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, Feb. 2015. https://arxiv.org/abs/1502.03167

[48] M. Marge, S. Banerjee, and A. I. Rudnicky, "Using the Amazon Mechanical Turk to transcribe and annotate meeting speech for extractive summarization," in *Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon's Mechanical Turk*, Los Angeles, CA, June 2010, pp. 99–107.

[49] A. Gulati et al., "Conformer: Convolution-augmented transformer for speech recognition," arXiv:2005.08100, May 2020. https://arxiv.org/abs/2005.08100

[50] X. Song and D. Zhou, "A Fast WordPiece Tokenization System," Google Research Blog, Dec. 10, 2021. [Online]. Available: https://research.google/blog/a-fast-word-piece-tokenization-system/

[51] Hugging Face, "Byte-Pair Encoding tokenization," Hugging Face LLM Course. [Online]. Available: https://huggingface.co/learn/llm-course/en/chapter6/5

[52] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," arXiv:1808.06226, Aug. 2018. [Online]. Available: https://arxiv.org/abs/1808.06226

[53] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, "Deep convolutional neural networks (AlexNet)," in Dive into Deep Learning, version 1.0.3. http://d2l.ai/chapter_convolutional-modern/alexnet.html

[54] *The New Yorker*, "Is 'deep learning' a revolution in artificial intelligence?" https://www.newyorker.com/news/news-desk/is-deep-learning-a-revolution-in-artificial-intelligence

[55] *MIT Technology Review*, "Conversational interfaces." https://www.technologyreview.com/technology/conversational-interfaces/

[56] *International Conference on Machine Learning (ICML)*, "Paper review (Paper ID: 76)," 2016. https://icml.cc/2016/reviews/76.txt

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[57] D. Patel, "Ilya Sutskever (OpenAI chief scientist)—Why next-token prediction could surpass human intelligence," Dwarkesh Podcast. https://www.dwarkesh.com/p/ilya-sutskever

[58] YouTube, "Interview with Dr. Ilya Sutskever, co-founder of OPEN AI at the Open University studios, English," video. https://youtu.be/H1YoNlz2LxA?si=03KgI1XB_LxVb0V7&t=595

[59] Same as [58]

[60] L. Fridman, "#309–John Carmack: Doom, Quake, VR, AGI, programming, video games, and rockets," Lex Fridman Podcast, Aug. 4, 2022. https://lexfridman.com/john-carmack/

[61] Same as [46]

[62] REWORK, "Baidu's Adam Coates on Deep Speech 2, new open source AI tool & more," Jan. 14, 2016. https://blog.re-work.co/deep-learning-baidu-adam-coates

[63] T. Petzinger Jr., "Mathematician perceives mind as a company-intranet model," *The Wall Street Journal*, May 22, 1998. https://www.wsj.com/articles/SB895791428926727000.

[64] B. Goertzel, "Waking up from the economy of dreams," Apr. 9, 2001. https://goertzel.org/benzine/WakingUpFromTheEconomyOfDreams.htm.

[65] M. Cave, "One dumb baby with a brilliant future," *Australian Financial Review*, May 11, 2000. https://www.afr.com/politics/one-dumb-baby-with-a-brilliant-future-20000511-k9g4u

[66] Goertzel's new company, TrueAGI, will focus on enterprises to offer "mind as a service."

[67] S. Shead, "Alphabet's DeepMind losses soared to $570 million in 2018," *Forbes*, Aug. 7, 2019. https://www.forbes.com/sites/samshead/2019/08/07/deepmind-losses-soared-to-570-million-in -2018/; A. Thomson, "Google waives $1.5 billion DeepMind loan as AI costs mount," Bloomberg, Dec. 17, 2020. https://www.bloomberg.com/news/articles/2020-12-17/deepmind-says-2019-revenue-jumped-158-on-ai-research-work

Chapter 5

[1] Jakob Uszkoreit proposed replacing RNNs with self-attention and started the effort to evaluate the idea. J. Uszkoreit, "Transformer: A novel neural network architecture for language understanding," Google Research Blog, Aug. 31, 2017. https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," arXiv preprint, arXiv:1706.03762, 2017

[3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," arXiv:1409.0473, Sep. 2014. https://arxiv.org/pdf/1409.0473

[4] A. Graves, G. Wayne, and I. Danihelka, "Neural Turing machines," arXiv:1410.5401, Oct. 2014. https://arxiv.org/abs/1410.5401

[5] M.-T. Luong, H. Pham, and C. D. Manning, "Effective Approaches to Attention-based Neural Machine Translation," in Proc. 2015 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2015, pp. 1412–1421, doi: 10.18653/v1/D15-1166. [Online]. Available: https://arxiv.org/pdf/1508.04025

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[6] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proceedings of Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3104–3112. https://arxiv.org/abs/1409.3215

[7] This contrasts with models like GPT-4, which are pure decoder-only architectures designed for next-token prediction without a separate encoder. These models generate text sequentially, conditioned solely on previously generated tokens, without explicitly encoding a separate input sequence. OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, Mar. 2023. [Online]. Available: https://arxiv.org/abs/2303.08774

[8] Typically LSTMs

[9] Today, transformers use learned positional embeddings, where positional information is encoded through differentiable parameters that are randomly initialized and updated during training via backpropagation.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," arXiv:1810.04805, Oct. 2018. https://arxiv.org/pdf/1810.04805

[11] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research* 21, no. 140 (2020): 1–67. https://arxiv.org/pdf/1910.10683

[12] H. Touvron et al., "LLaMA: Open and efficient foundation language models," arXiv:2302.13971, Feb. 2023. https://arxiv.org/pdf/2302.13971

[13] Vaswani et al. (2017) report both 41.0 and 41.8 BLEU for WMT14 English to French; 41.0 refers to the primary single-model result, while the higher figure reflects their best reported performance using checkpoint averaging (a common technique that modestly boosts scores).

[14] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," arXiv:1701.06538, Jan. 2017. https://arxiv.org/abs/1701.06538

[15] Z. Dai et al., "Transformer-XL: Attentive language models beyond a fixed-length context," arXiv:1901.02860, Jan. 2019. https://arxiv.org/pdf/1901.02860

[16] M. Planck, *Scientific Autobiography and Other Papers,* New York: Philosophical Library, 1950.

[17] That challenge catalyzed a family of *efficient-attention.* (a) A. Roy, M. Saffar, A. Vaswani, and D. Grangier, "Efficient content-based sparse attention with routing Transformers," arXiv:2003.05997, Mar. 2020. https://arxiv.org/abs/2003.05997; (b) S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," arXiv:2006.04768, Jun. 2020. https://arxiv .org/abs/2006.04768

[18] S. Ruder, "NLP's ImageNet moment has arrived," *The Gradient*, Jul. 8, 2018. https://thegradient.pub/nlp-imagenet

[19] Z. Shen, Y. Zhou, C. Huang, H. Ko, and X. Zhang, "DiSAN: Directional self-attention network for RNN/CNN-free language understanding," arXiv:1709.04696, Sep. 2017. https://arxiv.org/pdf/1709.04696

[20] M. Chen et al., "The best of both worlds: Combining recent advances in neural machine translation," in *Proceedings of the 56th Annual Meeting of the Association for Computer Linguistics (ACL)*, 2018, pp. 76–86, doi: 10.18653/v1/P18-1008. https://aclanthology.org/P18-1008/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[21] J. Hao, X. Wang, B. Yang, L. Wang, J. Zhang, and Z. Tu, "Modeling recurrence for transformer," in *Proceedings of the 2019 Conference of the North American Chapter Association for Computer. Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 1198–1207, doi: 10.18653/v1/N19-1122. https://aclanthology.org/N19-1122/

[22] K. Tran, A. Bisazza, and C. Monz, "The importance of being recurrent for modeling hierarchical structure," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 4731–4736, doi: 10.18653/v1/D18-1503. https://aclanthology.org/D18-1503.pdf

[23] S. Abnar, "On the merits of recurrent inductive bias," blog post, May 16, 2020. https://samiraabnar.github.io/articles/2020-05/recurrence

[24] B. Yang, L. Wang, D. F. Wong, L. S. Chao, and Z. Tu, "Assessing the ability of self-attention networks to learn word order," arXiv:1906.00592, Jun. 2019. https://arxiv.org/abs/1906.00592

[25] Google Research, "Moving beyond translation with the universal transformer," Google Research Blog. https://research.google/blog/moving-beyond-translation-with-the-universal-transformer

[26] YouTube, "No priors Ep. 39 | With OpenAI co-founder & chief scientist Ilya Sutskever." https://www.youtube.com/watch?v=Ft0gTO2K85A

[27] TensorFlow, "tensor2tensor," GitHub repository. https://github.com/tensorflow/tensor2tensor

[28] Stanford University, "CS224U Podcast: Rush," course webpage. https://web.stanford.edu/class/cs224u/podcast/rush

[29] Rush, "The annotated Transformer," Harvard NLP, Apr. 3, 2018. https://nlp.seas.harvard.edu/2018/04/03/attention.html

[30] J. Alammar, "The illustrated transformer," blog post. https://jalammar.github.io/illustrated-trans former/

[31] A. Arora, "The Annotated GPT-2," blog post, Feb. 18, 2020. [Online]. Available: https://amaarora.github.io/posts/2020-02-18-annotatedGPT2.html

[32] Hugging Face, "The annotated diffusion model," Hugging Face Blog. https://huggingface.co/blog/annotated-diffusion

[33] Harvard NLP, "annotated-transformer," GitHub repository. https://github.com/harvardnlp/annotated-transformer

[34] Reddit, r/MachineLearning, "[P] The annotated Transformer line-by-line PyTorch," post. https://www.reddit.com/r/MachineLearning/comments/89uy75/p_the_annotated_transformer_linebyline_pytorch/

[35] Princeton University, "COS 484: Natural Language Processing," course webpage. https://princeton-nlp.github.io/cos484

[36] Same as [33]

[37] Same as [6]

[38] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," arXiv:1310.4546, Oct. 2013. https://arxiv.org/abs/1310.4546

[39] Pathmind, "Thought vectors," Pathmind Wiki. https://wiki.pathmind.com/thought-vectors

[40] Same as [39]

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[41] Reddit, r/MachineLearning, "AMA: We are the Google Brain team." https://www.reddit.com/r/MachineLearning/comments/4w6tsv/ama_we_are_the_google_brain_team_wed_love_to/

[42] *The Guardian*, "Google a step closer to developing machines with human-like intelligence," May 21, 2015. https://www.theguardian.com/science/2015/may/21/google-a-step-closer-to-developing-machines-with-human-like-intelligence

[43] Same as [40]

[44] H. Sheikh, C. Prins, and E. Schrijvers, "AI is leaving the lab and entering society," in *Mission AI,* Springer Nature, 2023, pp. 43–84, doi: 10.1007/978-3-031-21448-6_3. https://link.springer.com/chapter/10.1007/978-3-031-21448-6_3

[45] Same as [47].

[46] Same as [3]

[47] Same as [40].

[48] Diederik Kingma and Jimmy Ba, *Losing to Adam: A Method for Stochastic Optimization.* Adaptive Moment Estimation, or Adam, has widely become the default optimizer over methods like Stochastic Gradient Descent across a range of deep learning tasks. *International Conference on Learning Representations (ICLR),* "Announcing the Test-of-Time award winners from ICLR 2015," ICLR Blog, Apr. 14, 2025. https://blog.iclr.cc/2025/04/14/announcing-the-test-of-time-award-winners-from-iclr-2015

[49] Microsoft Research, "NIPS: Oral session 4–Ilya Sutskever," video (9:23). https://t.co/sJa4QJJVu2

[50] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," arXiv:1506.03134, Jun. 2015. https://arxiv.org/pdf/1506.03134

[51] Same as [6]

[52] A. See, P.J. Liu, and C.D. Manning, "Get to the point: Summarization with pointer-generator networks," in R. Barzilay and M.-Y. Kan (eds.),P*roceedings of the 55th Annual Meeting of the Association for Computational Linguistics, Vol. 1: Long Papers. Association for Computational Linguistics*, pp. 1073–1083. https://doi.org/10.18653/v1/P17-1099

[53] Same as [4]

[54] They drew inspiration from classical Turing machines, theoretical models known for their universal computational abilities. *Stanford Encyclopedia of Philosophy*, "Turing machine," https://plato.stanford.edu/entries/turing-machine/

[55] Happy Scribe, "Lex Fridman podcast #94—Ilya Sutskever: Deep learning," transcript. https://www.happyscribe.com/public/lex-fridman-podcast-artificial-intelligence-ai/94-ilya-sutskever-deep-learning

[56] Same as [59].

[57] W. Zaremba and I. Sutskever, "Reinforcement learning neural Turing machines–revised," arXiv:1505.00521, May 2015. https://arxiv.org/abs/1505.00521

[58] D. Britz, "Attention and memory in deep learning and NLP," WildML Blog. https://dennybritz.com/posts/wildml/attention-and-memory-in-deep-learning-and-nlp

[59] O. Vinyals, S. Bengio, and M. Kudlur, "Order matters: Sequence to sequence for sets," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. https://arxiv.org/abs/1511.06391

[60] Same as [6]

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[61] *Sequence to Sequence Learning with Neural Networks* won the NeurIPS 2024 Test of Time Award. NeurIPS, "Announcing the NeurIPS 2024 Test of Time paper awards," NeurIPS Blog, Nov. 27, 2024. https://blog.neurips.cc/2024/11/27/announcing-the-neurips-2024-test-of-time-paper-awards/

[62] https://aclanthology.org/D15-1166.pdf

[63] J. Jordan, "Attention," blog post. https://www.jeremyjordan.me/attention

[64] D. Britz, A. Goldie, M.-T. Luong, and Q. Le, "Massive exploration of neural machine translation architectures," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, doi: 10.18653/v1/D17-1151. https://aclanthology.org/D17-1151.pdf

[65] A. Eriguchi, K. Hashimoto, and Y. Tsuruoka, "Incorporating source-side phrase structures into neural machine translation," *Computational Linguistics* 45, no. 2 (2019): 267–292, doi: 10.1162/coli_a_00348. https://direct.mit.edu/coli/article/45/2/267/1633/Incorporating-Source-Side-Phrase-Structures-into

[66] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623, doi: 10.1145/3442188.3445922. https://dl.acm .org/doi/pdf/10.1145/3442188.3445922

[67] I. Sutskever (@ilyasut), post on X. https://x.com/ilyasut/status/1675007883824758785

[68] Google Search, results for query "attention is not all you need," Sep. 13, 2025. https://www.google.com/search?q=attention+is+not+all+you+need. A Google search for "attention is not all you need" yields 583,000 results.

[69] Related: K. Nishi, "awesome-all-you-need-papers," GitHub repository. https://github.com/Kento Nishi/awesome-all-you-need-papers

[70] A. Alyakin, "All You Need" is not all you need for a paper title: "On the origins of a scientific meme," arXiv:2512.19700, Dec. 2025. https://arxiv.org/abs/2512.19700

[71] Know Your Meme, "Attention is all you need." https://knowyourmeme.com/memes/attention-is-all-you-need

[72] K. Chang et al., "Data is all you need: Finetuning LLMs for chip design via an automated design-data augmentation framework," arXiv:2403.11202v2, 2024. https://arxiv.org/abs/2403.11202v2

[73] M. Harmel, A. Paras, A. Pasternak, N. Roy, and G. Linscott, "Scaling is all you need: Autonomous driving with JAX-accelerated reinforcement learning," arXiv:2312.15122v4, 2024. https://arxiv.org/abs/2312.15122v4

[74] Y. Lian, "Reinforcement learning is all you need," arXiv:2503.09512v1, Mar. 2025. https://arxiv.org/abs/2503.09512v1

[75] TechEngines, "Deep learning: AI money is all you need," blog post, May 2021. https://techengines.github.io/2021/05/24/deep-learning-ai-money-is-all-you-need

[76] S. Albanie, D. Campbell, and J. F. Henriques, "A 23 MW data centre is all you need," arXiv:2203.17265, Mar. 2022. https://ar5iv.labs.arxiv.org/html/2203.17265v1

[77] OpenAI, "Microsoft invests in and partners with OpenAI," OpenAI. https://openai.com/index/microsoft-invests-in-and-partners-with-openai/

[78] VentureBeat, "Sakana AI's CTO says he's absolutely sick of transformers, the tech that powers …," VentureBeat. https://venturebeat.com/ai/sakana-ais-cto-says-hes-absolutely-sick-of-transformers-the-tech-that-powers

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[79] O. Kuchaiev and B. Ginsburg, "Factorization tricks for LSTM networks," arXiv:1703.10722, Mar. 2017.

[80] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," arXiv preprint, arXiv:1701.06538, 2017

[81] N. Kalchbrenner, L. Espeholt, K. Simonyan, A. van den Oord, A. Graves, and K. Kavukcuoglu, "Neural machine translation in linear time," arXiv:1610.10099, Oct. 2016. https://arxiv.org/pdf/1610.10099

[82] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," May 2017. https://arxiv.org/pdf/1705.03122

[83] "Exclusive: The most-cited papers of the twenty-first century," *Nature,* Apr. 15, 2025. https://www.nature.com/articles/d41586-025-01125-9

[84] Makebot AI, "Survey: Half of U.S. adults now use AI large language models like ChatGPT," Makebot AI Blog. https://www.makebot.ai/blog-en/survey-half-of-u-s-adults-now-use-ai-large-language-models-like-chatgpt

[85] I. Sutskever (@ilyasut), post on X. https://x.com/ilyasut/status/1597655903809208320?lang=hr

Chapter 6

[1] D. Meyer, "AI training costs: How much is too much?" *Fortune,* Apr. 4, 2024. https://fortune.com/2024/04/04/ai-training-costs-how-much-is-too-much-openai-gpt-anthropic-microsoft

[2] Wei et al., "Emergent abilities of large language models," arXiv:2206.07682, 2022. https://arxiv.org/abs/2206.07682

[3] J. Wei and Y. Tay, "Characterizing emergent phenomena in large language models," Google Research Blog, Nov. 10, 2022. https://research.google/blog/characterizing-emergent-phenomena-in-large-language-models

[4] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," arXiv:2201.11903, 2022. https://arxiv.org/abs/2201.11903

[5] T.B. Brown et al., "Language models are few-shot learners," arXiv:2005.14165, 2020. https://arxiv.org/abs/2005.14165

[6] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv:2303.12712, 2023. https://arxiv.org/abs/2303.12712

[7] J. Kaplan et al., "Scaling laws for neural language models," arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361

[8] "AI winter," Wikipedia. https://en.wikipedia.org/wiki/AI_winter

[9] J. Hestness et al., "Deep learning scaling is predictable, empirically," arXiv:1712.00409, 2017. https://arxiv.org/abs/1712.00409

[10] P. J. Liu et al., "Generating Wikipedia by summarizing long sequences," in *Proceedings of ICLR*, 2018. https://arxiv.org/pdf/1801.10198

[11] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, technical report, 2018. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[12] To estimate total training compute, the authors included the cost of the backward pass and multiply-accumulate operations (for example, matrix multiplications for projections, attention scores, and attention weights), same as in [7].

[13] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," arXiv:1804.04235, 2018. https://arxiv.org/pdf/1804.04235

[14] S. McCandlish, J. Kaplan, D. Amodei, and OpenAI Dota Team, "An empirical model of large-batch training," arXiv:1812.06162, 2018. https://arxiv.org/abs/1812.06162.

[15] Y. Yao, L. Rosasco, and A. Caponnetto, "On early stopping in gradient descent learning," Constructive Approximation, 2007. https://yao-lab.github.io/publications/YaoCapRos07_EarlyStop.pdf

[16] The authors correctly speculated that scaling laws would apply to other generative tasks, such as image, audio, or video modeling, though the exact numbers would ultimately depend on the data and representation. J. Kaplan et al., "Scaling laws for neural language models," arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361

[17] G. Branwen, "The scaling hypothesis," Gwern.net, n.d. https://gwern.net/scaling-hypothesis

[18] Same as [17]

[19] The GPT-3 paper cited the Kaplan scaling law when outlining their training strategy. T. B. Brown et al., "Language models are few-shot learners," arXiv:2005.14165, 2020. https://arxiv.org/abs/2005.14165

[20] The Decoder, "GPT-4 architecture, datasets, costs and more leaked," Jul. 11, 2023. https://the-decoder.com/gpt-4-architecture-datasets-costs-and-more-leaked

[21] Same as [6]

[22] R. T. McCoy, S. Yao, D. Friedman, M. Hardy, and T. L. Griffiths, "Embers of autoregression: Understanding large language models through the problem they are trained to solve," arXiv:2309.13638, 2023. https://arxiv.org/pdf/2309.13638

[23] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Transactions on Evolutionary Computation* 1, no. 1 (Apr. 1997): 67–82, doi: 10.1109/4235.585893.

[24] Richard S. Sutton, "The bitter lesson," 2019. http://www.incompleteideas.net/IncIdeas/BitterLesson.html

[25] J. Hoffmann et al., "Training compute-optimal large language models," *Advances in Neural Information Processing Systems* 35 (2022). https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf

[26] R. Hossam, "Chinchilla scaling laws for large language models (LLMs)," Medium, Sep. 7, 2023. https://medium.com/@raniahossam/chinchilla-scaling-laws-for-large-language-models-llms-40c434e4e1c1

[27] Same as [19]

[28] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023. https://arxiv.org/pdf/2307.09288

[29] Epoch AI, "AI models (notable AI models)," Epoch AI Data, n.d. https://epoch.ai/data/ai-models

[30] Espadrine, "Chinchilla's death," Espadrine's Blog, n.d. https://espadrine.github.io/blog/posts/chinchilla-s-death.html

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[31] P. Zhang et al., "TinyLlama: An open-source small language model," arXiv:2401.02385, 2024. https://arxiv.org/abs/2401.02385

[32] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024. https://arxiv.org/pdf/2407.21783

[33] Reuters, "What is DeepSeek and why is it disrupting the AI sector?" Jan. 27, 2025. https://www.reuters.com/technology/artificial-intelligence/what-is-deepseek-why-is-it-disrupting-ai-sector-2025-01-27

[34] DeepSeek-AI et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," arXiv:2501.12948, 2025. https://arxiv.org/pdf/2501.12948

[35] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," arXiv:2101.03961, 2021. https://arxiv.org/abs/2101.03961

[36] Model compression, distillation, sparse models like mixture-of-experts, and algorithmic optimizations can all cheat scaling costs. Y. Cheng, D. Wang, P. Zhou, and T. Zhang, "A survey of model compression and acceleration for deep neural networks," arXiv:1710.09282, 2017. https://arxiv.org/abs/1710.09282

[37] "Position: Enough of scaling LLMs! Let's focus on downscaling," arXiv:2505.00985, 2025. https://arxiv.org/abs/2505.00985

[38] For example, H. Pathak, "Navigating the frontier: A curated list of essential LLM recsys and papers (2018–2025)," Medium, n.d. https://medium.com/@harshnpathak/navigating-the-frontier-a-curated-list-of-essential-llm-recsys-and-papers-2018-2025-2fd7861d9ce1

[39] G. Marcus, "Deep learning is hitting a wall," Nautilus, n.d. https://nautil.us/deep-learning-is-hitting-a-wall-238440

[40] G. Marcus, "Post," X, Aug. 7, 2025. https://x.com/GaryMarcus/status/1953585116284170278

[41] What Marcus was really attacking was Gwern's bolder reframing of scale as a hypothesis about intelligence, though he aimed his critique at the scaling laws themselves. Same as [37]

[42] N. Lourie, M. Y. Hu, and K. Cho, "Scaling laws are unreliable for downstream tasks: A reality check," arXiv:2507.00885, 2025. https://arxiv.org/pdf/2507.00885

[43] S. Y. Gadre et al., "Language models scale reliably with over-training and on downstream tasks," arXiv:2403.08540, 2024. https://arxiv.org/abs/2403.08540

[44] E. Caballero, K. Gupta, I. Rish, and D. Krueger, "Broken neural scaling laws," arXiv:2210.14891, 2022. https://www.arxiv.org/abs/2210.14891

[45] GDB, "Post," X, Dec. 9, 2023. https://x.com/gdb/status/1733553161884127435

[46] A. Srivastava et al., "Beyond the imitation game: Quantifying and extrapolating the capabilities of language models," arXiv:2206.04615, 2022. https://arxiv.org/abs/2206.04615

[47] P. Liang et al., "Holistic evaluation of language models," arXiv:2211.09110, 2022. https://arxiv.org/abs/2211.09110

[48] D. Hendrycks et al., "Measuring massive multitask language understanding," arXiv:2009.03300, 2020. https://arxiv.org/pdf/2009.03300

[49] M. Kazemi et al., "BIG-Bench extra hard," arXiv:2502.19187, 2025. https://arxiv.org/abs/2502.19187

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[50] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021. https://arxiv.org/abs/2110.14168

[51] R. Schaeffer et al., "Why has predicting downstream capabilities of frontier AI models with scale remained elusive?" arXiv:2406.04391, 2024. https://arxiv.org/abs/2406.04391

[52] Stanford Law School, "GPT-4 passes the bar exam: What that means for artificial intelligence tools in the legal industry," Apr. 19, 2023. https://law.stanford.edu/2023/04/19/gpt-4-passes-the-bar-exam-what-that-means-for-artificial-intelligence-tools-in-the-legal-industry/

[53] *Ars Technica*, "What does 'PhD-level AI' mean? OpenAI's rumored 20,000-agent plan explained," Mar. 2025. https://arstechnica.com/ai/2025/03/what-does-phd-level-ai-mean-openais-rumored-20000-agent-plan-explained

[54] *The New York Times*, "AI jobs: Radiologists, Mayo Clinic," May 14, 2025. https://www.nytimes.com/2025/05/14/technology/ai-jobs-radiologists-mayo-clinic.html

[55] "Magic" is a reference to Karpathy's description of recurrent neural networks. A. Karpathy, "The unreasonable effectiveness of recurrent neural networks," May 21, 2015. https://karpathy.github.io/2015/05/21/rnn-effectiveness.

[56] N. Carlini, "My benchmark for large language models," 2024. https://nicholas.carlini.com/writing/2024/my-benchmark-for-large-language-models.html

[57] "Inverse Scaling Prize," GitHub repository, n.d. https://github.com/inverse-scaling/prize

[58] Jason Wei, Najoung Kim, Yi Tay, and Quoc V. Le, "Inverse scaling can become U-shaped," slides, 2023. https://docs.google.com/presentation/d/1JKpqsbkr5Fg-bj1iElPaC-ToTVpRmRLKZmN89krwl04/edit?pli=1&resourcekey=0-VPgp_Yc4krPPW3Mxv6UjgQ#slide=id.g29eef76a3f0_0_0

[59] Kiela et al. (2023), with minor processing by Our World in Data. "Test scores of AI systems on various capabilities relative to human performance" [dataset]. Kiela et al., "Dynabench: Rethinking benchmarking in NLP" [original data], Sep. 23, 2025. https://archive.ourworldindata.org/20250915-140209/grapher/test-scores-ai-capabilities-relative-human-performance.html (archived on September 15, 2025).

[60] GPT-4 technical report. https://arxiv.org/pdf/2303.08774

[61] E. Martinez, "Re-evaluating GPT-4's bar exam performance," *SSRN Electronic Journal*, 2023

[62] "LM contamination," HITZ Zentroa (Basque Center for Language Technology), n.d. https://hitz-zentroa.github.io/lm-contamination/

[63] "ARC-AGI," ARC Prize, n.d. https://arcprize.org/arc-agi

[64] Goodhart's Law; C.A.E. Goodhart, "Problems of monetary management: The U.K. experience," in *Papers in Monetary Economics*, vol. 1. Reserve Bank of Australia, 1975.

[65] J. Dean et al., "Large scale distributed deep networks," *Advances in Neural Information Processing Systems* 25 (2012). https://papers.nips.cc/paper_files/paper/2012/file/6aca97005c68f1206823815f66102863-Paper.pdf

[66] A. Krizhevsky, "One weird trick for parallelizing convolutional neural networks," arXiv:1404.5997, 2014. https://arxiv.org/abs/1404.5997

[67] S. Rajbhandari et al., "ZeRO: Memory optimizations toward training trillion parameter models," arXiv:1910.02054, 2019. https://arxiv.org/abs/1910.02054

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[68] AmoebaNet-B is a convolutional network architecture discovered using neural architecture search. E. Real et al., "Regularized evolution for image classifier architecture search," arXiv:1802.01548, 2018. https://arxiv.org/abs/1802.01548

[69] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" arXiv:1805.08974, 2018. https://arxiv.org/abs/1805.08974.

[70] A popular paraphrased version of Kierkegaard's journal entry: "Life can only be understood backwards, but it must be lived forwards." P. Jorgensen, "Life can only be understood backwards; but it must be lived forwards (Kierkegaard source note)," University of Iowa, n.d. https://homepage.divms.uiowa.edu/~jorgen/quotes/abbreviation.html

[71] "Visualizing 6D." https://main-horse.github.io/posts/visualizing-6d

[72] D. Arfeen et al., "PipeFill: Using GPUs during bubbles in pipeline-parallel LLM training," arXiv:2410.07192, 2024. https://www.pdl.cmu.edu/PDL-FTP/BigLearning/Pipefill-2410.07192v1.pdf

[73] Same as [72]

[74] M. Shoeybi et al., "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv:1909.08053, 2019. https://arxiv.org/abs/1909.08053

[75] Same as [69]

[76] J. Rosset, "Turing-NLG: A 17-billion parameter language model by Microsoft," Microsoft Research Blog, Feb. 11, 2020. https://www.microsoft.com/en-us/research/blog/turing-nlg-a-17-billion-parameter-language-model-by-microsoft/

[77] "Breadth-first pipeline parallelism," arXiv:2211.05953, 2022. https://arxiv.org/abs/2211.05953

[78] One effort that encapsulates these challenges is "MegaScale: Scaling large language model training to more than 10,000 GPUs," arXiv:2402.15627, 2024. https://arxiv.org/pdf/2402.15627

[79] Anson Ho, Tamay Besiroglu, Ege Erdil, David Owen, Robi Rahman, Zifan Carl Guo, David Atkinson, Neil Thompson, and Jaime Sevilla, "Algorithmic progress in language models," arXiv preprint, arXiv:2403.05812, 2024. https://arxiv.org/abs/2403.05812

[80] G. Branwen, "Clippy" (fiction), Gwern.net, n.d. https://gwern.net/fiction/clippy

[81] OpenAI, "Fine-tuning GPT-2 from human preferences," OpenAI, n.d. https://openai.com/index/fine-tuning-gpt-2/

[82] N. Bilton, "Artificial intelligence may be humanity's most ingenious invention—and its last," *Vanity Fair,* Sep. 13, 2023. https://www.vanityfair.com/news/2023/09/artificial-intelligence-industry-future?srsltid=AfmBOooDFNXFYIHUkwc4Muom9FbnCLSHYVa6y6XBwwFcXXO-nzpIMzQz

[83] In the "Fine-Tuning GPT-2 from Human Preferences" study, a flipped reward bug led the model to produce text that was as bad as possible while still fluent. This shows that large language models can preserve surface competence while pursuing misaligned goals. It also illustrates that neural networks don't fail noisily; they adapt, often in subtle and misleading ways, to whatever signal they're actually given. The same is true of [84].

[84] Mind Matters News, "Podcast guest claims he won't die, says we're creating God with AI," Jan. 2025. https://mindmatters.ai/2025/01/podcast-guest-claims-he-wont-die-says-were-creating-god-with-ai/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[85] Future of Life Institute, "Pause giant AI experiments: An open letter," Mar. 22, 2023. https://futureoflife.org/open-letter/pause-giant-ai-experiments/

[86] E. Bender et al., "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623

[87] Radiology Business, "NY Times revisits Nobel Prize winner's prediction that AI will render radiologists obsolete," n.d. https://radiologybusiness.com/topics/artificial-intelligence/ny-times-revisits-nobel-prize-winners-prediction-ai-will-render-radiologists-obsolete

[88] T. Chiang, "ChatGPT is a blurry JPEG of the web," *The New Yorker*, Feb. 9, 2023. https://www.newyorker.com/tech/annals-of-technology/chatgpt-is-a-blurry-jpeg-of-the-web

[89] A. Fawzi et al., "Discovering faster matrix multiplication algorithms with reinforcement learning," *Nature*, 2022. https://www.nature.com/articles/s41586-022-05172-4

Chapter 7

[1] Singh, Aaditya, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, et al. "OpenAI GPT-5 system card," arXiv [Cs.CL], 2025. http://arxiv.org/abs/2601.03267

[2] Reuters, "OpenAI and rivals seek new path to smarter AI as current methods hit limitations," Nov. 11, 2024. https://www.reuters.com/technology/artificial-intelligence/openai-rivals-seek-new-path-smarter-ai-current-methods-hit-limitations-2024-11-11/

[3] To keep gradients flowing, use the reparameterization trick. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," arXiv preprint arXiv:1312.6114, 2013. [Online]. Available: https://arxiv.org/abs/1312.6114.

[4] X. Chen, D. P. Kingma, T. Salimans, Y. Duan, P. Dhariwal, J. Schulman, I. Sutskever, and P. Abbeel, "Variational lossy autoencoder," arXiv preprint, arXiv:1611.02731, 2017

[5] I. Goodfellow et al., "Generative adversarial nets," arXiv preprint, arXiv:1406.2661, 2014. https://arxiv.org/pdf/1406.2661

[6] In the paper, the authors show that the Kullback–Leibler (KL) divergence term rises slightly. KL divergence measures how one probability distribution differs from another. S. Kullback and R. A. Leibler, "On information and sufficiency," *Annals of Mathematical Statistics* 22, no. 1 (1951): 79–86

[7] Graph-based approaches and memory-augmented models have made progress, but they assume a graph structure ahead of time or introduce complexity. Z. Wu et al., "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems* (Jan 2021): 4–24; A. Graves, G. Wayne, and I. Danihelka, "Neural Turing machines," arXiv preprint, arXiv:1410.5401, 2014. https://arxiv.org/abs/1410.5401

[8] Adam Santoro, et al, "A simple neural network module for relational reasoning," 2017. https://arxiv .org/abs/1706.01427

[9] J. Johnson et al., "CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1988–1997, doi: 10.1109/CVPR.2017.215.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

https://openaccess.thecvf.com/content_cvpr_2017/papers/Johnson_CLEVR_A_Diagnostic_CVPR_2017_paper.pdf

[10] J. Weston, A. Bordes, S. Chopra, and T. Mikolov, "Towards AI-complete question answering: A set of prerequisite toy tasks," arXiv preprint, arXiv:1502.05698, 2015. https://arxiv.org/abs/1502.05698

[11] K. Quach, "DeepMind teaches AI to reason," *The Register*, Jun. 9, 2017, https://www.theregister.com/2017/06/09/deepmind_teaches_ai_to_reason

[12] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein, "Deep compositional question answering with neural module networks," arXiv preprint, arXiv:1511.02799, 2015. https://arxiv.org/pdf/1511.02799

[13] R. Hu, J. Andreas, M. Rohrbach, T. Darrell, and K. Saenko, "Learning to reason: End-to-end module networks for visual question answering," arXiv preprint, arXiv:1704.05526, 2017. https://arxiv.org/pdf/1704.05526

[14] J. J. Irwin, "Staring off into chemical space," *Nature Chemical Biology* 5, no. 8 (2009): 536–537, doi: 10.1038/nchembio0809-536

[15] P. Kirkpatrick and C. Ellis, "Chemical space." *Nature* 432, no. 7019 (2004): 823–865. Bibcode: 2004Natur.432.823K. doi:10.1038/432823a

[16] R. Ramakrishnan, P. Dral, M. Rupp et al. Quantum chemistry structures and properties of 134 kilo-molecules. *Scientific Data* 1 (2014): 140022. https://doi.org/10.1038/sdata.2014.22

[17] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," arXiv preprint, arXiv:1704.01212, 2017

[18] P. Hohenberg and W. Kohn, "Inhomogeneous electron gas," *Physical Review* 136 (1964): B864

[19] W. Kohn and L. J. Sham, "Self-consistent equations including exchange and correlation effects," *Physical Review* 140 (1965): A1133

[20] Li, Yujia, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel, "Gated graph sequence neural networks," *arXiv* [*cs.LG*], 2017. http://arxiv.org/abs/1511.05493

[21] F. A. Faber et al., "Machine learning prediction errors better than DFT accuracy," arXiv preprint, arXiv:1702.05532, 2017. https://arxiv.org/pdf/1702.05532

[22] Kristof T. Schütt, Pieter-Jan Kindermans, Huziel E. Sauceda, Stefan Chmiela, Alexandre Tkatchenko, and Klaus-Robert Müller, "SchNet: A continuous-filter convolutional neural network for modeling quantum interactions," arXiv [stat.ML], 2017. http://arxiv.org/abs/1706.08566; K.T. Schütt, F. Arbabzadah, S. Chmiela, K.-R. Müller, and A. Tkatchenko, "Quantum-chemical insights from deep tensor neural networks," *Nature Communications* 8 (2017): 13890

[23] Johannes Gasteiger, Janek Groß, and Stephan Günnemann, "Directional message passing for molecular graphs," arXiv [Cs.LG], 2022. http://arxiv.org/abs/2003.03123

[24] Alex Graves, Greg Wayne, Malcolm Reynolds, et al., "Hybrid computing using a neural network with dynamic external memory," *Nature* 538, no. 7626 (2016): 471

[25] W. Zaremba and I. Sutskever, "Learning to execute," arXiv preprint, arXiv:1410.4615, 2015. https://arxiv.org/abs/1410.4615

[26] Mikael Henaff, Jason Weston, Arthur Szlam, Antoine Bordes, and Yann LeCun, "Tracking the world state with recurrent entity networks," in *Fifth International Conference on Learning Representations*, 2017.

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[27] Edouard Grave, Armand Joulin, and Nicolas Usunier, "Improving neural language models with a continuous cache," arXiv preprint, arXiv:1612.04426, 2016

[28] S. Bai, J. Z. Kolter, and V. Koltun, "Convolutional sequence modeling revisited," OpenReview (ICLR Workshop), 2018. https://openreview.net/forum?id=rk8wKk-R-

[29] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, "Language modeling with gated convolutional networks," arXiv preprint, arXiv:1612.08083, 2016. http://arxiv.org/abs/1612.08083

[30] J. W. Rae, C. Dyer, P. Dayan, and T. P. Lillicrap, "Fast parametric learning with activation memorization," arXiv preprint, arXiv:1803.10049, 2018. http://arxiv.org/abs/1803.10049.

[31] S. Merity, N. S. Keskar, J. Bradbury, and R. Socher, "Scalable language modeling: WikiText-103 on a single GPU in 12 hours," 2018, doi: 10.1002/j.2334-1234.2018.12159.x. https://mlsys.org/Confer ences/doc/2018/50.pdf

[32] H. Liu et al., "Evaluating the logical reasoning ability of ChatGPT and GPT-4," arXiv preprint, arXiv:2304.03439, 2023. https://arxiv.org/pdf/2304.03439

[33] OpenAI, "GPT-4 technical report," arXiv preprint, arXiv:2303.08774, 2023. https://arxiv.org/pdf/2303.08774

[34] V. Kocijan, E. Davis, T. Lukasiewicz, G. Marcus, and L. Morgenstern, "The defeat of the Winograd Schema Challenge," *Artificial Intelligence* 325 (2023): art. no. 103971. https://www.sciencedirect.com/science/article/abs/pii/S0004370223001170

[35] Qihuang Zhong, Kang Wang, Ziyang Xu, Juhua Liu, Liang Ding, and Bo Du, 'Achieving >97% on GSM8K: "Deeply understanding the problems makes LLMs better solvers for math word problems," arXiv [cs.CL], 2024. http://arxiv.org/abs/2404.14963

[36] A. Lewkowycz et al., "Minerva: Solving quantitative reasoning problems with language models," arXiv preprint, arXiv:2206.14858, 2022. https://arxiv.org/pdf/2206.14858

[37] J. Hart, "OpenAI took gold at a math competition, and it's fueling AGI speculation," *Business Insider*, Jul. 2025. https://www.businessinsider.com/openai-gold-iom-math-competition-2025-7

[38] C. Yang et al., "Large language models as optimizers," arXiv preprint, arXiv:2309.03409, 2023. https://arxiv.org/abs/2309.03409

[39] Wikipedia contributors, "Wolf, goat and cabbage problem," Wikipedia. https://en.wikipedia.org/wiki/Wolf,_goat_and_cabbage_problem

[40] S. Newman, "GPT-4 doesn't figure things out, it already knows them," Second Thoughts, Apr. 13, 2023. https://amistrongeryet.substack.com/p/gpt-4-capabilities

[41] S. Amarel, "On representations of problems of reasoning about actions," in *Machine Intelligence 3*, D. Michie (ed.). Amsterdam, The Netherlands: Elsevier/North Holland, 1968, pp. 131–171

[42] J. McCarthy, "Circumscription: A form of non-monotonic reasoning," *Artificial Intelligence* 13, no. 1–2 (1980): 27–39, doi: 10.1016/0004-3702(80)90011-9

[43] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," arXiv preprint, arXiv:1810.04805, 2018. https://research.google/blog/open-sourcing-bert-state-of-the-art-pre-training-for-natural-language-processing/

[44] L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans, "The reversal curse: LLMs trained on 'A is B' fail to learn 'B is A'," arXiv

Licensed to Hermilo Cortés <hermilocg@gmail.com>

preprint, arXiv:2309.12288, 2023. https://research.google/blog/open-sourcing-bert-state-of-the-art-pre-training-for-natural-language-processing/

[45] J. McCarthy, "Applications of circumscription to formalizing common-sense knowledge," *Artificial Intelligence* 28, no. 1 (1986): 89–116, doi: 10.1016/0004-3702(86)90032-9

[46] J. McCarthy, "Elaborating MCP (Part I)," Stanford University, technical report, n.d. http://jmc.stanford.edu/articles/missionaries1/missionaries1.pdf

[47] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou, "Chain-of-thought prompting elicits reasoning in large language models," arXiv [Cs.CL], 2023. https://arxiv.org/abs/2201.11903

[48] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, "Large language models are zero-shot reasoners," arXiv [cs.CL], 2023. https://arxiv.org/abs/2205.11916

[49] Google Research, "Language models perform reasoning via chain of thought," May 11, 2022. https://research.google/blog/language-models-perform-reasoning-via-chain-of-thought

[50] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," arXiv preprint, arXiv:2203.11171, 2022. https://arxiv.org/abs/2203.11171

[51] Again, the prompt is not unlocking hidden intelligence so much as supplying a lightweight formal method embedded in text. It forces the model to expose intermediate commitments that can be checked. This is why prompting can shift performance, sometimes substantially, while still failing on unfamiliar variants.

[52] For example, in the 1988 revised edition of their seminal book *Perceptrons: An Introduction to Computational Geometry*, Marvin Minsky and Seymour Papert wrote, "The movement of research interest between the poles of connectionist learning and symbolic reasoning may provide a fascinating subject for the sociology of science, but workers in those fields must understand that these poles are artificial simplifications." M. Minsky, S. Papert and L. Bottou, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 2017

[53] J. Haugeland, *Artificial Intelligence, The Very Idea*. Cambridge, MA: MIT Press 1985

[54] A. Newell and H. Simon, "Computer science as empirical inquiry: Symbols and search," *Communications of the Association for Computing Machinery* 19, no. 3 (1976): 113–126. http://dx.doi.org/10.1145/360018.360022

[55] J. A. Fodor and Z. W. Pylyshyn, "Connectionism and cognitive architecture: A critical analysis," *Cognition* 28, no. 1–2 (Mar. 1988): 3–71

[56] P. Smolensky, "On the proper treatment of connectionism," *Behavioral and Brain Sciences* 11 (1988): 1–23

[57] J. Pavlus, "Google's AI Wizard unveils a new twist on neural networks," *WIRED*, Nov. 1, 2017. https://www.wired.com/story/googles-ai-wizard-unveils-a-new-twist-on-neural-networks/

[58] G. Marcus, "Deep learning: A critical appraisal," arXiv preprint, arXiv:1801.00631, 2018. https://arxiv.org/pdf/1801.00631

[59] McDermott wrote, "A common idiocy in AI research is to suppose that having identified the shortcomings of Version I of a program is equivalent to having written

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Version II." Drew McDermott, "Artificial intelligence meets natural stupidity," *ACM SIGART Bulletin* 57 (Apr. 1976): 4–9. https://doi.org/10.1145/1045339.1045340

[60] F. Chollet, "On the measure of intelligence," arXiv, 2019. https://arxiv.org/abs/1911.01547

[61] ARC Prize Foundation, 2026. https://arcprize.org

[62] In current ARC Prize settings, evaluation also includes resource constraints such as bounded compute, no internet access for private competitions, runtime limits, and cost caps, making efficiency part of the benchmark's practical significance.

[63] F. Chollet, M. Knoop, G. Kamradt, and B. Landers, "ARC Prize 2024: Technical report," arXiv preprint, arXiv:2412.04604, 2025. https://arxiv.org/abs/2412.04604

[64] https://arcprize.org/leaderboard

[65] ARC Prize Foundation, "OpenAI o3 breakthrough high score on ARC-AGI-Pub," Dec. 20, 2024. https://arcprize.org/blog/oai-o3-pub-breakthrough

[66] Greg Kamradt, "Analyzing o3 and o4-mini with ARC-AGI," ARC Prize Foundation Blog, Apr. 22, 2025. https://arcprize.org/blog/analyzing-o3-with-arc-agi

[67] Future of Life Institute, "Pause giant AI experiments: An open letter," Mar. 22, 2023. https://futureoflife.org/open-letter/pause-giant-ai-experiments/

[68] In fairness, ARC-AGI-2 is not just newer. It was also tougher for today's AI reasoning systems, yet it kept human difficulty roughly similar. ARC Prize Foundation, "Announcing ARC-AGI-2 and ARC Prize 2025," Mar. 24, 2025. https://arcprize.org/blog/announcing-arc-agi-2-and-arc-prize-2025.

[69] C. A. E. Goodhart, "Problems of monetary management: The U.K. experience," in *Papers in Monetary Economics*, vol. 1. Sydney, Australia: Reserve Bank of Australia, 1975

[70] S. Sagawa, P.W. Koh, T.B. Hashimoto, and P. Liang, *Distributionally Robust Neural Networks for Group Shifts: On the Importance of Regularization for Worst-Case Generalization,* ICLR, 2020

[71] E.M. Bender and A. Koller, *Climbing Towards NLU: On Meaning, Form, and Understanding in the Age of Data.* ACL, 2020

[72] X. Bouthillier et al., "Accounting for variance in machine learning benchmarks," in *Proceedings of the 4th Conference on Machine Learning and Systems (MLSys)*, 2021. https://proceedings.mlsys.org/paper_files/paper/2021/file/0184b0cd3cfb185989f858a1d9f5c1eb-Paper.pdf

[73] W. James, *Pragmatism: A New Name for Some Old Ways of Thinking.* New York: Longmans, Green, 1907, lecture VI ("Pragmatism's Conception of Truth")

Chapter 8

[1] Mind Cathedral, "Conference Jensen Huang (NVIDIA) and Ilya Sutskever (OpenAI). AI Conferenza SUB ITA," YouTube video, 2023. https://www.youtube.com/watch?v=ZZ0atq2yYJw

[2] A. Radford, I. Sutskever, R. Józefowicz, J. Clark, and G. Brockman, "Unsupervised sentiment neuron," OpenAI, Apr. 6, 2017. https://openai.com/index/unsupervised-sentiment-neuron/

[3] The second law of thermodynamics is the principle that entropy does not decrease for an isolated system. Following Aaronson, Carroll, and Ouellette, the coffee-cup model

Licensed to Hermilo Cortés <hermilocg@gmail.com>

is treated as a "closed system" for the purposes of studying how entropy can increase while apparent complexity temporarily rises and falls.

[4] A. Shen, V. A. Uspensky, and N. Vereshchagin, *Kolmogorov Complexity and Algorithmic Randomness, Mathematical Surveys and Monographs*, vol. 220. Providence, RI: American Mathematical Society, 2017. https://bookstore.ams.org/surv-220

[5] F. Green, "The book review column," *ACM SIGACT News* 50, no. 4 (2019.) https://mathcs.clarku.edu/~fgreen/SIGACTReviews/bookrev/50-4.pdf

[6] L. Fortnow, "Compressing in Moscow," Computational Complexity Blog, Jun. 12, 2019. https://blog.computationalcomplexity.org/2019/06/compressing-in-moscow.html

[7] For instance, they proved that every linear inequality true for classical Shannon entropy also holds for Kolmogorov complexity, cementing a strong link between probabilistic information theory and algorithmic information. Shen is also known for a notable result in computational complexity (an alternate proof of IP = PSPACE) but has spent decades on algorithmic information theory. By 2017, Shen was at CNRS in France and Vereshchagin at Moscow State. They, along with Uspensky, distilled their collective knowledge into this volume.

[8] Claude E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal* 27, no. 3 (Jul. 1948): 379–423

[9] Note that nonrepetition alone is not algorithmic randomness. The digits of 0.0% or $\sqrt{2}$ do not settle into a repeating pattern, but they are computable because a short algorithm can generate their digits. In Kolmogorov terms, their finite prefixes are therefore highly compressible compared with typical random strings of the same length

[10] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal* 27, no. 3 (Jul 1948): 379–423; 27, no. 4 (Oct. 1948): 623–656

[11] R. J. Solomonoff, "Ray Solomonoff's home page," raysolomonoff.com, n.d. https://raysolomonoff.com

[12] The halting problem asks whether there is a general algorithm that can determine, for any program and input, whether the program will eventually stop or run forever. Alan Turing showed that no such algorithm exists and some computational questions are undecidable. A. M. Turing, "On computable numbers, with an application to the Entscheidungs problem," *Proceedings of the London Mathematical Society Series 2* 42 (1937): 230–265. https://doi.org/10.1112/plms/s2-42.1.230

[13] Hacker News, "Ilya Sutskever: 'If you learn all of these, you'll know 90% of what matters'," May 18, 2024. https://news.ycombinator.com/item?id=40397806

[14] ErdosProblems.com Forum, "Erdős problem #481—Discussion thread," last edited Dec. 3, 2025. https://www.erdosproblems.com/forum/thread/481#post-1934

[15] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in A. Platzer and G. Sutcliffe (eds.), *Automated Deduction—CADE 28*. Lecture Notes in *Computer Science*, vol 12699. Cham, Switzerland: Springer, 2021. https://doi.org/10.1007/978-3-030-79876-5_37

[16] Same as [1]

[17] M. Chen et al., "Generative pretraining from pixels," in *Proceedings of the 37th International Conference on Machine Learning*, vol. 119, 2020, pp. 1691–1703. https://cdn.openai.com/papers/Generative_Pretraining_from_Pixels_V2.pdf

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[18] C. S. Smith, interviewer, and I. Sutskever, interviewee, "Ilya Sutskever: The mastermind behind GPT-4 and the future of AI," Eye on A.I. Podcast, Season 2, Episode 118, Mar. 15, 2023, 43 min. (08:36, 26:02, 29:40). https://podcasts.apple.com/us/podcast/ilya-sutskever-the-mastermind-behind-gpt-4-and/id1438378439?i=1000604382855

[19] Meta AI, "V-JEPA: The next step toward advanced machine intelligence," Feb. 15, 2024. https://ai.meta.com/blog/v-jepa-yann-lecun-ai-model-video-joint-embedding-predictive-architecture/

[20] R. Caruana, S. Lawrence, and C. L. Giles, "Overfitting in neural nets: Backpropagation, conjugate gradient, and early stopping," in *Advances in Neural Information Processing Systems,* vol. 13, 2000, pp. 402–408. https://proceedings.neurips.cc/paper_files/paper/2000/file/059fdcd96baeb75112f09fa1dcc740cc-Paper.pdf

[21] A. Krogh and J. A. Hertz, "A simple weight decay can improve generalization," in *Advances in Neural Information Processing Systems,* vol. 4, 1991, pp. 950–957. https://proceedings.neurips.cc/paper/1991/file/8eefcfdf5990e441f0fb6f3fad709e21-Paper.pdf

[22] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra, "Grokking: Generalization beyond overfitting on small algorithmic datasets," arXiv:2201.02177, Jan. 6, 2022. https://arxiv.org/abs/2201.02177

[23] Robert Heinlein coined "grok" for an understanding native to an alien conceptual scheme. It conveys a language one can speak without inhabiting the world that gives it meaning.

[24] B. DeMoss, S. Sapora, J. Foerster, N. Hawes, and I. Posner, "The complexity dynamics of grokking," arXiv:2412.09810v2, Aug. 21, 2025. https://arxiv.org/pdf/2412.09810

[25] N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt, "Progress measures for grokking via mechanistic interpretability," arXiv:2301.05217, 2023. https://arxiv.org/pdf/2301.05217

[26] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," arXiv:1611.03530, 2016. https://arxiv.org/abs/1611.03530

[27] M. Belkin, D. Hsu, S. Ma, and S. Mandal, "Reconciling modern machine-learning practice and the classical bias-variance trade-off," arXiv:1812.11118, 2019. https://arxiv.org/abs/1812.11118

[28] P. Nakkiran, G. Kaplun, Y. Bansal, T. Yang, B. Barak, and I. Sutskever, "Deep double descent: Where bigger models and more data hurt," arXiv:1912.02292, 2019. https://arxiv.org/abs/1912.02292

[29] A. Karpathy, "A recipe for training neural networks," Apr. 25, 2019. http://karpathy.github.io/2019/04/25/recipe/

[30] Happy Scribe, "Transcript of #94—Ilya Sutskever: Deep learning," Lex Fridman Podcast, May 8, 2020. https://podcasts.happyscribe.com/lex-fridman-podcast-artificial-intelligence-ai/94-ilya-sutskever-deep-learning

[31] I. Sutskever (@ilyasut), "The double descent phenomenon occurs in practical deep learning settings," X, Dec. 5, 2019. https://x.com/ilyasut/status/1202646723140734977

[32] OpenAI, "Deep double descent," Dec. 5, 2019. https://openai.com/index/deep-double-descent/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[33] T. Chiang, "ChatGPT is a blurry JPEG of the web," *The New Yorker*, Feb. 9, 2023. https://www.newyorker.com/tech/annals-of-technology/chatgpt-is-a-blurry-jpeg-of-the-web

Chapter 9

[1] "Definitions of intelligence," University of Illinois, Department of Psychology. https://labs.psychology.illinois.edu/~lyubansk/IQdef.htm

[2] As Legg puts it, "If it is able to perform well in a wide range of environments, that is all that matters." S. Legg and M. Hutter, "Universal intelligence: A definition of machine intelligence," *Minds and Machines* 17, no. 4 (2007): 391–444, doi: 10.1007/s11023-007-9079-x

[3] S. Legg, "Machine super intelligence," Ph.D. dissertation, Università della Svizzera italiana, Lugano, Switzerland, Jun. 2008. http://www.vetta.org/documents/Machine_Super_Intelligence.pdf

[4] P. A. Carpenter, M. A. Just, and P. Shell, "What one intelligence test measures: A theoretical account of the processing in the Raven Progressive Matrices Test," *Psychological Review* 97, no. 3 (1990): 404–431, doi: 10.1037/0033-295X.97.3.404

[5] Bayesians reply that with enough data and a reasonable prior, the posterior is dominated by the likelihood term P(D|h), so the exact prior often matters little in practice, and classical (non-Bayesian) methods smuggle in their own priors anyway. But the problem becomes sharper in infinite hypothesis spaces, where naive choices lead to "improper" priors and further conceptual unease.

[6] This makes it a kind of grand unified theory of inductive inference. Indeed, if it were not for the tiny problem of its incomputability, the problem of induction might be considered solved.

[7] Solomonoff and L. A. Levin extended the idea in 1973. L. A. Levin, "Universal sequential search problems," *Problems of Information Transmission* 9 (1973): 265–266

[8] Dr. Biology, "Life 3.0," Ask A Biologist, Arizona State University, Feb. 28, 2018. https://askabiologist.asu.edu/listen-watch/life-30

[9] Quote Investigator, "Quote origin: The brain is merely a meat machine," May 6, 2020 (updated Mar. 28, 2025). https://quoteinvestigator.com/2020/05/06/machines

[10] P. Mirowski, *Machine Dreams: Economics Becomes a Cyborg Science*. Cambridge, U.K.: Cambridge University Press, 2001

[11] S. Armstrong, "Defining AI wireheading," LessWrong, Nov. 21, 2019. https://www.lesswrong.com/posts/vXzM5L6njDZSf4Ftk/defining-ai-wireheading

[12] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in AI safety," arXiv preprint, arXiv:1606.06565, 2016. https://arxiv.org/abs/1606.06565

[13] L. Cecco, "Air Canada ordered to pay customer who was misled by airline's chatbot," *The Guardian*, Feb. 16, 2024. https://www.theguardian.com/world/2024/feb/16/air-canada-chatbot-lawsuit

[14] National Transportation Safety Board, "Collision between vehicle controlled by developmental automated driving system and pedestrian," Investigation ID HWY18MH010, n.d. https://www.ntsb.gov/investigations/Pages/HWY18MH010.aspx

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[15] I. J. Good, "Speculations concerning the first ultraintelligent machine," *Advances in Computers* 6 (1965): 31–88, 1965, doi: 10.1016/S0065-2458(08)60418-0. https://doi.org/10.1016/S0065-2458(08)60418-0

[16] V. Vinge, "The coming technological singularity: How to survive in the post-human era," 1993. https://mindstalk.net/vinge/vinge-sing.html

[17] R. Kurzweil, *The Singularity Is Near: When Humans Transcend Biology*. New York: Viking, 2005. https://philpapers.org/rec/KURTSI

[18] D. Bergmann and C. Stryker, "What is artificial general intelligence (AGI)?" IBM Think, Sep. 17, 2024. https://www.ibm.com/think/topics/artificial-general-intelligence

[19] B. Goertzel and C. Pennachin, eds., *Artificial General Intelligence*. Berlin, Germany: Springer, 2007, doi: 10.1007/978-3-540-68677-4. https://link.springer.com/book/10.1007/978-3-540-68677-4

[20] Hubert L. Dreyfus, "A history of first step fallacies," *Minds and Machines* 22, no. 2 (2012): 87–99

[21] Google Scholar lists 7,000 to 9,000 citations, an extraordinary count for a work in analytic philosophy and AI risk. N. Bostrom, "Google Scholar profile," Google Scholar Citations. https://scholar.google.es/citations?user=oQwpz3QAAAAJ&hl

[22] Yudkowsky's essay has accumulated about 425 academic citations, a substantial number for a contribution to an edited volume. E. Yudkowsky, "Artificial intelligence as a positive and negative factor in global risk," in N. Bostrom and M. M. Ćirković (eds.), *Global Catastrophic Risks*. Oxford, U.K.: Oxford University Press, 2008, pp. 308–345. https://www.semanticscholar.org/paper/Artificial-Intelligence-as-a-Positive-and-Negative-Yudkowsky/fcf7368061a544a09d16826eb4c5a8463ee5482e

[23] E. Yudkowsky, "Pausing AI developments isn't enough. We need to shut it all down," *TIME*, Mar. 29, 2023. https://time.com/6266923/ai-eliezer-yudkowsky-open-letter-not-enough/

[24] E. Yudkowsky, "Creating friendly AI 1.0: The analysis and design of benevolent goal architectures," The Singularity Institute for Artificial Intelligence, 2001. https://intelligence.org/files/CFAI.pdf

[25] XiXiDu, "Q&A with Shane Legg on risks from AI," LessWrong, Jun. 17, 2011. https://www.lesswrong.com/posts/No5JpRCHzBrWA4jmS/q-and-a-with-shane-legg-on-risks-from-ai

[26] W. Henshall, "Shane Legg," *TIME* (TIME100 AI). https://time.com/collection/time100-ai/6310659/shane-legg/

[27] M. Hutter, "Google Scholar profile," Google Scholar Citations. https://scholar.google.com/citations?user=7hmCntEAAAAJ&hl

[28] That said, Shane Legg and Hutter published a paper in *Minds and Machines* titled "Universal Intelligence: A Definition of Machine Intelligence" in 2007, which is a precursor to "Machine Super Intelligence." It has accumulated over 1,200 citations.

[29] Same as [1].

[30] M. Kaufman, *First Contact: Scientific Breakthroughs in the Hunt for Life Beyond Earth*, reprint ed. New York: Simon & Schuster, 2012

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[31] For example, if you "made a computer out of old beer cans powered by windmills; if it had the right program, it would have to have a mind." J. R. Searle, "Minds, brains and science: 2. Beer cans and meat machines," *The Listener* 112 (Nov. 15, 1984): 14–16

[32] John Watson sought to reframe psychology as a natural science by focusing on observable behavior, in his famous work titled, *Behaviorist Manifesto*. Watson spearheaded the scientific movement in the first half of the 20th century.

[33] David Silver, Satinder Singh, Doina Precup, and Richard Sutton argue in "Reward is Enough" that "the reinforcement learning problem differs from behaviorism in allowing an agent to construct and use internal state."

[34] University of California, Berkeley, "Philosophy of mind, lecture #3 (Philosophy 132, spring 2011)," YouTube video, 2011. https://youtu.be/qJ9YQ5IHzrI?t=1745.

[35] Hilary Putnam provides a more serious counterexample, named Super-Spartans, that attacks behaviorism. H. Putnam, "Brains and behavior," in *Readings in Philosophy of Psychology*, vol. 1, N. Block (ed.), Cambridge, MA: Harvard University Press, 2013, pp. 24–36, doi: 10.4159/harvard.9780674594623.c4

[36] B. F. Skinner, *The Behavior of Organisms: An Experimental Analysis*. New York: D. Appleton-Century Company, 1938.

[37] Skinner Box. https://www.ebsco.com/research-starters/history/skinner-box

[38] B. F. Skinner, *Verbal Behavior*. New York: Appleton-Century-Crofts, 1957

[39] N. Chomsky, "A review of B. F. Skinner's *Verbal Behavior*," *Language* 35, no. 1 (1959): 26–58

[40] Skinner argued that language was acquired through behaviorist reinforcement learning, whereas Chomsky argued that language was innate and evolved. S. Pinker, *The Language Instinct: How the Mind Creates Language*. New York: William Morrow, 1994

[41] "Fallacy of definition," Wikipedia. https://en.wikipedia.org/wiki/Fallacy_of_definition.

[42] J. Baggini, "The enduring brilliance of Ludwig Wittgenstein," *Prospect Magazine*, May 7, 2018. https://www.prospectmagazine.co.uk/ideas/philosophy/41153/the-enduring-brilliance-of-ludwig-wittgenstein

[43] A. M. Turing, "Computing machinery and intelligence," *Mind* 59, no. 236 (1950): 433–460, doi: 10.1093/mind/LIX.236.433. https://courses.cs.umbc.edu/471/papers/turing.pdf

[44] P. A. Murphy, "Alan Turing believed the question 'Can machines think?' to be meaningless," *Becoming Human: Artificial Intelligence Magazine* (Medium), Jan. 30, 2025. https://becominghuman.ai/alan-turing-believed-the-question-can-machines-think-to-be-meaningless-7a4a8887b220

[45] "Allen Iverson 'practice' press conference," YouTube video, https://youtu.be/eGDBR2L5kzI?si=NQ3cf5U9TjC7LjEX

[46] Emerson M. Pugh, quoted in George Edgin Pugh's *The Biological Origin of Human Values*, observed that "if the human brain were so simple that we could understand it, we would be so simple that we couldn't" (p. 154).

[47] C. R. Jones and B. K. Bergen, "Large language models pass the Turing test," arXiv:2503.23674, Mar. 31, 2025, doi: 10.48550/arXiv.2503.23674. https://arxiv.org/abs/2503.23674

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[48] Turing famously said that "Nevertheless I believe that at the end of the century the use of words and general educated opinion will have altered so much that one will be able to speak of machines thinking without expecting to be contradicted." Same as [43]

[49] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv:2303.12712, Mar. 2023, doi: 10.48550/arXiv.2303.12712. https://arxiv.org/abs/2303.12712

[50] Hitchens's razor: Christopher Hitchens, *God Is Not Great: How Religion Poisons Everything*, Kindle Edition. Twelve Books, 2009

[51] OpenAI, "GPT-4," Mar. 14, 2023. https://openai.com/index/gpt-4-research/

[52] The dubious part: E. Martínez, "Re-evaluating GPT-4's bar exam performance," *Artificial Intelligence and Law* 33 (2025): 581–604, doi: 10.1007/s10506-024-09396-9

[53] Mike Alder, "Newton's flaming laser sword," *Philosophy Now* 46 (2004): 29–33

[54] "Conversations with history: Hubert Dreyfus," UCTV, YouTube video, 2008.https://www.youtube.com/watch?v=-CHgt2Szk-I.

[55] John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude E. Shannon, "A proposal for the Dartmouth Summer Research Project on Artificial Intelligence, August 31, 1955," *AI Magazine* 27 (2006): 12–14

[56] The phrase "the hard problem of consciousness" was coined by David Chalmers. David J. Chalmers, *The Conscious Mind: In Search of a Fundamental Theory*. New York: Oxford University Press, 1996

[57] Johansson in Hermann Haken, *The Machine as Metaphor and Tool*. Berlin, Germany: Springer, 1993, p. 38

[58] Ludwig Wittgenstein, *Philosophical Investigations,* G. E. M. Anscombe (ed.). New York: Wiley-Blackwell, 1953

[59] Anthropic, "Claude Opus 4 and 4.1 can now end a rare subset of conversations," Aug. 15, 2025. https://www.anthropic.com/research/end-subset-conversations

Epilogue

[1] Volodymyr Mnih et al., "Playing Atari with deep reinforcement learning," 2013.

[2] DeepMind, "Deep reinforcement learning," Google DeepMind Blog, Jun. 17, 2016. https://deepmind.google/blog/deep-reinforcement-learning/

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Human-level control through deep reinforcement learning," *Nature* 518 (2015): 529–533. https://doi.org/10.1038/nature14236

[4] AI Rev, "AlphaGo: A technical deep dive," AI Rev, https://airev.us/alpha-go

[5] D. Silver, J. Schrittwieser, K. Simonyan, et al., "Mastering the game of Go without human knowledge," *Nature* 550, no. 7676 (Oct. 2017): 354–359

[6] J. Schrittwieser, I. Antonoglou, T. Hubert, K. et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature* 588 (Dec. 2020): 604–609

[7] Google Research, "Open Sourcing BERT: State-of-the-art pre-training for natural language processing," Google Research Blog, Nov. 2, 2018. https://research.google/blog/open-sourcing-bert-state-of-the-art-pre-training-for-natural-language-processing/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional Transformers for language understanding," arXiv preprint, arXiv:1810.04805, 2018. https://arxiv.org/abs/1810.04805

[9] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text Transformer," *Journal of Machine Learning Research 21*, no. 140 (2020): 1–67. https://arxiv.org/abs/1910.10683

[10] N. Shibuya, "ViT: Vision Transformer (2020)," Naoki Shibuya Blog, Nov. 2, 2022. https://naokishibuya.github.io/blog/2022-11-02-vit-vision-transformer-image-classifier-2020/

[11] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," arXiv preprint, arXiv:2010.11929, 2020. https://arxiv.org/abs/2010.11929

[12] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2020, pp. 1597–1607. https://arxiv.org/abs/2002.05709

[13] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Visual Pattern Recognition (CVPR)*, 2020, pp. 9729–9738. https://arxiv.org/abs/1911.05722

[14] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," arXiv [cs.LG], 2020

[15] J.-B. Grill, F. Strub, F. Altché, et al., "Bootstrap your own latent: A new approach to self-supervised learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. https://arxiv.org/abs/2006.07733

[16] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments," in *Advances in Neural Information Processing Systems*, vol. 33, 2020. https://arxiv.org/abs/2006.09882

[17] A. Radford et al., "Learning transferable visual models from natural language supervision," arXiv preprint, arXiv:2103.00020, 2021. https://arxiv.org/abs/2103.00020

[18] A. Nichol, P. Dhariwal, P. Ramesh, P. Shyam, M. Mishkin, B. McGrew, and I. Sutskever, "GLIDE: Towards photorealistic image generation and editing with text-guided diffusion models," arXiv preprint, arXiv:2112.10741, 2021. https://arxiv.org/abs/2112.10741

[19] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, "Hierarchical text-conditional image generation with CLIP latents," arXiv preprint, arXiv:2204.06125, 2022. https://arxiv.org/abs/2204.06125

[20] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Visual Pattern Recognition (CVPR)*, 2022, pp. 10684–10695. https://arxiv.org/abs/2112.10752

[21] J. Jumper, R. Evans, A. Pritzel, T. Green, et al., "Highly accurate protein structure prediction with AlphaFold," *Nature*. 596, no. 7873 (Aug. 2021): 583–589

[22] DeepMind, "AlphaFold: A solution to a 50-year-old grand challenge in biology," Google DeepMind Blog, Nov. 30, 2020. https://deepmind.google/blog/alphafold-a-solution-to-a-50-year-old-grand-challenge-in-biology/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[23] J. Jumper et al., "Highly accurate protein structure prediction with AlphaFold," *Nature* 596 (2021): 583–589

[24] K. Tunyasuvunakool, J. Adler, Z. Wu, et al., "Highly accurate protein structure prediction for the human proteome," *Nature* 596, no. 7873 (Aug. 2021): 590–596

[25] E. Callaway, "'It will change everything': DeepMind's AI makes gigantic leap in solving protein structures," *Nature* 588 (Dec. 2020): 203–204

[26] Q. V. Le et al., "Building high-level features using large scale unsupervised learning," arXiv preprint, arXiv:1112.6209, 2011. https://arxiv.org/abs/1112.6209

[27] J. Schmidhuber, "Learning complex, extended sequences using the principle of history compression", 1992

[28] C. Bucila, R. Caruana, and A. Niculescu-Mizil, "Model compression", on Knowledge Discovery and Data Mining conference, 2006.

[29] L. J. Ba and R. Caruana, "Do deep nets really need to be deep?" arXiv [cs.LG], 2014

[30] G. Hinton, O. Vinyals, J. Dean (2015). "Distilling the Knowledge in a Neural Network"

[31] G. Hinton, "Dark knowledge," YouTube video, TTIC Distinguished Lecture Series, 2014. https://www.youtube.com/watch?v=F5gXXubLo9s

[32] G. Hinton, "Dark knowledge," TTIC Distinguished Lecture Series, 2014. https://www.ttic.edu/dl/dark14.pdf

[33] W. Knight, "How distillation makes AI models smaller and cheaper," *WIRED,* Sep. 20, 2025. https://www.wired.com/story/how-distillation-makes-ai-models-smaller-and-cheaper/

[34] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv preprint, arXiv:1910.01108, 2019. https://arxiv.org/abs/1910.01108

[35] OpenAI, "Aligning language models to follow instructions," OpenAI, Jan. 27, 2022. https://openai.com/index/instruction-following/

[36] L. Ouyang et al., "Training language models to follow instructions with human feedback," arXiv preprint, arXiv:2203.02155, 2022. https://arxiv.org/abs/2203.02155

[37] C. R. Wolfe, "Direct Preference Optimization (DPO)," Deep (Learning) Focus, Substack, Jul. 28, 2025. https://cameronrwolfe.substack.com/p/direct-preference-optimization

[38] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your language model is secretly a reward model," arXiv preprint, arXiv:2305.18290, 2023. https://arxiv.org/abs/2305.18290

[39] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," arXiv preprint, arXiv:2212.08073, 2022. https://arxiv.org/abs/2212.08073

[40] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019

[41] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," arXiv [cs.CL], 2023

[42] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," arXiv [cs.CL], 2023

[43] https://github.com/deepseek-ai/DeepSeek-R1/blob/main/DeepSeek_R1.pdf

[44] OpenAI, "Introducing OpenAI o1," OpenAI, Sep. 12, 2024. https://openai.com/o1/

Licensed to Hermilo Cortés <hermilocg@gmail.com>

[45] OpenAI, "Learning to reason with LLMs," OpenAI, Sep. 12, 2024. https://openai.com/index/learning-to-reason-with-llms/

[46] OpenAI, "OpenAI o1 system card," OpenAI, Dec. 5, 2024. https://openai.com/index/openai-o1-system-card/

[47] K. Robison, "OpenAI releases o1, its first model with 'reasoning' abilities," *The Verge*, Sep. 12, 2024. https://www.theverge.com/2024/9/12/24242439/openai-o1-model-reasoning-strawberry-chatgpt

[48] Same as [47]

Licensed to Hermilo Cortés <hermilocg@gmail.com>

index

Numerics

1F1B (one-forward, one-backward) schedule 151

A

ablation studies 52–55
activations 52
AGI (artificial general intelligence) 4, 73, 93, 126, 138, 217
  as field 227
AI (artificial intelligence)
  complexity, blurry JPEGs 215
  deep learning, long short-term memory (LSTM) networks 74–79
  Neural Turing Machines (NTMs) 118
  Sutskever, Ilya, rise of 3–7
  Sutskever's List, defined 1–2
AI safety 225–231
  AGI (as field) 227
  AI foom 229
  effects 229–231
  intelligence explosion 226
  superintelligence 228
  technological singularity 227
AlexNet 17–20, 30–41, 63–64, 80–81, 144, 183
  challenges of training 27–30
  data augmentation 36
  effect of 39–41
  efficient and scalable training 37–39
  feature engineering 20–22
  ImageNet 25–27
  network architecture 30–33
  pre-AlexNet skepticism 22–24
  ResNet vs. 49
  training innovations 33–36
algorithmic randomness 195
all-reduce routine 86
AlphaFold 65
AlphaGo 65
Altman, Sam 7, 185
Annotated Transformer 107
ANNs (artificial neural networks) 18, 159
  keeping simple 202–205
apparent complexity 191
ARC-AGI (Abstraction and Reasoning Corpus) 184–185
architecture comparisons 137
ASR (automatic speech recognition) 89
attention 95–96
  Annotated Transformer 107
  Bahdanau attention 107–113
  Neural Turing Machines (NTMs) 118
  Ptr-Net (Pointer Networks) 113–117
  reversing input sentences 122–125
Attention Is All You Need (Vaswani et al.), effects of 125–128
autoencoders, VLAE 154–159
  autoregressive prior 158

Licensed to Hermilo Cortés <hermilocg@gmail.com>

autoencoders, VLAE *(continued)*
overview 154–156
results 157

B

bAbI (bAbI dataset) 162
backpropagation 18
Bahdanau attention 107–113
experimental results 110–112
model architecture 109
significance 113
batch dispatch 88
behaviorism 234
Bengio, Yoshua 20
Biederman, Irving 25
BIG-bench 142
binary sequence prediction task 220
Bitter Lesson, The (Sutton) 63
Blockhead 233
blurry JPEGs 215
BPE (byte-pair encoding) 90
broken scaling laws 141–143
Busy Beaver function 195

C

Carmack, John, reading lists 12–16
channels, defined 47
char-rnn model 68–69
ChatGPT 5, 68
Chinchilla 138–140
Chollet, François 184
Christofides algorithm 116
Ciresan, Dan 26
CLEVR (Compositional Language and Elementary Visual Reasoning) 161
CNNs (convolutional neural networks) 3, 19, 61, 67, 69
coffee automaton 188–194
methodology 189–191
results 192–194
combinatorial tasks 121
complexity
blurry JPEGs 215
coffee automaton 188–194
keeping artificial neural networks simple 202–205
Complexity Dynamics of Grokking (DeMoss et al.) 206
compression 208–209
constituency parsing 121
contamination 143–144
content-based addressing 119
context module 56
controller network 118
convergence 136
convex hulls 114
convolutional networks 183
CoT (chain-of-thought) prompting 181
CS231n course 62
CTC (Connectionist Temporal Classification) 84
CVPR (Conference on Computer Vision and Pattern Recognition) 64

D

DanNet 18, 82
data augmentation 36
data parallelism 145
debates about reasoning 182–186
deep learning 66
AlexNet 30–41
CS231n course 62
long short-term memory (LSTM) networks 74–79
scaling 130–140
Delaunay triangulation 115
Dennett, Daniel 222
dense prediction 55–58
deterministic dynamics 223
DFT (density functional theory) 163, 168
DimeNet 169
DNC (differentiable neural computer) 172
Dota 2 4
double descent 212–214
DPM (Deformable Parts Model) 22
DS2 (Deep Speech 2) 67, 83–94
core architecture 83
engineering shift 90–94

Licensed to Hermilo Cortés <hermilocg@gmail.com>

language models and decoding 88
significance and broader impact 89
training techniques 86–87

E

Efros, Alyosha 64
emergent abilities 130
ergodic environments 223
exploding gradients 80

F

FAIR (Facebook AI Research) 40
FC (fully connected) layers 33
feature engineering 20–22
FFN (feed-forward network) 99
FLOPs (floating-point operations) 47, 150
Fodor, Jerry A. 182
forget gate 76
FP16 (half-precision) 86
functionalism 232–233

G

GAN (generative adversarial network) 10
GGNN (gated graph neural network) 164
GitHub 72
GLUE 144
Gödel's incompleteness theorem 224
GOFAI (Good Old-Fashion AI) 182
Goldberg, Yoav 72–73
GoogLeNet (Google Network), ResNet vs. 50
GPipe 146
GPT-1 4
GPT-2 (Generative Pre-trained Transformer 2) 5–6, 65
controversy 8
GPT-3 model 5, 184
GPT-4.1 model 184
gradient clipping 80
gradient descent 18
graphical model likelihood estimation 121
Graves attention 119
grokking 205–214
compression 208
double descent 212–214
theory to practice to theory 210–212
GRUs (gated recurrent units) 110
GSM8K (Grade School Math 8K) 142

H

Hacker News 71
Hinton, Geoffrey 18, 20, 183
van Camp, Drew, keeping artificial neural networks simple 202–205
Hochreiter, Sepp 75
HOG (Histogram of Oriented Gradients) 20
Hopfield Networks 18
human performance threshold 59–62
Humanity's Last Exam 142
hyperscale, rounding errors 152

I

IAPR (International Association for Pattern Recognition) 26
ICDAR (International Conference on Document Analysis and Recognition) 26
ICLM (International Conference on Machine Learning) 91
identity function 58
iGPT (image generation pre-trained transformer) 200
IJCNN (International Joint Conference on Neural Networks) 26
ILSVRC (ImageNet Large Scale Visual Recognition Challenge) 3, 18, 26
ImageNet 25–27, 64, 144
infinite data limit 134
input gate 76
intelligence explosion 226
internal covariate shift 85
IoU (intersection-over-union) 57
ISBI (IEEE International Symposium on Biomedical Imaging) 26

Licensed to Hermilo Cortés <hermilocg@gmail.com>

J

JPEGs 215

K

Kaplan scaling laws 131–133
Karpathy, Andrej 5, 69, 74
  blog post 67–74
KL (Kullback–Leibler) divergence 203
Know Your Meme 126
Kolmogorov Complexity and Algorithmic Randomness (Uspensky, Shen, Vereshchagin) 194–198
  algorithmic statistics 196–198
  significance of 198
Koltun, Vladlen 56

L

language modeling, defined 121
language models 87–88
latent code 154
LaTeX 71
LBP (Local Binary Patterns) 25
LDA (Latent Dirichlet Allocation) 81
LeCun, Yann 5, 20
Legg's definition of intelligence 231–244
  behaviorism 234
  definitions masquerading as explanations 238–240
  dirty hands, clean(er) claims 242–244
  functionalism 232–233
  meaning in use 235
  public tests 240
  Turing test 237–238
lemmas 71
LeNet 18–19
Li, Fei-Fei 25, 64
limit to the amount of data 135
living doubts 184
LLMs (large language models) 97
LM Contamination Index 144
location-based addressing 119
LRN (local response normalization) 36
LSTM (long short-term memory) 30, 67, 137, 161, 188
  networks 14, 74–79, 82

M

machine superintelligence 217–225
  agents and limits 224–225
  AIXI 221–223
  defining intelligence 218
  environment taxonomy 223
  universal intelligence measure 219–221
Malik, Jitendra 64, 183
Marcus, Gary 184
masked self-attention 100
MDL (minimum description length) principle 199–202, 220
Megatron-LM 150
Mesh-TensorFlow 150
message passing neural networks 164–166
MHDPA (multihead dot-product attention) 170
Mikolov, Tomas 73
Minsky, Marvin 222
MMLU (Measuring Massive Multitask Language Understanding) 142
MNIST (Modified National Institute of Standards and Technology) 64
model degradation 44
model parallelism 145
model size 134
models, reasoning 176–182
  distributional brittleness 177–179
  prompting as reasoning interface 180–182
  reversal curse 179
MPNNs (message passing neural networks) 154
multiple explanations 219

N

neural message passing for quantum chemistry 163–169
  accuracy as bottleneck 163
  influence 168
  message passing neural networks 164–166
  towers architecture 168

Licensed to Hermilo Cortés <hermilocg@gmail.com>

training and results 166
NLL (negative log-likelihood) 157, 174
NMT (neural machine translation) 124
NTMs (Neural Turing Machines) 3, 118, 171

O

o3-preview system 184
off-manifold sampling 155
Olah, Chris 70, 72
OpenAI Gym 4
optimal batch size 135
order matters 119–122
output gate 76
overfitting 199

P

paper doubts 184
parallelism 145–152
GPipe 146
pipeline parallelism 149–152
results 147–149
PASCAL VOC (Visual Object Classes) challenge 20
peephole connections 78
Peirce, Charles S. 184
perplexity, defined 79
physical symbol system 182
polynomial curve-fitting 200
posterior collapse 156
pre-activation and post-activation 52
prompting, as reasoning interface 180–182
proofs, defined 71
PTB (Penn Treebank) 79
Ptr-Net (Pointer Networks) 3, 113–117
experimental results 114–116
significance 117
Pylyshyn, Zenon 182

Q

Q* (Q-star) 6
quantization 86
quantization-aware training 209
quantum chemistry, neural message passing for 163–169
accuracy as bottleneck 163
influence 168
message passing neural networks 164–166
towers architecture 168
training and results 166

R

reading lists, John Carmack 12–16
real-world adoption of ResNet 51
reasoning 153
debates about 182–186
models 176–182
pivot to, neural message passing for quantum chemistry 163–169
relational reasoning 159–162
relational recurrent neural networks 170–176
VLAE 154–159
reasoning models 176–182
distributional brittleness 177–179
prompting as reasoning interface 180–182
reversal curse 179
ReLU (rectified linear unit) 33
ResNet (Residual Networks) 18, 42, 44–49, 144
comparison with other major architectures 49–50
extending with dense prediction 55–58
human performance threshold 59–62
real-world adoption of 51
ResNet v2 51–55
scaling 63–65
ResNet v2 51–55
ablation studies 52–55
post-activation to pre-activation 52
reversing input sentences 122–125
RF (receptive field) 158
RL (reinforcement learning) 15, 221
RMCs (relational memory cores) 154, 170
RNNencdec (RNN encoder–decoder) 110–111

Licensed to Hermilo Cortés <hermilocg@gmail.com>

RNNs (recurrent neural networks) 67, 97, 113, 127, 164
  Karpathy's blog post 67–74
  regularization 79–82
  relational 170–176
RNNsearch 110–112
RNs (relational networks) 159
RoPE (rotary positional embeddings) 102
rounding errors 152
RPW (Read-Process-Write) 120, 166
Rumelhart, David 18

S

s2s (set2set) model 166
safe superintelligence, machine superintelligence 217–225
  agents and limits 224–225
  AIXI 221–223
  defining intelligence 218
  environment taxonomy 223
  universal intelligence measure 219–221
scaling
  deep learning 130–140
  hypothesis 137
  ResNet 63–65
scaling laws 129
  broken 141–143
  contamination 143
Schmidhuber, Jürgen 20, 75
SchNet 169
Searle, John 233
seq2seq (sequence-to-sequence) models 97, 113
  order matters 119–122
sequence-wise batch normalization 85
SGD (stochastic gradient descent) 38, 212
Shanahan, Murray 5
shape, defined 133
SIFT (Scale-Invariant Feature Transform) 20, 25, 63
simplicity 187
Skinner, B.F. 234
Smolensky, Paul 182
Sort-of-CLEVR 162
SortaGrad 86
SOTA (state-of-the-art) 183
sparse prediction task 55
SQuAD 144
state descriptions 161
stochastic dynamics 223
Superalignment team 5
superintelligence 216, 228
  Legg's definition of intelligence 231–244
  machine superintelligence 217–225
SURF (Speeded-Up Robust Features) 20
Sutskever, Ilya 64, 79, 185
  rise of 3–7
Sutton, Richard 63
SVMs (support vector machines) 19, 25

T

tanh (hyperbolic tangent) 28
technological singularity 227
Tegmark, Max 222
Tensor2Tensor library 107
TensorFlow 3
theory to practice to theory 210–212
towers architecture 168
TPU (tensor processing unit) 147
training
  AlexNet 37–39
  challenges of 27–30
  efficient and scalable 37–39
Transformer-XL 176
transformers 96–106
  empirical results 102
  Universal Transformer 103–106
Traveling Salesman Problem (TSP) 116
Turing test 237–238

U

universal intelligence measure 219–221
UT (Universal Transformer) 103–106

V

VAEs (Variational autoencoders) 156
vanishing gradient problem 44, 80

Licensed to Hermilo Cortés <hermilocg@gmail.com>

Vaswani et al. (2017), *Attention Is All You Need*, effects of 125–128
VGGNet (Visual Geometry Group Network) 18
  ResNet vs. 49
Vinyals, Oriol 79
VLAEs (Variational Lossy Autoencoders) 154–159
  autoregressive prior 158
  overview 154–156
  results 157

W

WER (word error rates) 73
Williams, Ronald 18
wireheading 224
Wittgenstein, Ludvig 255–256
word2vec 73

Y

Yu, Fisher 56

Z

Zaremba, Wojciech 79
ZeRO (Zero Redundancy Optimizer) 151
ZFNet (Zero-Feedforward Network), ResNet vs. 49
ZPVE (zero-point vibrational energy) 167

Licensed to Hermilo Cortés <hermilocg@gmail.com>

(continued from back cover)

for hype or cynicism. Complex concepts are clarified through relevant examples, vivid anecdotes, and practical engineering insights.

Each of the core papers examined in *Sutskever's List* represents a crucial steppingstone in the evolution of the AI. You'll love how Richard Heimann combines a deep technical background with a journalistic eye, never losing sight of practical considerations and providing a stepping off point to understand where the technology goes next.

Sutskever's List features nine chapters, an epilogue, and a practical appendix, smoothly blending technical instruction with cultural and historical context. The result is a logically flowing book that remains highly accessible, navigable, and technically deep without requiring the reader to have a specialist's background.

"A clear, insightful overview of foundational papers of AI, connecting theory to practical modern applications."

—Anirban Majumder, Amazon

"If you've ever wanted to understand the intellectual DNA of modern AI, this is the book to read."

—Oliver Roskill, The Hacking Games

"Perfectly summarizes the highlights of the Sutskever's list while providing the historical and technical context."

—Nicolas Bievre, Meta

"A working reconstruction of the field's foundational ideas, the kind of understanding that only implementation produces."

—Amarda Shehu
George Mason University

"Transforms a decade of research into a powerful toolkit, codifying a philosophy of minimum innovation for maximum leverage."

—I-Sah Hsieh
State of North Carolina

"Gets across and connects most of the foundational ideas leading up to modern AI. A remarkable achievement."

—Peter Grünwald, Leiden University

Licensed to Hermilo Cortés <hermilocg@gmail.com>

"A perspective the field has needed. *Sutskever's List* delivers it with care and historical accuracy."

—Yanping Huang, Google

Sutskever's List is a guided intellectual journey through the ideas that made modern AI suddenly possible. Each chapter is anchored in specific papers, books, or other sources from Sutskever's list. The papers themselves are not the focus. Instead, the author uses them as entry points into the larger breakthroughs, arguments, interconnections, and shifts in thinking that transformed the field.

It begins with AlexNet, where data, GPUs, and training craft made neural networks impossible to dismiss, then moves to ResNet, where depth becomes a superpower rather than a liability. From there, the story accelerates through sequence models, speech systems, attention, Transformers, and hyperscale, showing how AI escaped older bottlenecks and became built to grow.

Later chapters ask whether these systems can reason, why simplicity can emerge from complexity, and what intelligence and safety mean once AI capabilities begin to feel uncanny. Reviewers praise Heimann's "exquisitely deep, detailed, and nuanced knowledge" and the "massive amount of gold material" gathered here. Yet the book remains remarkably easy to read, turning difficult papers into a "guided initiation those papers were never designed to provide on their own."

As you go, you'll understand how abstract lab results have translated into real-world consequences, including shifting architectures and internal organizational politics. With lucid explanations of the core technologies of AI as defined in Sutskever's collection of seminal papers, Heimann explores common engineering choices, evaluating the strengths and limits of deep learning without falling

(continued on last page)

The book covers

- Decoding landmark AI papers from AlexNet to transformers
- Understanding scaling laws, reasoning models, and AI safety
- Engineering patterns that scale from research to real-world systems

About the reader

For anyone interested in modern AI and deep learning. No specialist knowledge required.

About the author

Richard Heimann has honed his deep AI and machine learning expertise across technical and strategic roles in industry, academia, and government. He excels at translating complex ideas into clear, engaging insights for audiences from practitioners to policymakers.

FREE eBook
see first page

or go to
https://www.manning.com/freebook

ISBN-13: 978-1-63343-479-0

9 781633 434790 90000

MANNING

Licensed to Hermilo Cortés <hermilocg@gmail.com>